Seq2seq and Attention

materials: Lena Voita

speaker: George Yakushev

Lecture-blog and lots of additional materials are here:
https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Sequence to Sequence Task

- Translation between natural languages
- More generally, translation between any sequences



What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

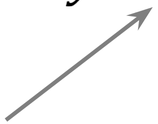
What is going to happen:

- Seq2seq Basics
  - Machine Translation Task
  - Encoder-Decoder Framework
  - Conditional LMs
  - The Simplest RNN Model
  - Training
  - Inference
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$



The “probability” is intuitive and is given by a human translator’s expertise

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The “probability” is intuitive and is given by a human translator’s expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

How does the model for $p(y|x, \theta)$ look like?

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

How does the model for $p(y|x, \theta)$ look like?

- **learning**

How to find θ?

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

  How does the model for $p(y|x, \theta)$ look like?

- **learning**

  How to find θ?

- **search**

  How to find the argmax?

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

How does the model for $p(y|x, \theta)$ look like?

- **learning**

How to find θ?

- **search**

How to find the argmax?

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

  How does the model for $p(y|x, \theta)$ look like?

- **learning**

  How to find θ?

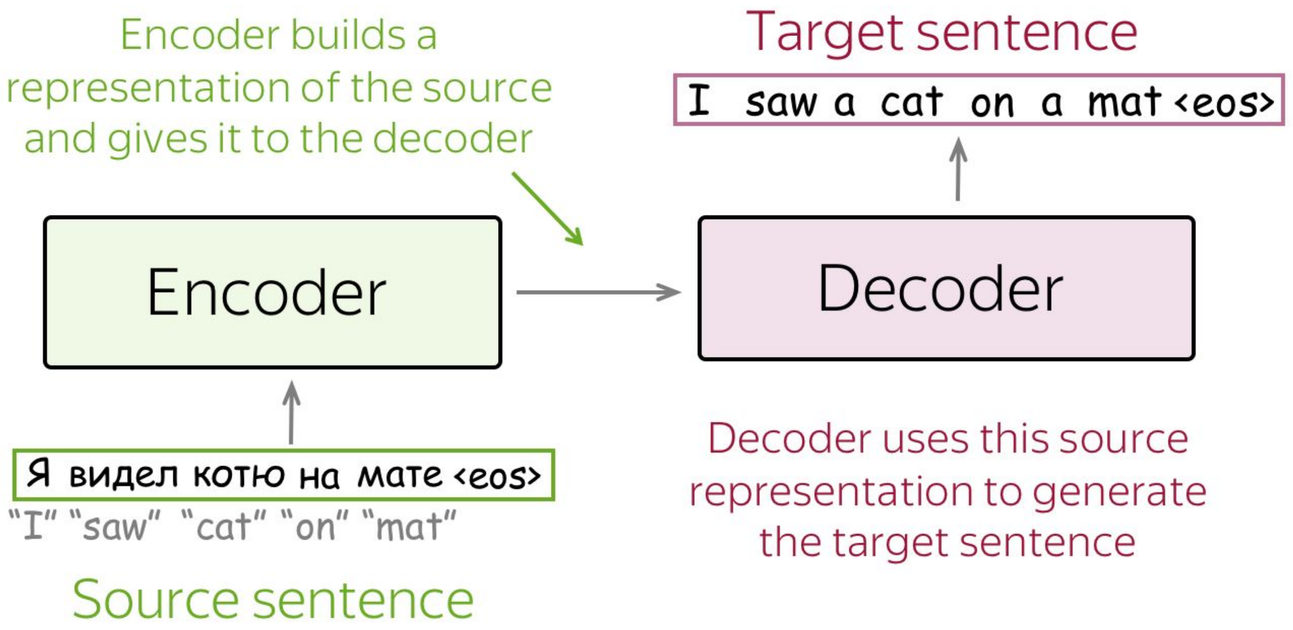
- **search**

  How to find the argmax?

Encoder-Decoder Framework

The standard modeling paradigm:

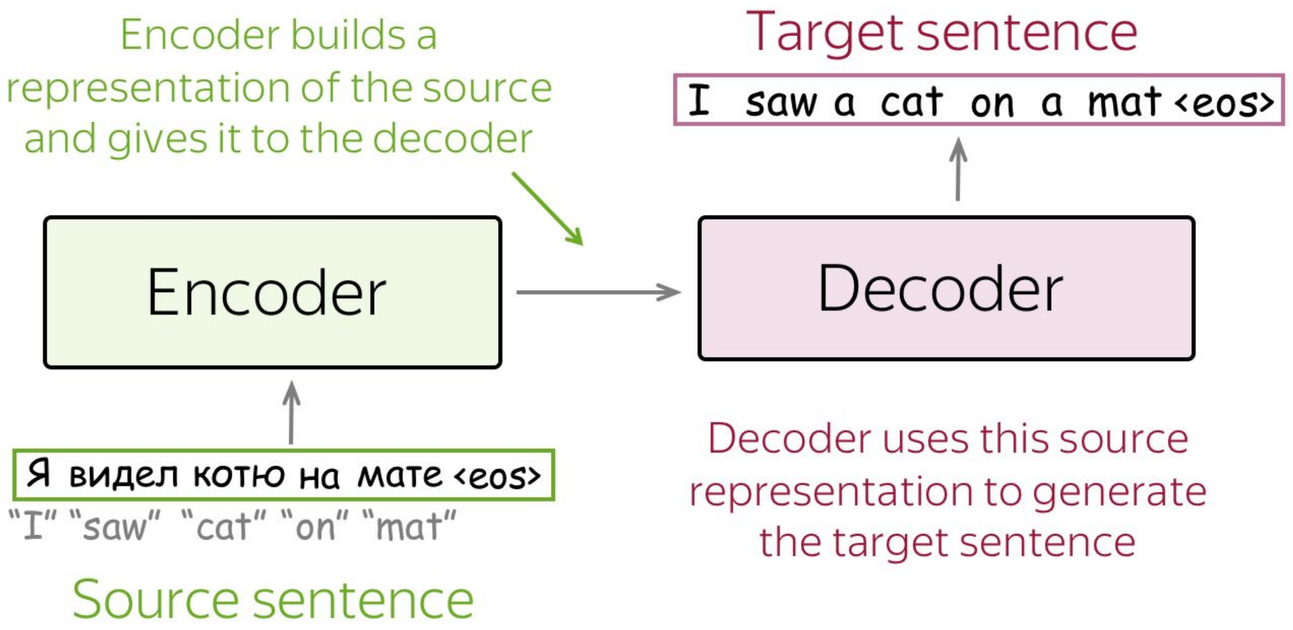
- **Encoder** – reads the source sentence and produces its representation



Encoder-Decoder Framework

The standard modeling paradigm:

- **Encoder** – reads the source sentence and produces its representation
- **Decoder** - uses source representation from the encoder to generate the target sequence.



Conditional Language Models

Language Models: $P(y_1, y_2, \ldots, y_n) = \prod_{t=1}^{n} p(y_t | y_{<t})$

(left-to-right)

Conditional Language Models

Language Models: $P(y_1, y_2, \ldots, y_n) = \prod_{t=1}^{n} p(y_t | y_{<t})$

(left-to-right)

Conditional Language Models: $P(y_1, y_2, \ldots, y_n, | x) = \prod_{t=1}^{n} p(y_t | y_{<t}, x)$

condition on source x

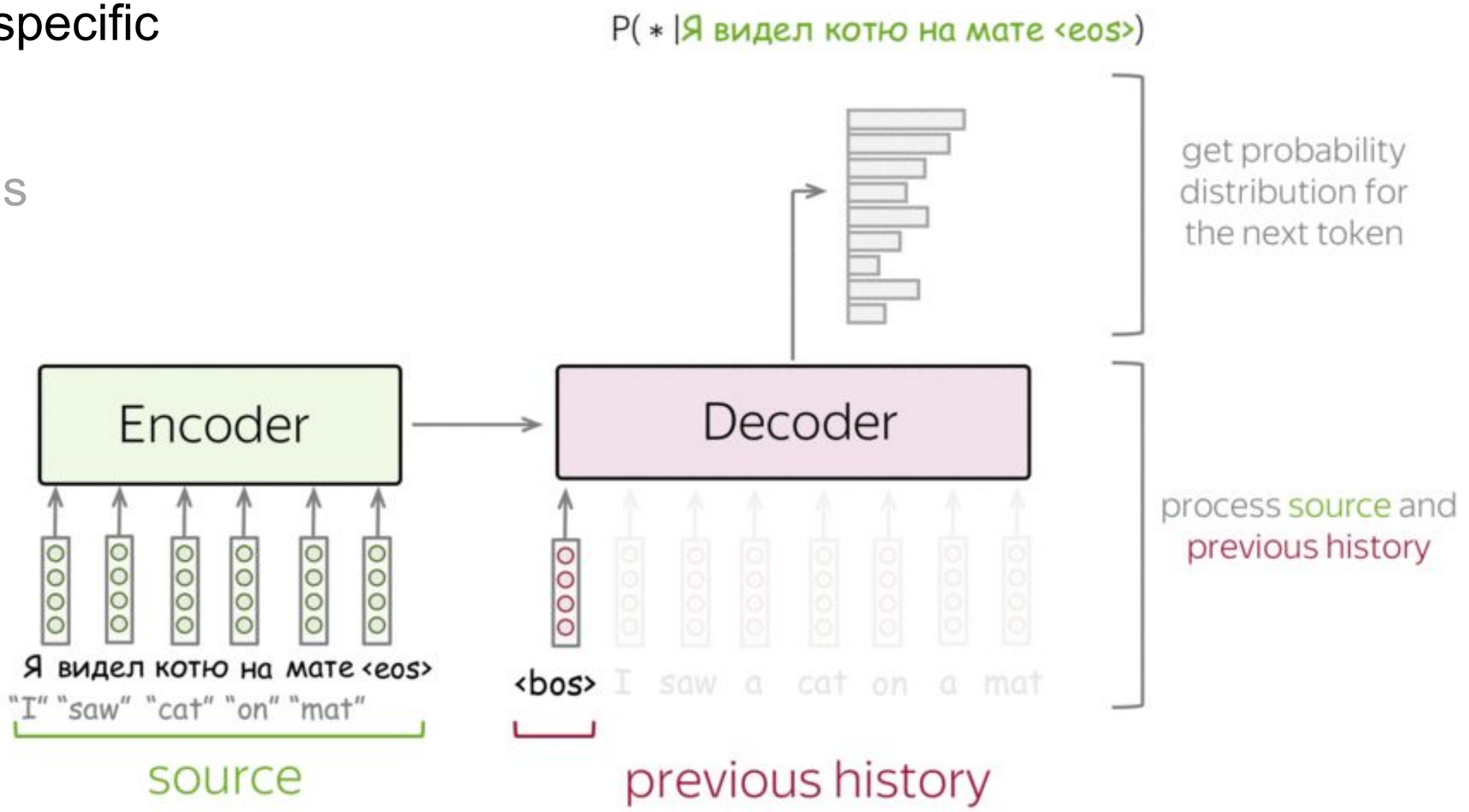
General View

- process context – model-specific

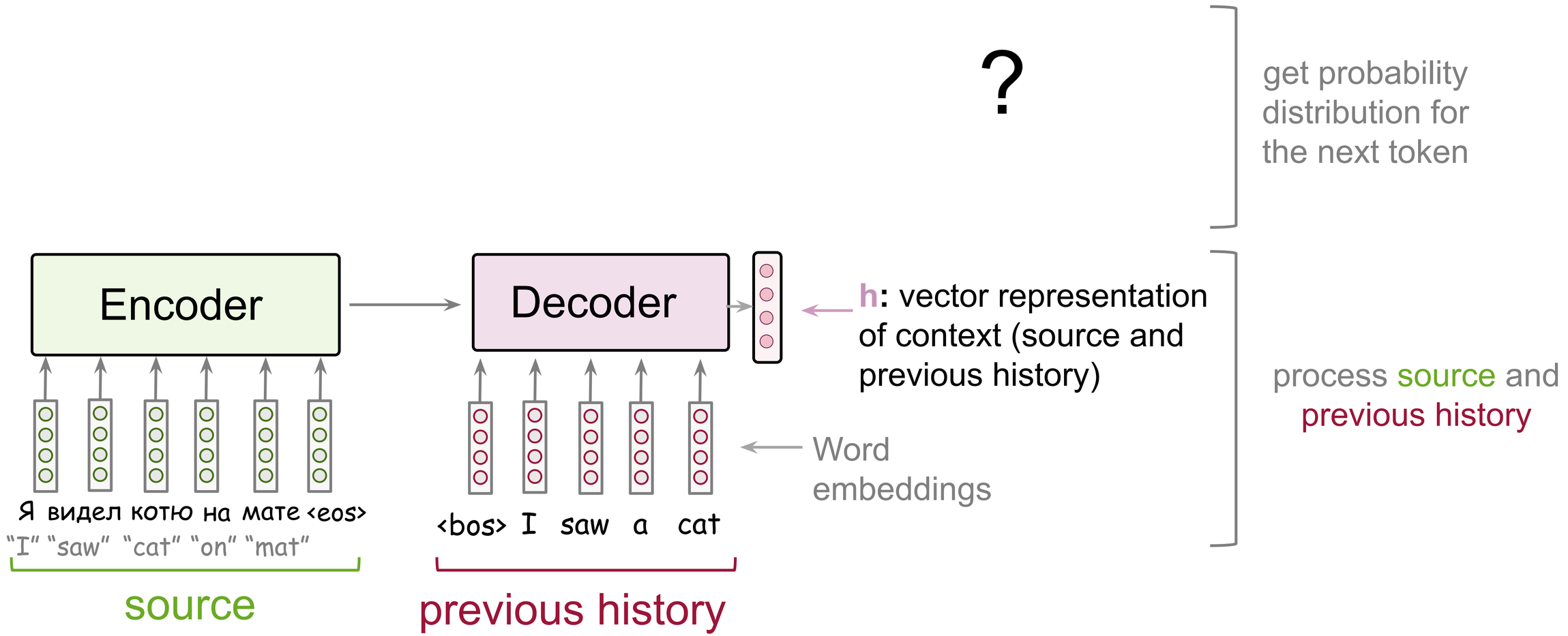
  Get vector representation of the source and previous target tokens

- evaluate probabilities – model-agnostic

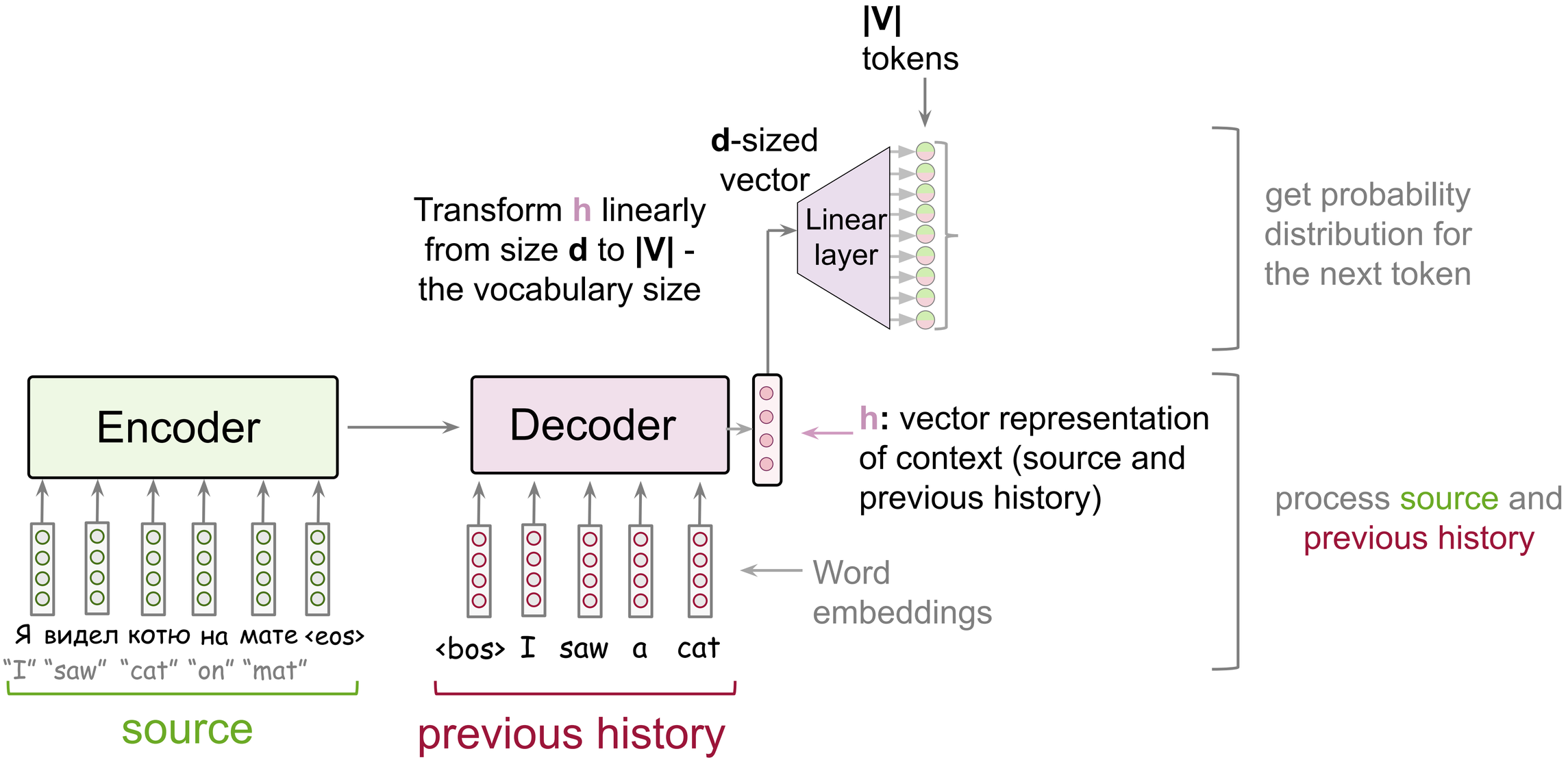
  Predict probability distribution for the next target token



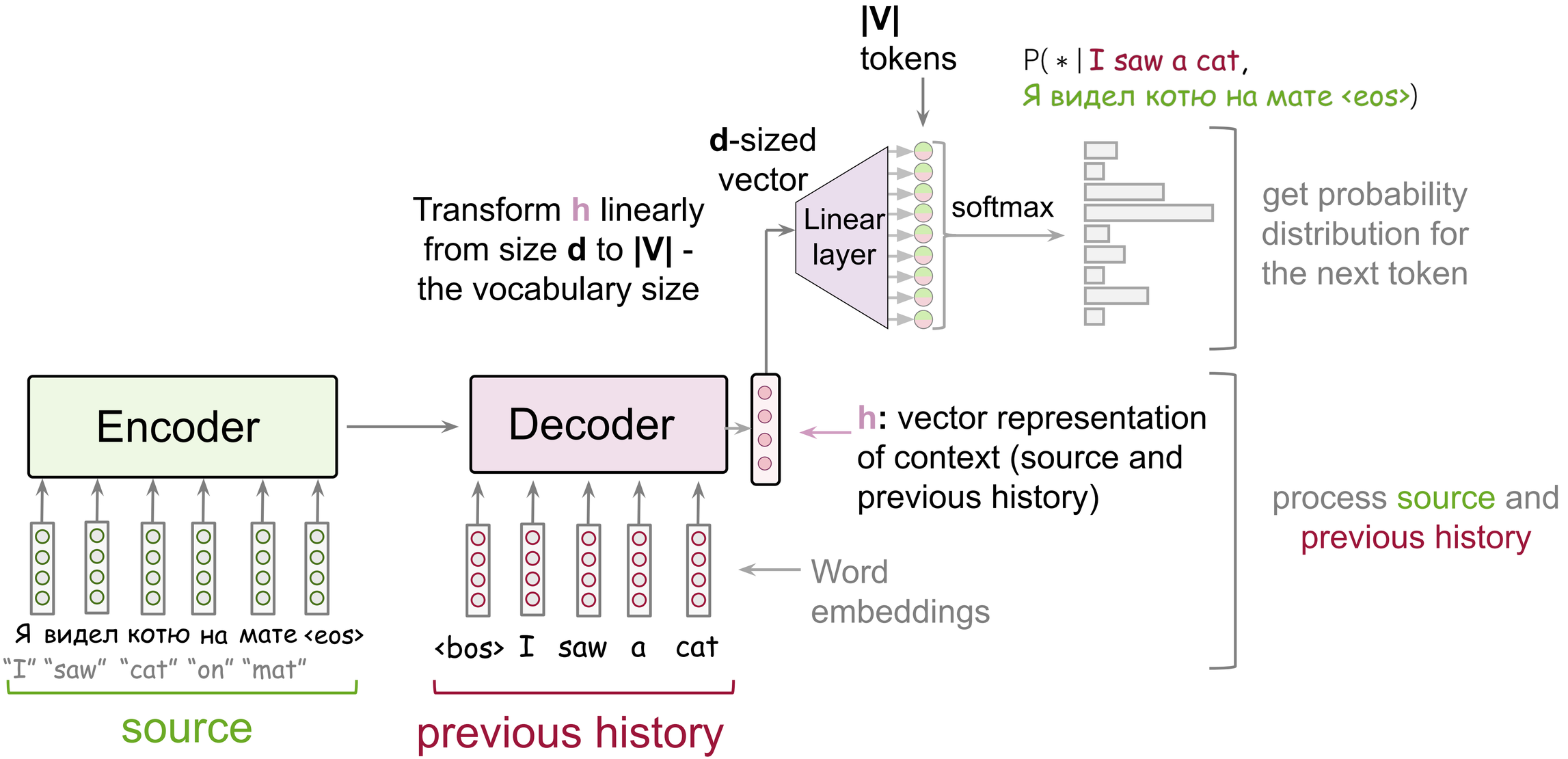
High-Level Pipeline



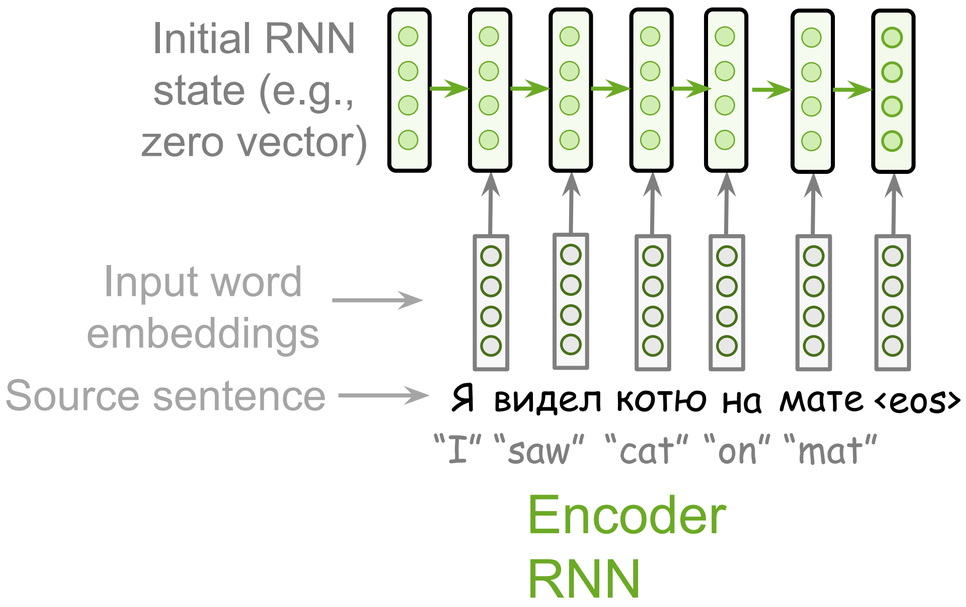
High-Level Pipeline



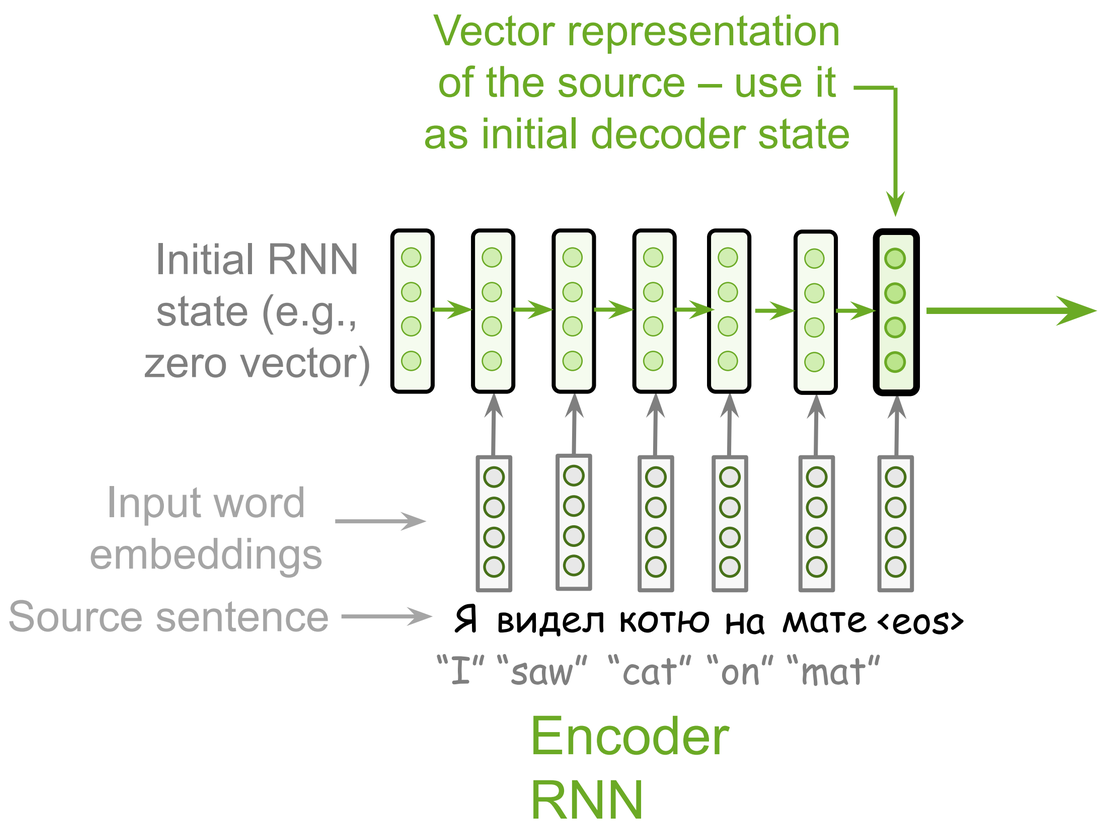
High-Level Pipeline



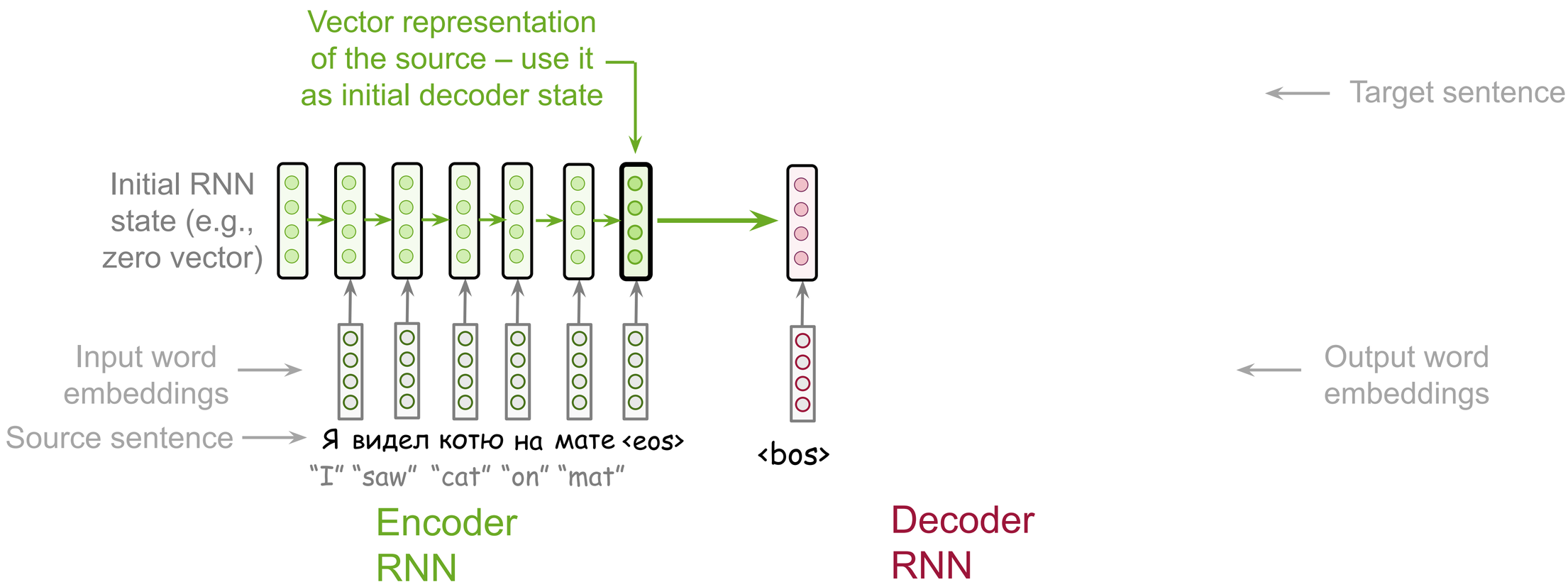
The Simplest Model: RNN Encoder and Decoder



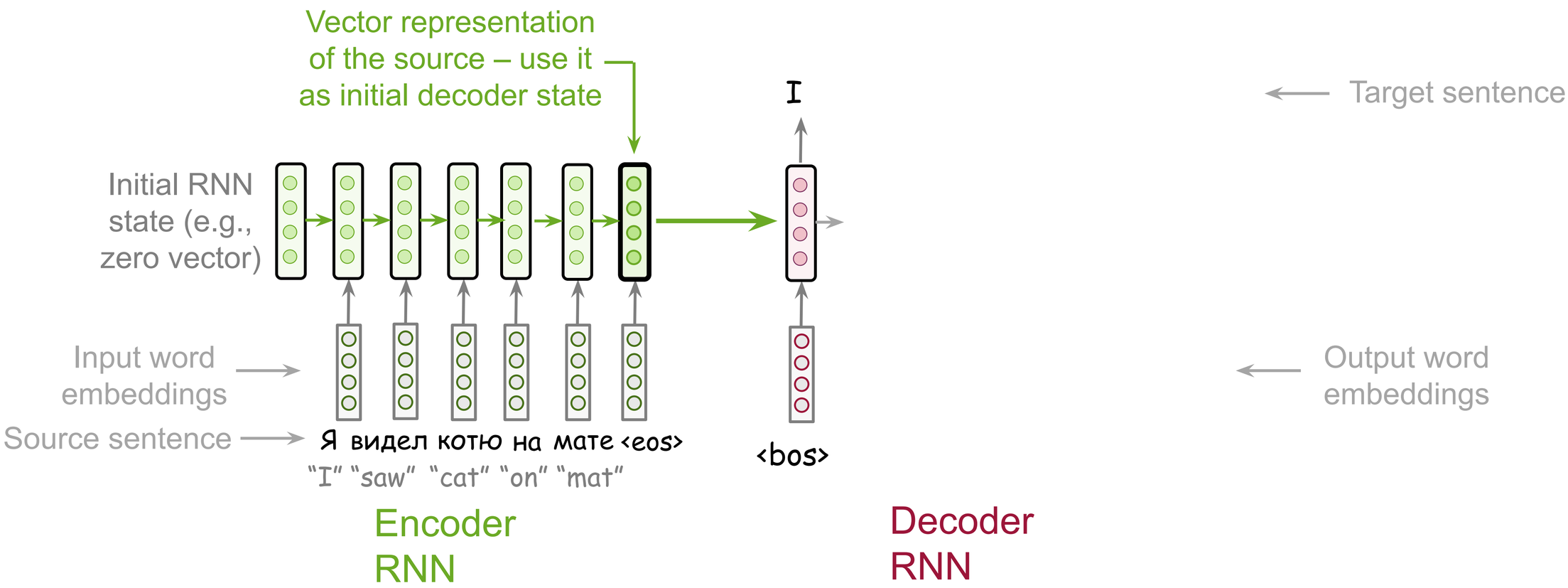
The Simplest Model: RNN Encoder and Decoder



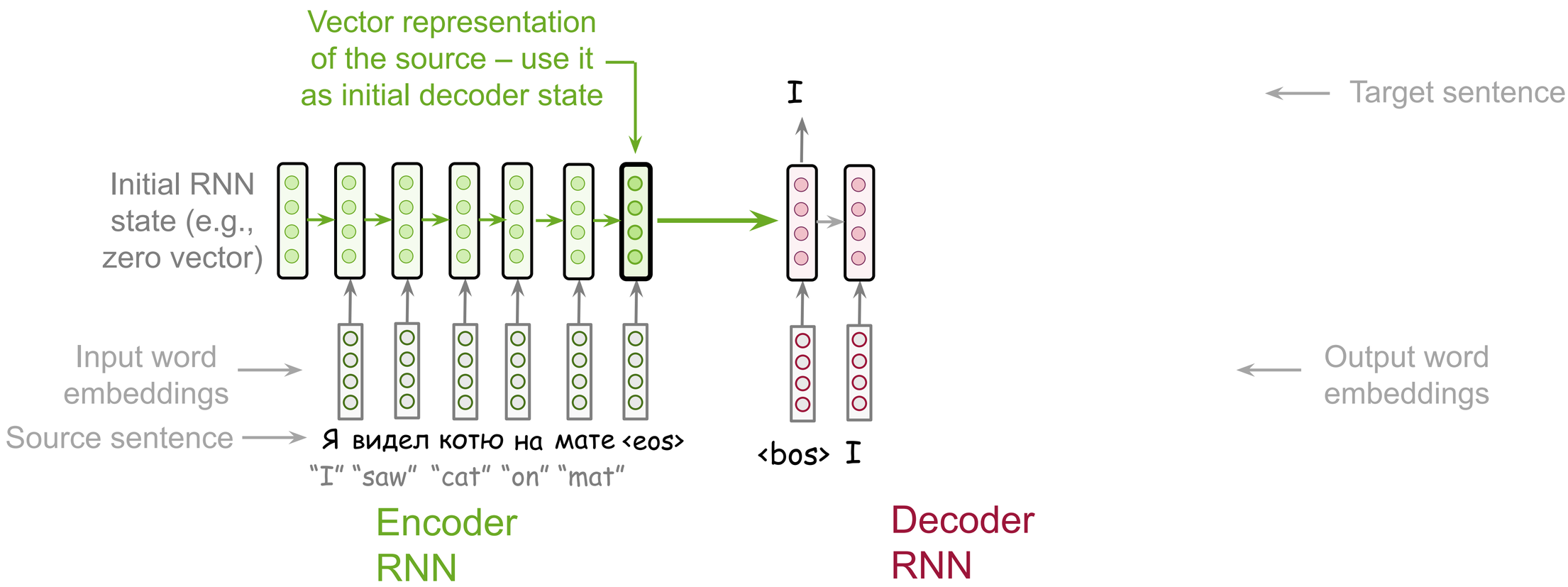
The Simplest Model: RNN Encoder and Decoder



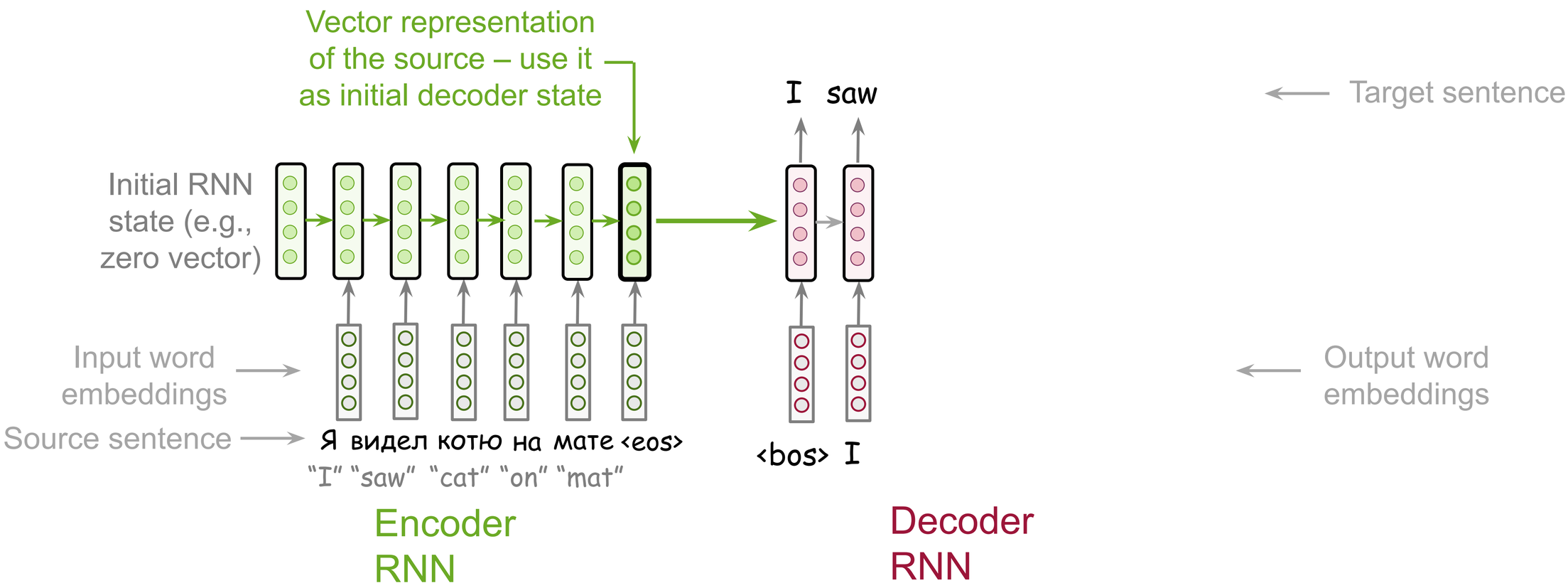
The Simplest Model: RNN Encoder and Decoder



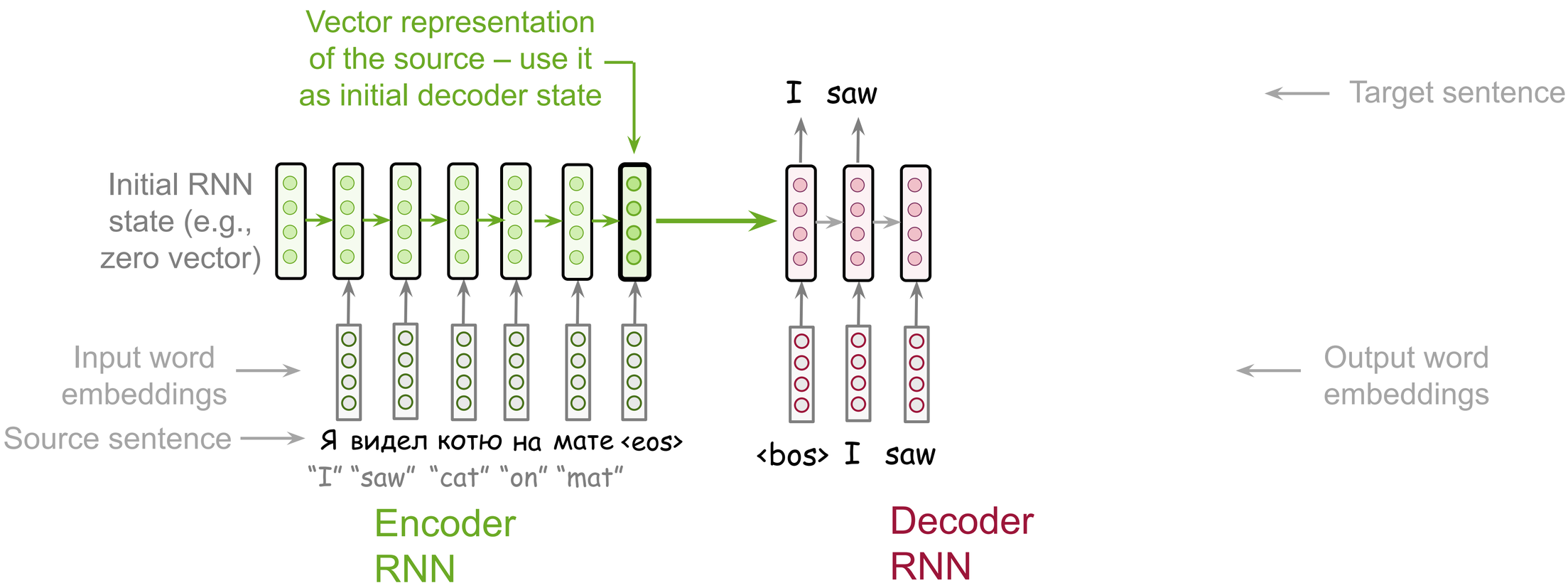
The Simplest Model: RNN Encoder and Decoder



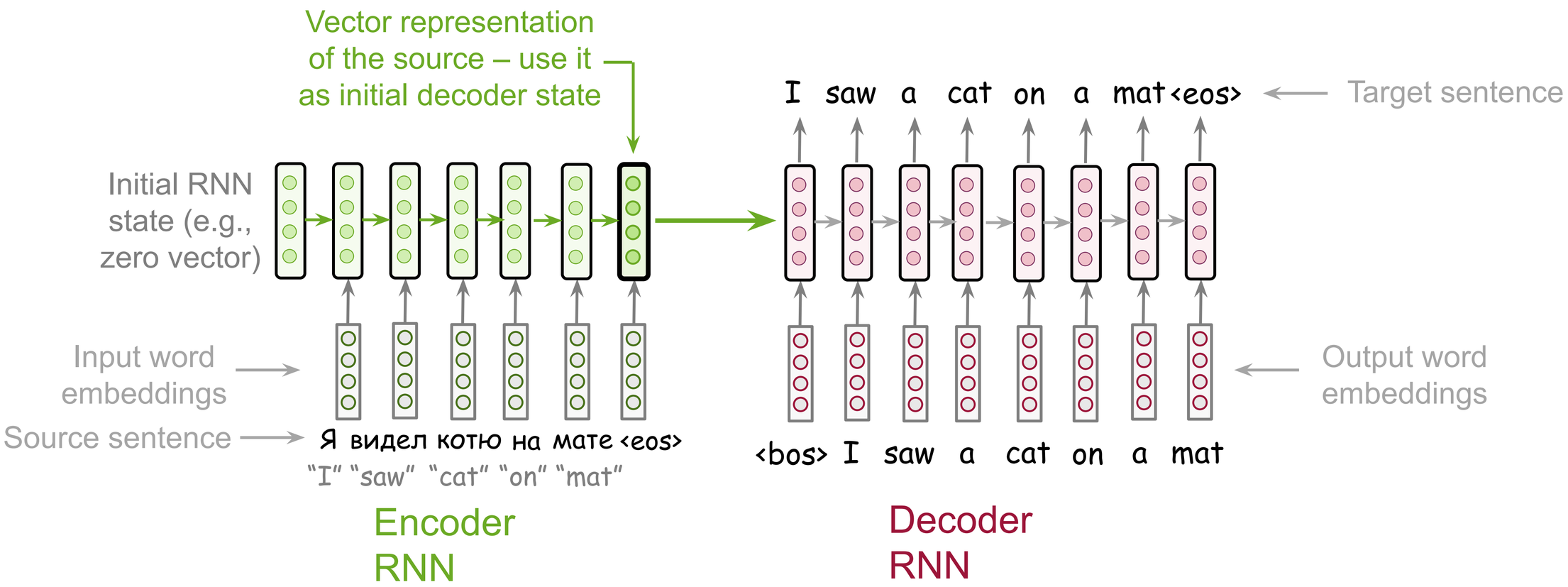
The Simplest Model: RNN Encoder and Decoder



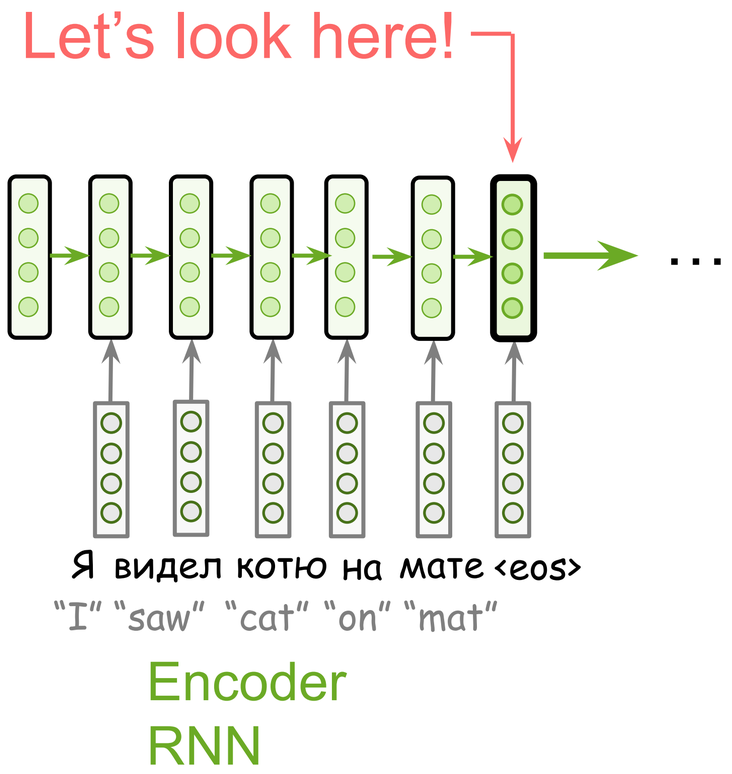
The Simplest Model: RNN Encoder and Decoder

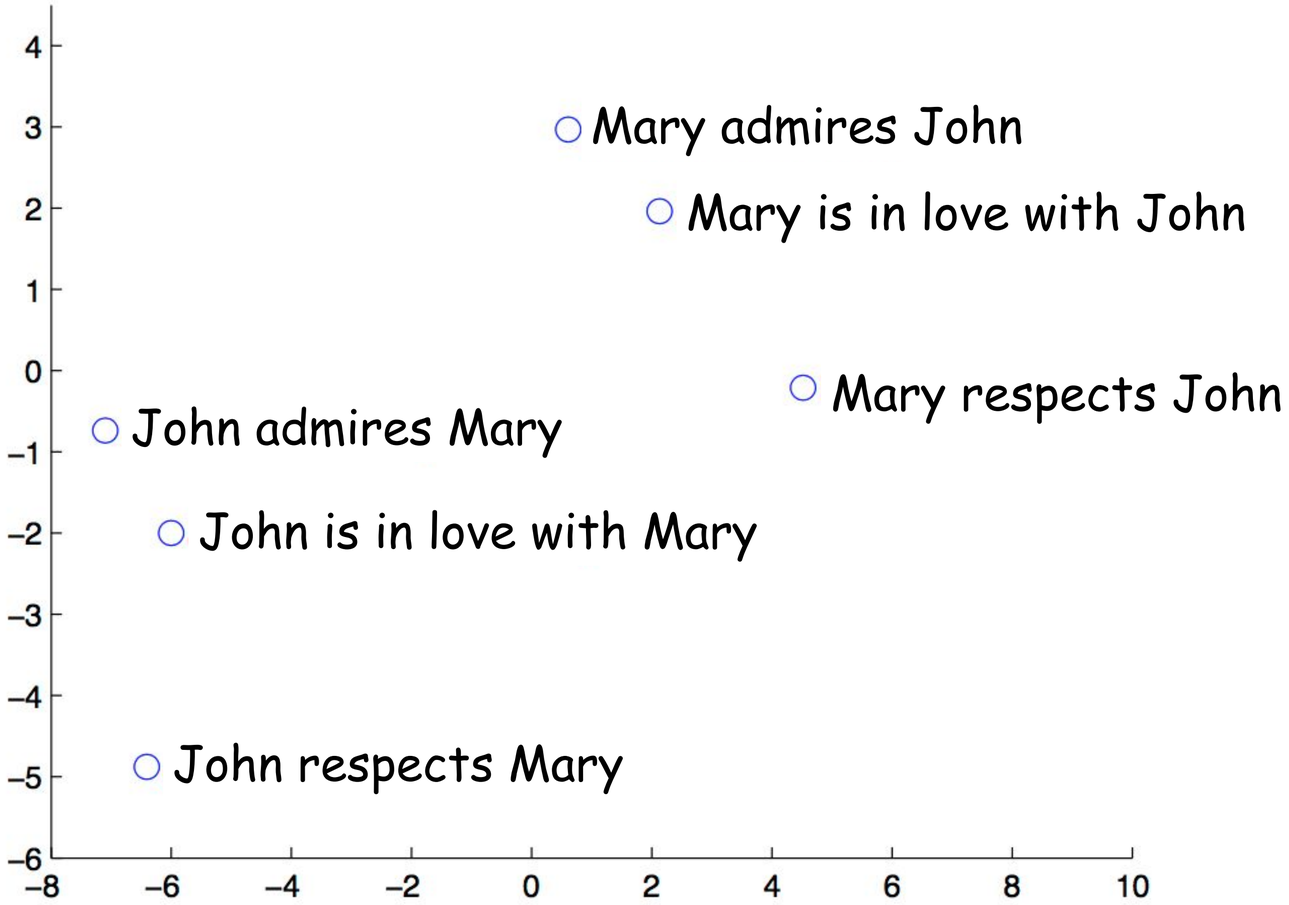


The Simplest Model: RNN Encoder and Decoder



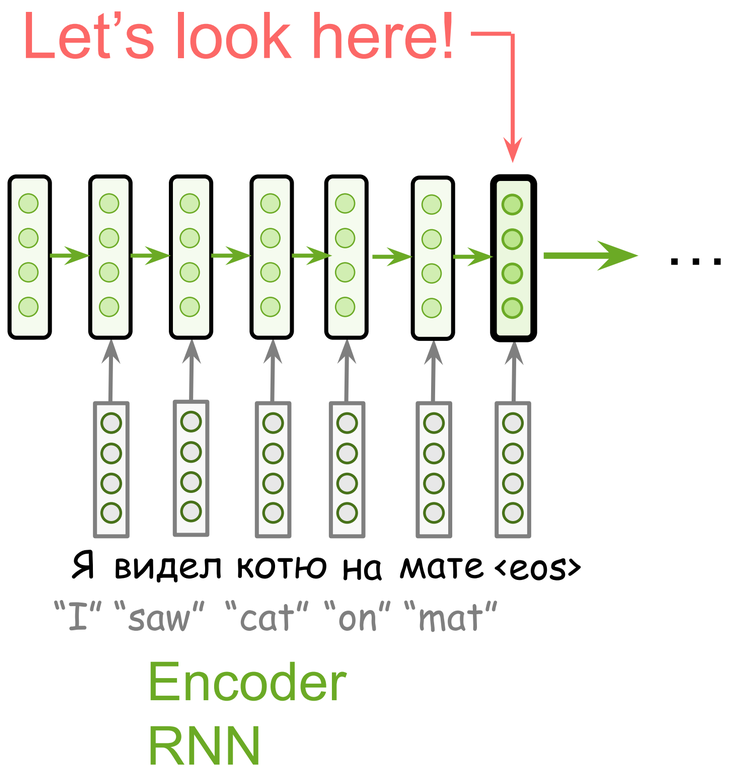
What does final encoder state represent?

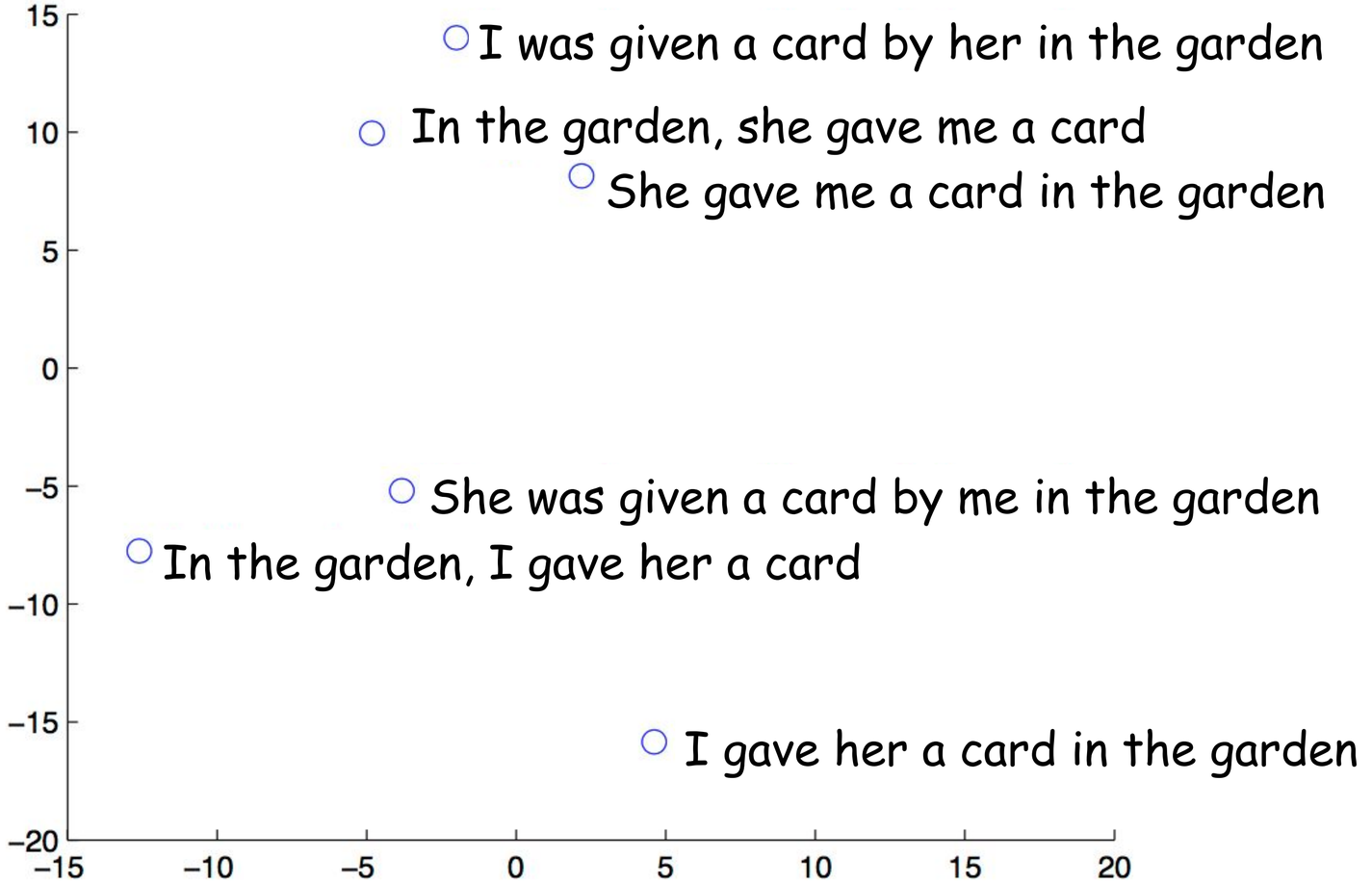




The examples are from the paper Sequence to Sequence Learning with Neural Networks

What does final encoder state represent?





The examples are from the paper Sequence to Sequence Learning with Neural Networks

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

  How does the model for $p(y|x, \theta)$ look like?

- **learning**

  How to find θ?

- **search**

  How to find the argmax?

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

  How does the model for $p(y|x, \theta)$ look like?

- **learning**

  How to find θ?

- **search**

  How to find the argmax?

What is going to happen:

- Seq2seq Basics
  - Machine Translation Task
  - Encoder-Decoder Framework
  - Conditional LMs
  - The Simplest RNN Model
  - Training
  - Inference
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

Cross-Entropy – again!

Source sequence:

Я видел котю на мате <eos>

"I" "saw" "cat" "on" "mat"

Target sequence:

I saw a cat on a mat <eos>

one training example

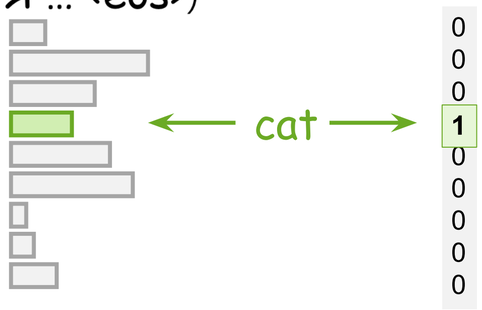
one step for this example

previous tokens

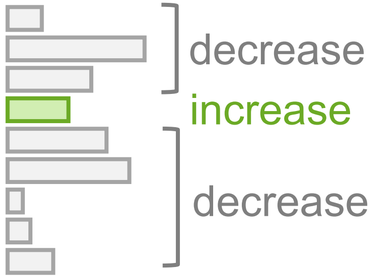
we want the model to predict this

Model prediction: p(* |I saw a, Я ... <eos>)

Target



Loss = -log (p(cat)) → min



Cross-Entropy – again!

Encoder: read source



we are here

Source: Я видел котю на мате <eos>

"I" "saw" "cat" "on" "mat"

Target: I saw a cat on a mat <eos>

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

  How does the model for $p(y|x, \theta)$ look like?

- **learning**

  How to find θ?

- **search**

  How to find the argmax?

Translation

Human Translation

$$y^* = \arg\max_y p(y|x)$$

The "probability" is intuitive and is given by a human translator's expertise

Machine Translation

model parameters

$$y' = \arg\max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

  How does the model for $p(y|x, \theta)$ look like?

- **learning**

  How to find θ?

- **search**

  How to find the argmax?

What is going to happen:

- Seq2seq Basics
  - Machine Translation Task
  - Encoder-Decoder Framework
  - Conditional LMs
  - The Simplest RNN Model
  - Training
  - Inference
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

How To Generate a Translation?

$$y' = \arg\max_{y} p(y|x) = \arg\max_{y} \prod_{t=1}^{n} p(y_t|y_{<t}, x)$$

Greedy Decoding

$$y' = \arg\max_{y} p(y|x) = \arg\max_{y} \prod_{t=1}^{n} p(y_t|y_{<t}, x)$$

Straightforward:

- **greedy** - at each step, pick token with the highest probability

Greedy Decoding

$$y' = \arg\max_y p(y|x) = \arg\max_y \prod_{t=1}^{n} p(y_t|y_{<t}, x)$$

Straightforward:

- **greedy** - at each step, pick token with the highest probability

Wait a minute…

Greedy Decoding

$$y' = \arg\max_y p(y|x) = \arg\max_y \prod_{t=1}^{n} p(y_t|y_{<t}, x)$$

Straightforward:

- **greedy** - at each step, pick token with the highest probability

$$\max_y \prod_{t=1}^{n} p(y_t|y_{<t}, x) \neq \prod_{t=1}^{n} \max_{y_t} p(y_t|y_{<t}, x)$$

- this is bad!

Beam Search

- At each step, keep several best hypotheses

Beam Search

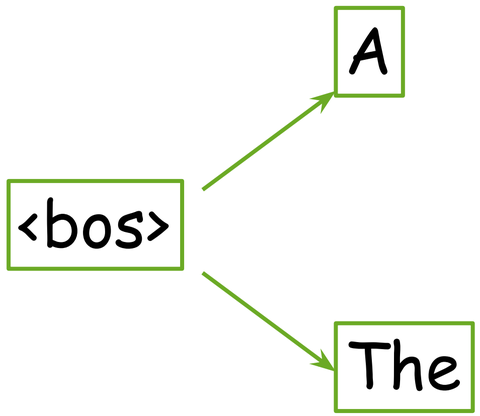
- At each step, keep several best hypotheses

<bos>

Start with the begin of sentence token or with an empty sequence

Beam Search

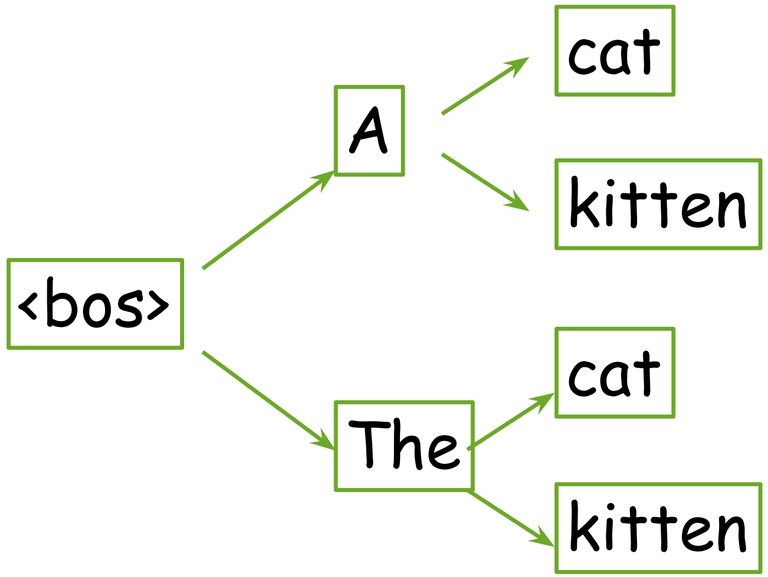
- At each step, keep several best hypotheses



Generate: `beam_size` most probable tokens

Beam Search

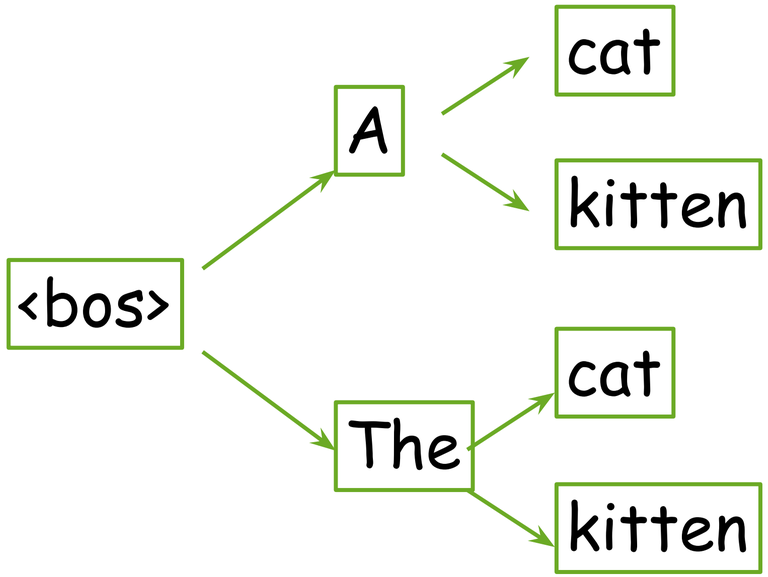
- At each step, keep several best hypotheses



Generate: `beam_size` most probable tokens

Beam Search

- At each step, keep several best hypotheses

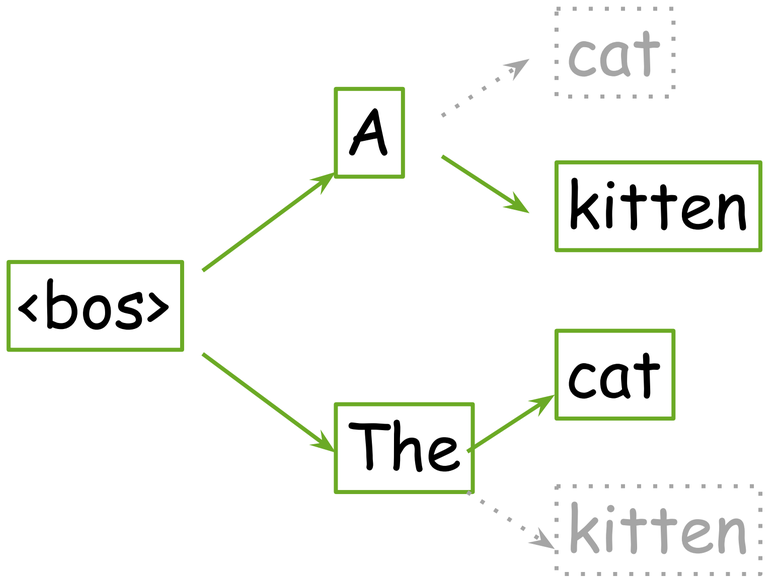


Look at probabilities: P(The cat) > P(A kitten) > P(A cat) > P(The kitten)

Top 2 hypos

Beam Search

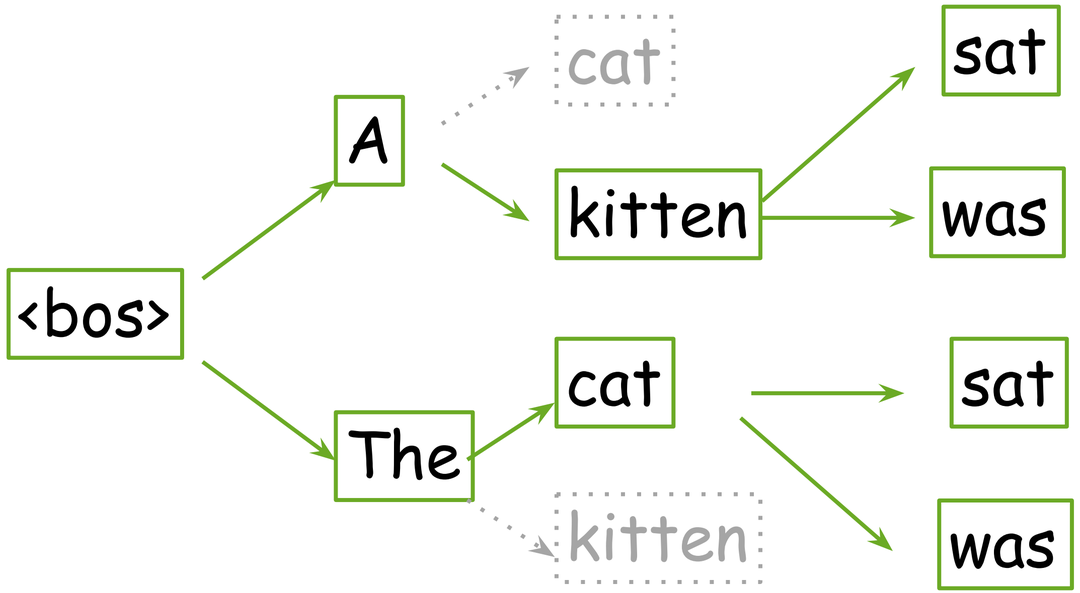
- At each step, keep several best hypotheses



Pick top `beam_size` hypos, terminate the rest

Beam Search

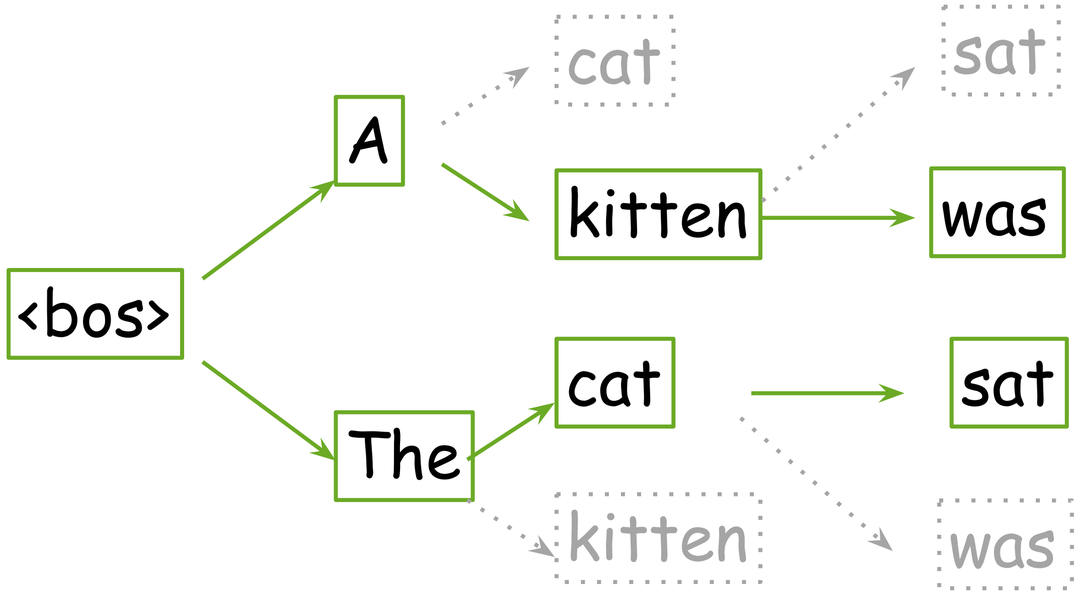
- At each step, keep several best hypotheses



Generate: `beam_size` most probable tokens

Beam Search

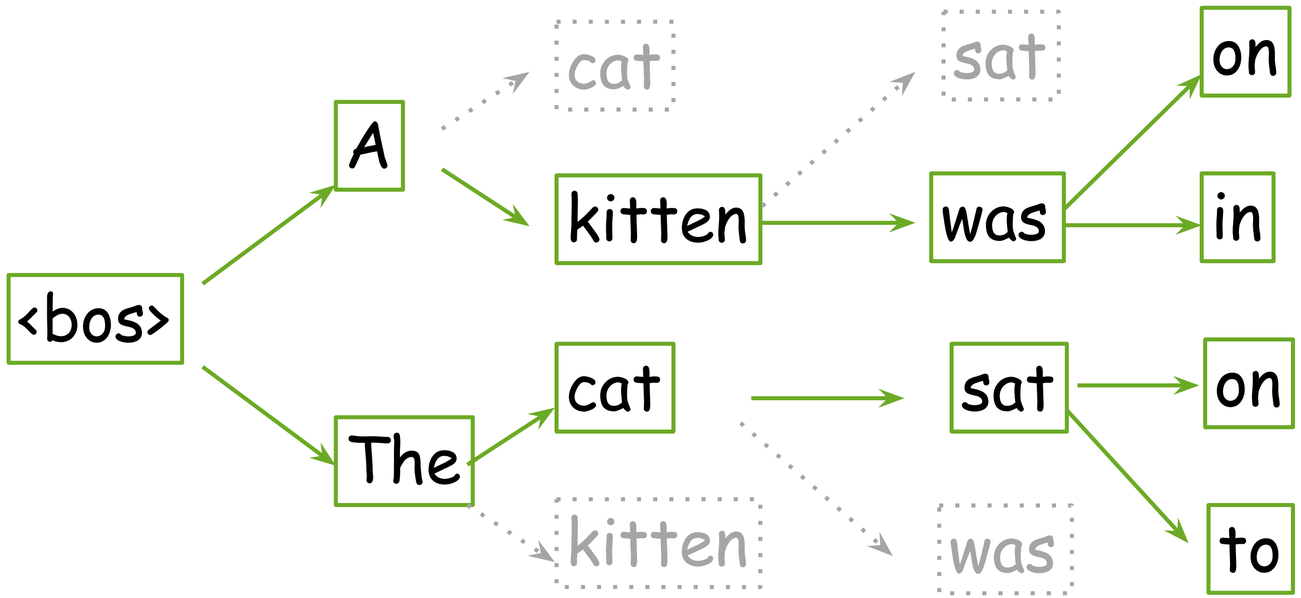
- At each step, keep several best hypotheses



Pick top `beam_size` hypos, terminate the rest

Beam Search

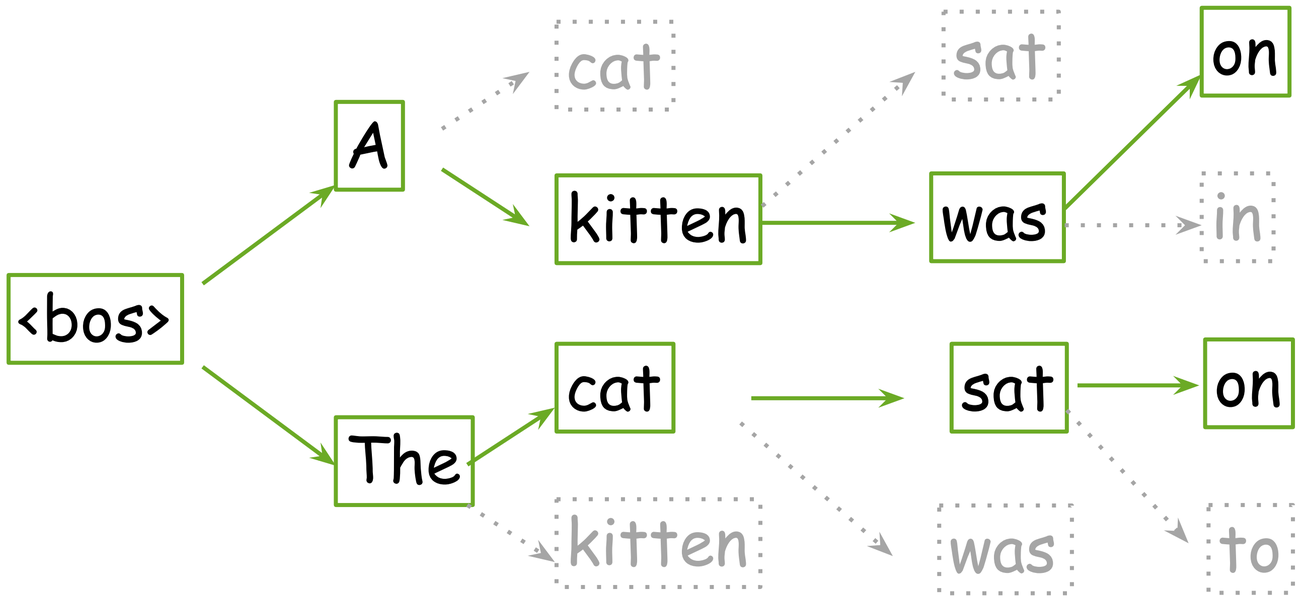
- At each step, keep several best hypotheses



Generate: `beam_size` most probable tokens

Beam Search

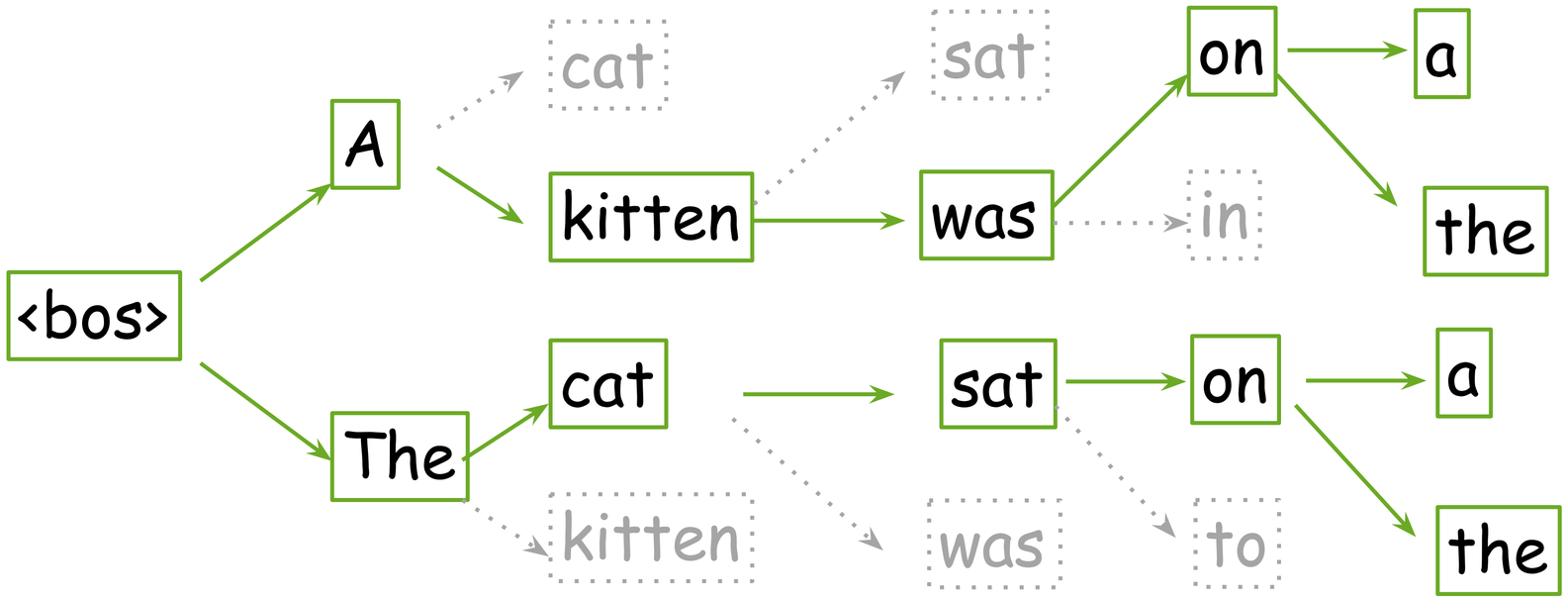
- At each step, keep several best hypotheses



Pick top `beam_size` hypos, terminate the rest

Beam Search

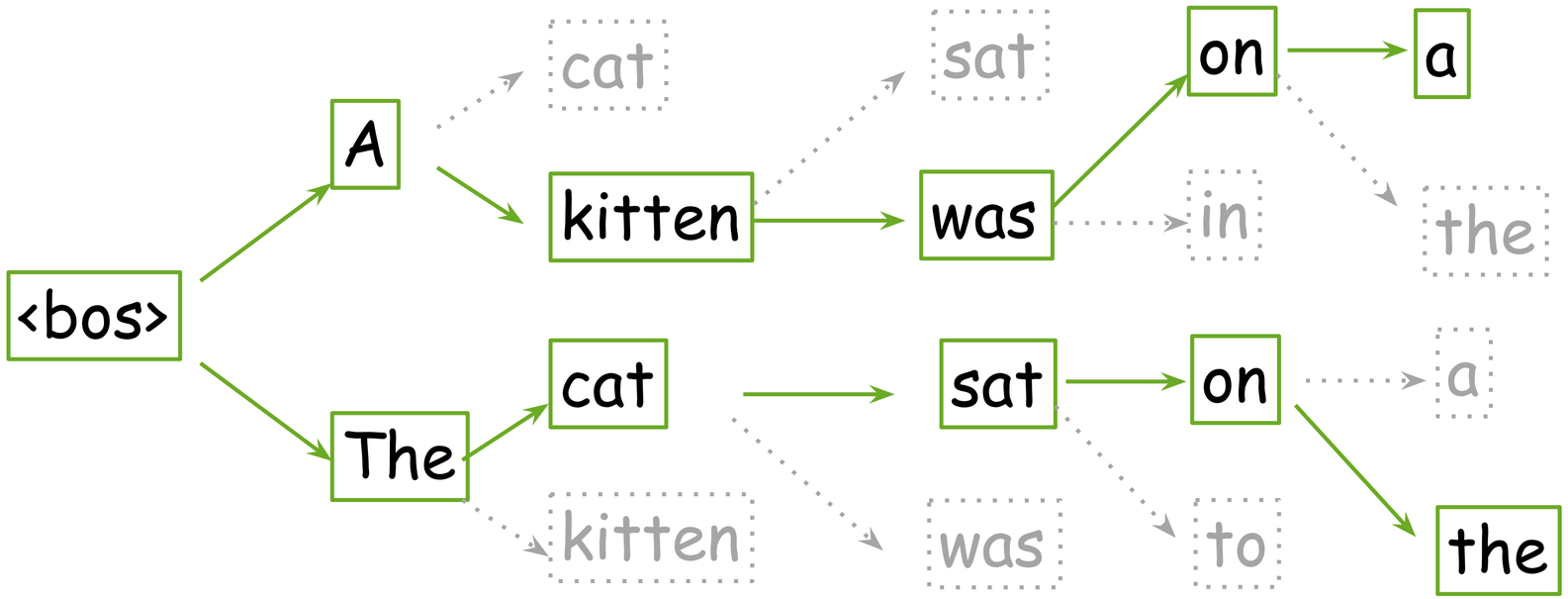
- At each step, keep several best hypotheses



Generate: `beam_size` most probable tokens

Beam Search

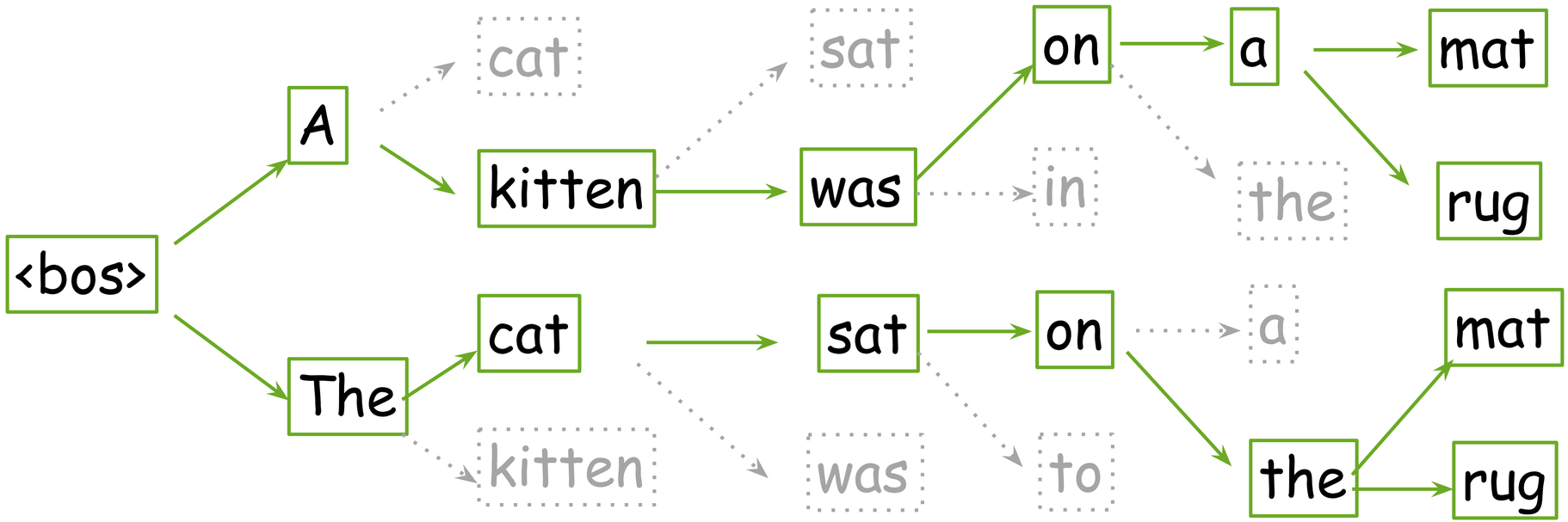
- At each step, keep several best hypotheses



Pick top `beam_size` hypos, terminate the rest

Beam Search

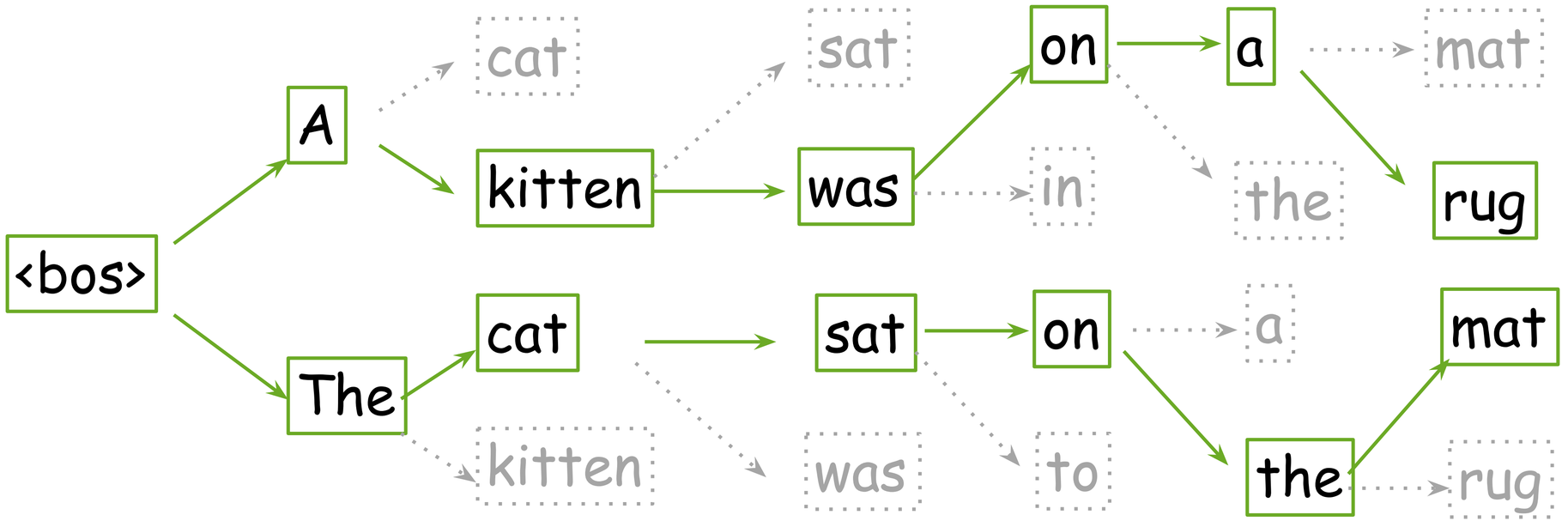
- At each step, keep several best hypotheses



Generate: `beam_size` most probable tokens

Beam Search

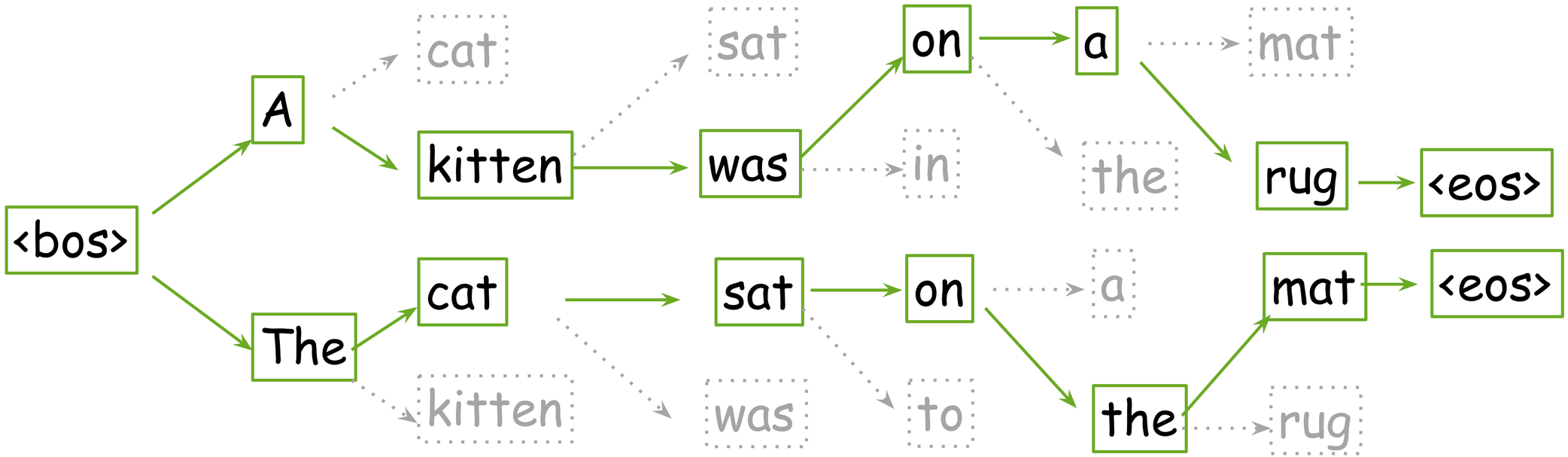
- At each step, keep several best hypotheses



Pick top `beam_size` hypos, terminate the rest

Beam Search

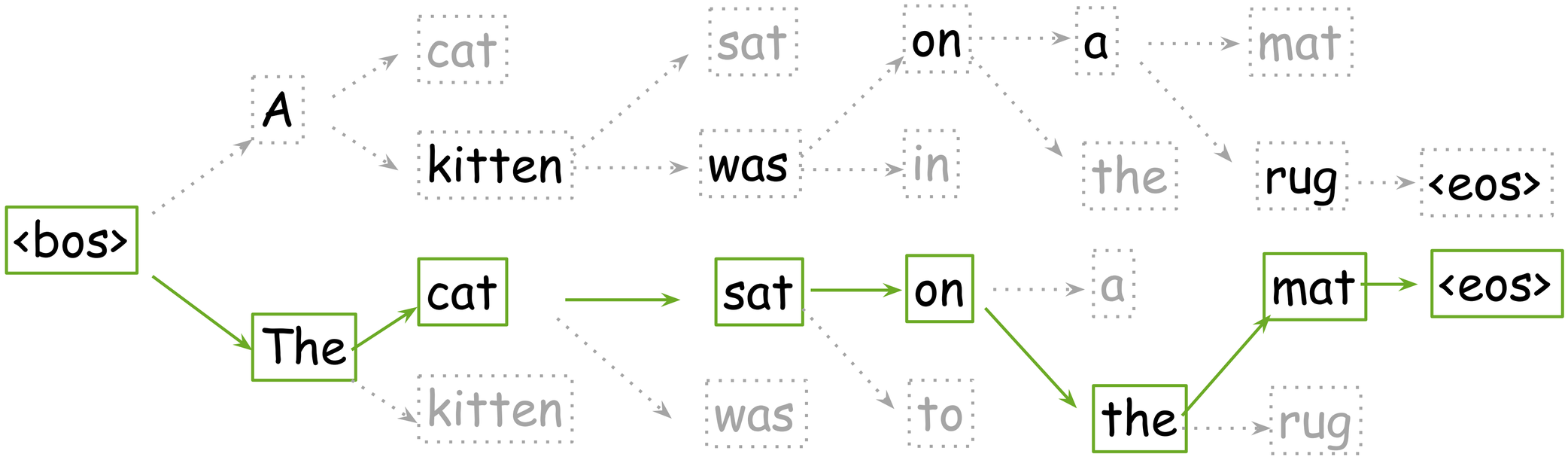
- At each step, keep several best hypotheses



All hypotheses are complete - generation ended

Beam Search

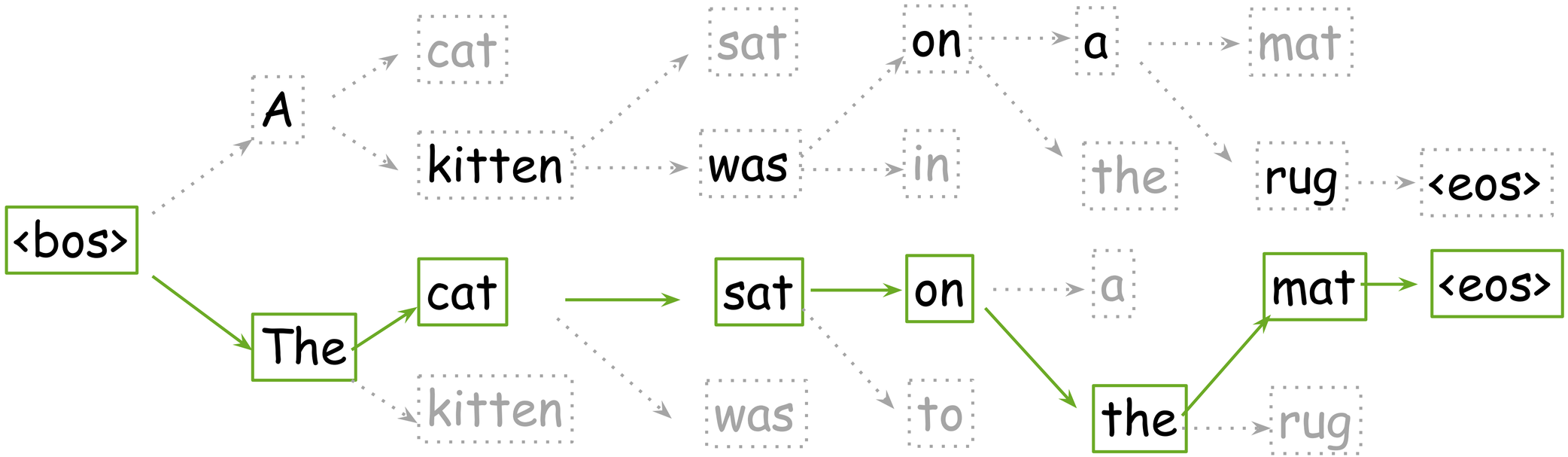
- At each step, keep several best hypotheses



Pick the hypothesis with the highest probability

Beam Search

- At each step, keep several best hypotheses



Result: The cat sat on the mat

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

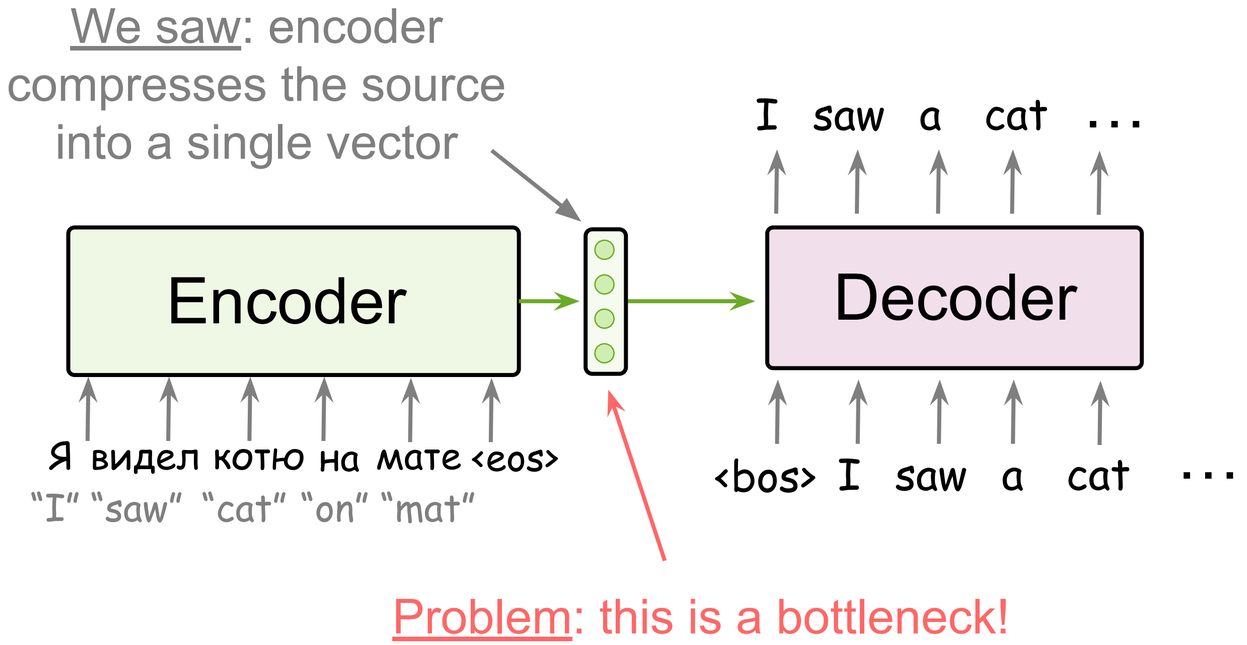
What is going to happen:

- Seq2seq Basics
- Attention
  - Why do we need it?
  - Attention: High-Level
  - Attention Score Functions
  - Models: Bahdanau vs Luong
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

The Problem of Fixed Encoder Representation

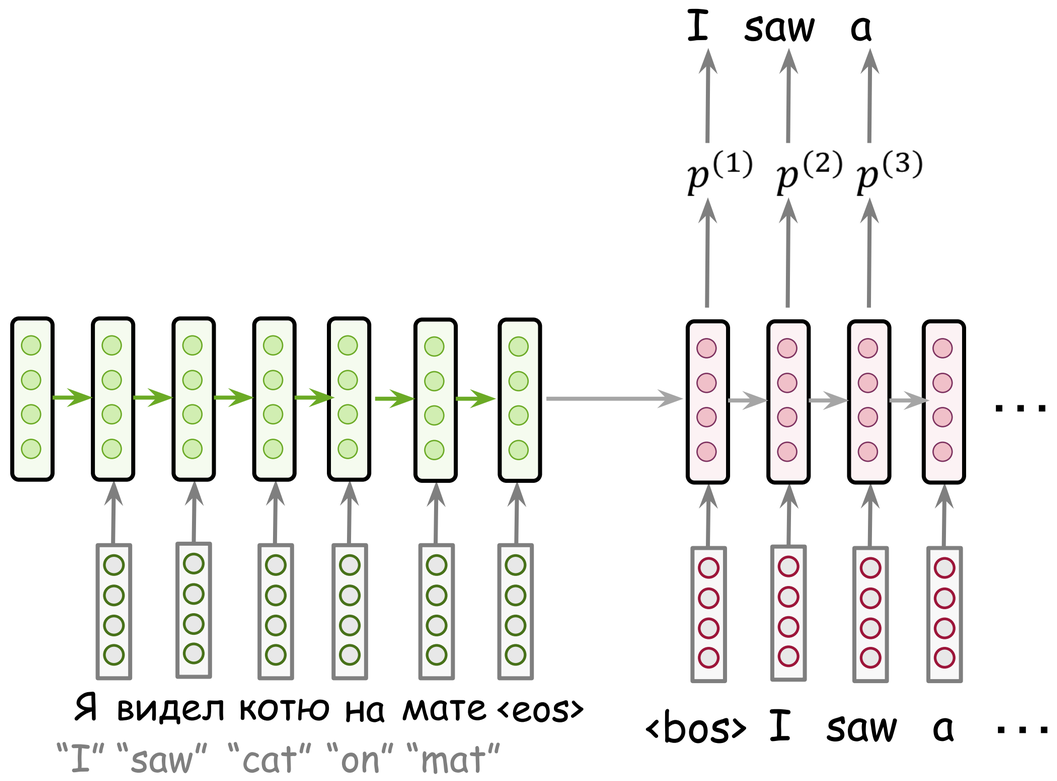
Fixed source representation is bad:

- for **encoder**, it is hard to compress a sentence
- for **decoder**, different information may be needed at different steps



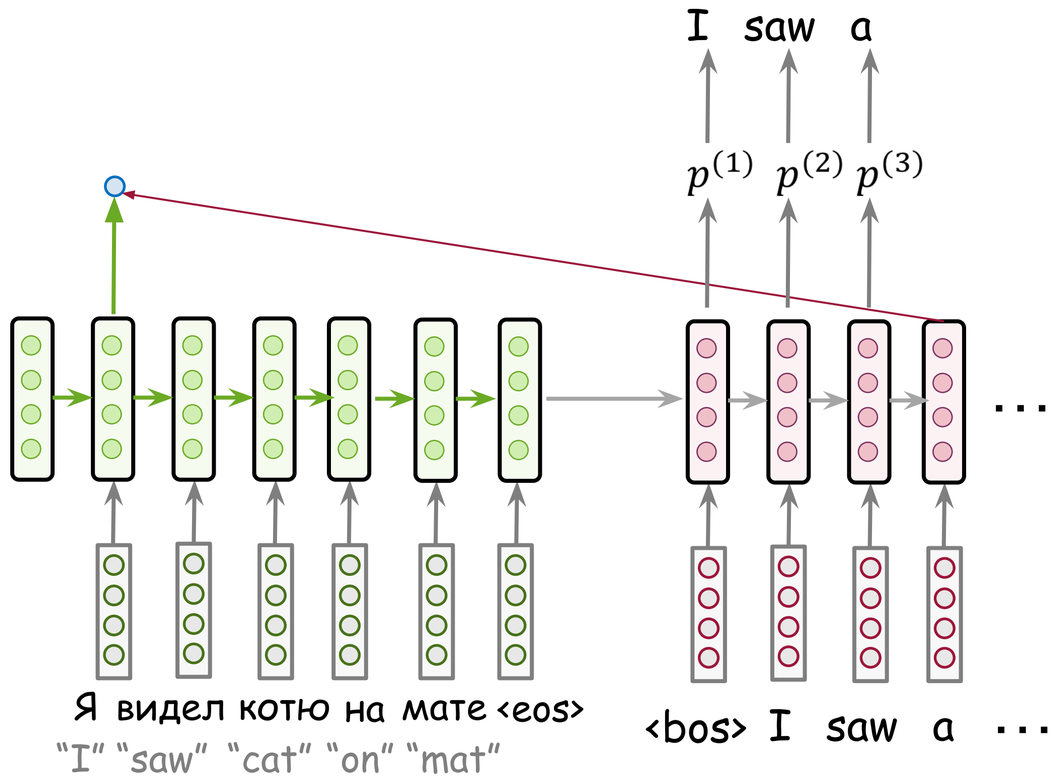
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



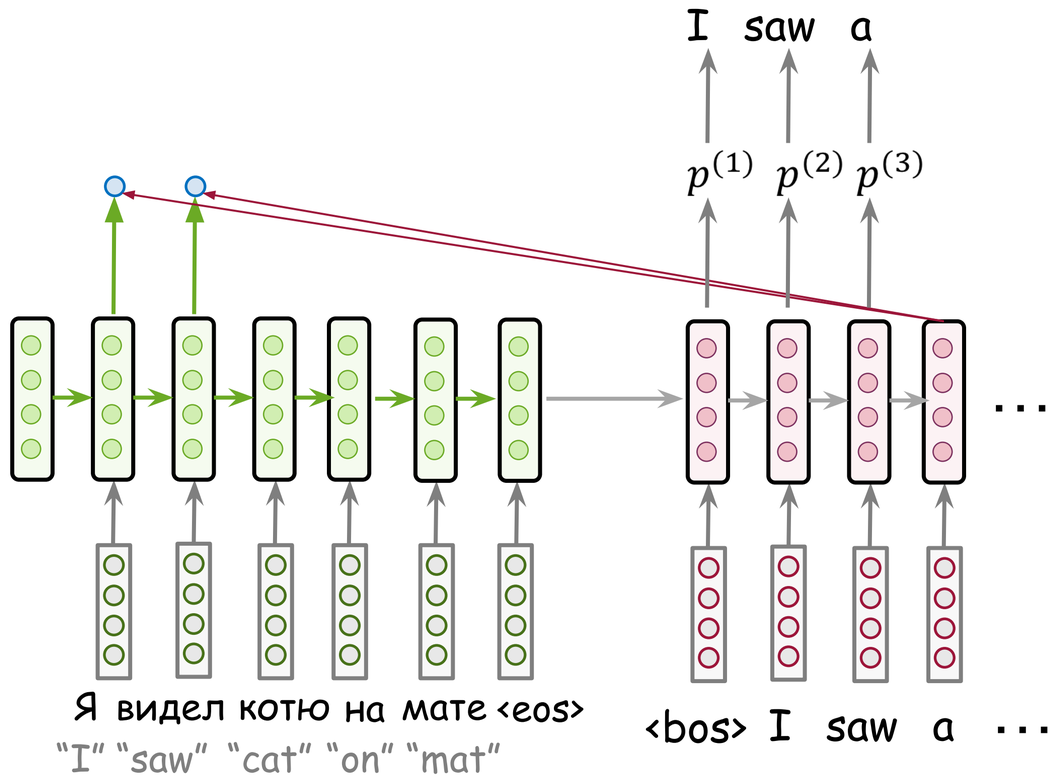
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



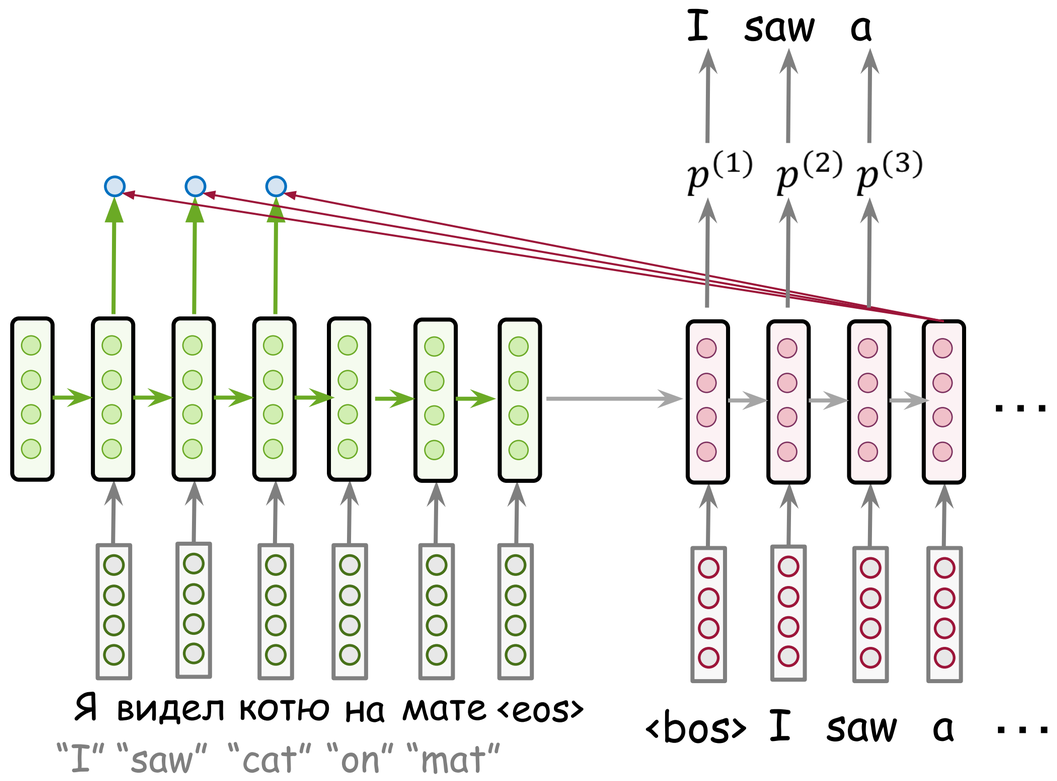
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



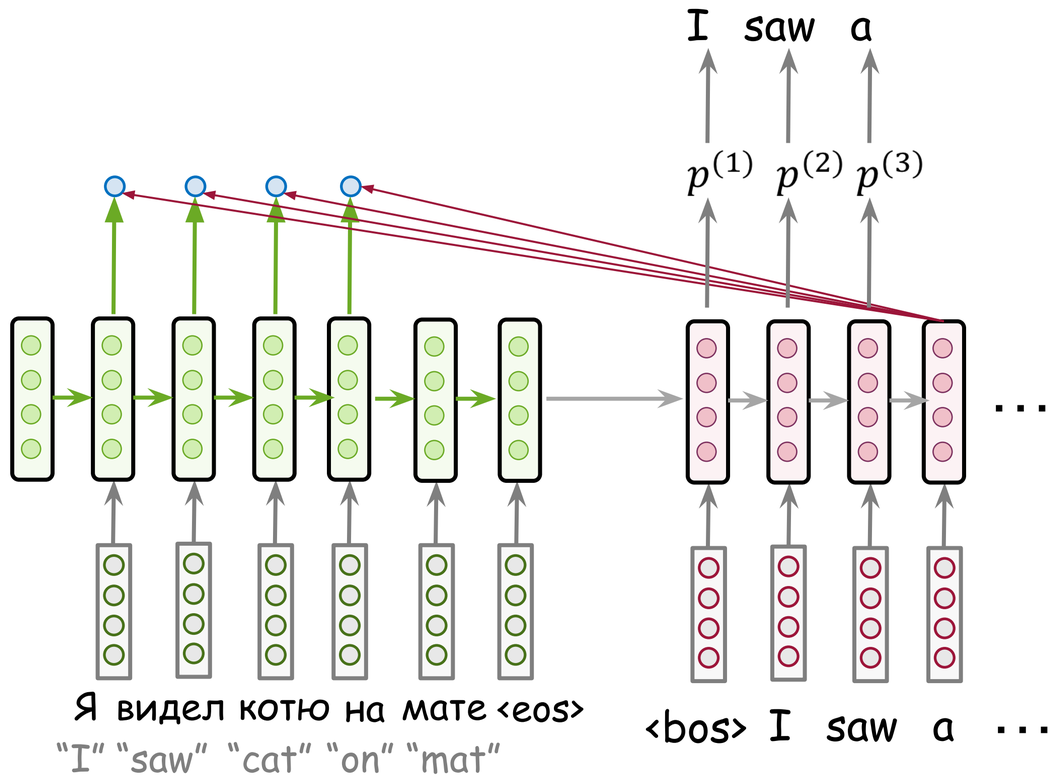
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



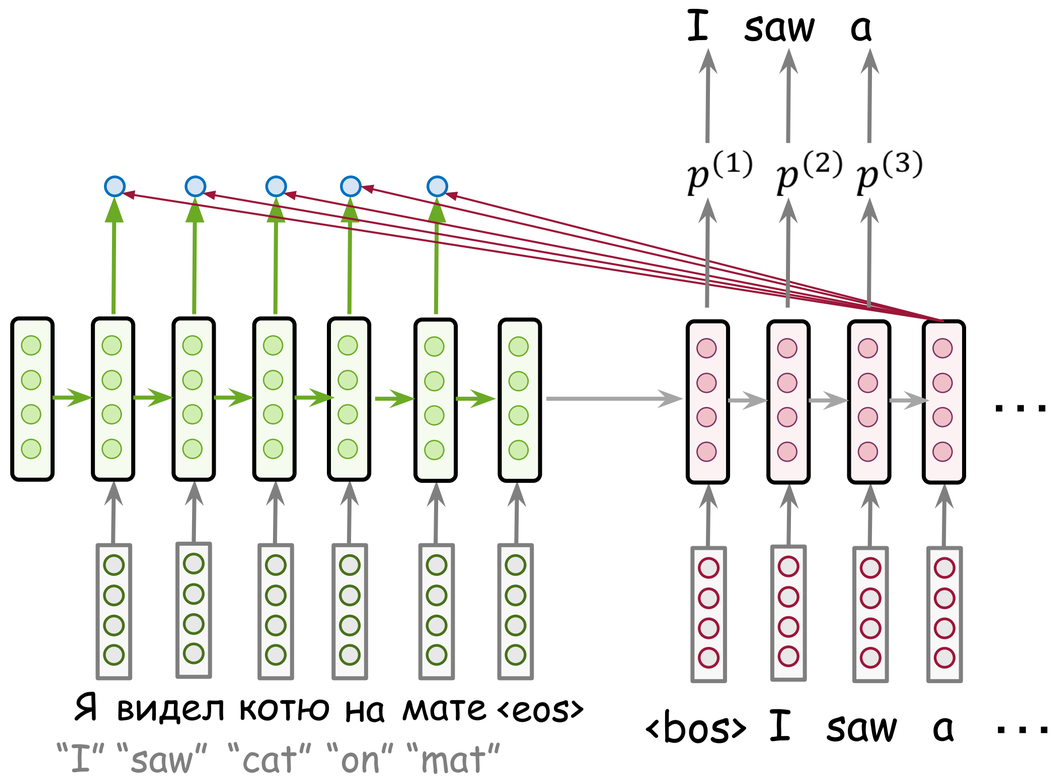
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



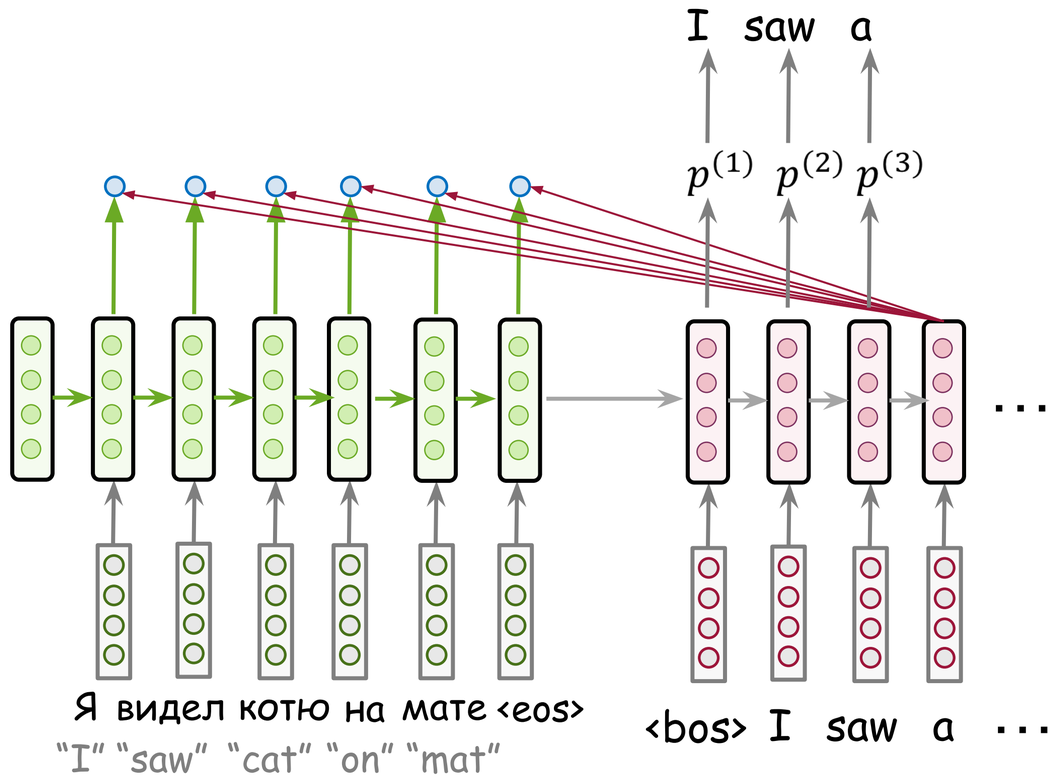
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



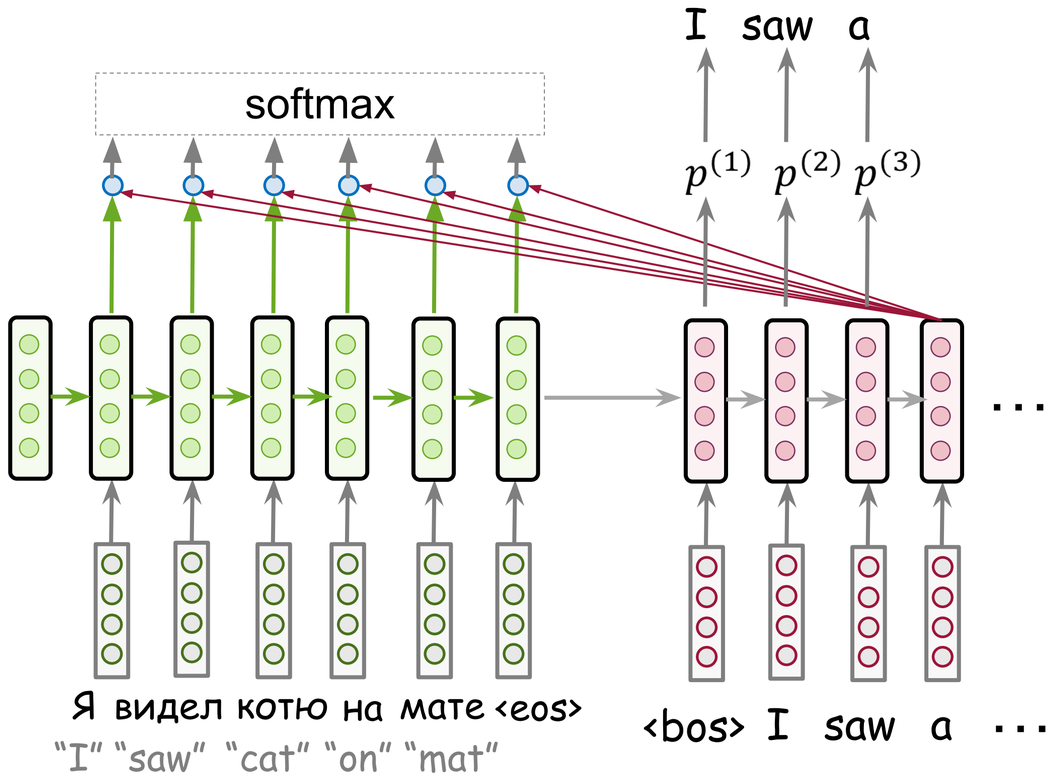
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



Attention: High-Level View

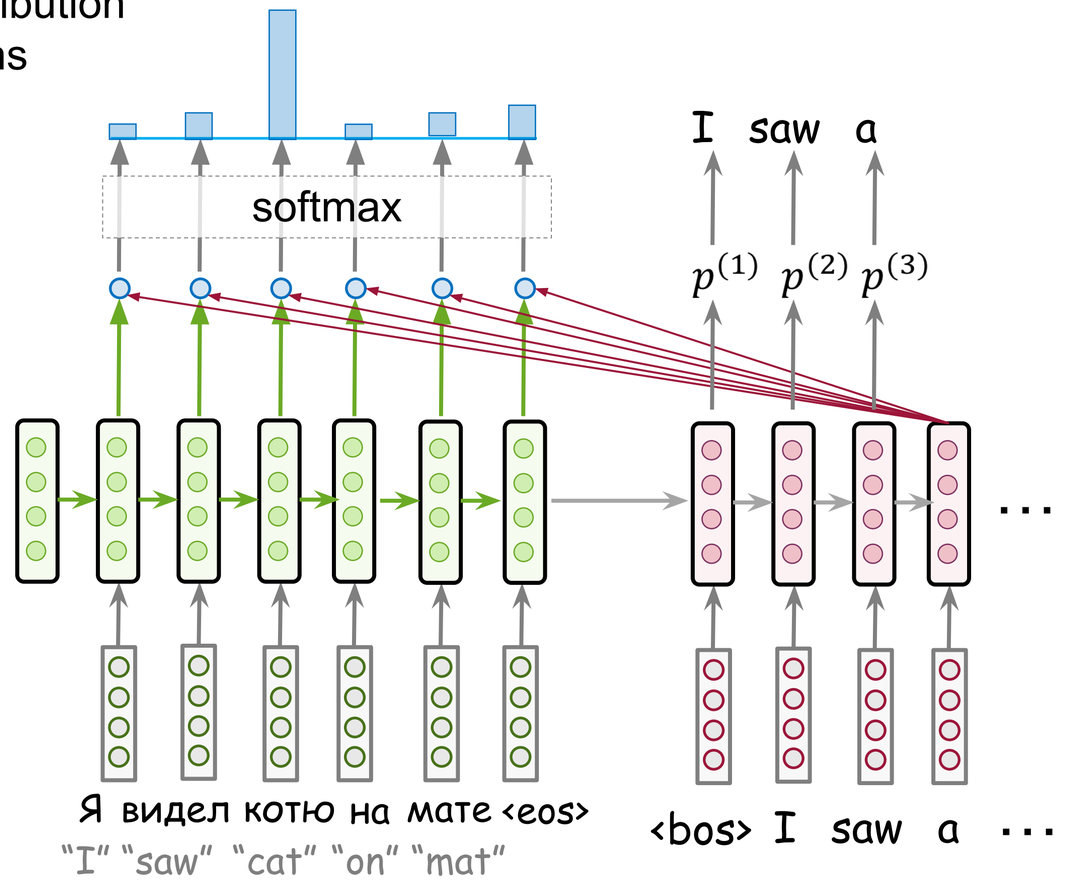
> Attention: At different steps, let a model "focus" on different parts of the input.



Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.

Attention weights: distribution over source tokens

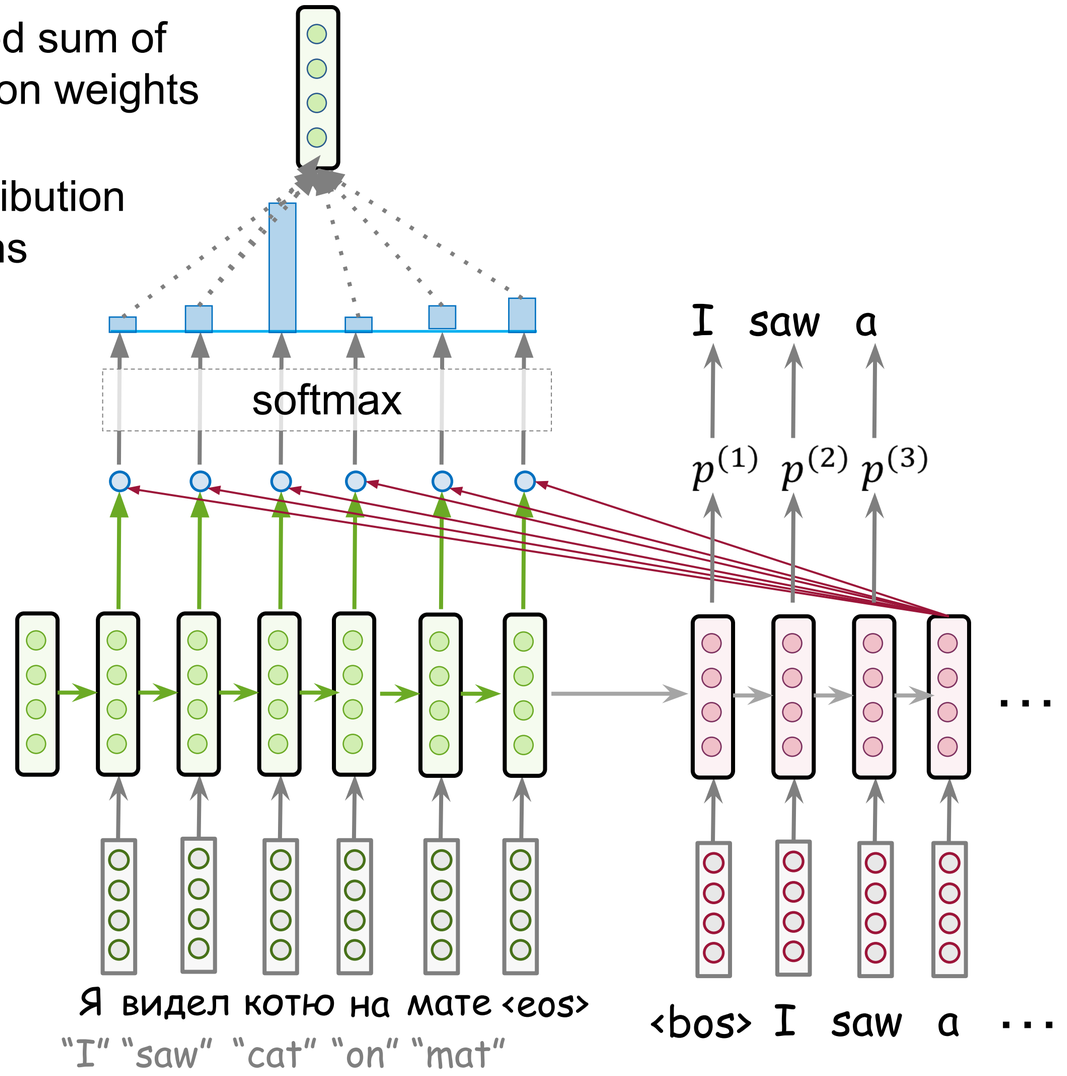


Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.

Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens

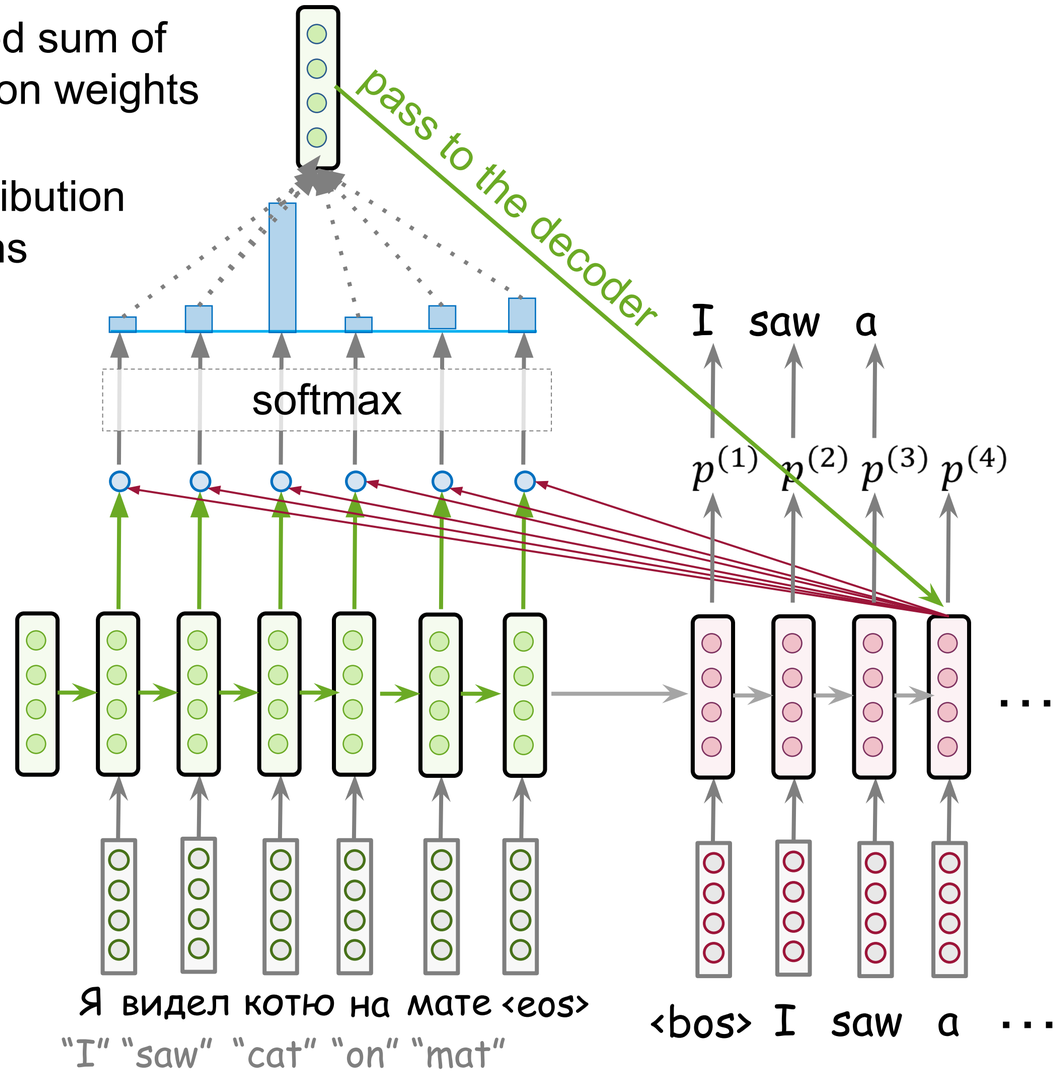


Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.

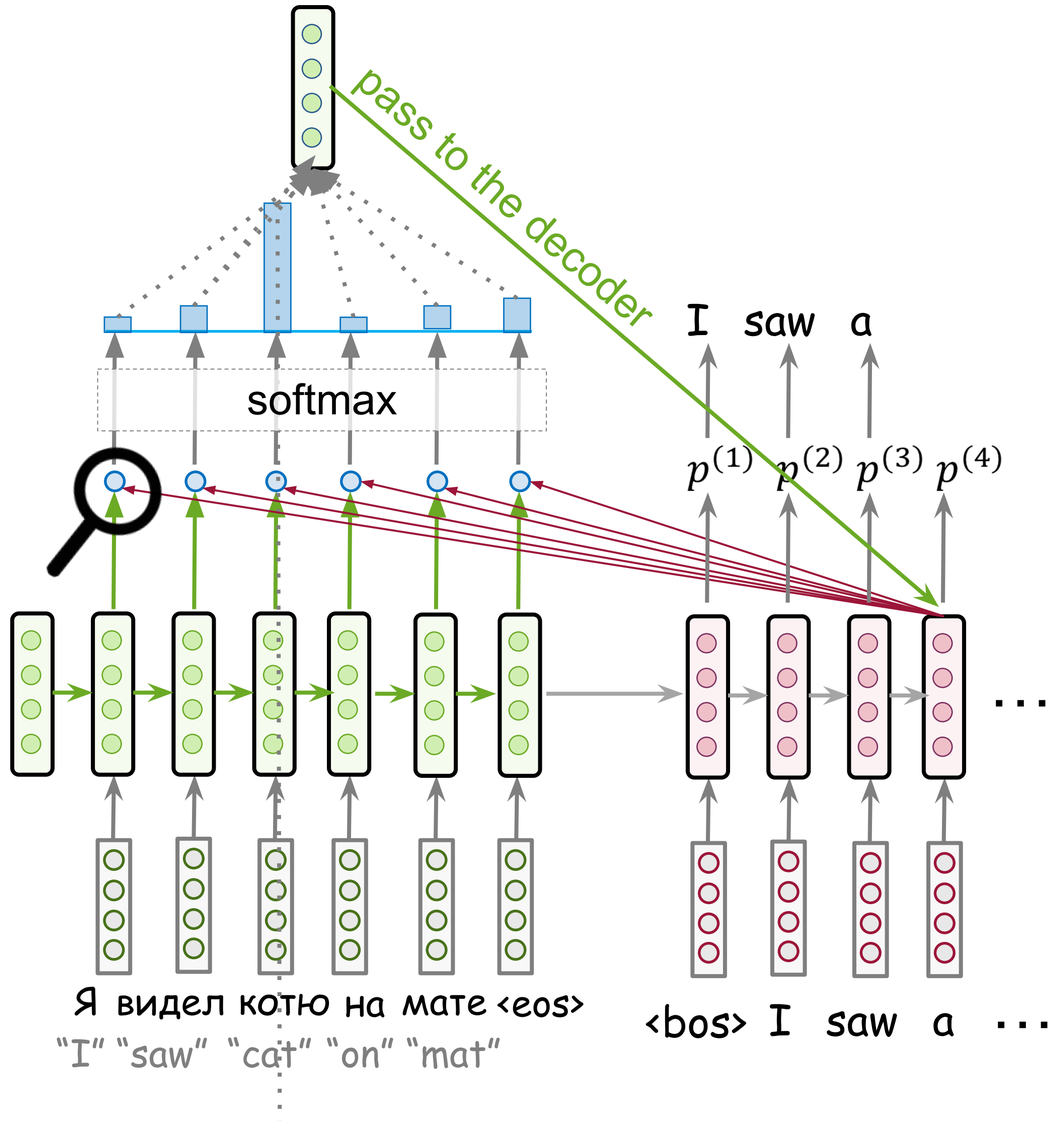
Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens



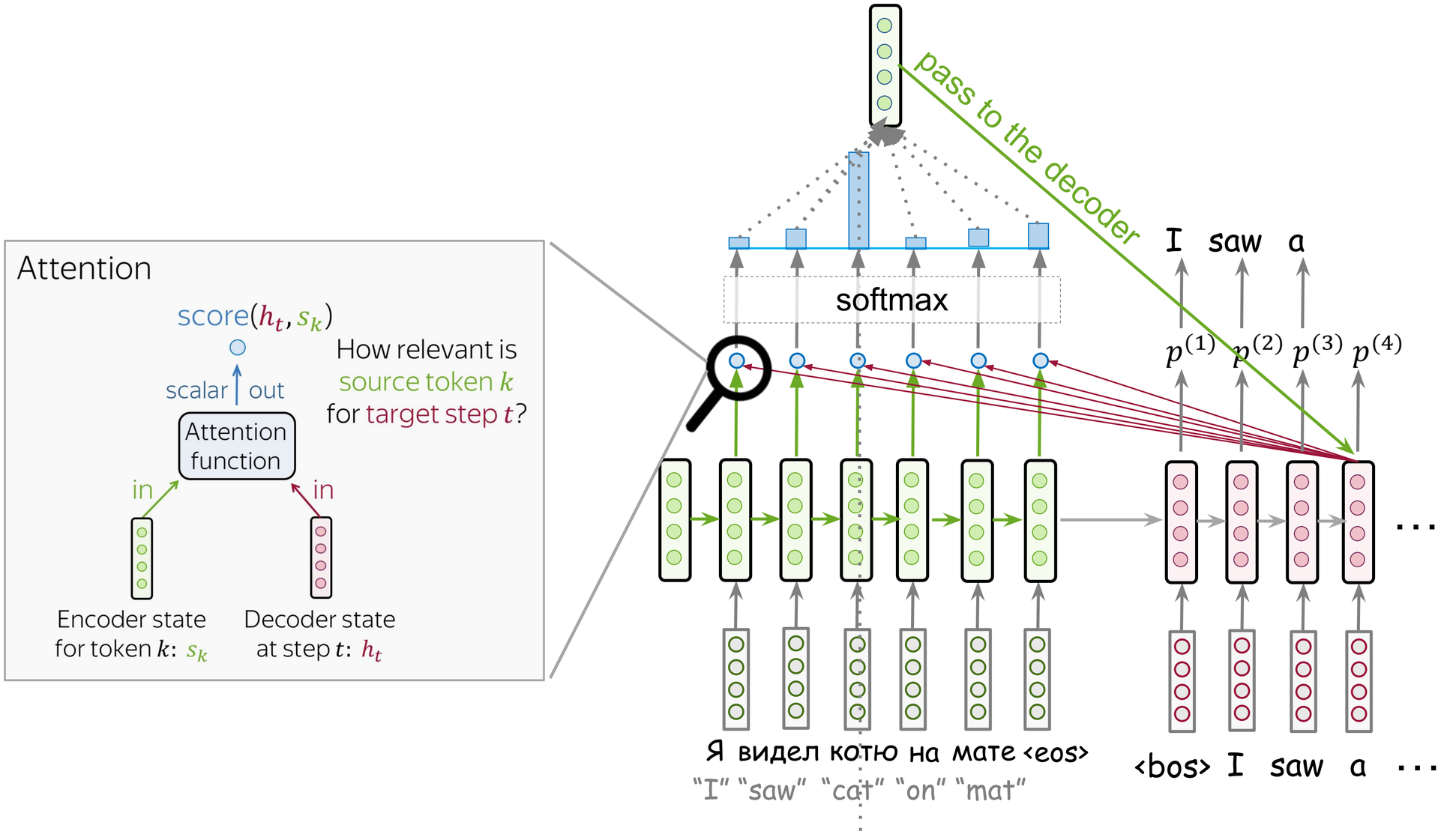
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.

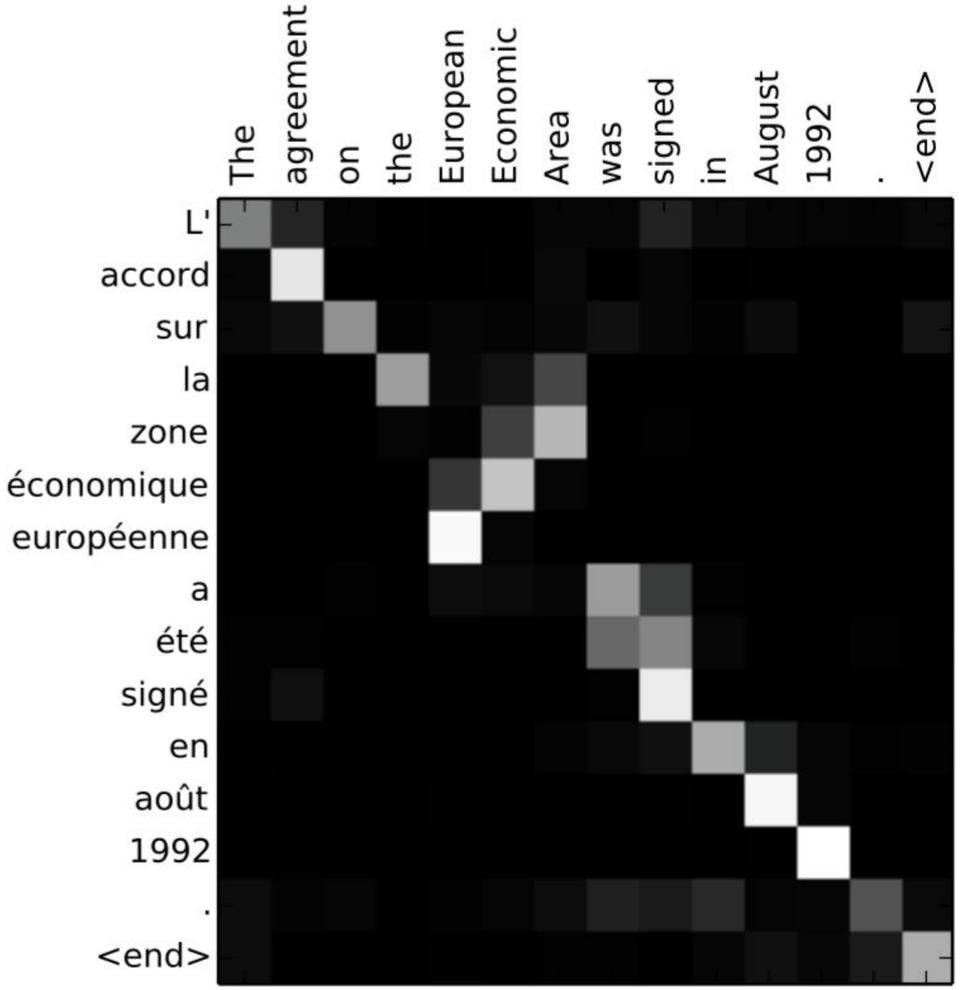


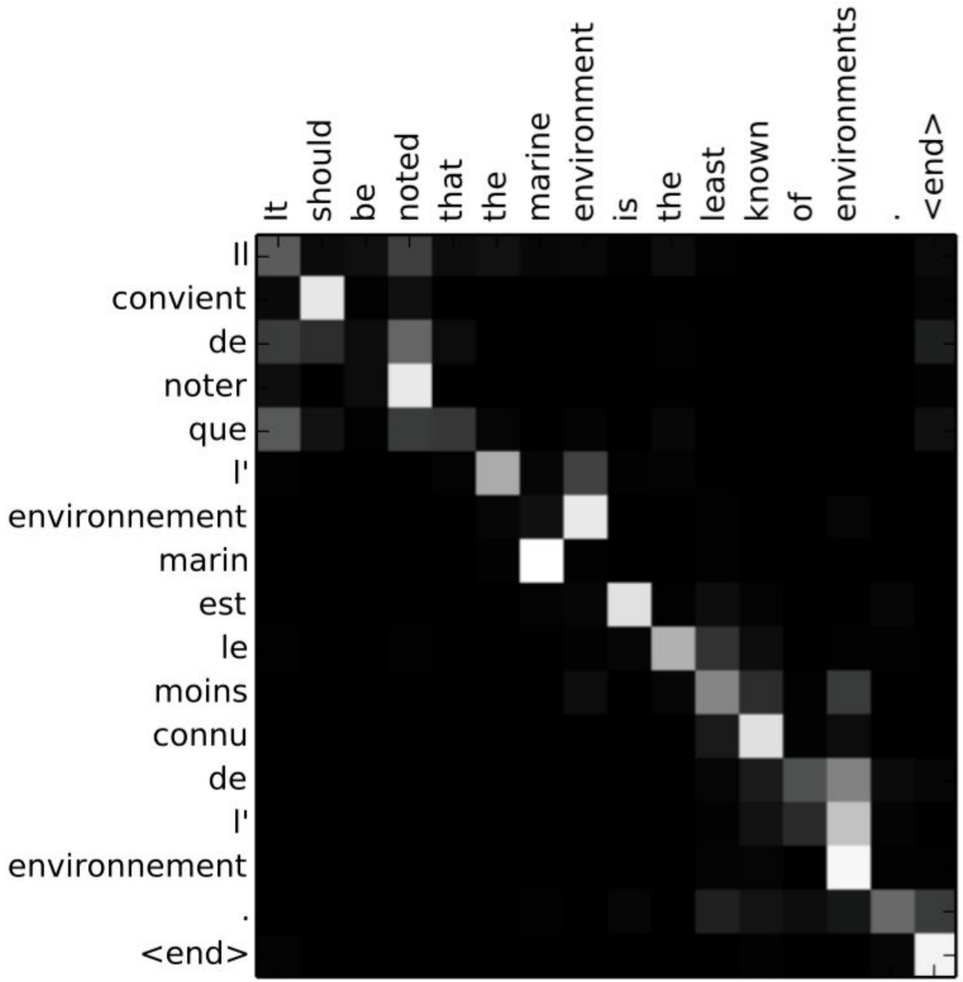
Attention: High-Level View

> Attention: At different steps, let a model "focus" on different parts of the input.



Attention Learned (almost) Alignment





The examples are from the paper Neural Machine Translation by Jointly Learning to Align and Translate.

Computation Pipeline

Attention input

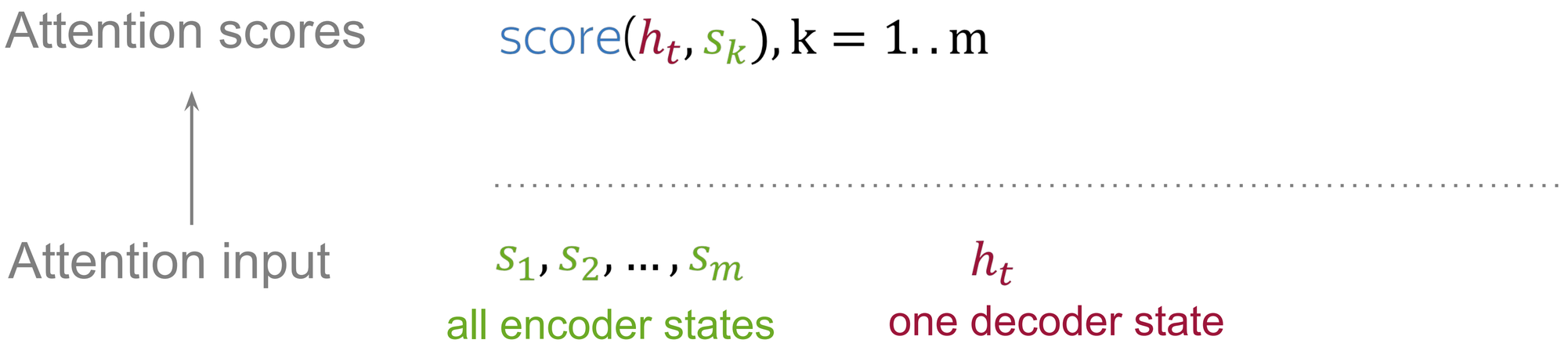
$s_1, s_2, \ldots, s_m$

all encoder states

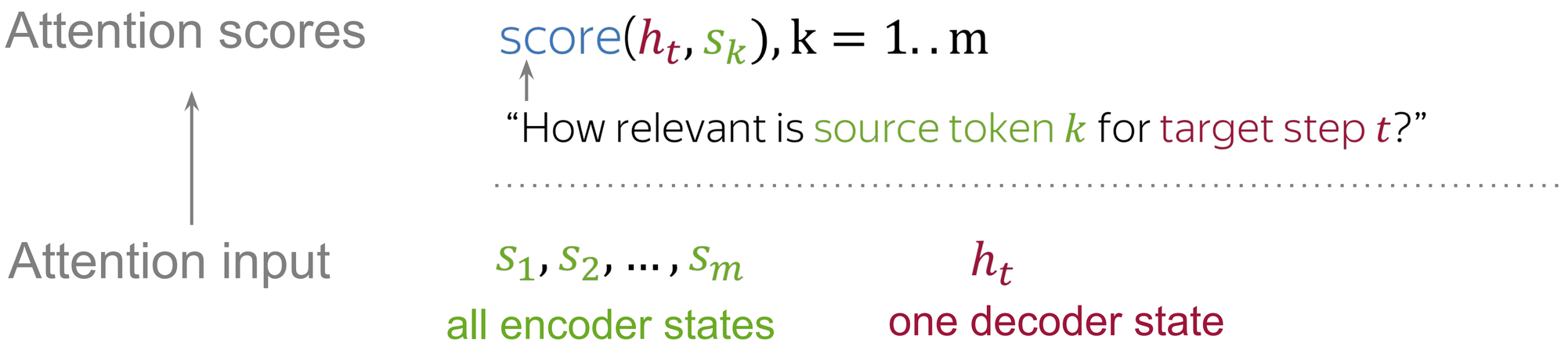
h_t

one decoder state

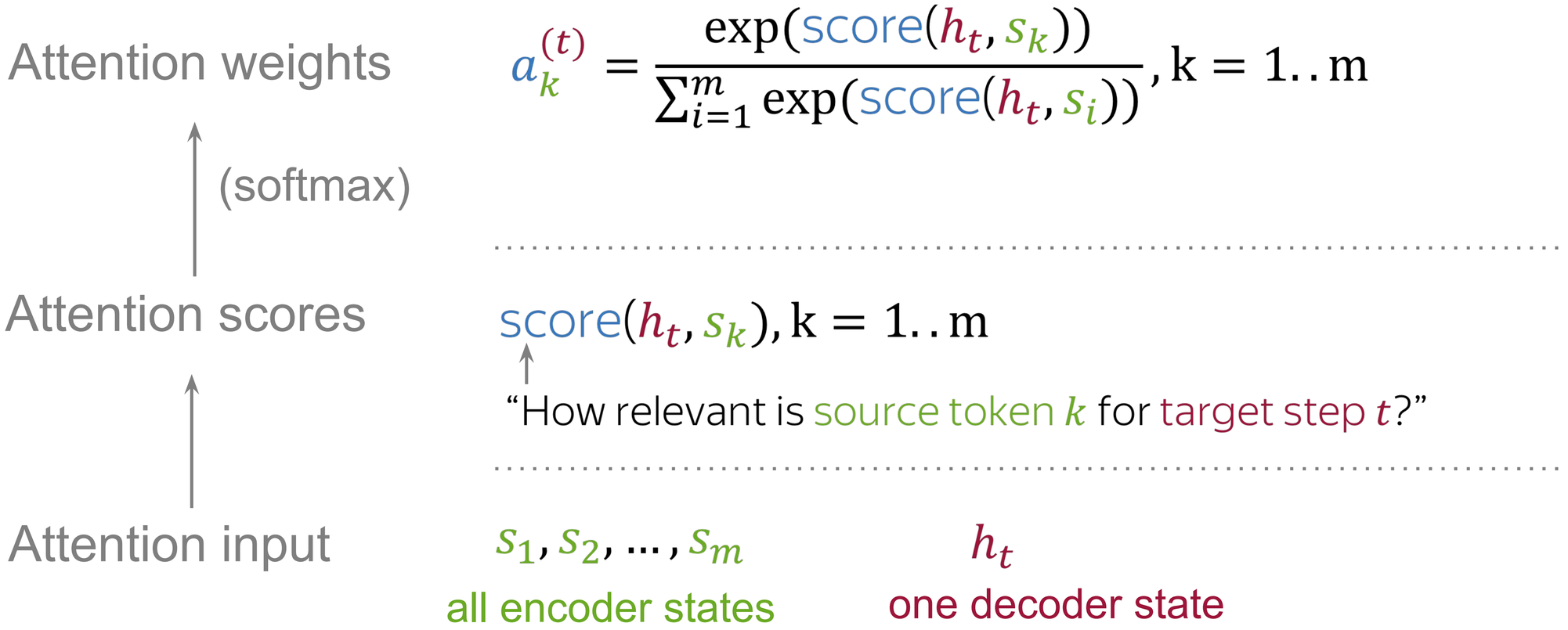
Computation Pipeline



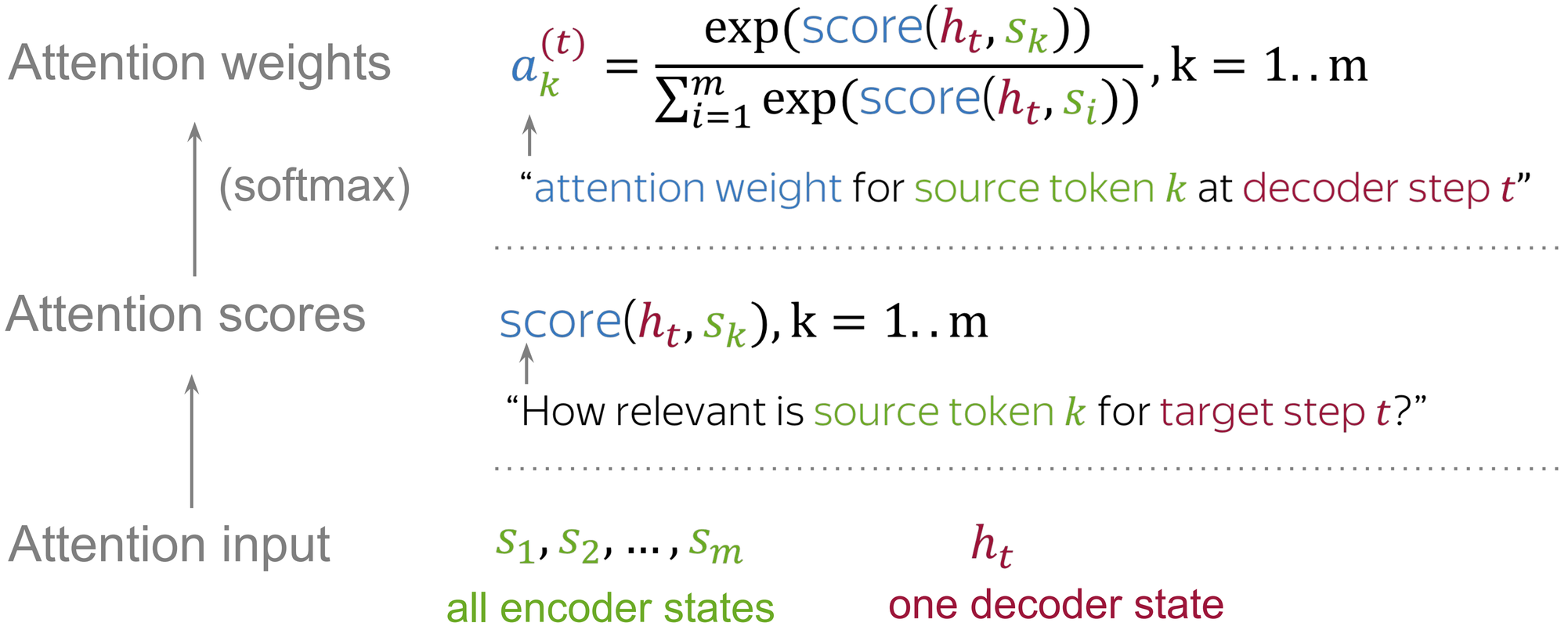
Computation Pipeline



Computation Pipeline



Computation Pipeline



Computation Pipeline

Attention output

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \cdots + a_m^{(t)} s_m = \sum_{k=1}^{m} a_k^{(t)} s_k$$

(weighted sum)

Attention weights

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_{i=1}^{m} \exp(\text{score}(h_t, s_i))}, \text{k} = 1..\text{m}$$

"attention weight for source token k at decoder step t"

(softmax)

Attention scores

$$\text{score}(h_t, s_k), \text{k} = 1..\text{m}$$

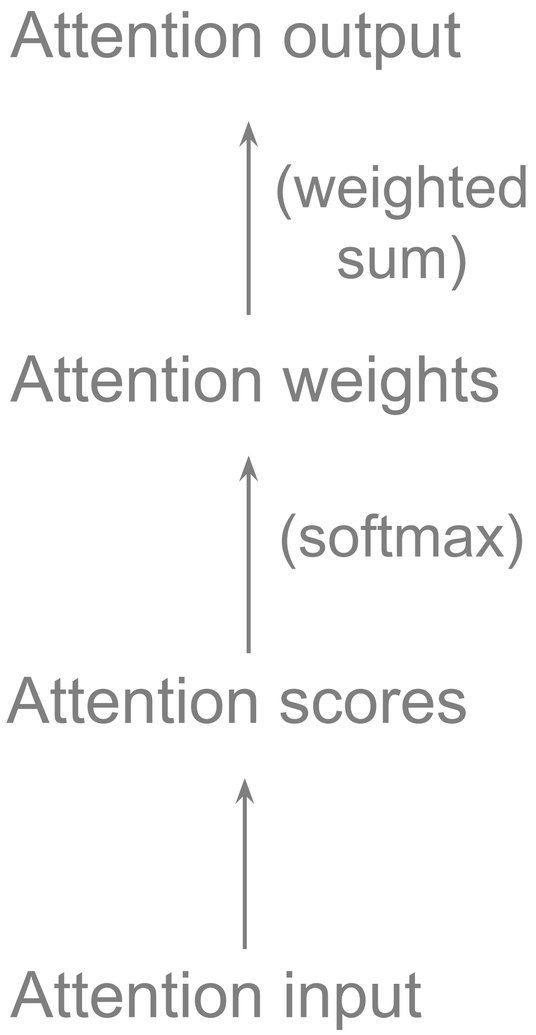
"How relevant is source token k for target step t?"

Attention input

$s_1, s_2, \ldots, s_m$ all encoder states

h_t one decoder state

Computation Pipeline



$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \cdots + a_m^{(t)} s_m = \sum_{k=1}^{m} a_k^{(t)} s_k$$

"source context for decoder step t"

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_{i=1}^{m} \exp(\text{score}(h_t, s_i))}, \text{k} = 1..\text{m}$$

"attention weight for source token k at decoder step t"

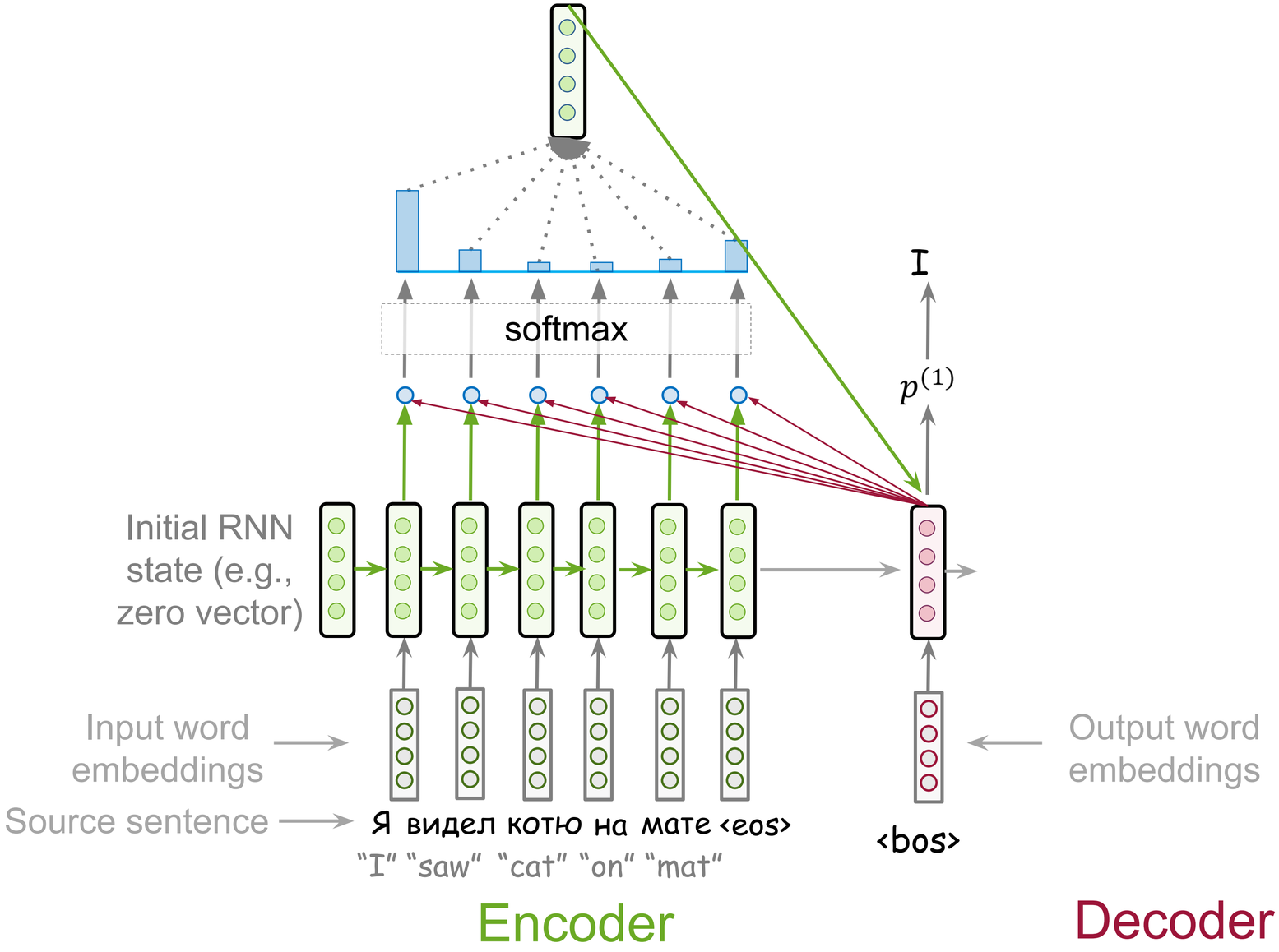
$$\text{score}(h_t, s_k), \text{k} = 1..\text{m}$$

"How relevant is source token k for target step t?"

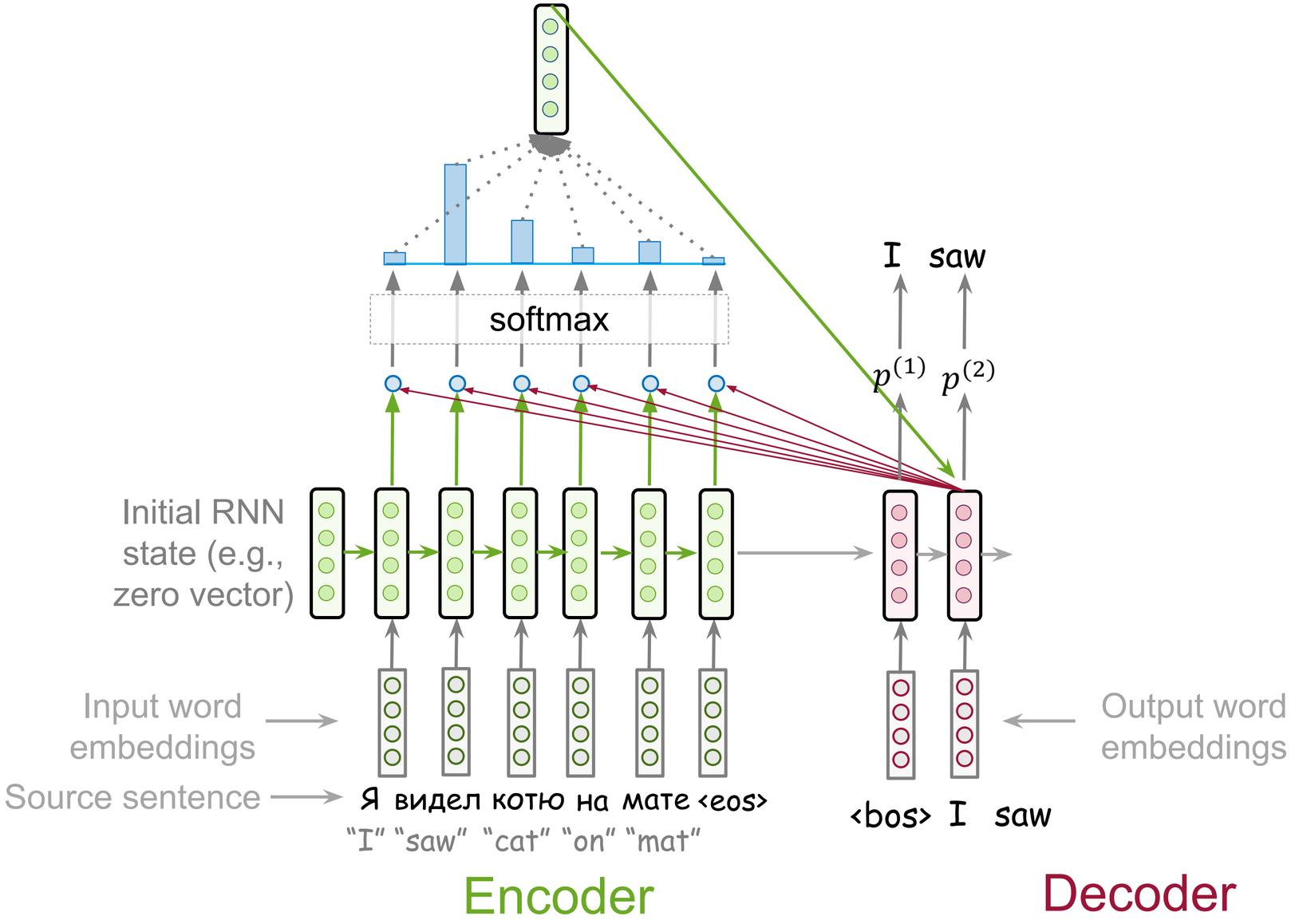
$s_1, s_2, \ldots, s_m$ all encoder states

h_t one decoder state

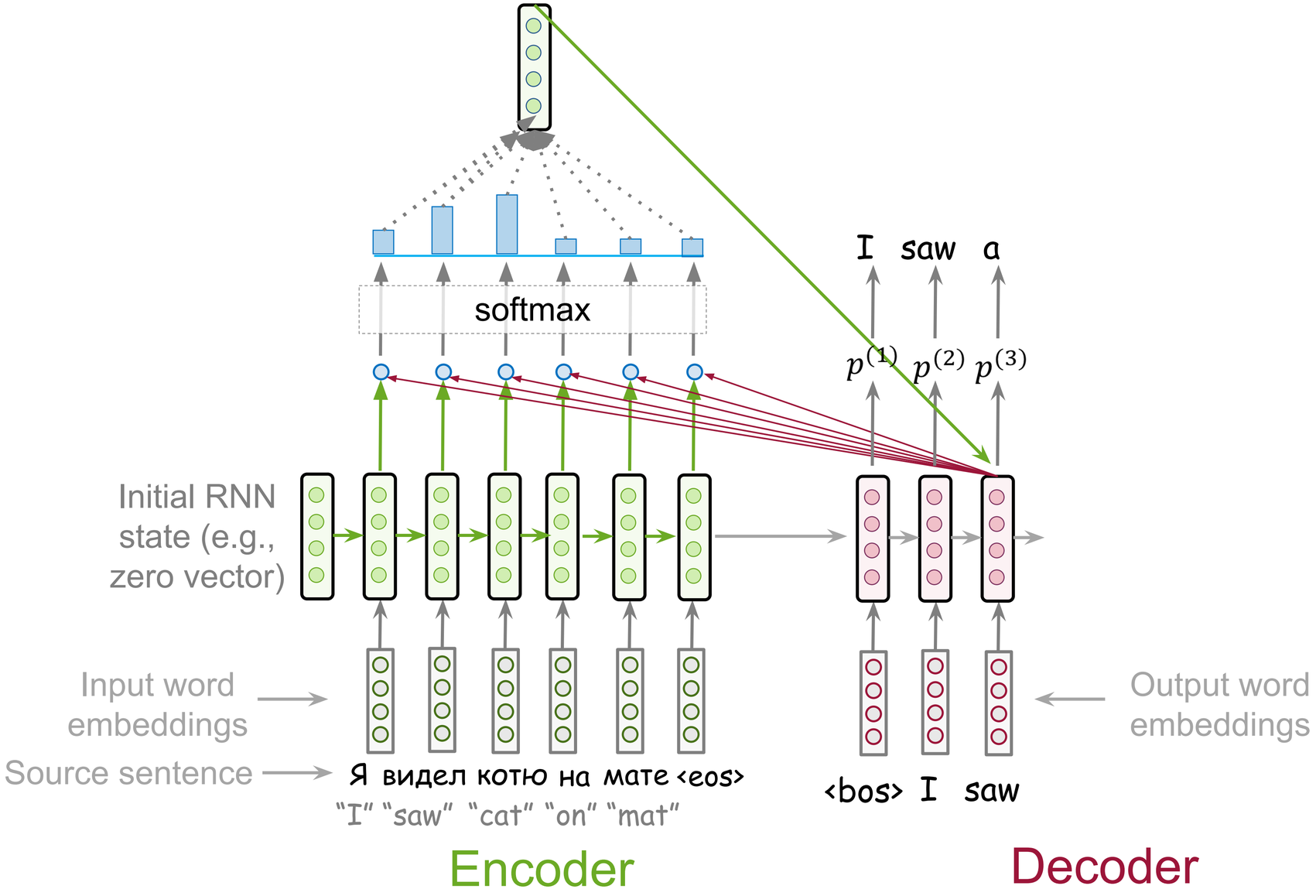
Model Learns to Pick Relevant Tokens



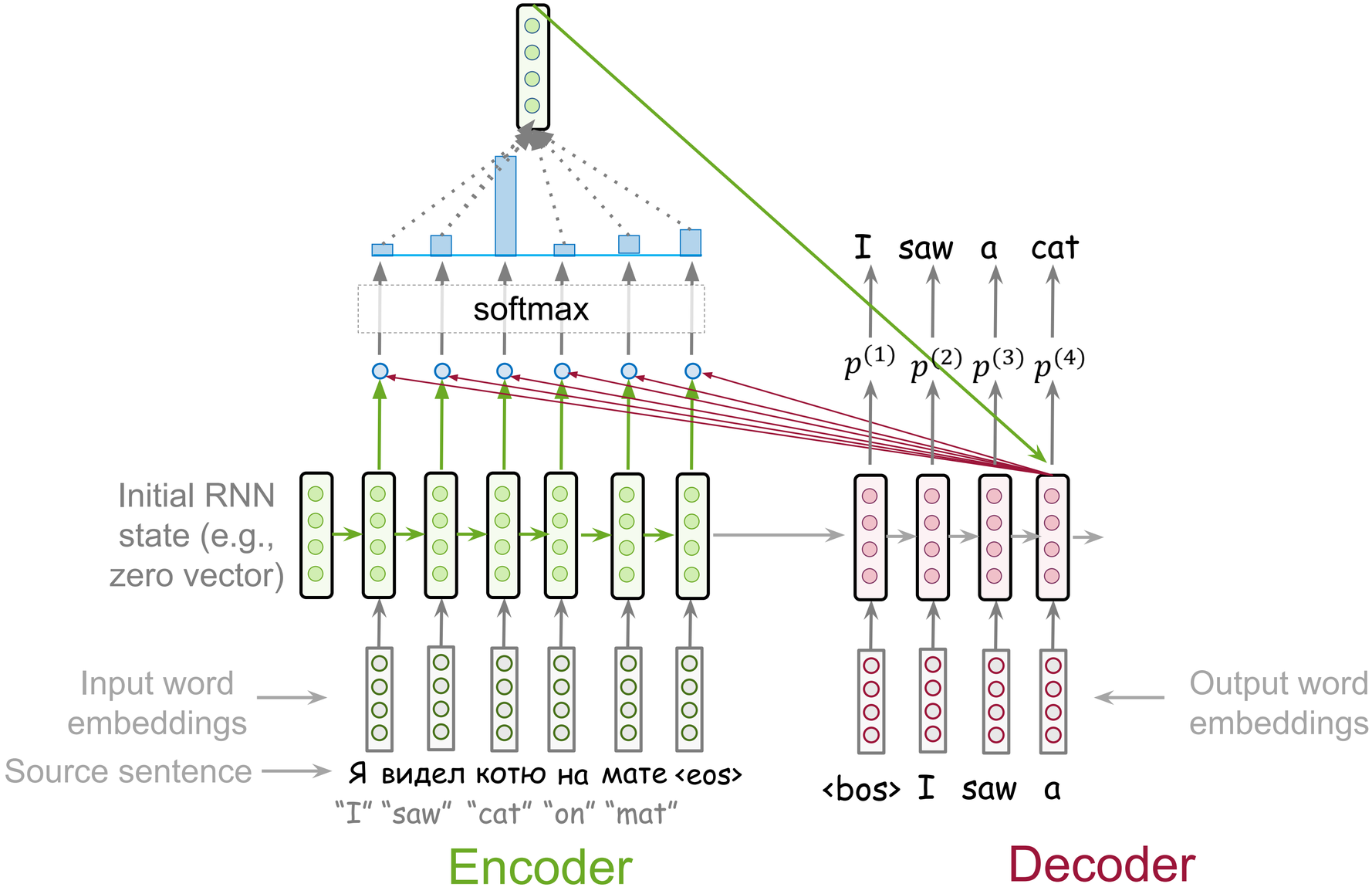
Model Learns to Pick Relevant Tokens



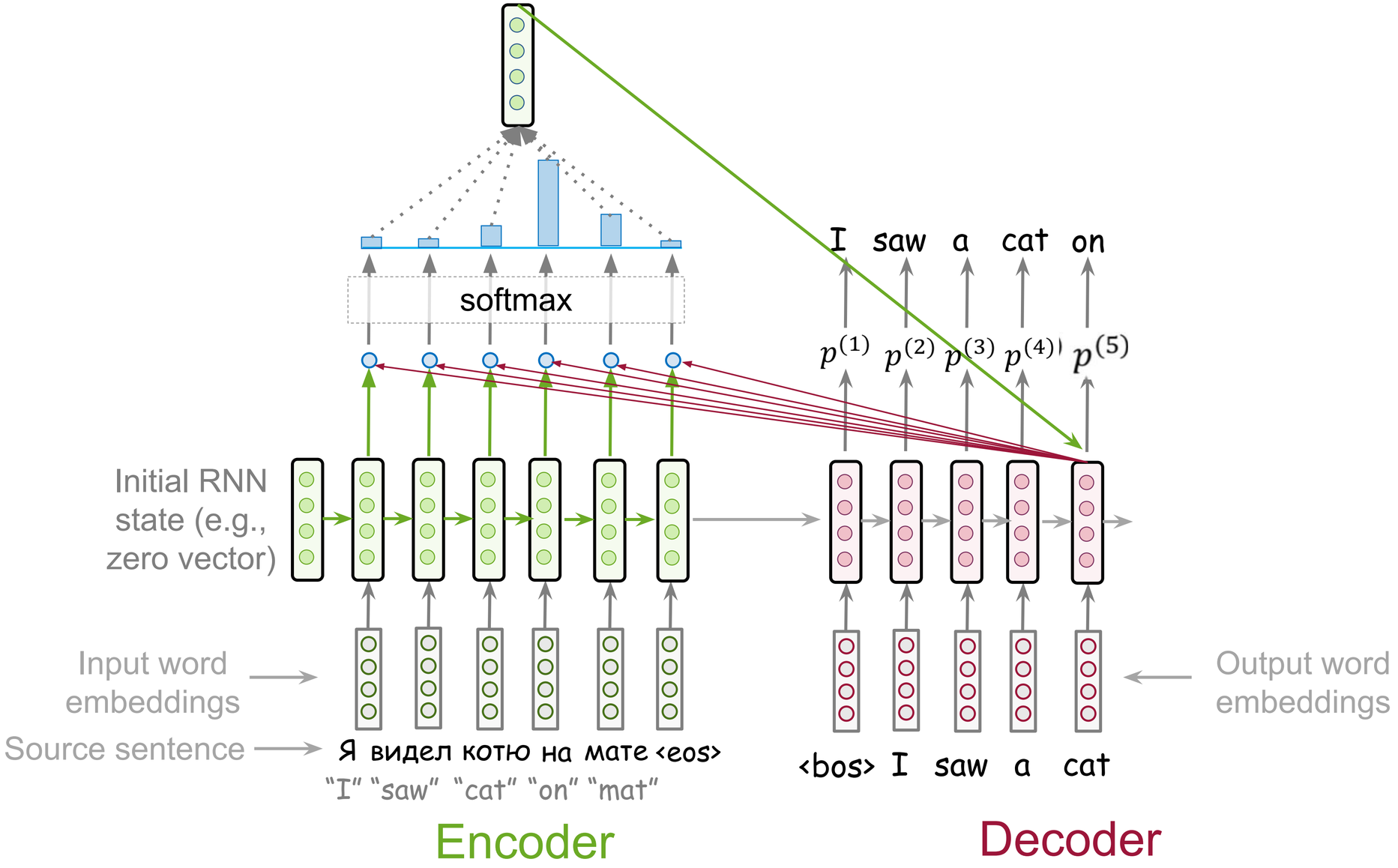
Model Learns to Pick Relevant Tokens



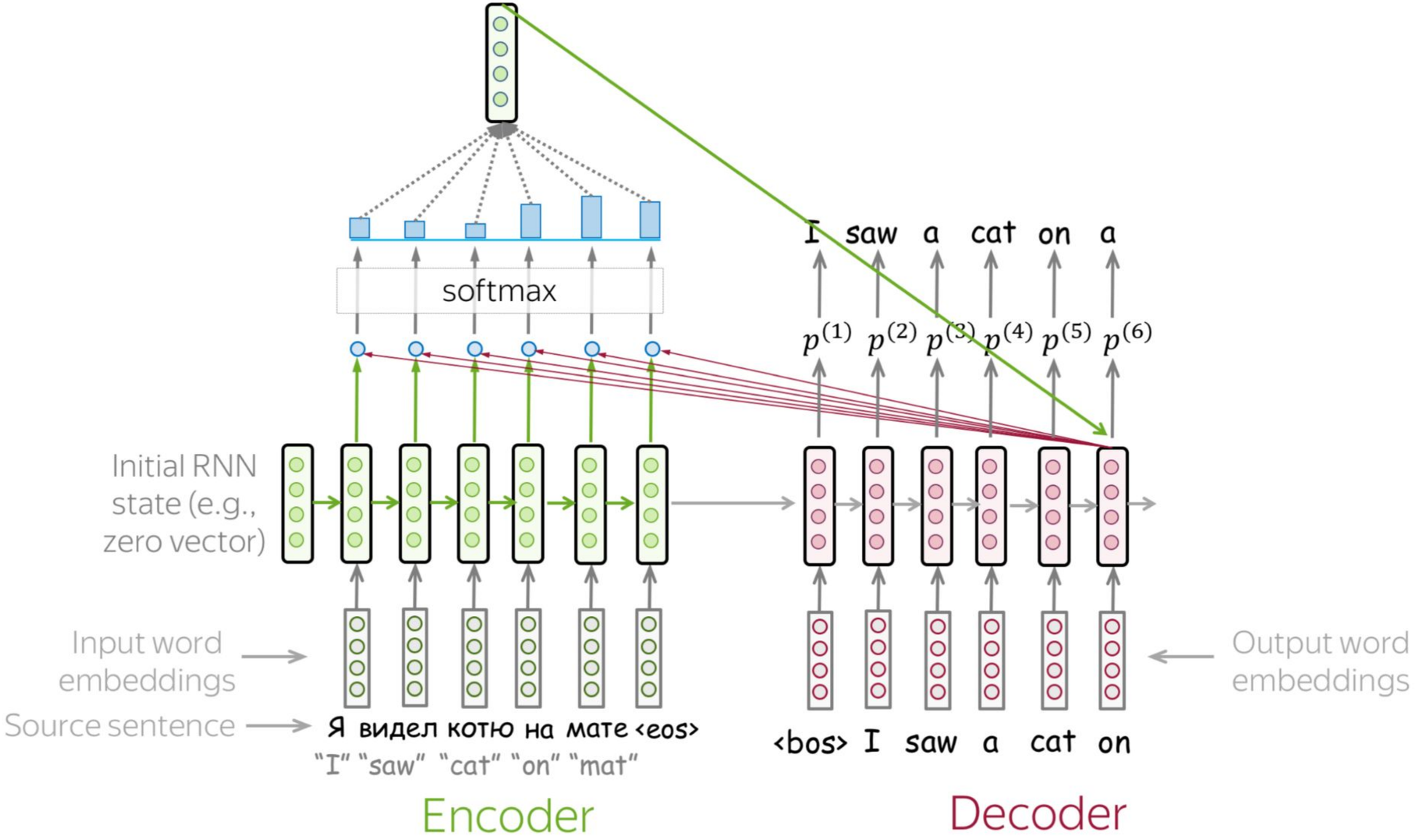
Model Learns to Pick Relevant Tokens



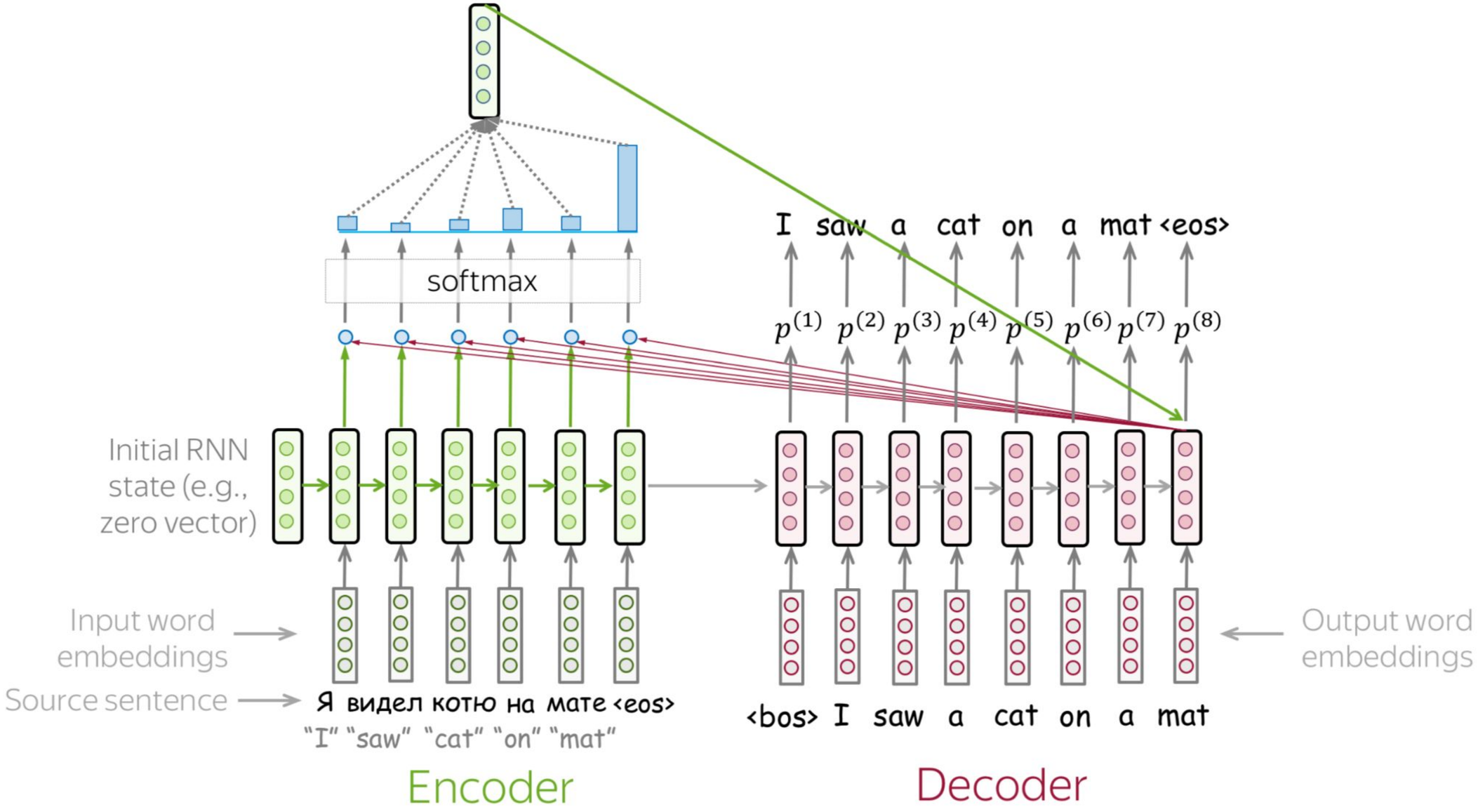
Model Learns to Pick Relevant Tokens



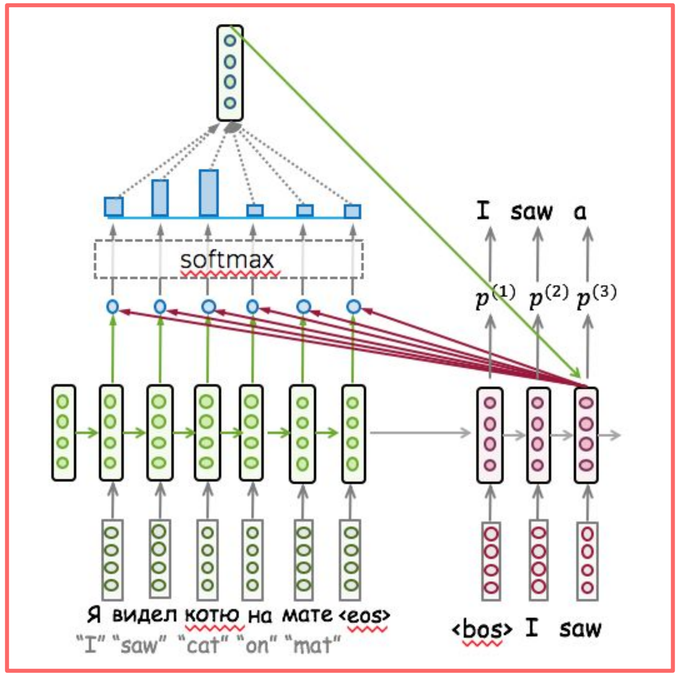
Model Learns to Pick Relevant Tokens



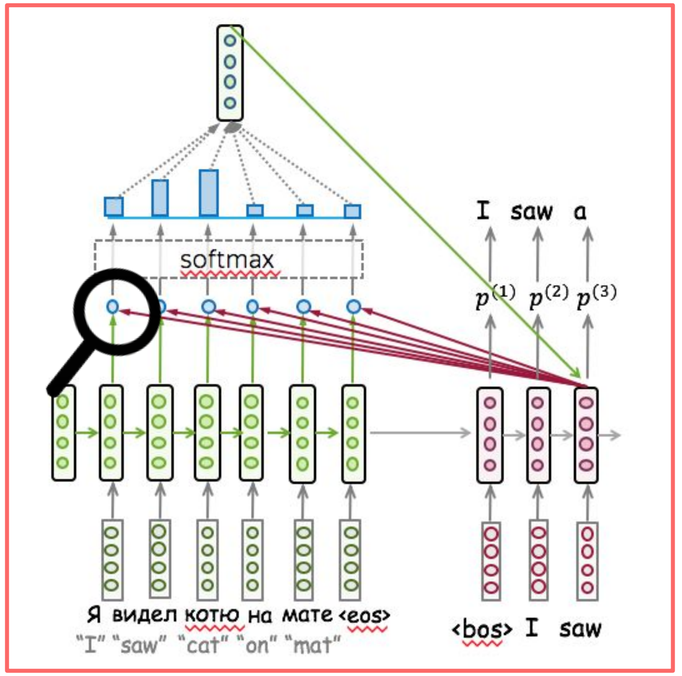
Model Learns to Pick Relevant Tokens



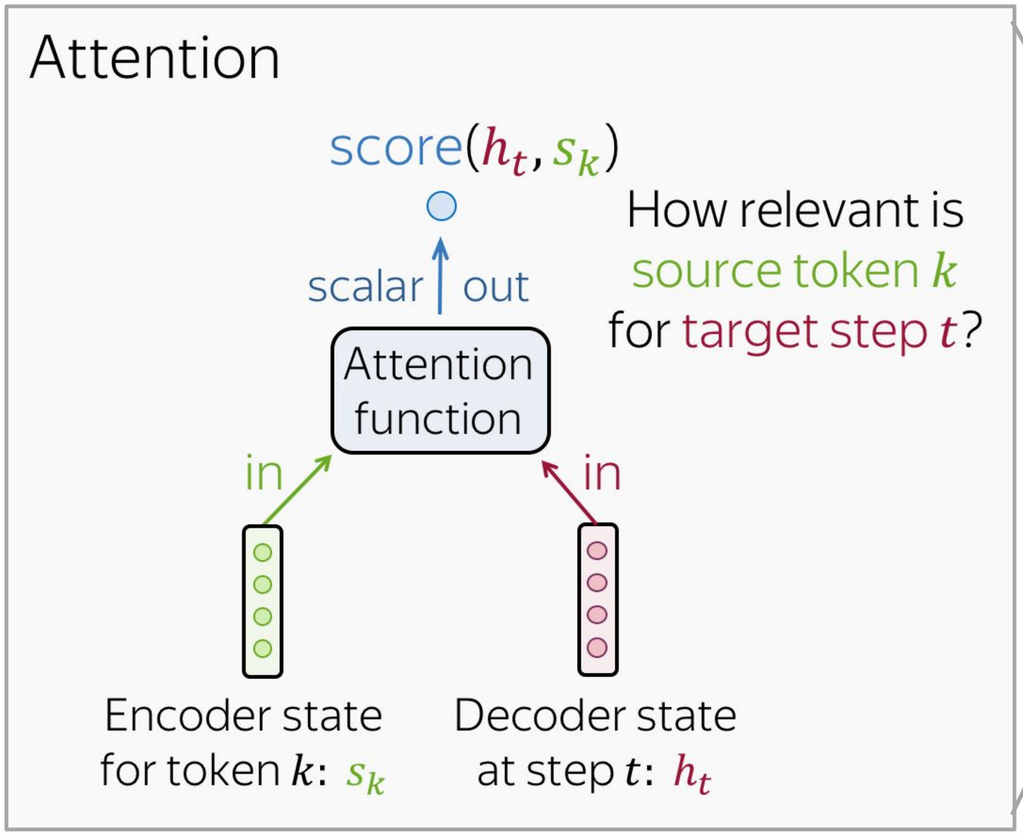
Attention Score Functions

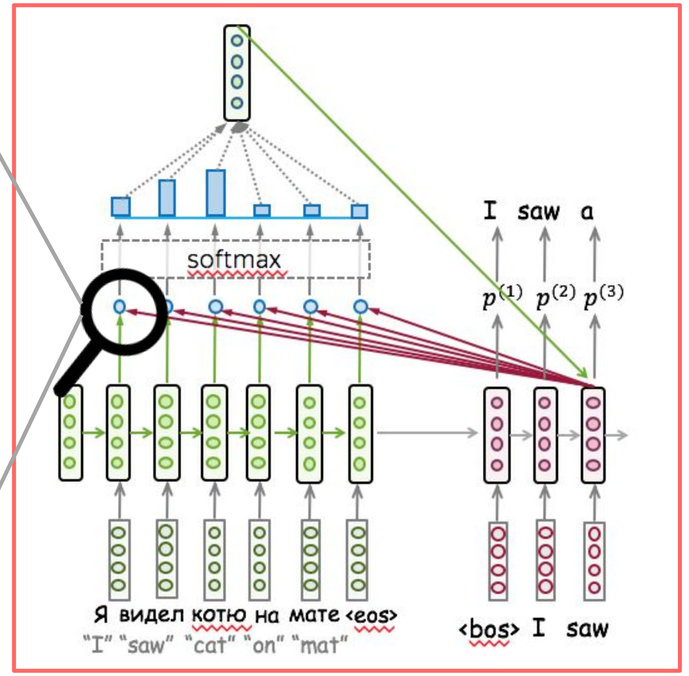


Attention Score Functions



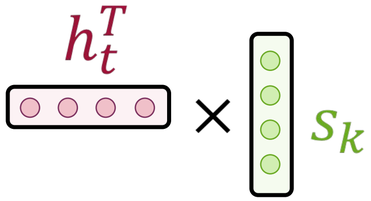
Attention Score Functions

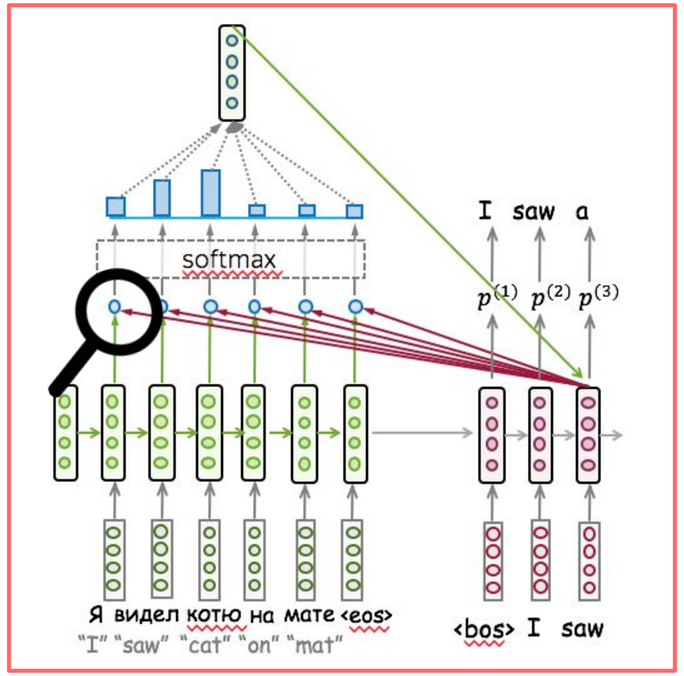




Attention Score Functions

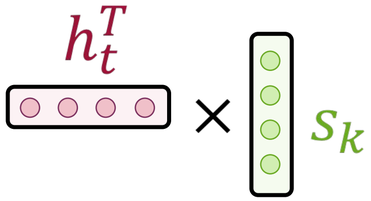
- **Dot-product:** $\text{score}(h_t, s_k) = h_t^T \, s_k$



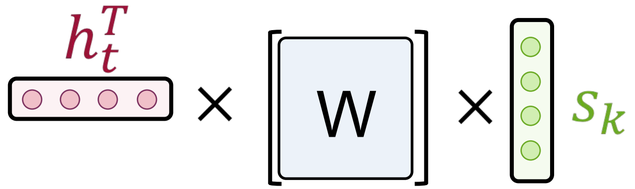


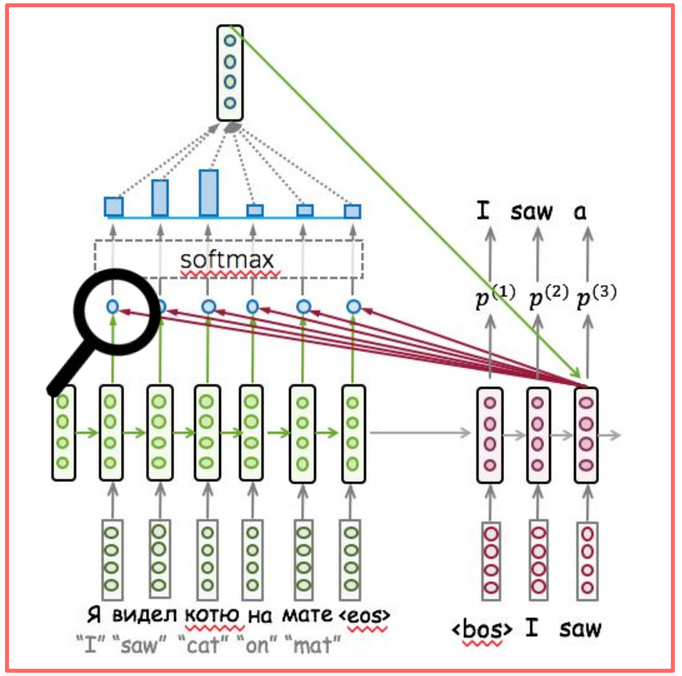
Attention Score Functions

- **Dot-product:** $\text{score}(h_t, s_k) = h_t^T \, s_k$



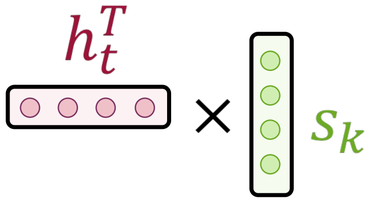
- **Bilinear:** $\text{score}(h_t, s_k) = h_t^T \, W \, s_k$



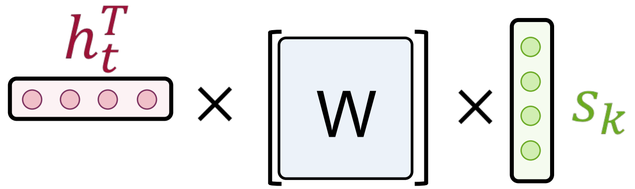


Attention Score Functions

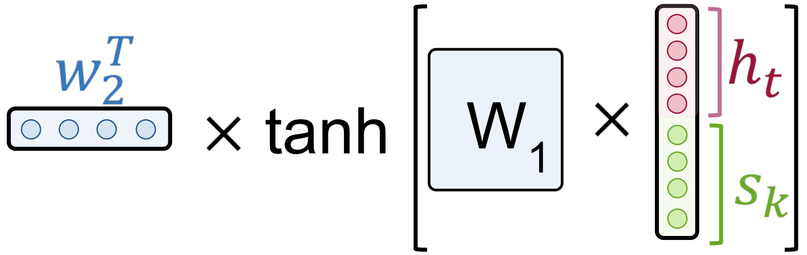
- **Dot-product:** $\text{score}(h_t, s_k) = h_t^T \, s_k$

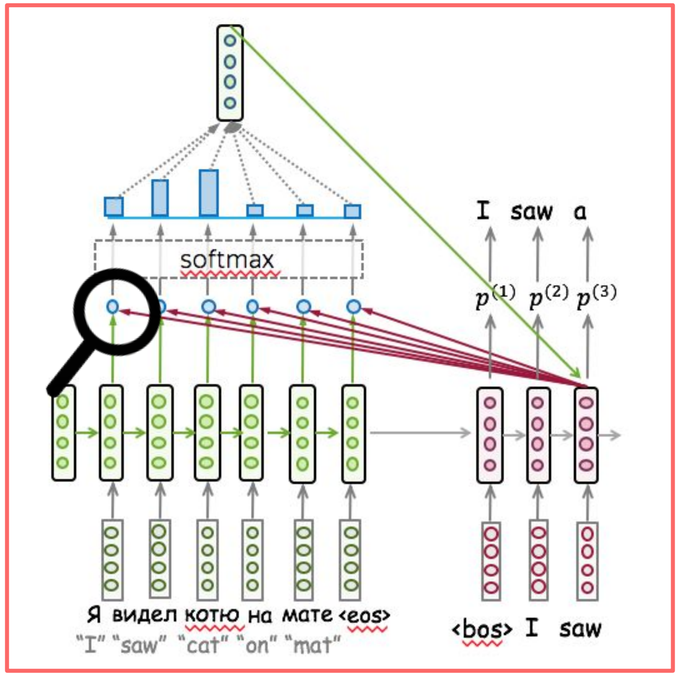


- **Bilinear:** $\text{score}(h_t, s_k) = h_t^T \, W \, s_k$



- **Multi-Layer Perceptron:** $\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1[h_t, s_k])$





What is going to happen:

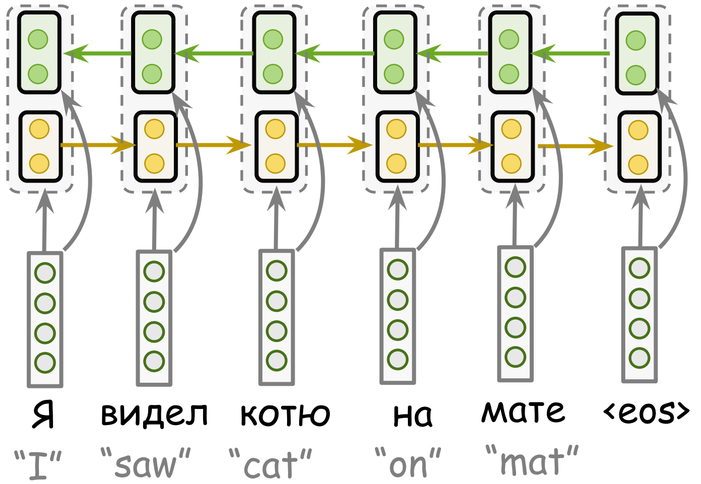
- Seq2seq Basics
- Attention
  - Why do we need it?
  - Attention: High-Level
  - Attention Score Functions
  - Models: Bahdanau vs Luong
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

- Seq2seq Basics
- Attention
  - Why do we need it?
  - Attention: High-Level
  - Attention Score Functions
  - Models: Bahdanau vs Luong
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

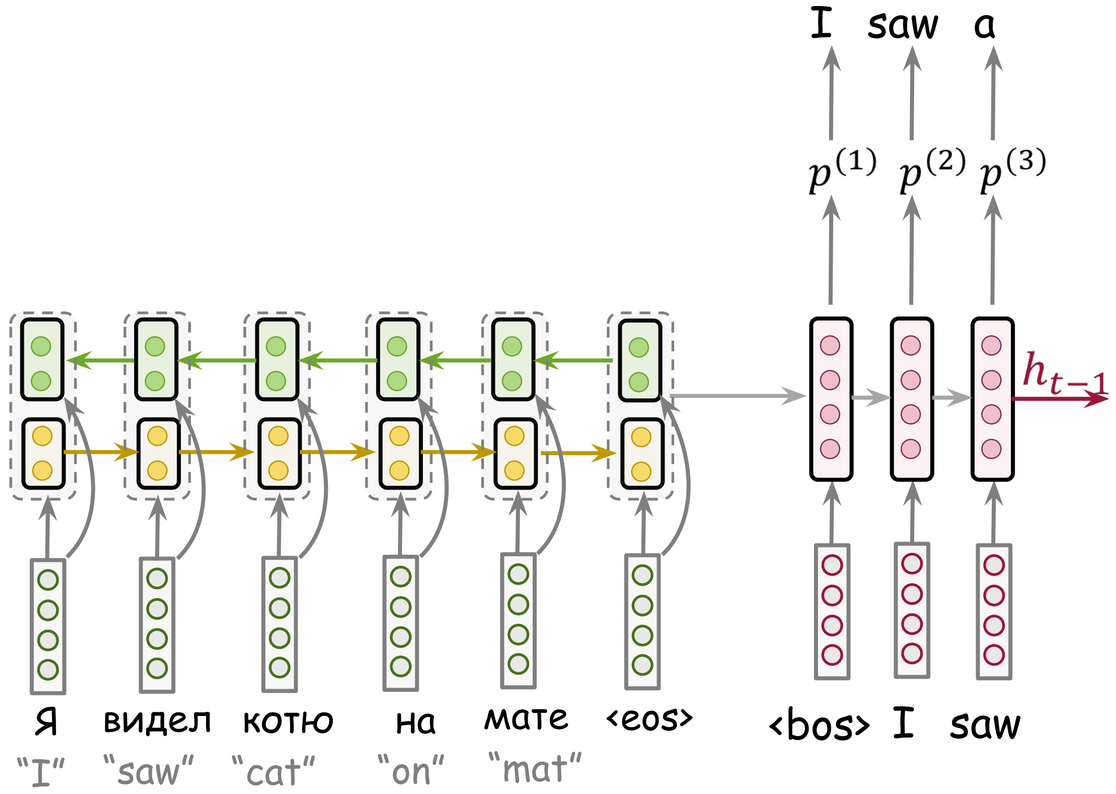
Bahdanau Model (the original attention model)

Bidirectional encoder
Concatenate states from
forward and backward RNNs



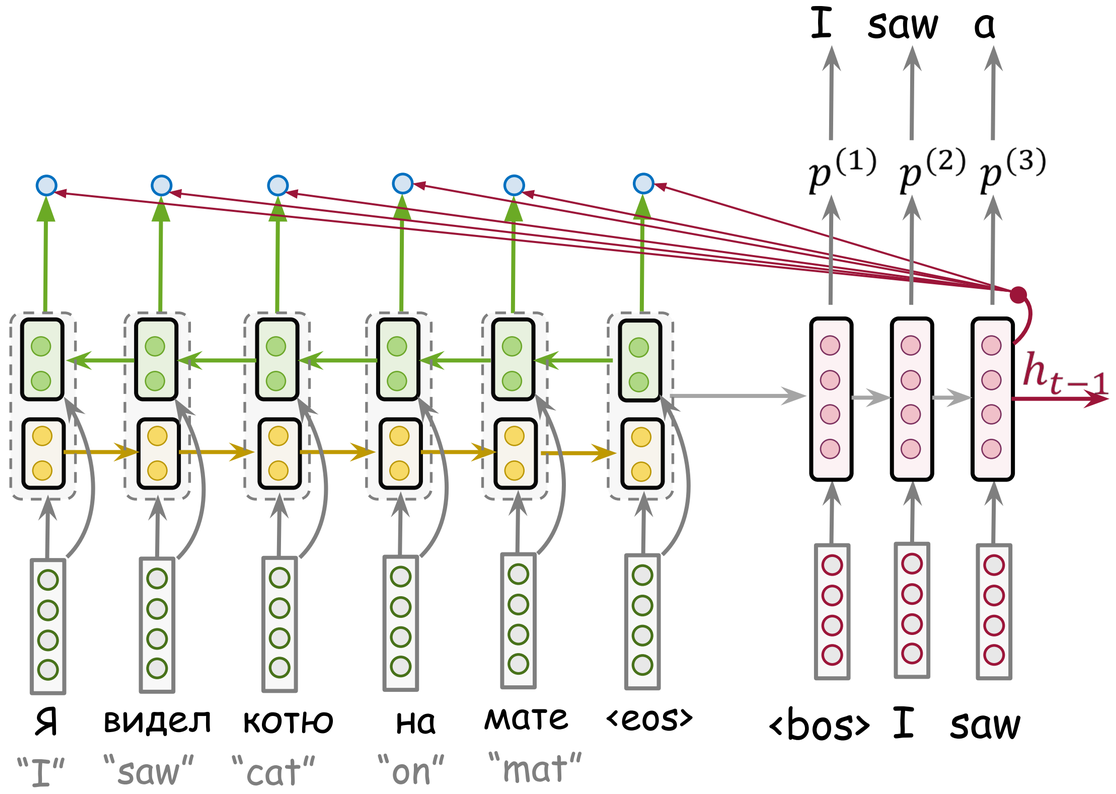
Bahdanau Model (the original attention model)

Bidirectional encoder
Concatenate states from
forward and backward RNNs



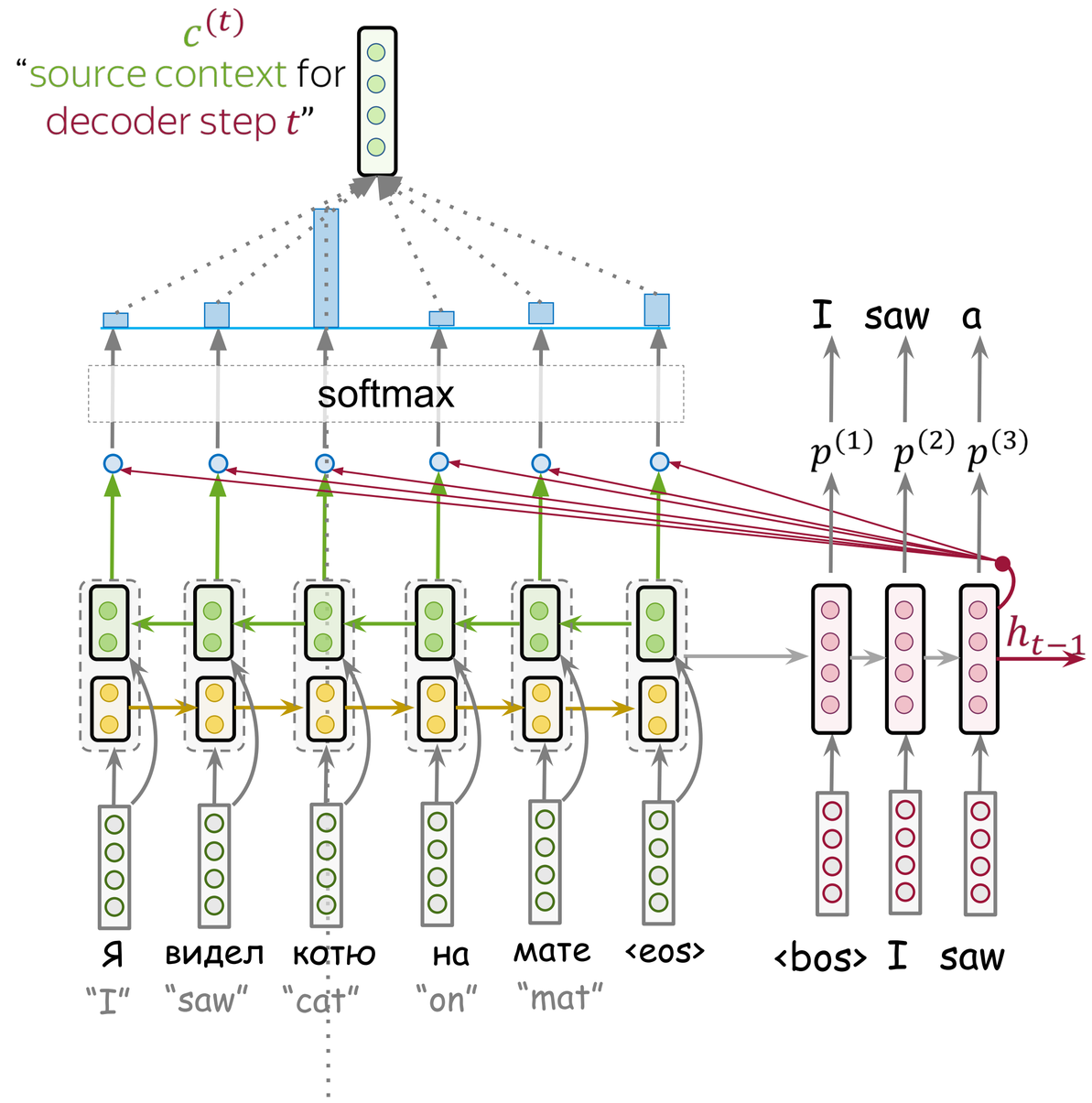
Bahdanau Model (the original attention model)

Bidirectional encoder
Concatenate states from
forward and backward RNNs



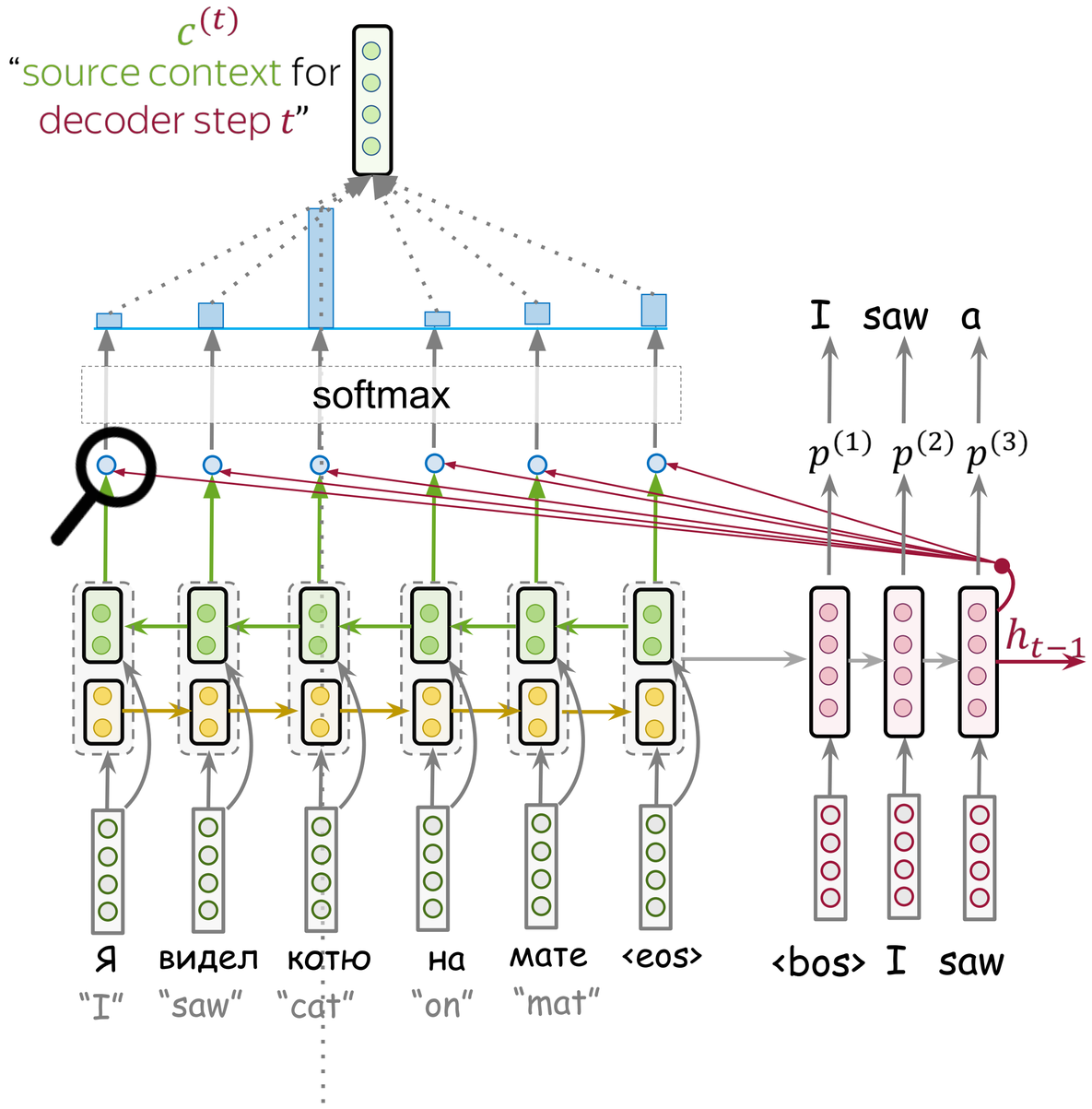
Bahdanau Model (the original attention model)

Bidirectional encoder
Concatenate states from
forward and backward RNNs

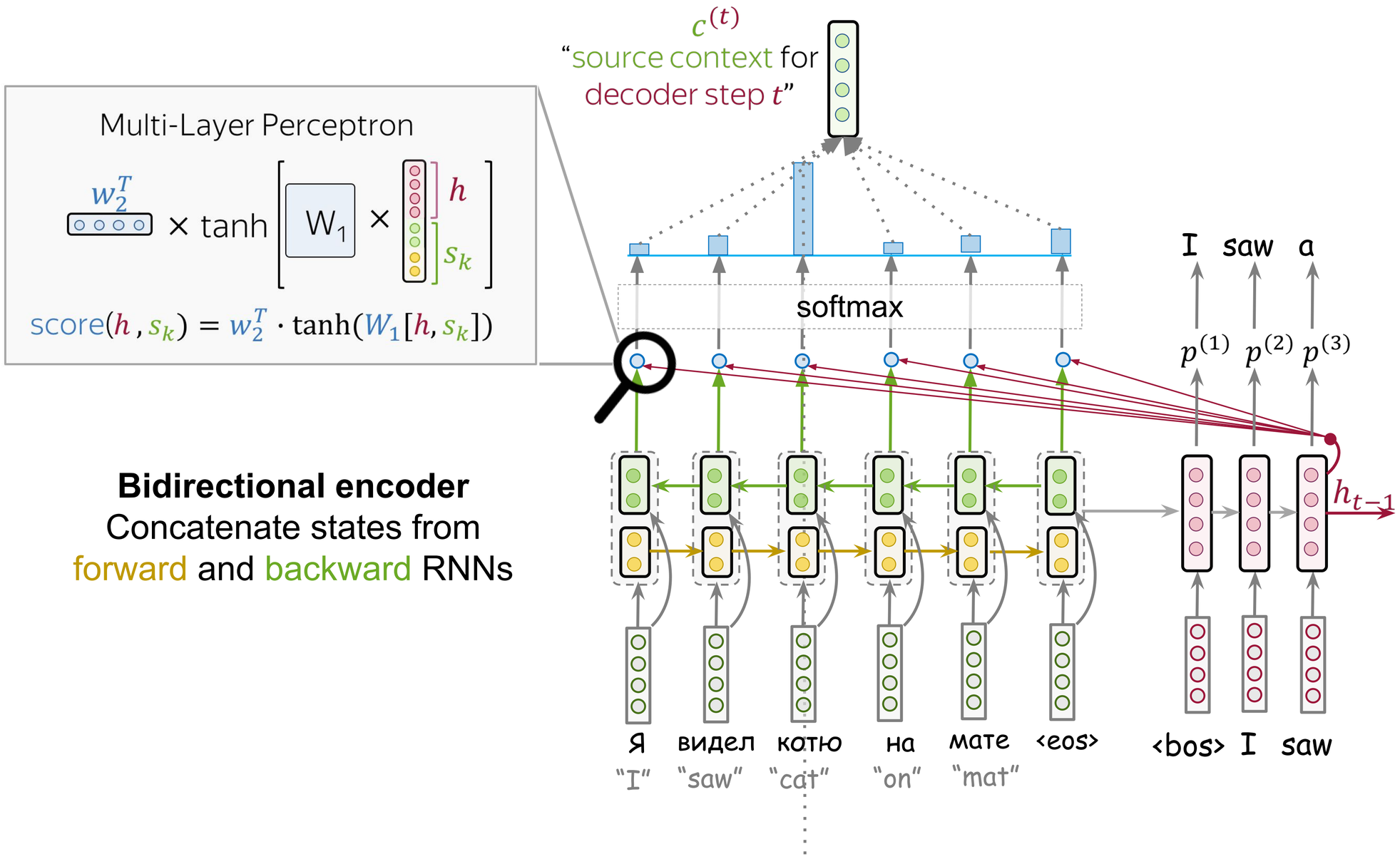


Bahdanau Model (the original attention model)

Bidirectional encoder
Concatenate states from
forward and backward RNNs

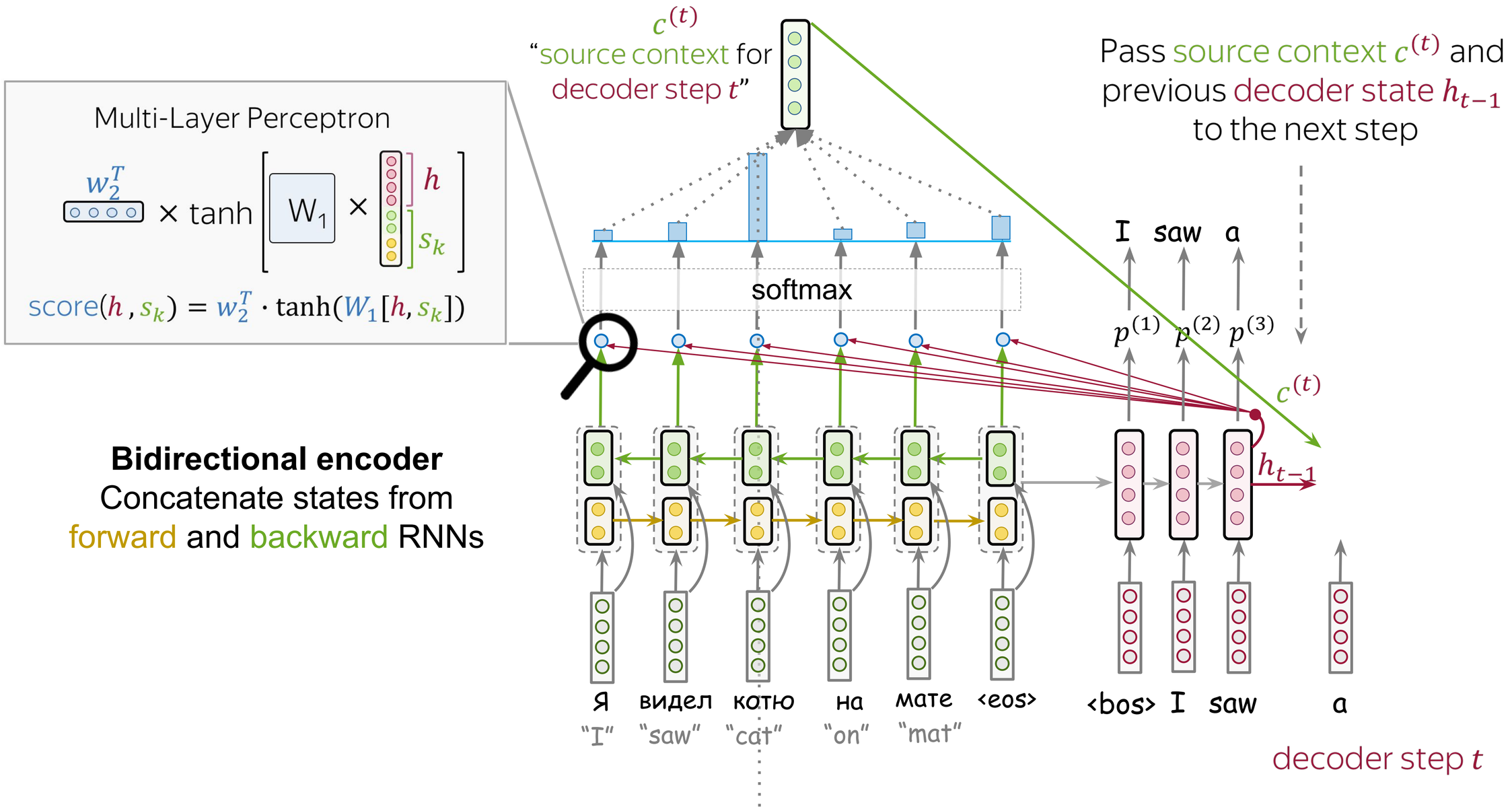


Bahdanau Model (the original attention model)



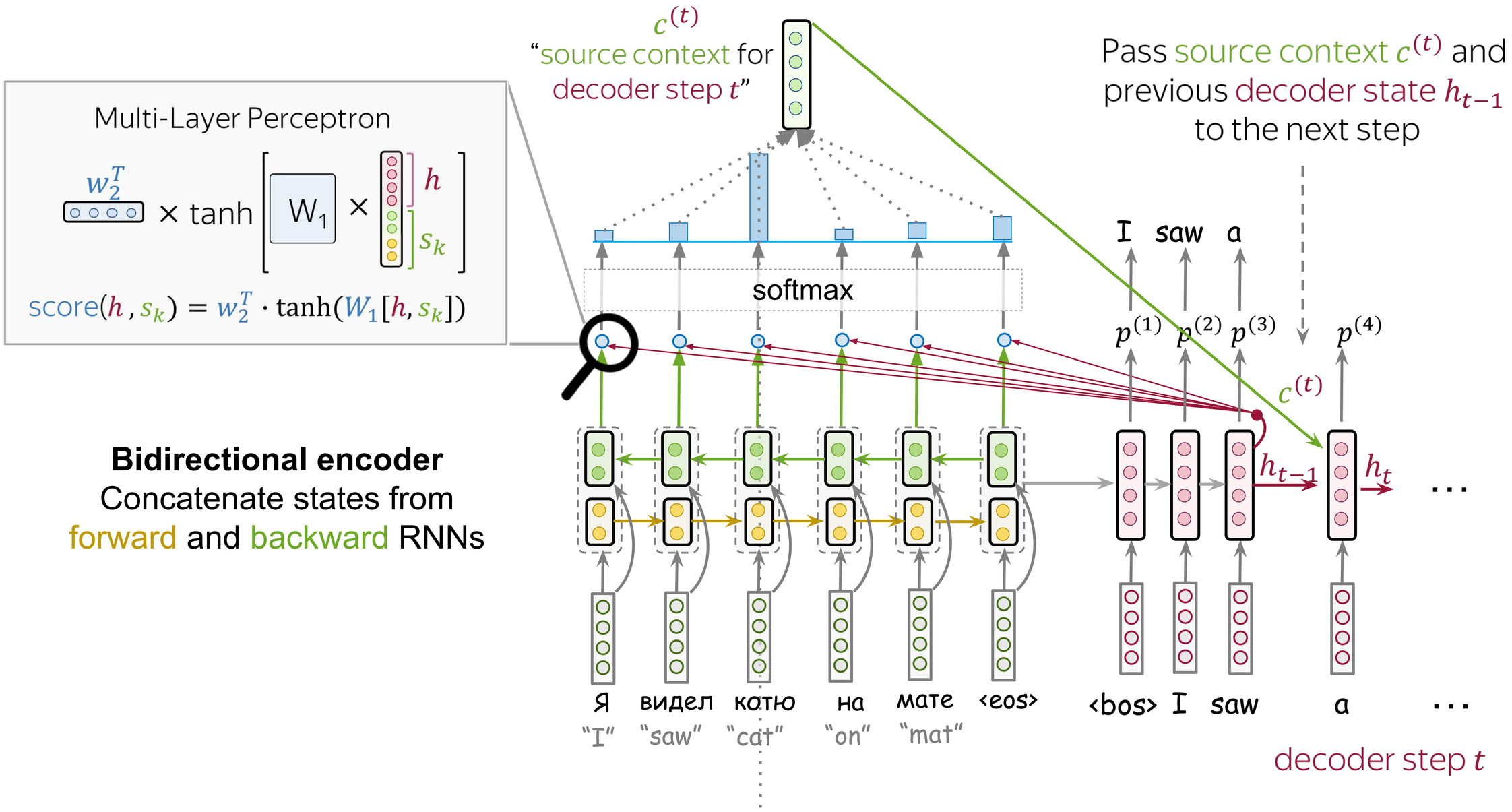
Bidirectional encoder
Concatenate states from forward and backward RNNs

Bahdanau Model (the original attention model)



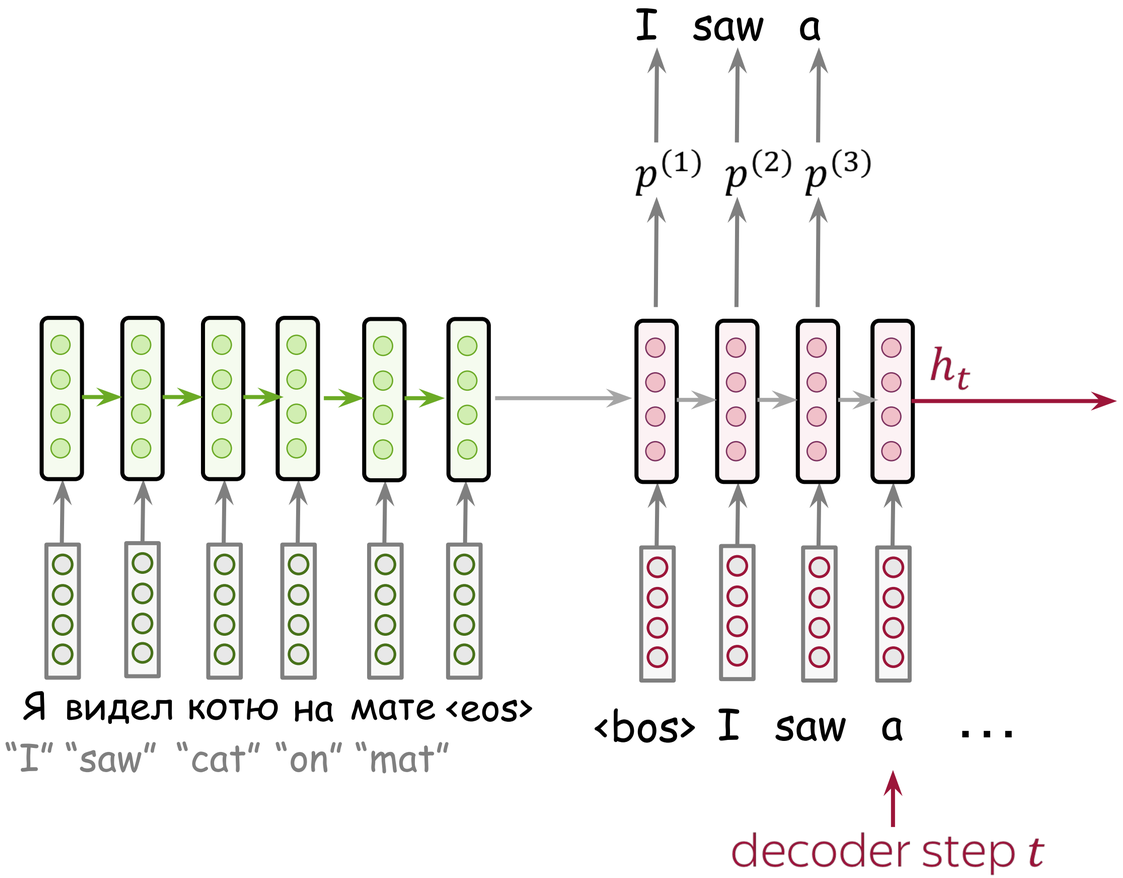
Bidirectional encoder
Concatenate states from forward and backward RNNs

Bahdanau Model (the original attention model)

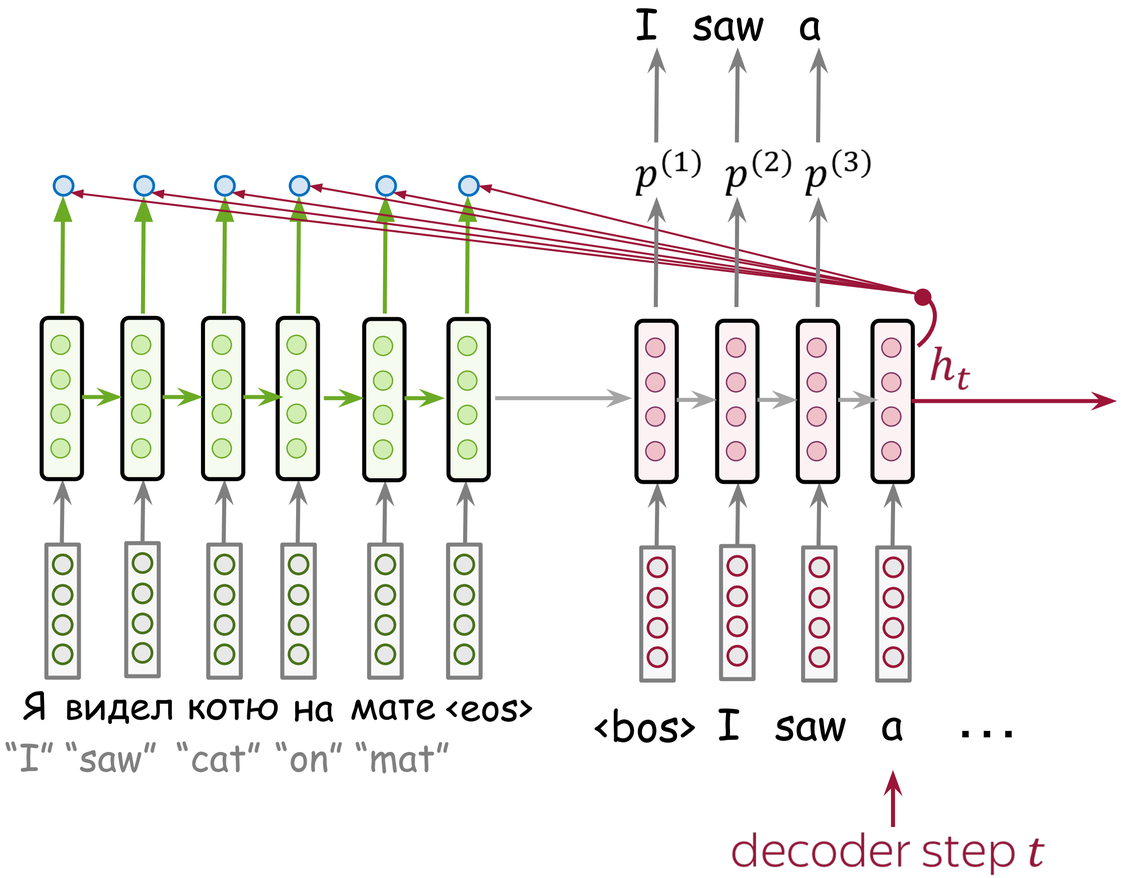


Bidirectional encoder
Concatenate states from forward and backward RNNs

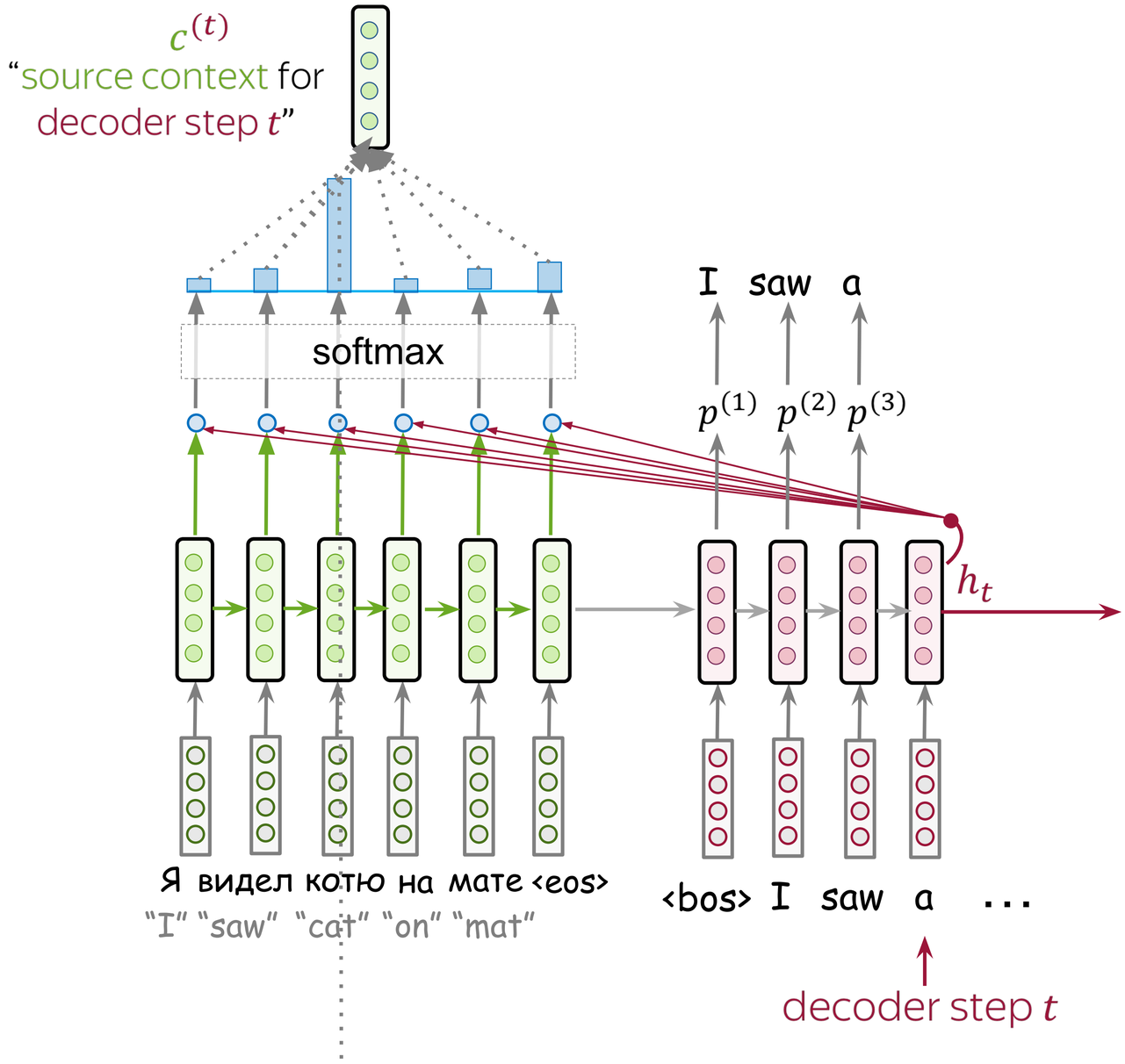
Luong Model


I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
h_t
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

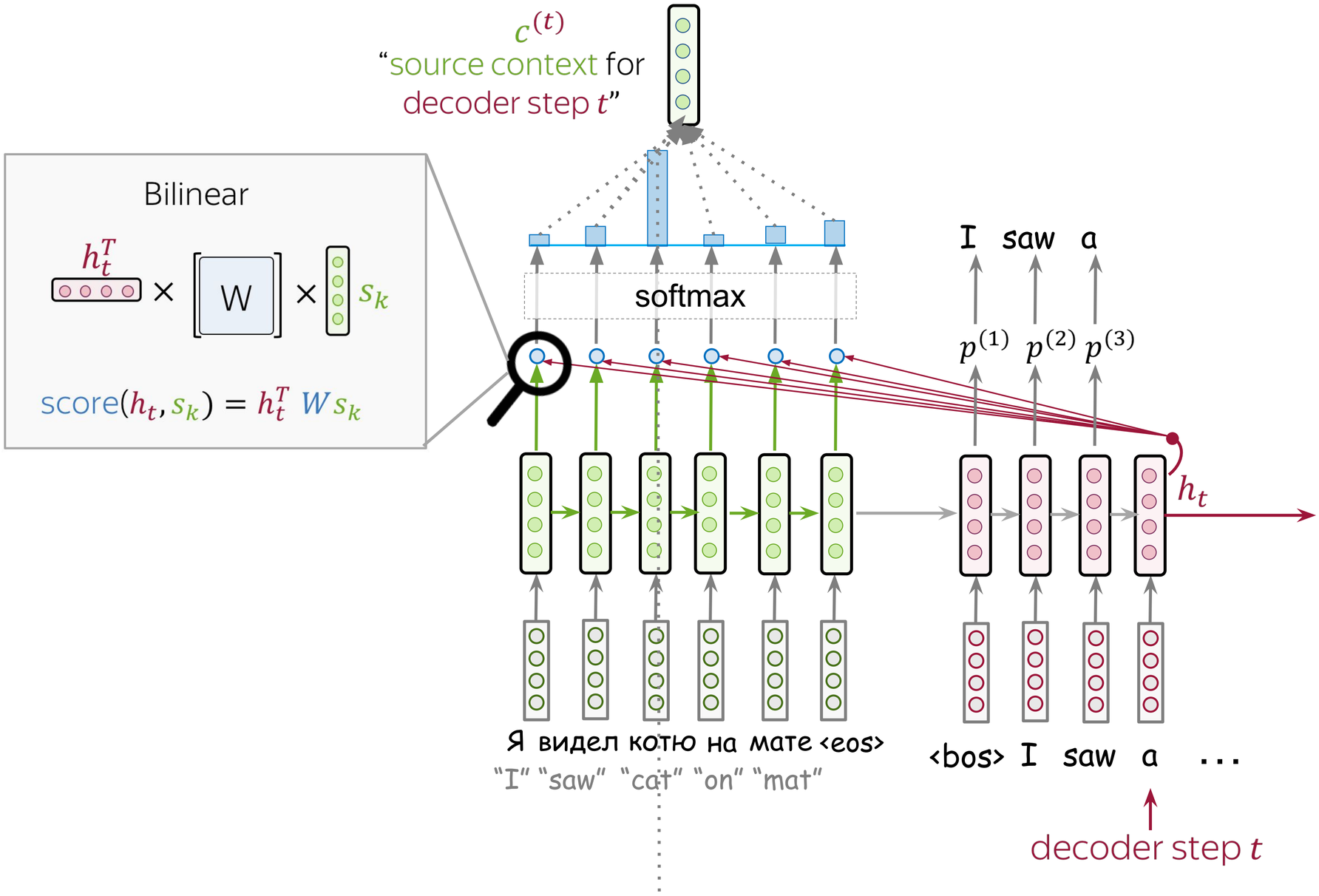
Luong Model


I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
h_t
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

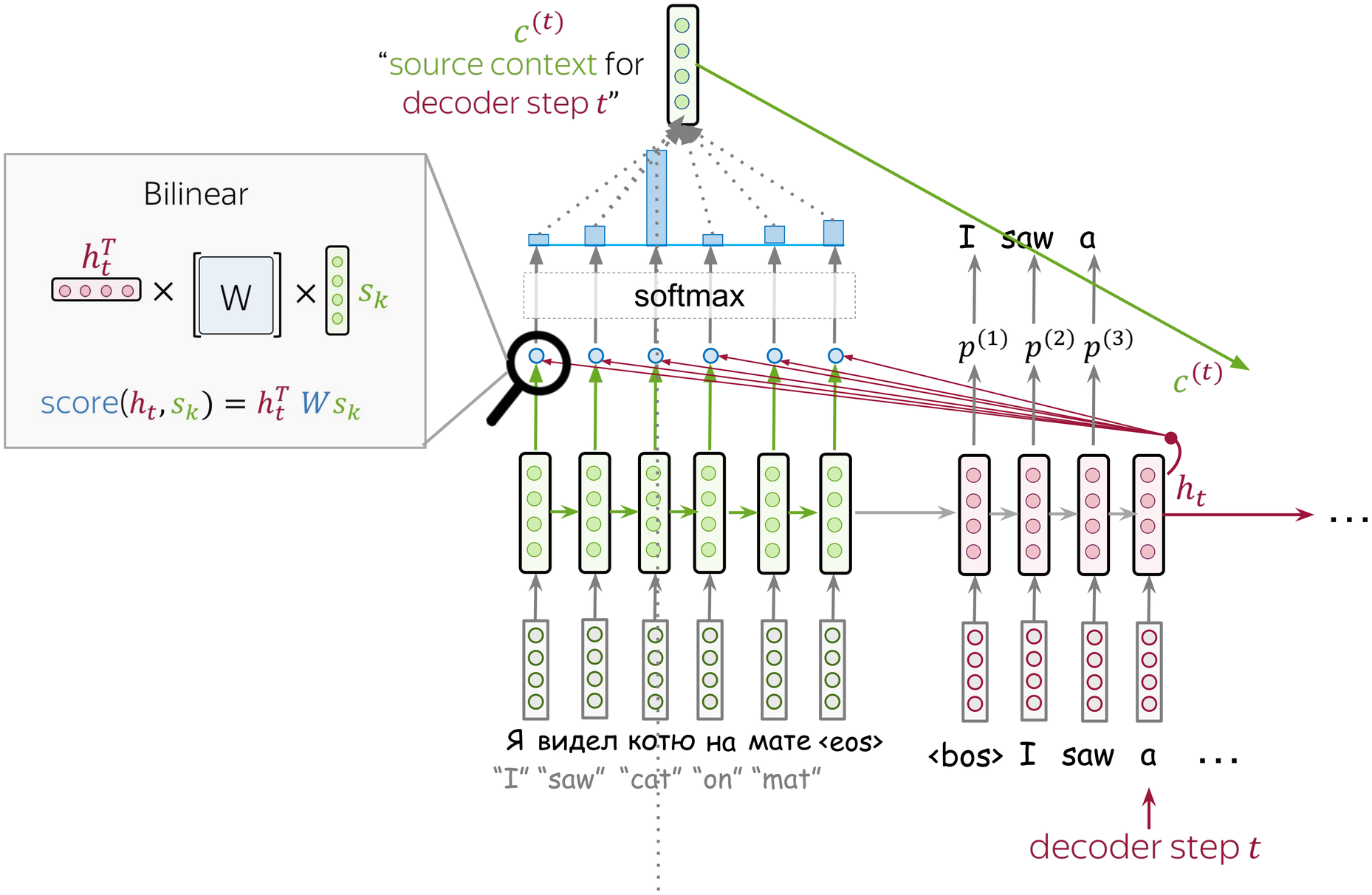
Luong Model


$c^{(t)}$
"source context for decoder step t"
softmax
I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
h_t
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

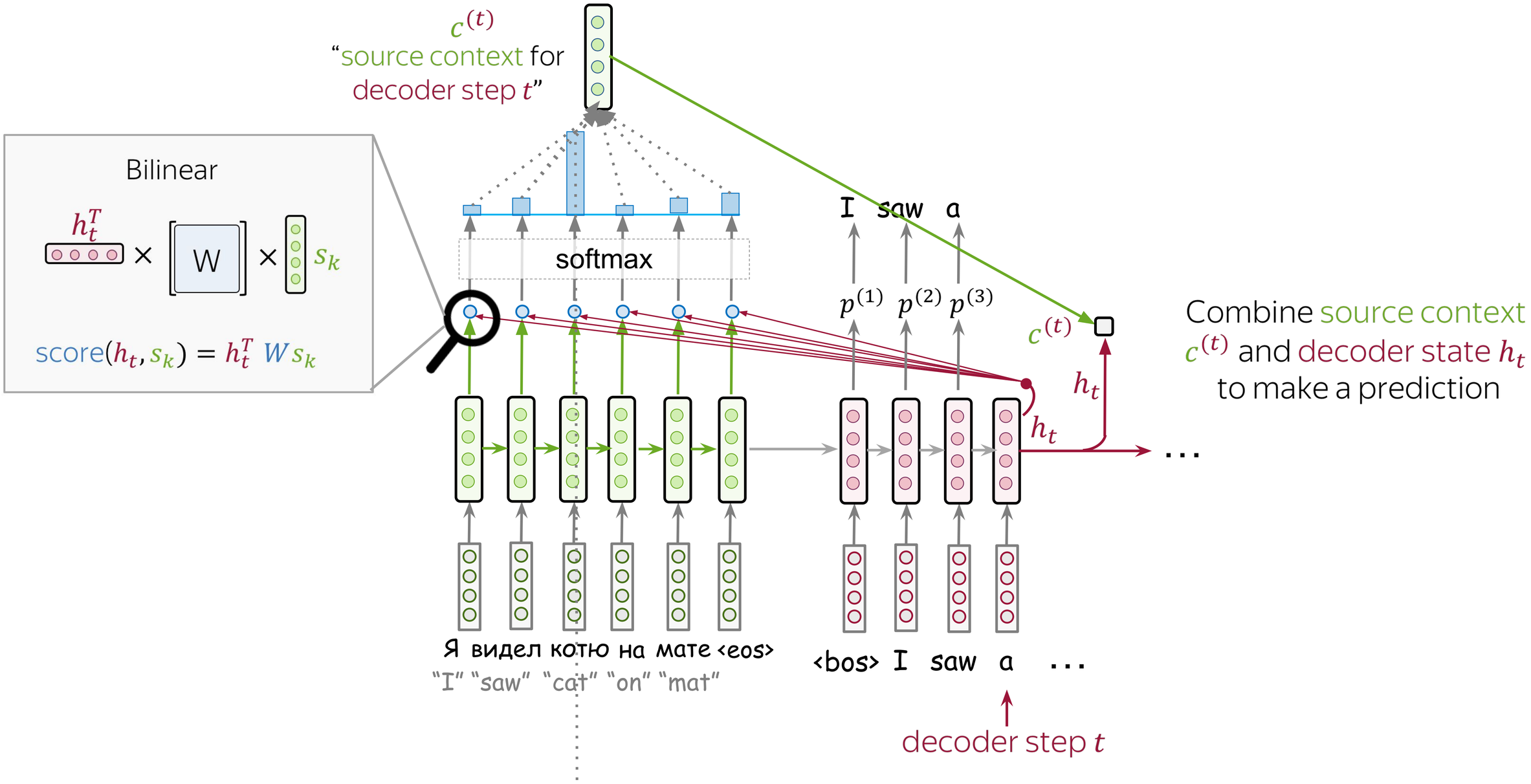
Luong Model


$c^{(t)}$
"source context for decoder step t"
Bilinear
h_t^T
W
s_k
score$(h_t, s_k) = h_t^T W s_k$
softmax
I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
h_t
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

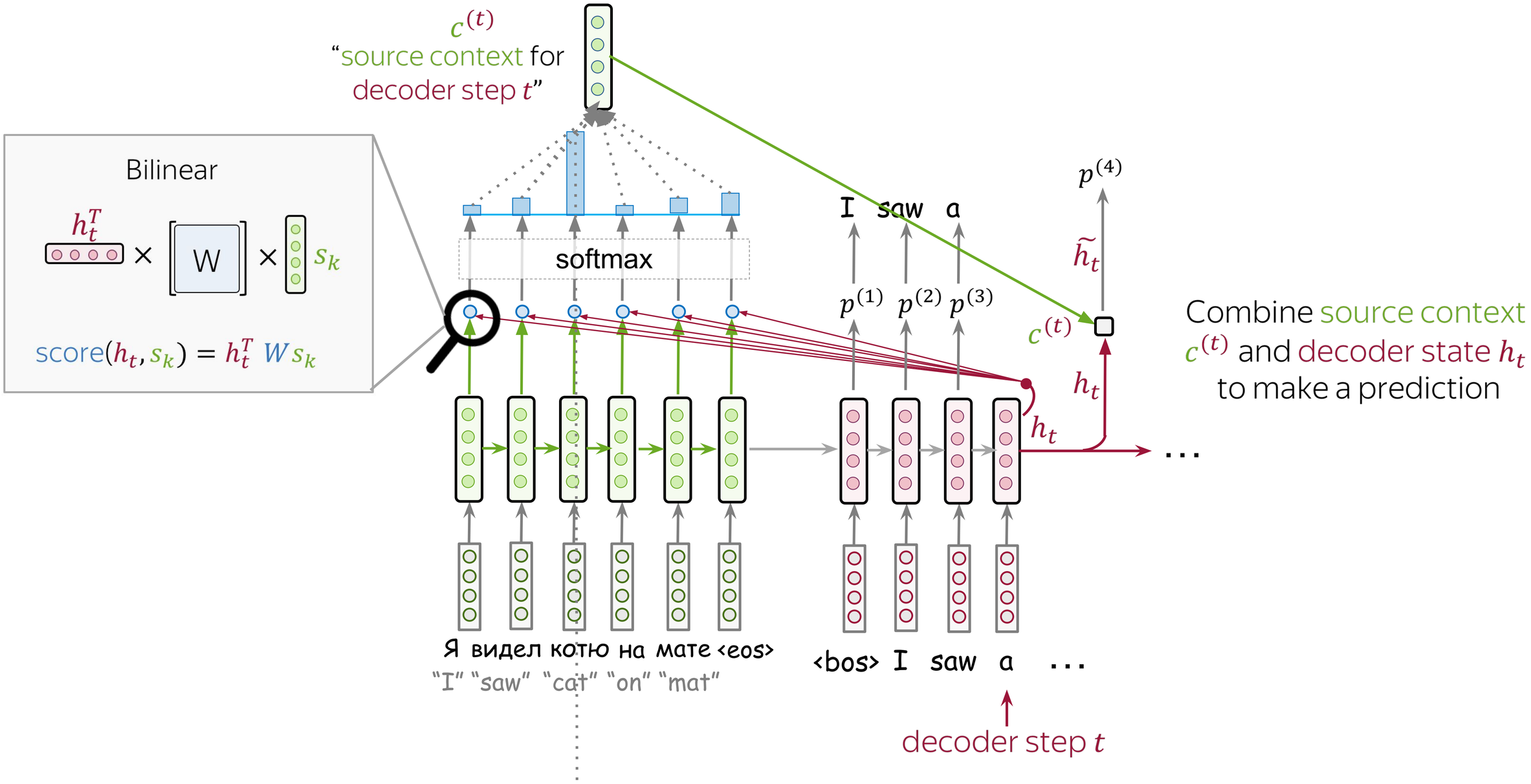
Luong Model


$c^{(t)}$
"source context for decoder step t"
Bilinear
h_t^T
W
s_k
score$(h_t, s_k) = h_t^T W s_k$
softmax
I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
$c^{(t)}$
h_t
...
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

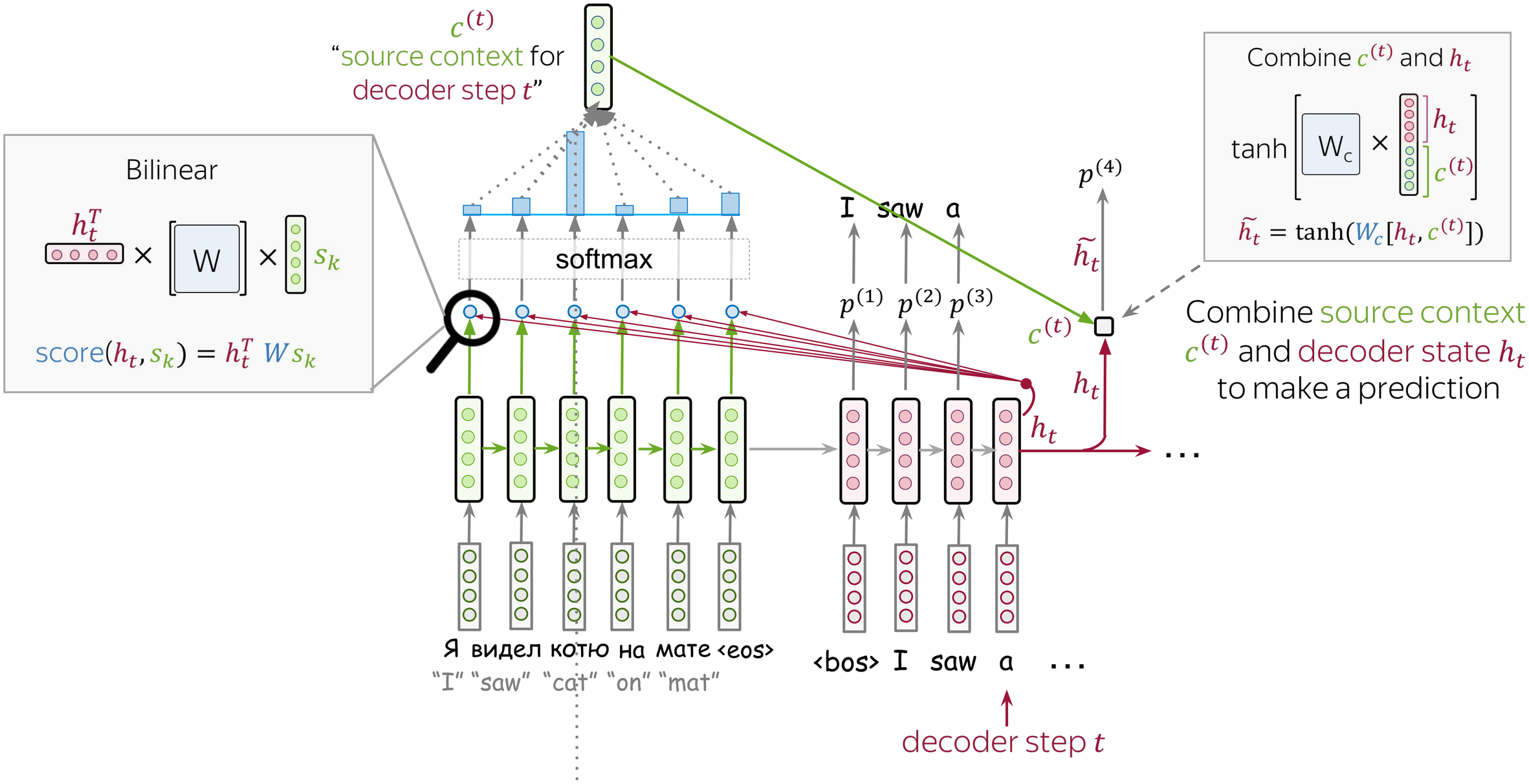
Luong Model


$c^{(t)}$
"source context for decoder step t"
Bilinear
h_t^T × W × s_k
score$(h_t, s_k) = h_t^T W s_k$
softmax
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
I saw a
$c^{(t)}$
h_t
h_t
...
Combine source context $c^{(t)}$ and decoder state h_t to make a prediction
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

Luong Model


$c^{(t)}$
"source context for decoder step t"
Bilinear
h_t^T × W × s_k
score$(h_t, s_k) = h_t^T W s_k$
softmax
I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
$p^{(4)}$
$\tilde{h}_t$
$c^{(t)}$
h_t
h_t
...
Combine source context $c^{(t)}$ and decoder state h_t to make a prediction
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

Luong Model


$c^{(t)}$
"source context for decoder step t"
Bilinear
h_t^T × W × s_k
score$(h_t, s_k) = h_t^T W s_k$
softmax
I saw a
$p^{(1)}$ $p^{(2)}$ $p^{(3)}$
$p^{(4)}$
$\tilde{h}_t$
$c^{(t)}$
h_t
Combine $c^{(t)}$ and h_t
tanh [W_c × h_t ; $c^{(t)}$]
$\tilde{h}_t = \tanh(W_c[h_t, c^{(t)}])$
Combine source context $c^{(t)}$ and decoder state h_t to make a prediction
...
Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"
<bos> I saw a ...
decoder step t

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

Idea: Attention is All You Need

	Seq2seq without attention	Seq2seq with attention
processing within encoder	RNN/CNN	RNN/CNN
processing within **decoder**	RNN/CNN	RNN/CNN
decoder -encoder interaction	static fixed-sized vector	attention

Idea: Attention is All You Need

	Seq2seq without attention	Seq2seq with attention	Transformer
processing within encoder	RNN/CNN	RNN/CNN	attention
processing within **decoder**	RNN/CNN	RNN/CNN	attention
decoder-encoder interaction	static fixed-sized vector	attention	attention

Idea: Attention is All You Need

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Encoder

Who is doing:

What they are doing:

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Encoder

Who is doing:

- all source tokens

What they are doing:

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Encoder

Who is doing:

- all source tokens

What they are doing:

- look at each other
- update representations

repeat N times

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Decoder

Who is doing:

What they are doing:

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Decoder

Who is doing:

- target token at the current step

What they are doing:

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Decoder

Who is doing:

- target token at the current step

What they are doing:

- looks at previous target tokens
- looks at source representations
- update representation

repeat N times

The animation is from the Google AI blog post.

Why can this be better than RNNs?

I arrived at the bank after crossing the …

…street?

…river?

What does bank mean in this sentence?

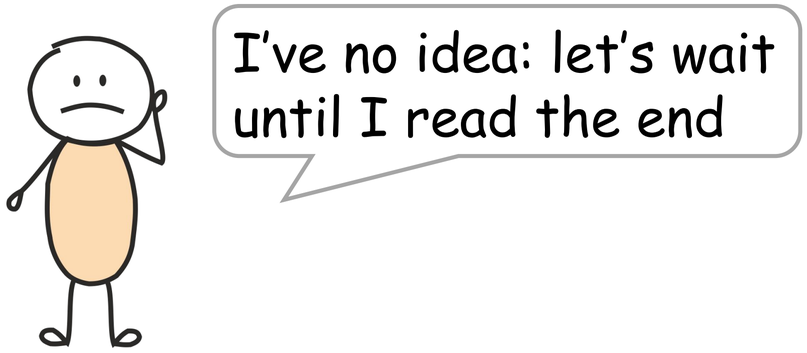
Why can this be better than RNNs?

I arrived at the bank after crossing the …

…street?

…river?

What does bank mean in this sentence?



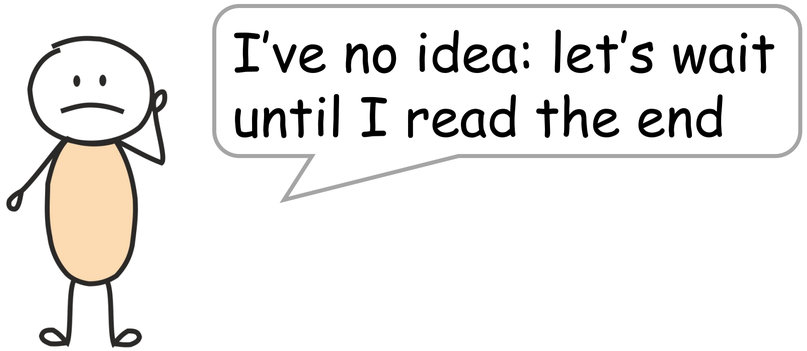
RNNs

O(N) steps to process a sentence with length N

Why can this be better than RNNs?

I arrived at the bank after crossing the … …street? …river?

What does bank mean in this sentence?



RNNs

O(N) steps to process a sentence with length N



Transformer

Constant number of steps to process any sentence

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

Self-Attention: Why “Self”?

Decoder-encoder attention is looking

- **from**: one current decoder state
- **at**: all encoder states

Self-Attention: Why “Self”?

Decoder-encoder attention is looking

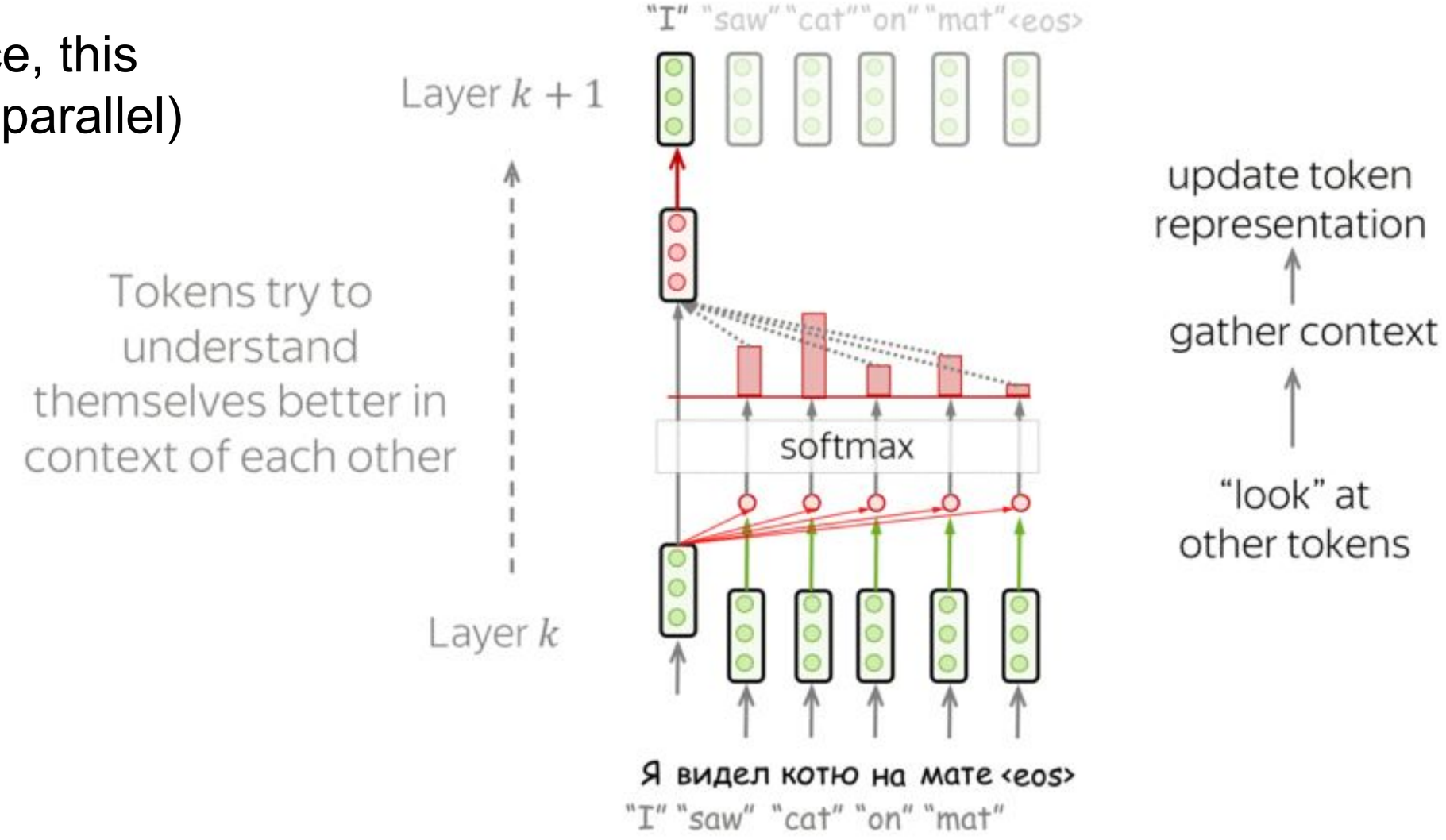
- **from**: one current decoder state
- **at**: all encoder states

Self-attention is looking

- **from**: each state from a set of states
- **at**: all other states in the same set

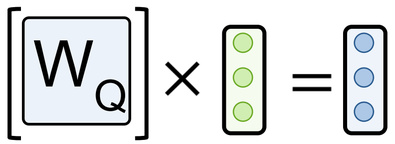
Self-Attention: The “Look at Each Other” Part

(in practice, this happens in parallel)



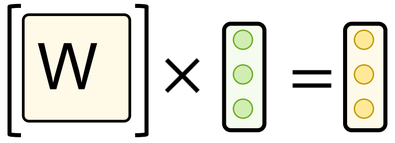
Query, Key, Value

Each vector receives three representations (“roles”)



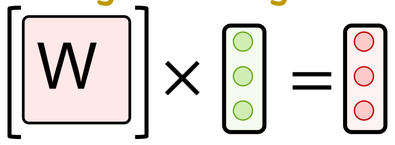
Query: vector **from** which the attention is looking

“Hey there, do you have this information?”



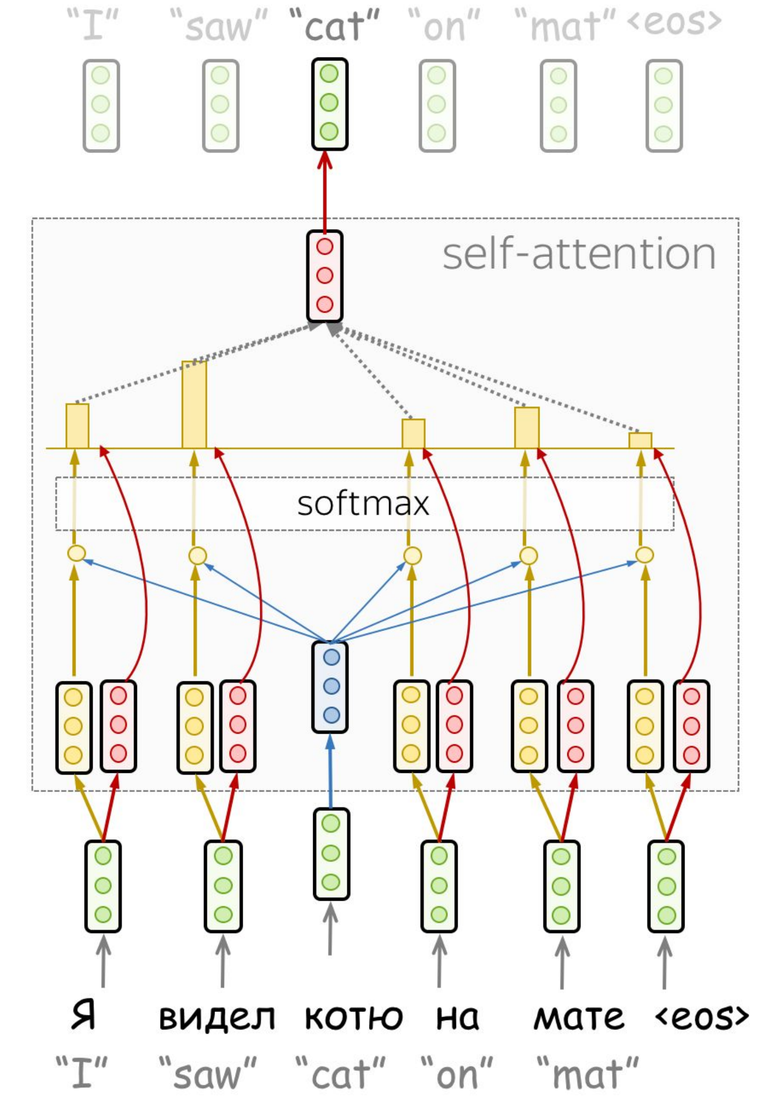
Key: vector **at** which the query looks to compute weights

“Hi, I have this information – give me a large weight!”

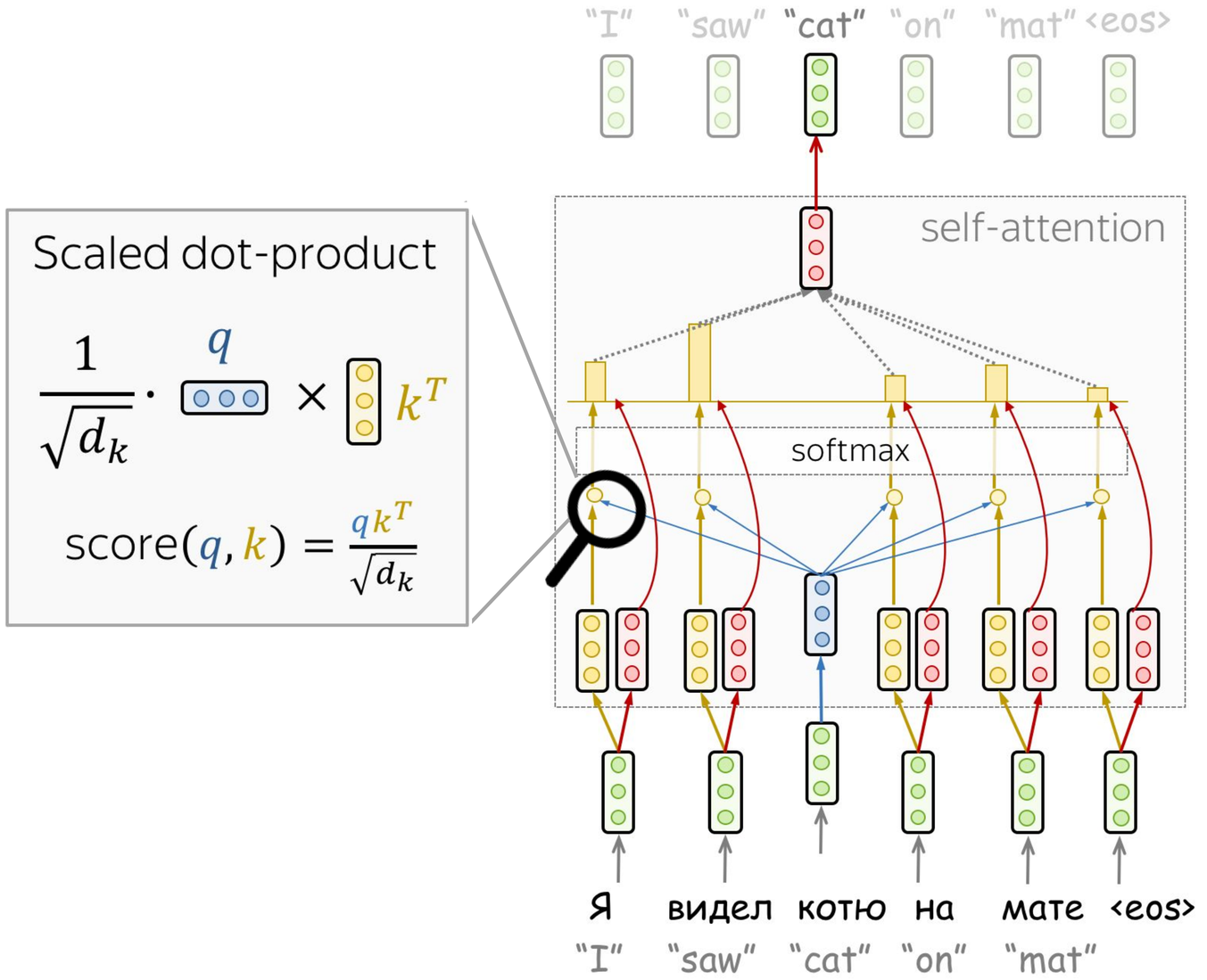


Value: their weighted sum is attention output

“Here’s the information I have!”



Query, Key, Value



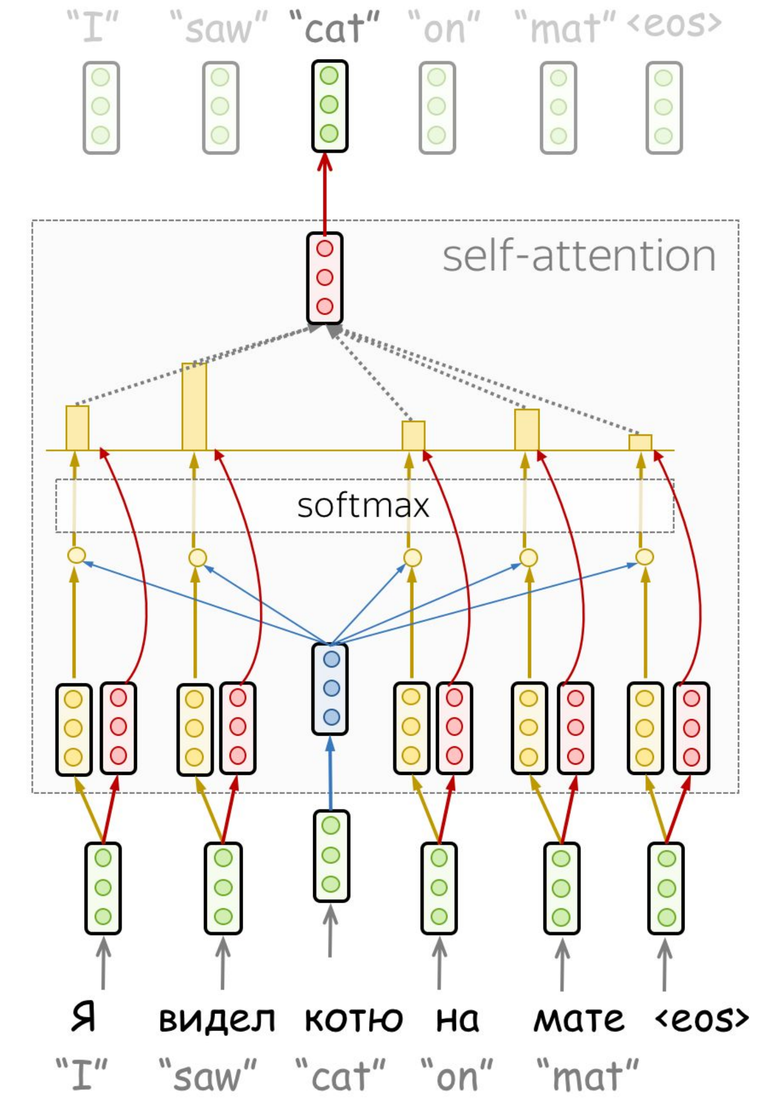
Query, Key, Value

$$Attention(q, k, v) = \overbrace{softmax\left(\frac{qk^T}{\sqrt{d_k}}\right)v}^{\text{Attention weights}}$$

from

to

vector dimensionality of K, V



$\sqrt{d_k}$?

Assumption: $0 = \mathbb{E}[k_i] = \mathbb{E}[q_i],\ 1 = Var[k_i] = Var[q_i]$

$$Var[XY] = \mathbb{E}[X^2Y^2] - \mathbb{E}^2[XY] \overset{why?}{=} \mathbb{E}[X^2]\mathbb{E}[Y^2] - \mathbb{E}^2[X]\mathbb{E}^2[Y]$$
$$= Var[X]Var[Y] + \mathbb{E}[X^2]\mathbb{E}^2[Y] + \mathbb{E}^2[X]\mathbb{E}[Y^2] - 2\mathbb{E}^2[X]\mathbb{E}^2[Y]$$
$$= Var[X]Var[Y] + Var[X]\mathbb{E}^2[Y] + Var[Y]\mathbb{E}^2[X]$$

Hence

$$Var[k^T q] = \sum Var[k_i q_i] = \sum Var[k_i]Var[q_i] = d_k$$

$\sqrt{d_k}$?

Assumption: $0 = \mathbb{E}[k_i] = \mathbb{E}[q_i],\ 1 = Var[k_i] = Var[q_i]$

$$Var[XY] = \mathbb{E}[X^2Y^2] - \mathbb{E}^2[XY] \overset{why?}{=} \mathbb{E}[X^2]\mathbb{E}[Y^2] - \mathbb{E}^2[X]\mathbb{E}^2[Y]$$
$$= Var[X]Var[Y] + \mathbb{E}[X^2]\mathbb{E}^2[Y] + \mathbb{E}^2[X]\mathbb{E}[Y^2] - 2\mathbb{E}^2[X]\mathbb{E}^2[Y]$$
$$= Var[X]Var[Y] + Var[X]\mathbb{E}^2[Y] + Var[Y]\mathbb{E}^2[X]$$

Hence

$$Var[k^T q] = \sum Var[k_i q_i] = \sum Var[k_i]Var[q_i] = d_k$$

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

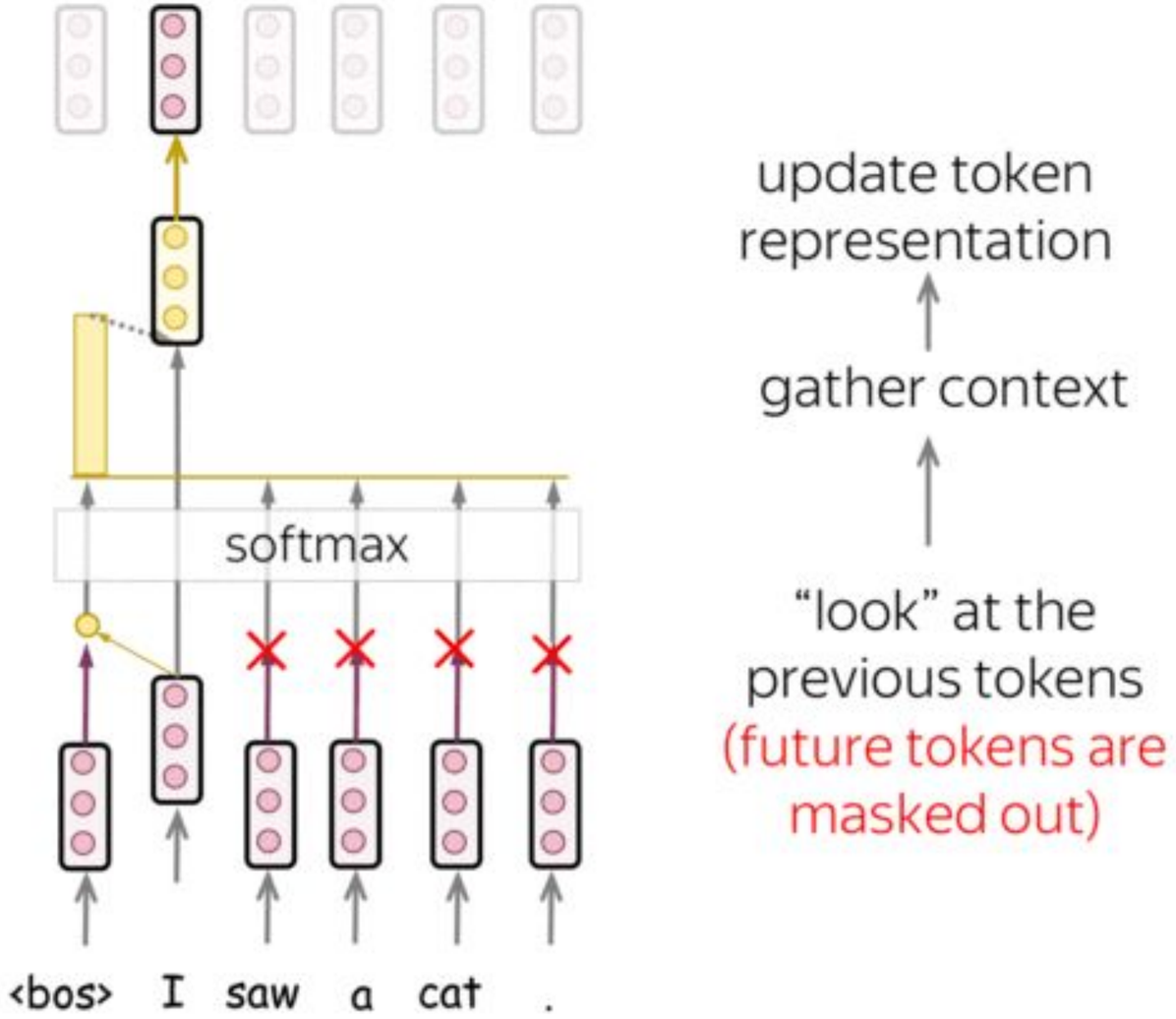
What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

Masked Self-Attention: “Don’t Look Ahead”

In the decoder, we forbid looking at future tokens – we don’t know them

Note: in training, decoder processes all target tokens at once – without masks, it would see future



What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

Multi-Head Attention

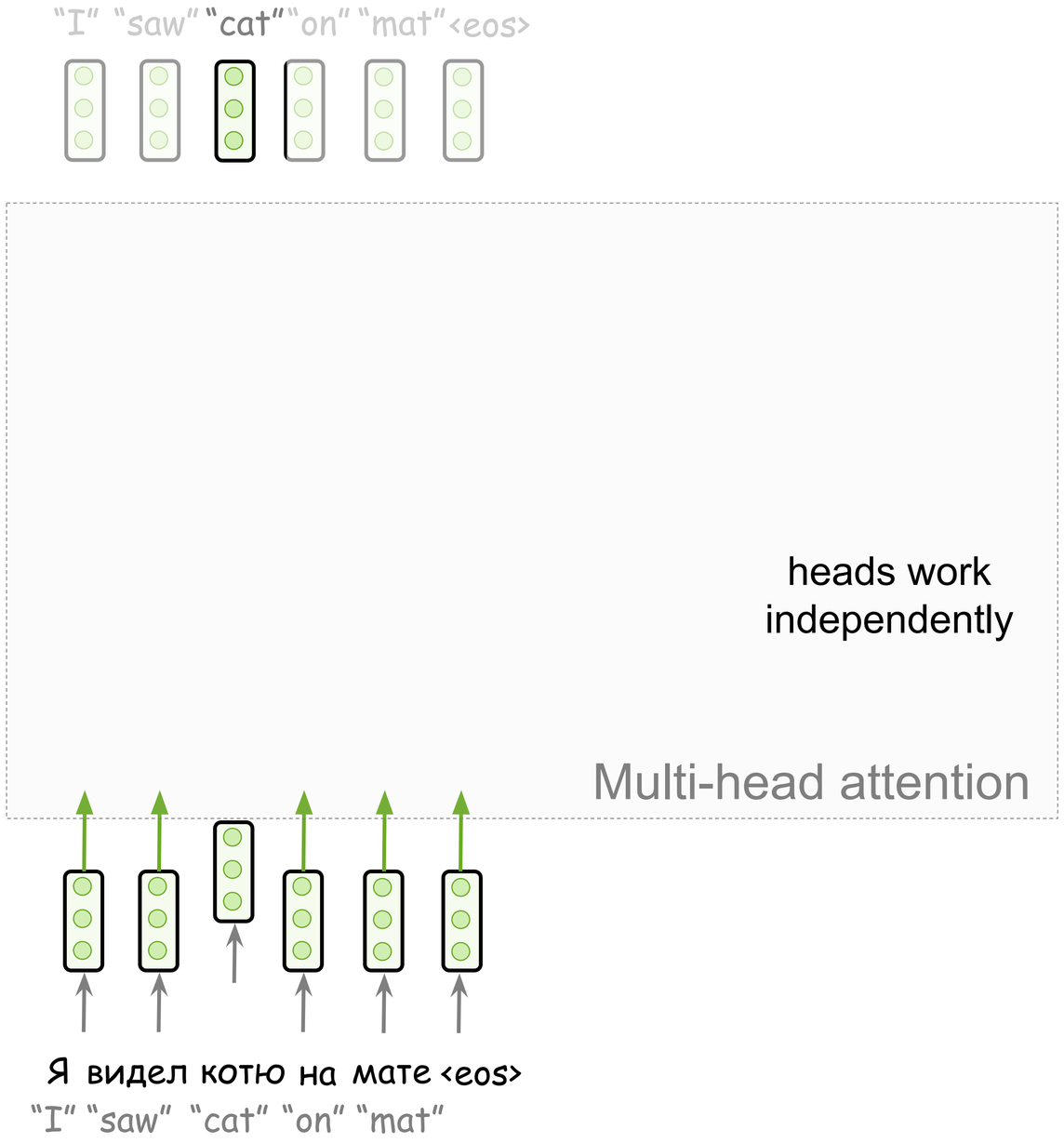
We need to track many different things at once!



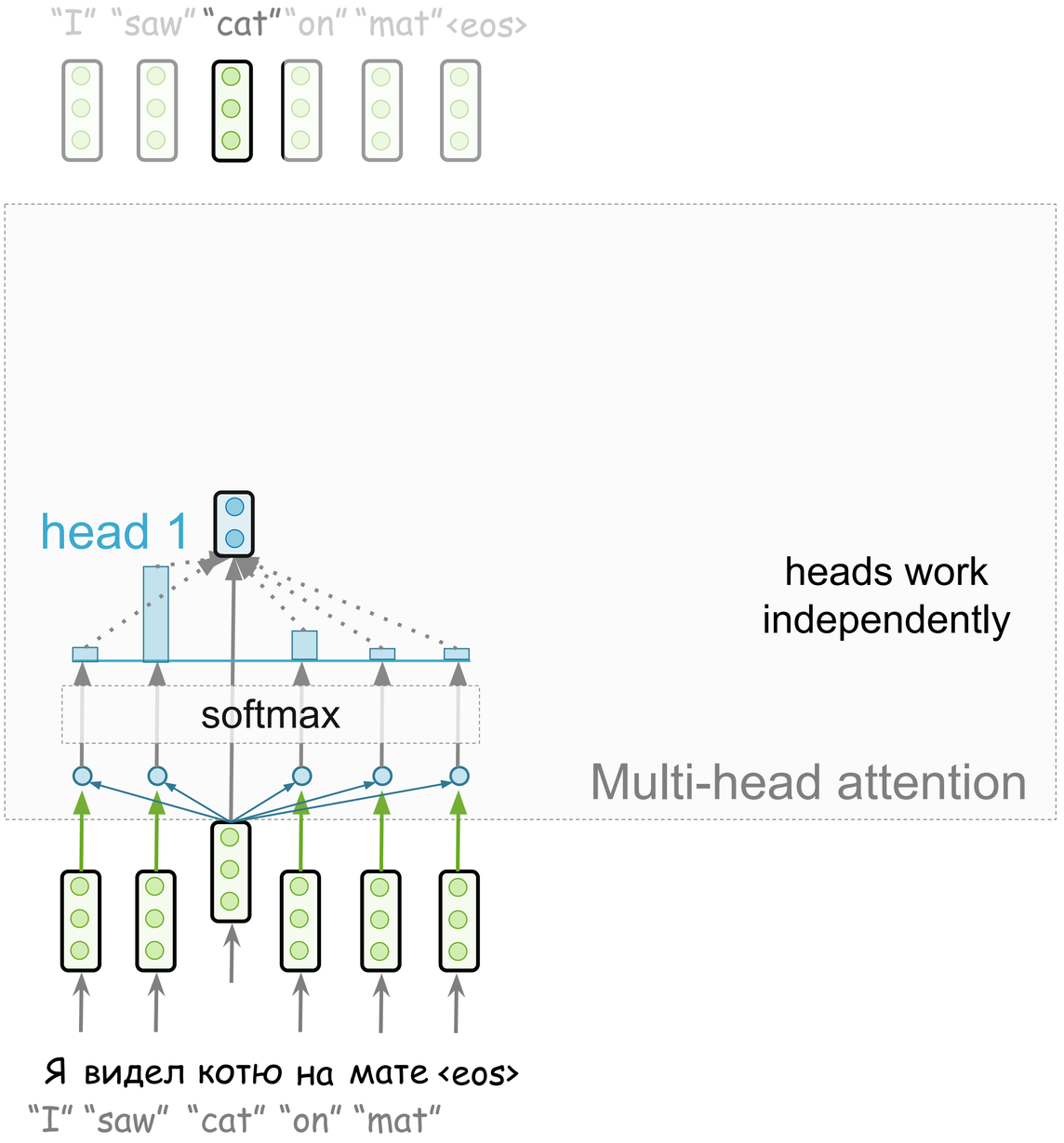
- Gender agreement
- Case government
- Lexical preferences
- ...

The example is from David Talbot

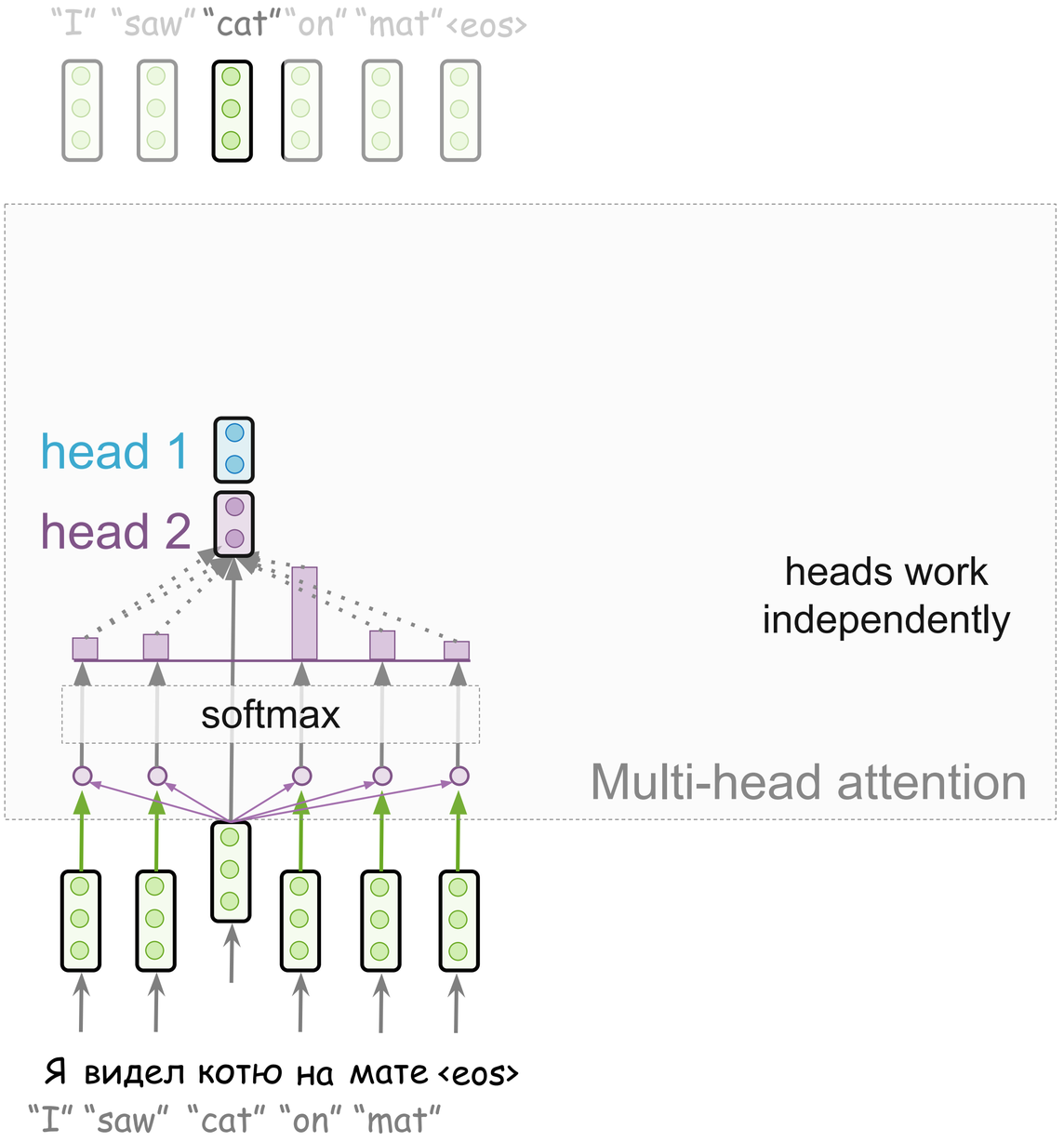
Multi-Head Attention



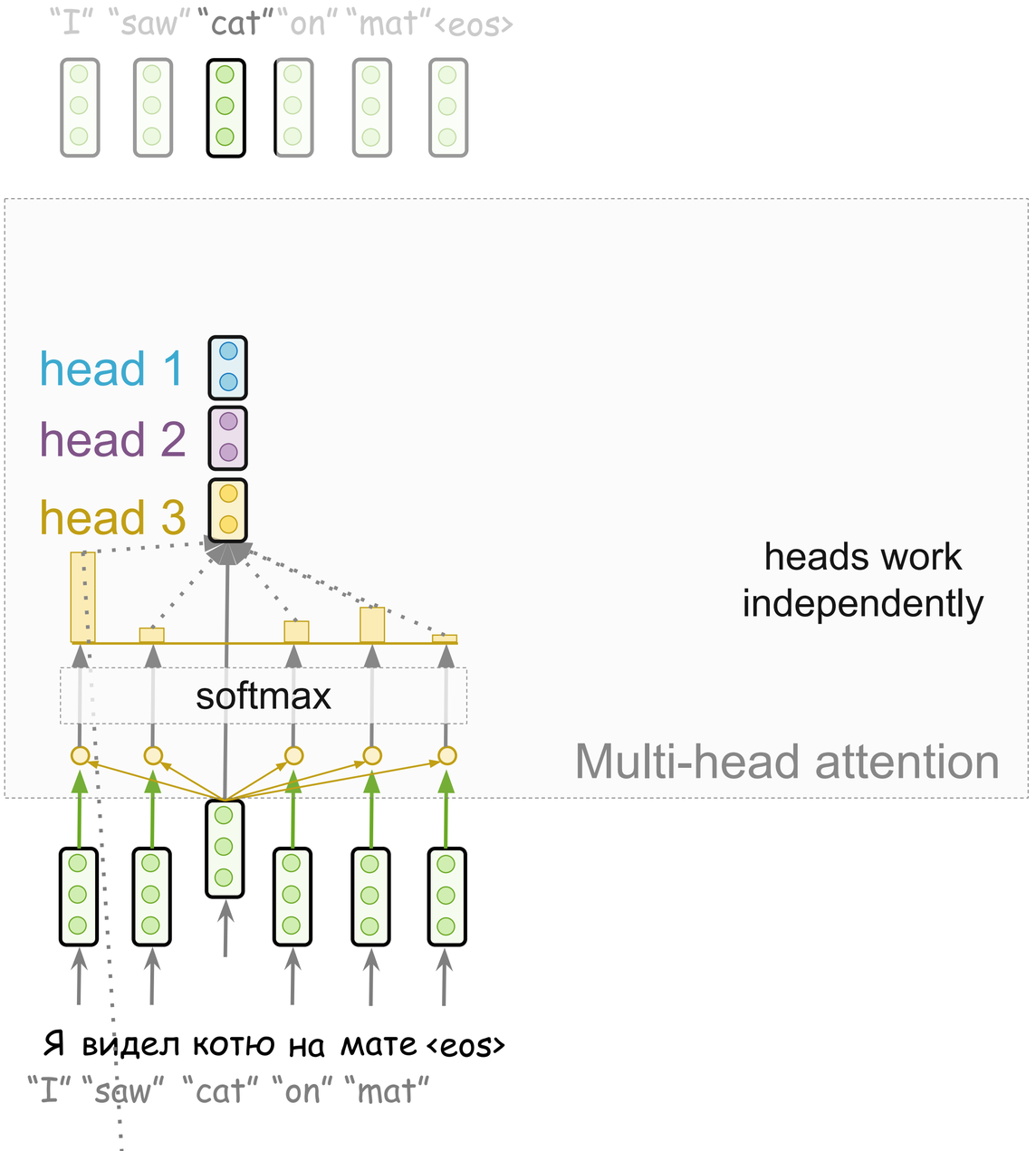
Multi-Head Attention



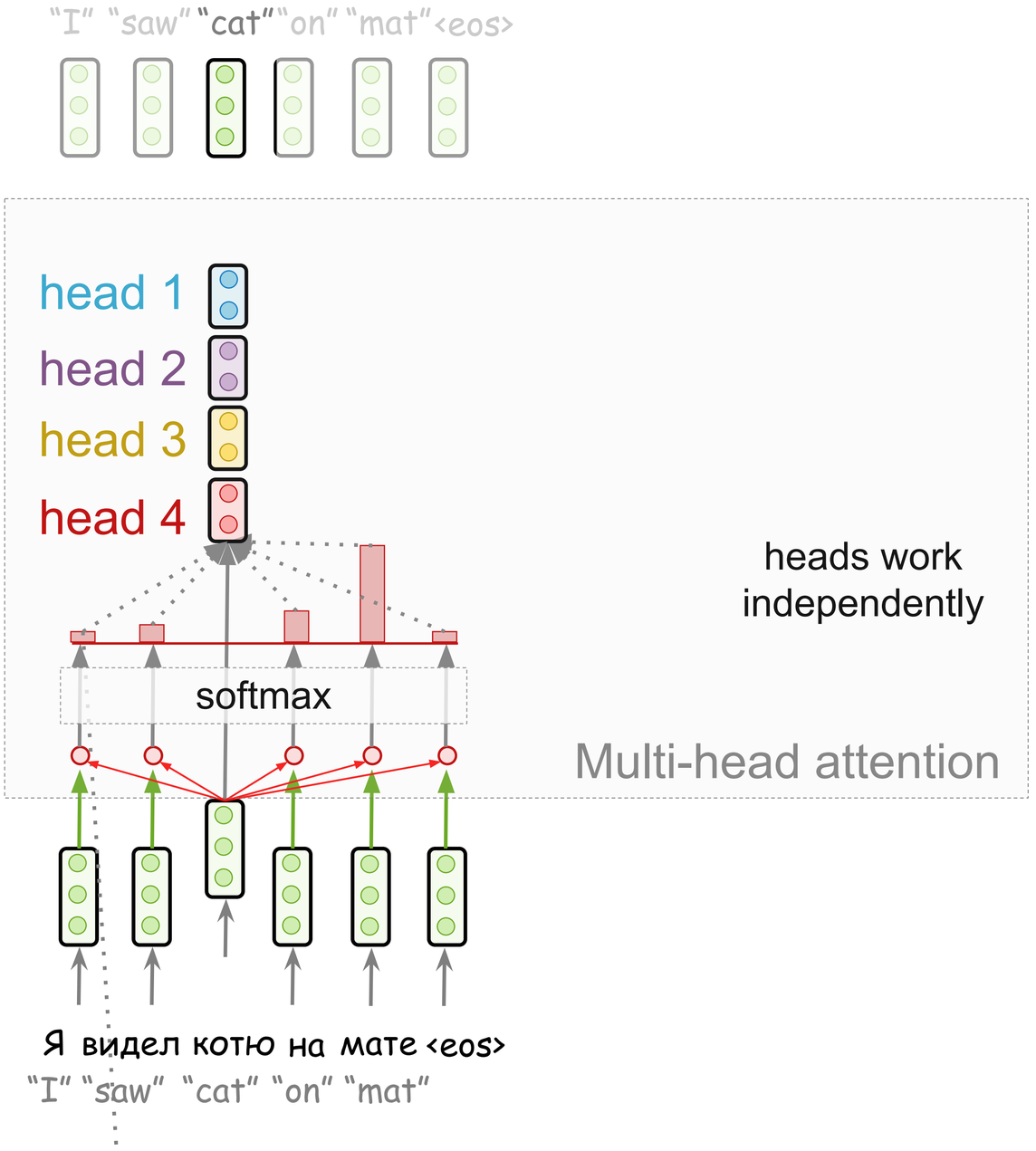
Multi-Head Attention



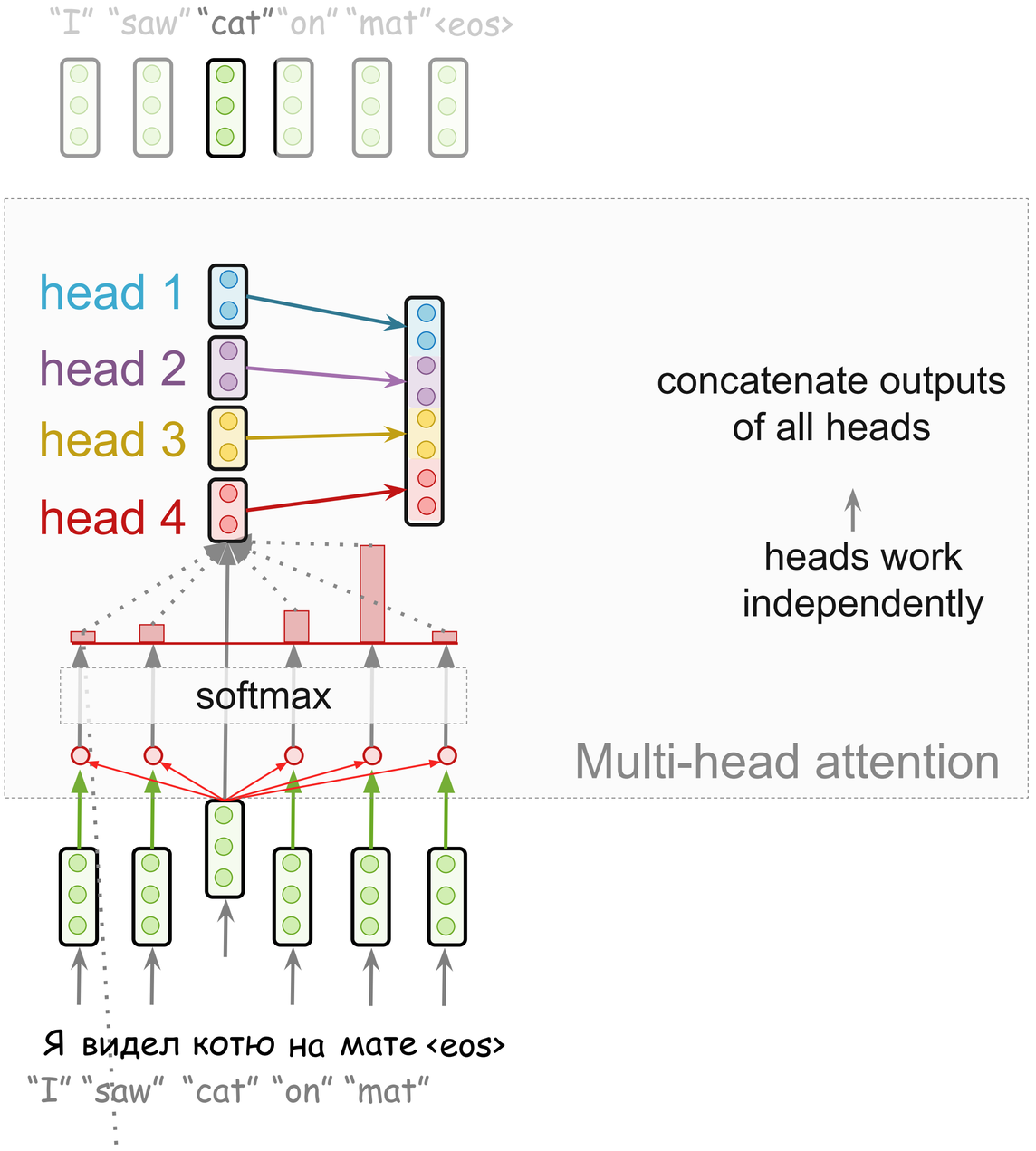
Multi-Head Attention



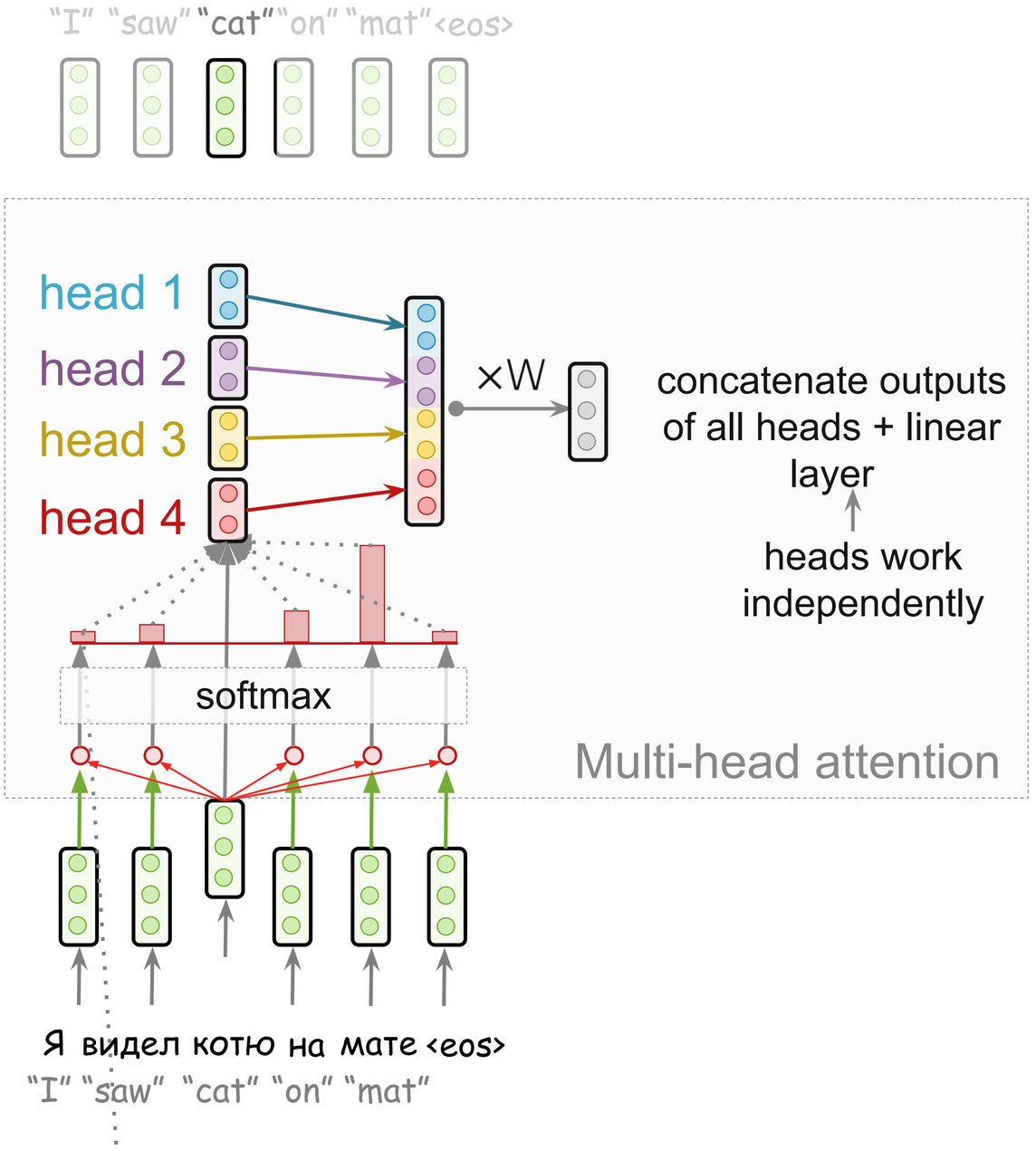
Multi-Head Attention



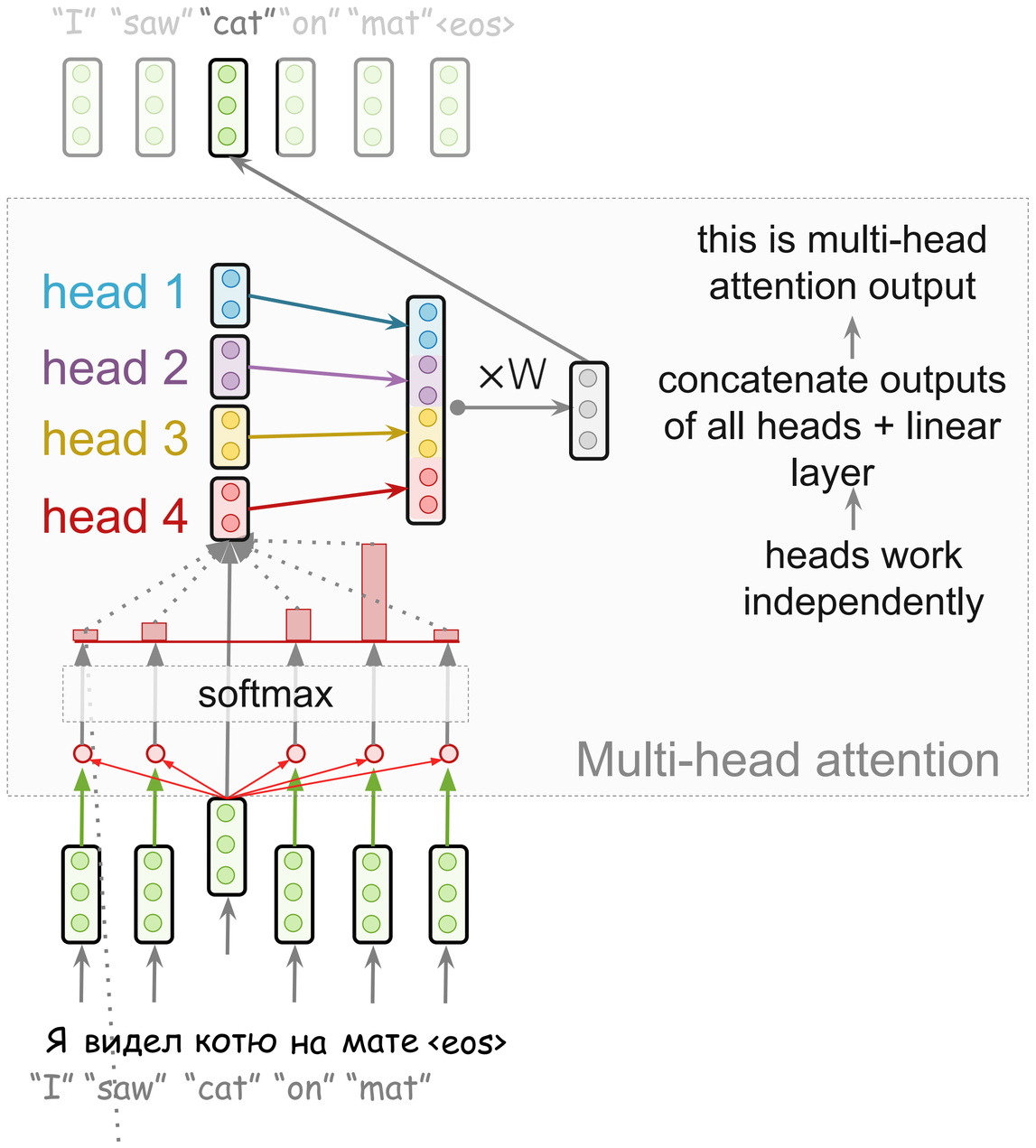
Multi-Head Attention



Multi-Head Attention



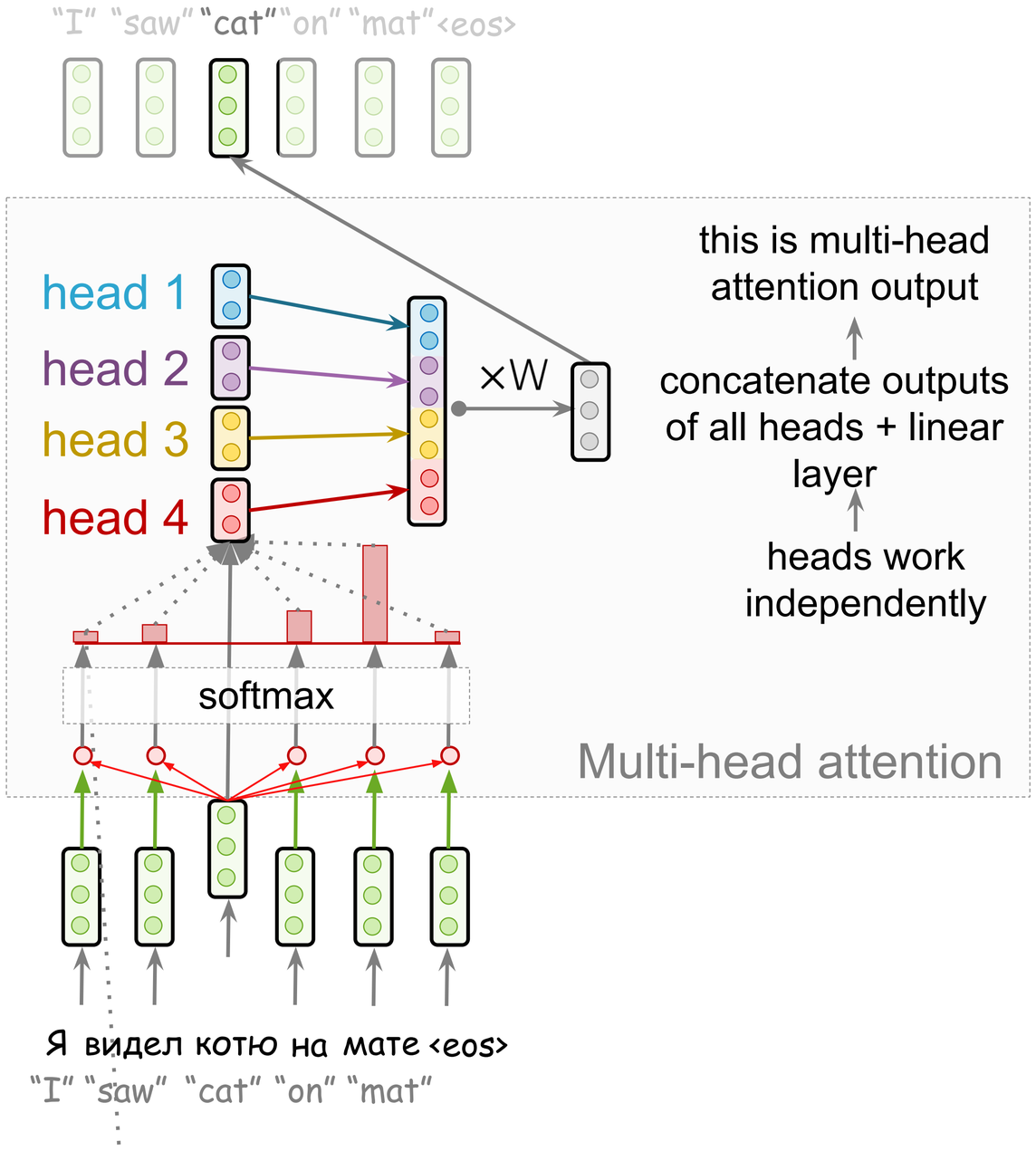
Multi-Head Attention



Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_n)W_o,$$

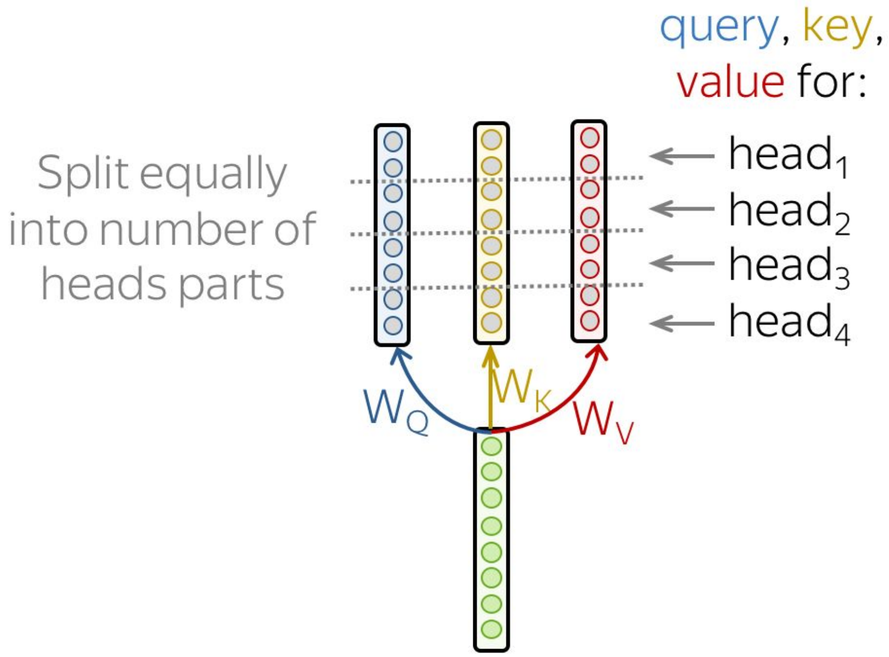
$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

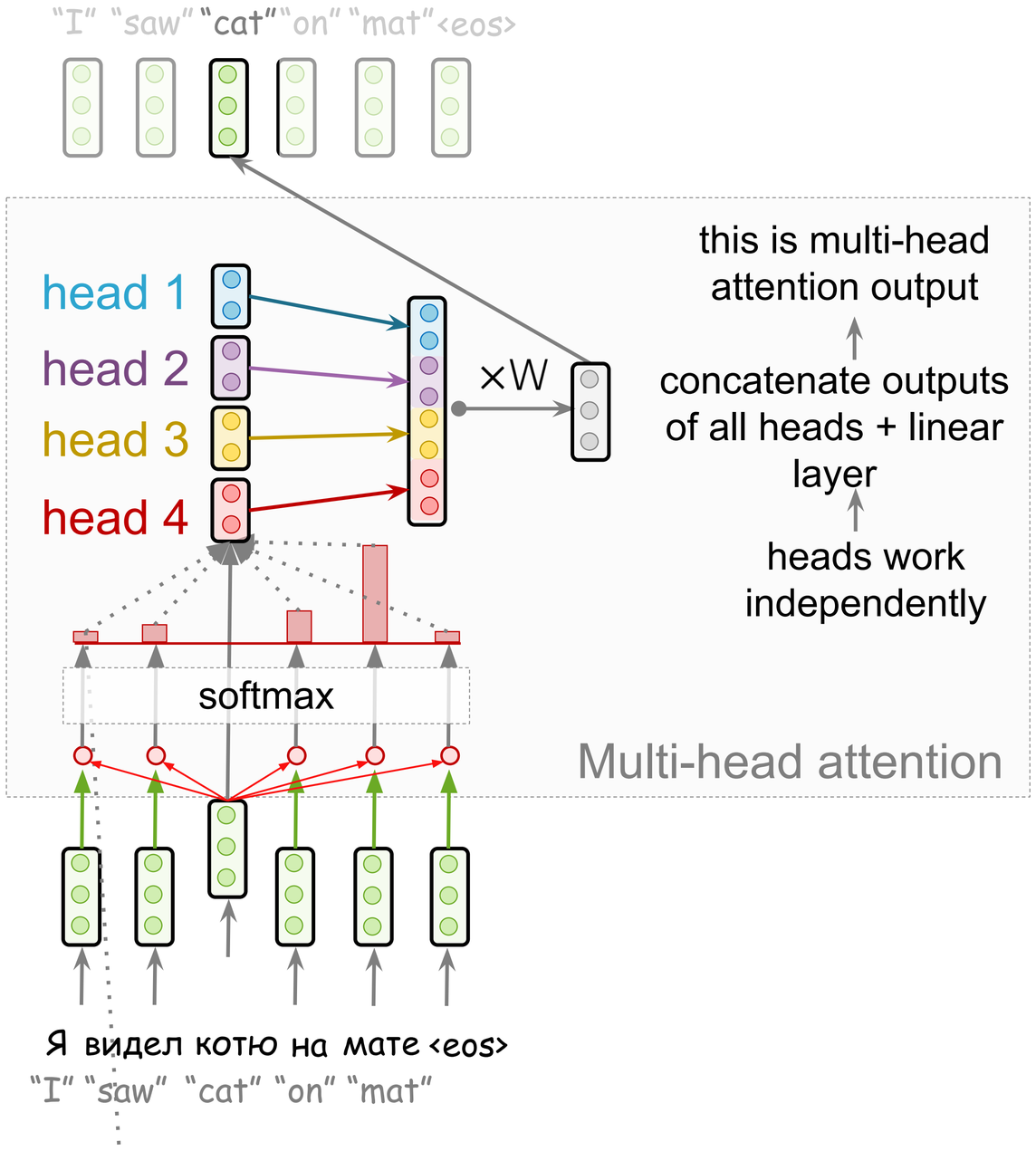


Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_n)W_o,$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$





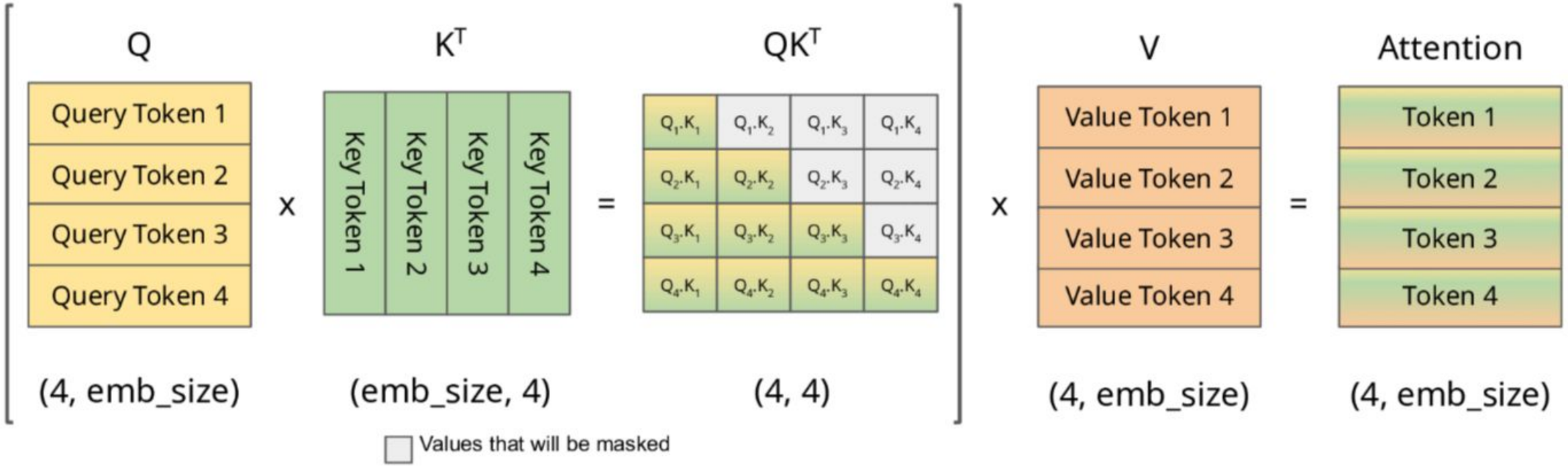
What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

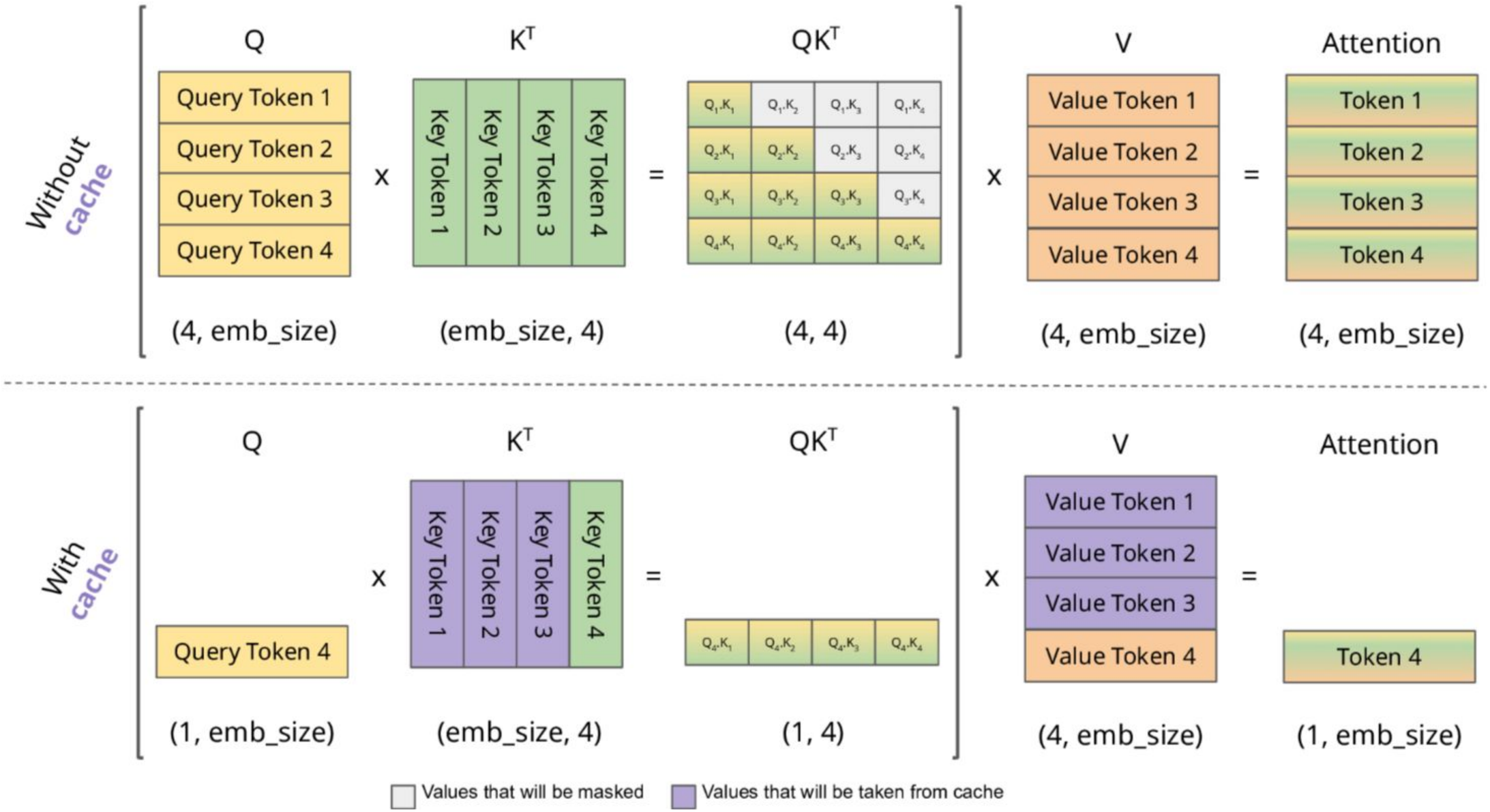
- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

KV caching


Q
K^T
QK^T
V
Attention
Query Token 1
Query Token 2
Query Token 3
Query Token 4
x
Key Token 1
Key Token 2
Key Token 3
Key Token 4
=
Q1.K1
Q1.K2
Q1.K3
Q1.K4
Q2.K1
Q2.K2
Q2.K3
Q2.K4
Q3.K1
Q3.K2
Q3.K3
Q3.K4
Q4.K1
Q4.K2
Q4.K3
Q4.K4
x
Value Token 1
Value Token 2
Value Token 3
Value Token 4
=
Token 1
Token 2
Token 3
Token 4
(4, emb_size)
(emb_size, 4)
(4, 4)
(4, emb_size)
(4, emb_size)
Values that will be masked


СИДИШЬ, ТРАНСФОРМИРУЕШЬ
И ЧЕТ УСТАЛ O(n^2)
ВСЕ ТАКИ КАК ТО ЛЕНЬ ВОТ БЫ O(n) БЫЛО

KV caching


Without cache
Q
Query Token 1
Query Token 2
Query Token 3
Query Token 4
(4, emb_size)
x
K^T
Key Token 1
Key Token 2
Key Token 3
Key Token 4
(emb_size, 4)
=
QK^T
(4, 4)
x
V
Value Token 1
Value Token 2
Value Token 3
Value Token 4
(4, emb_size)
=
Attention
Token 1
Token 2
Token 3
Token 4
(4, emb_size)
With cache
Q
Query Token 4
(1, emb_size)
x
K^T
Key Token 1
Key Token 2
Key Token 3
Key Token 4
(emb_size, 4)
=
QK^T
(1, 4)
x
V
Value Token 1
Value Token 2
Value Token 3
Value Token 4
(4, emb_size)
=
Attention
Token 4
(1, emb_size)
Values that will be masked
Values that will be taken from cache

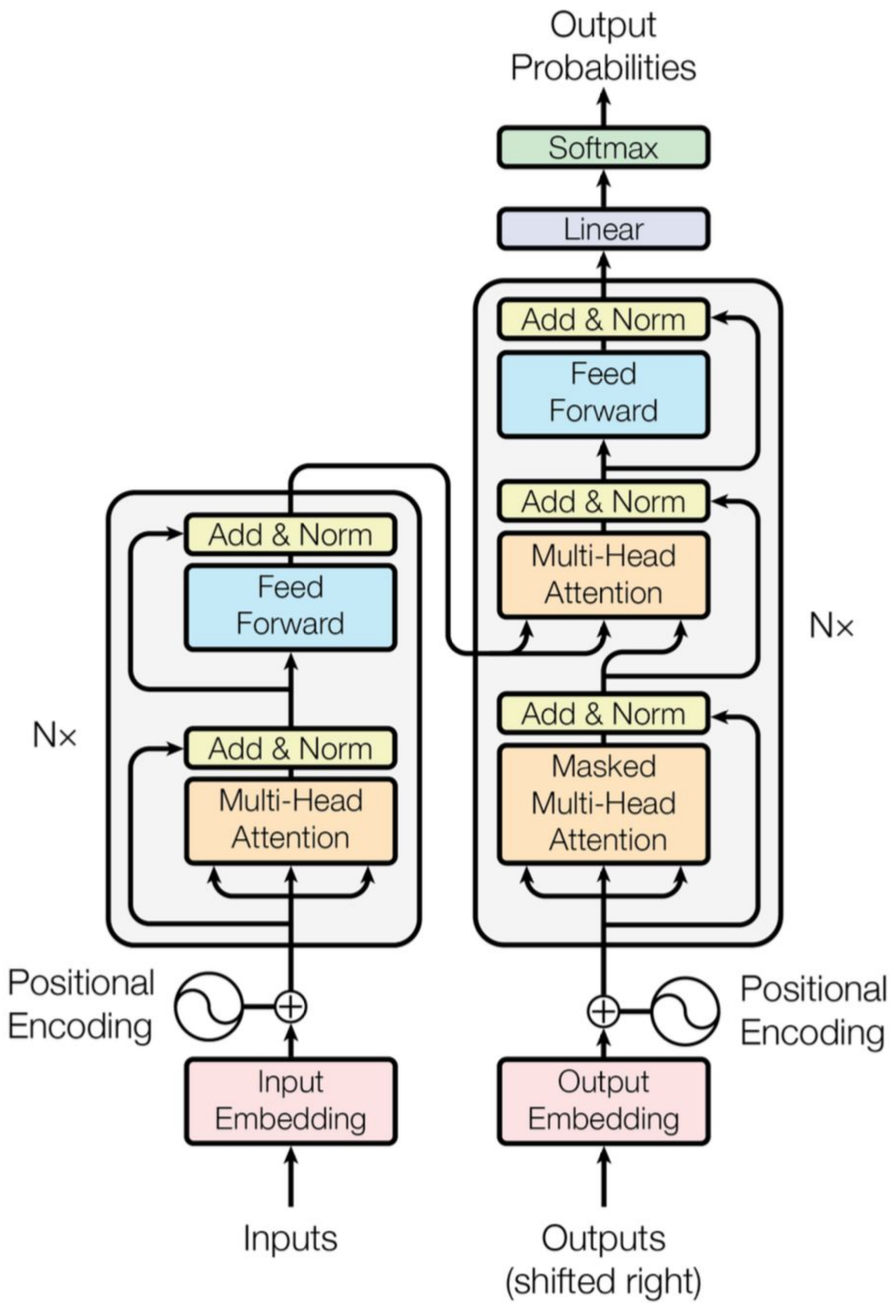
What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

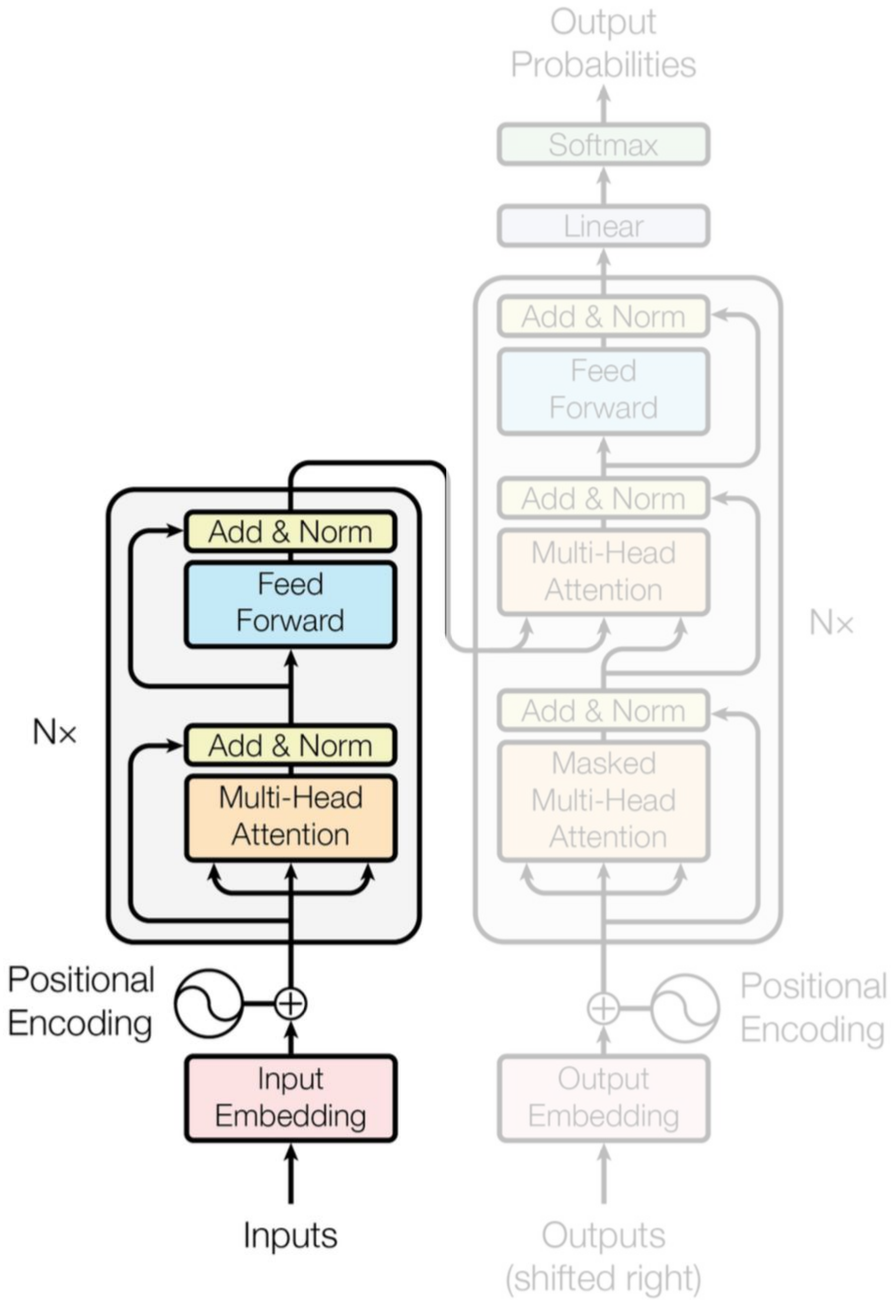
What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
  - Idea
  - Self-Attention
  - Masked Self-Attention
  - Multi-Head Attention
  - KV caching
  - Model architecture
- Subword Segmentation: BPE
-  Analysis and Interpretability

Transformer



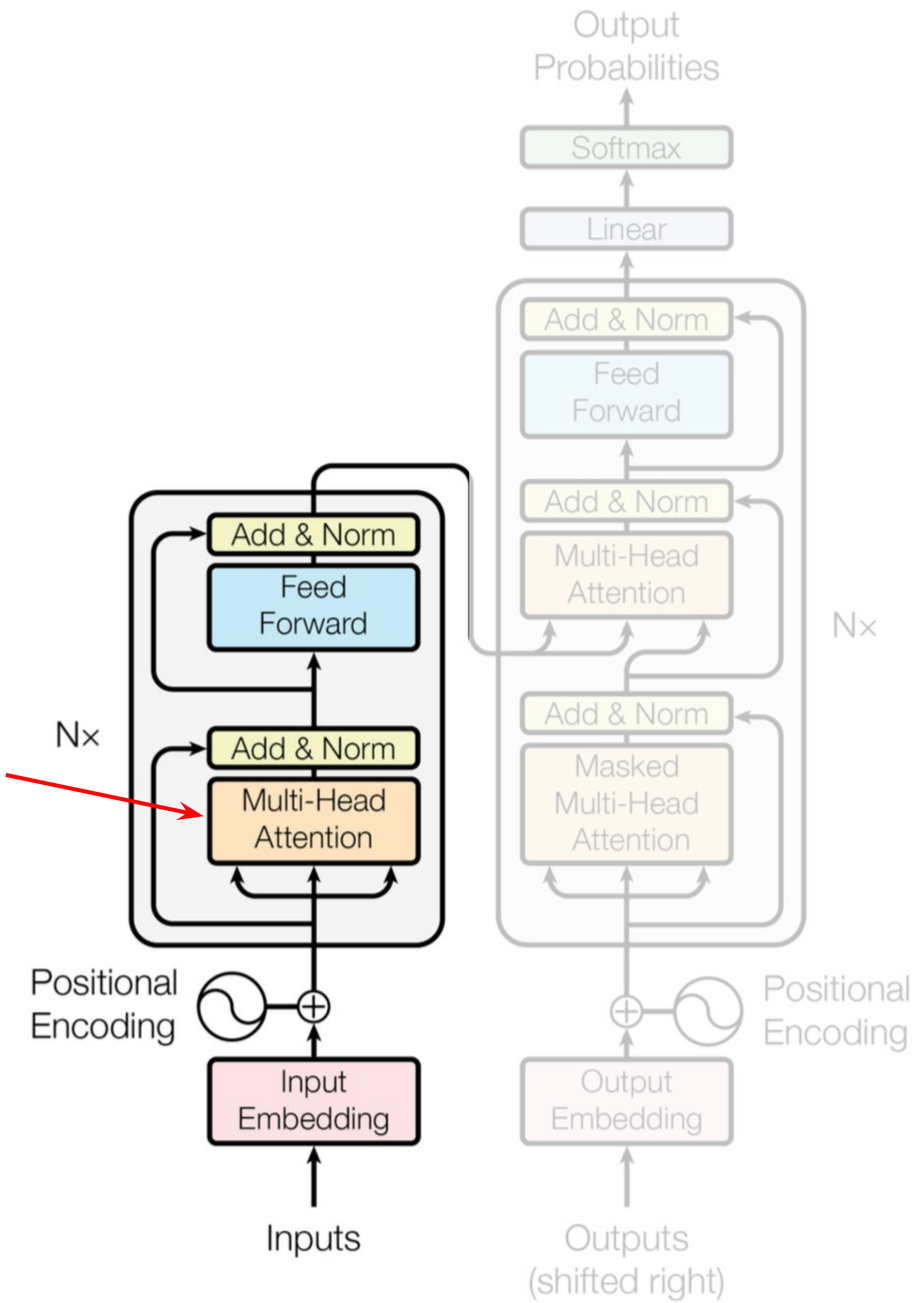
Transformer



Transformer

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states

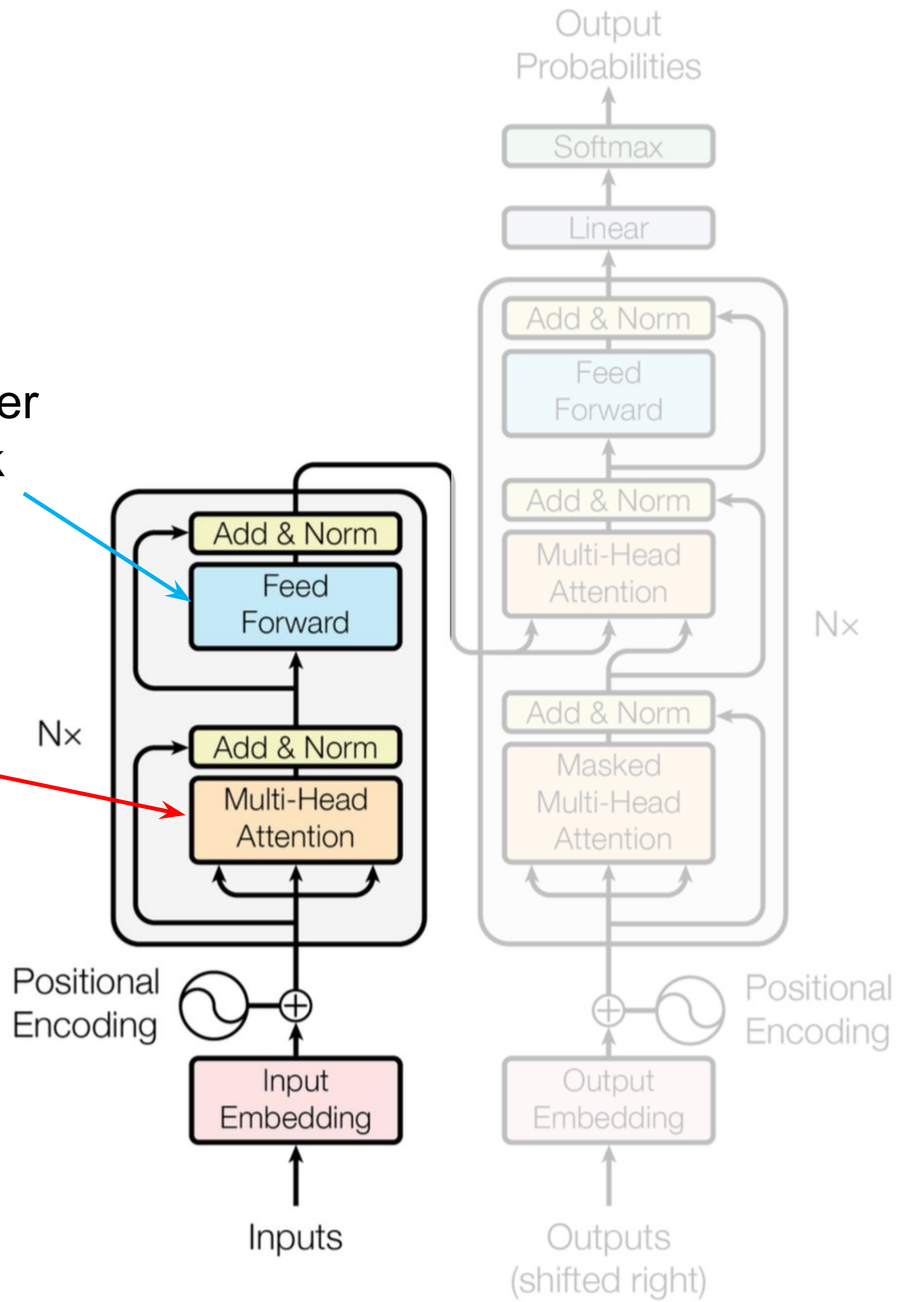


Transformer

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states

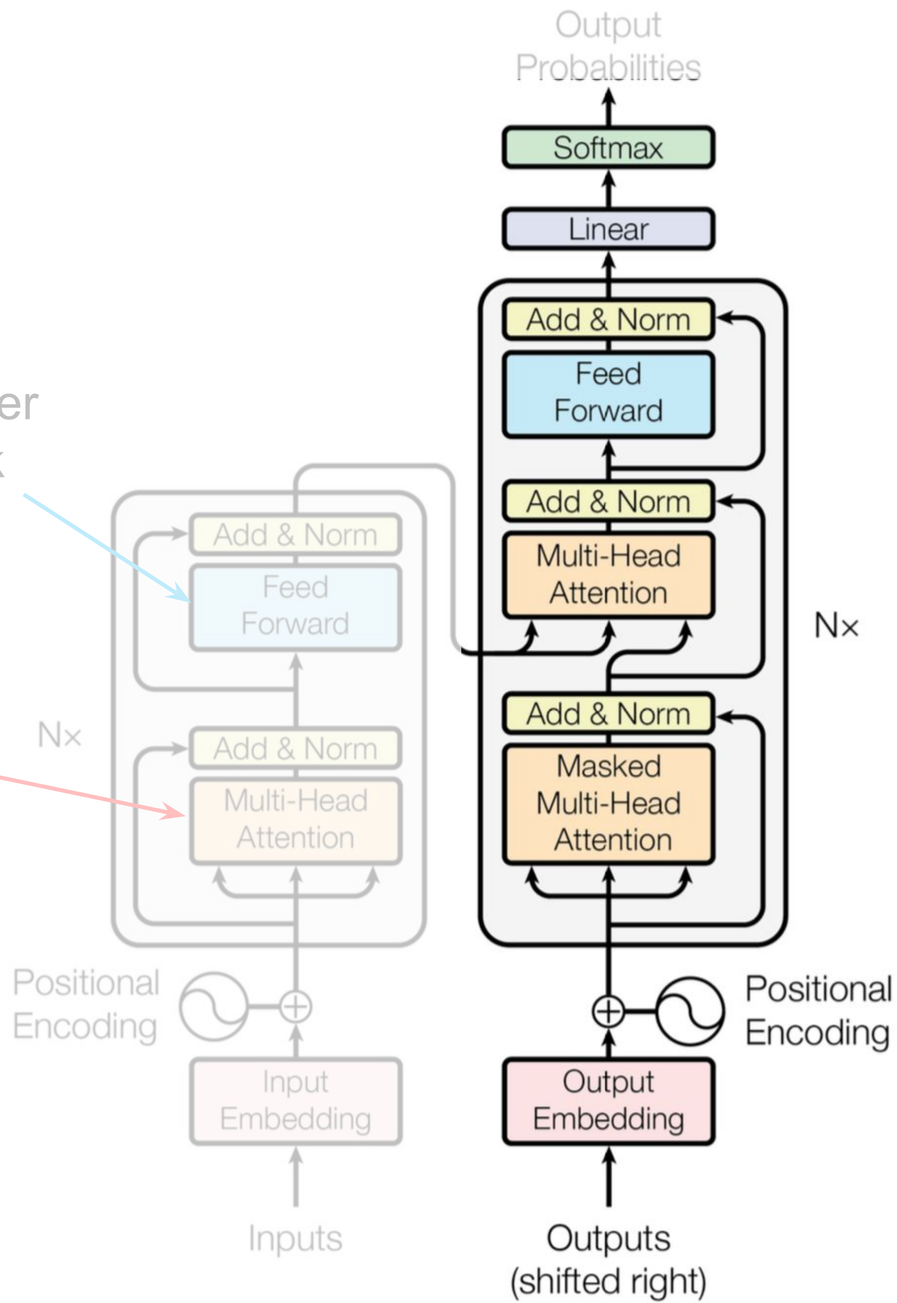


Transformer

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states

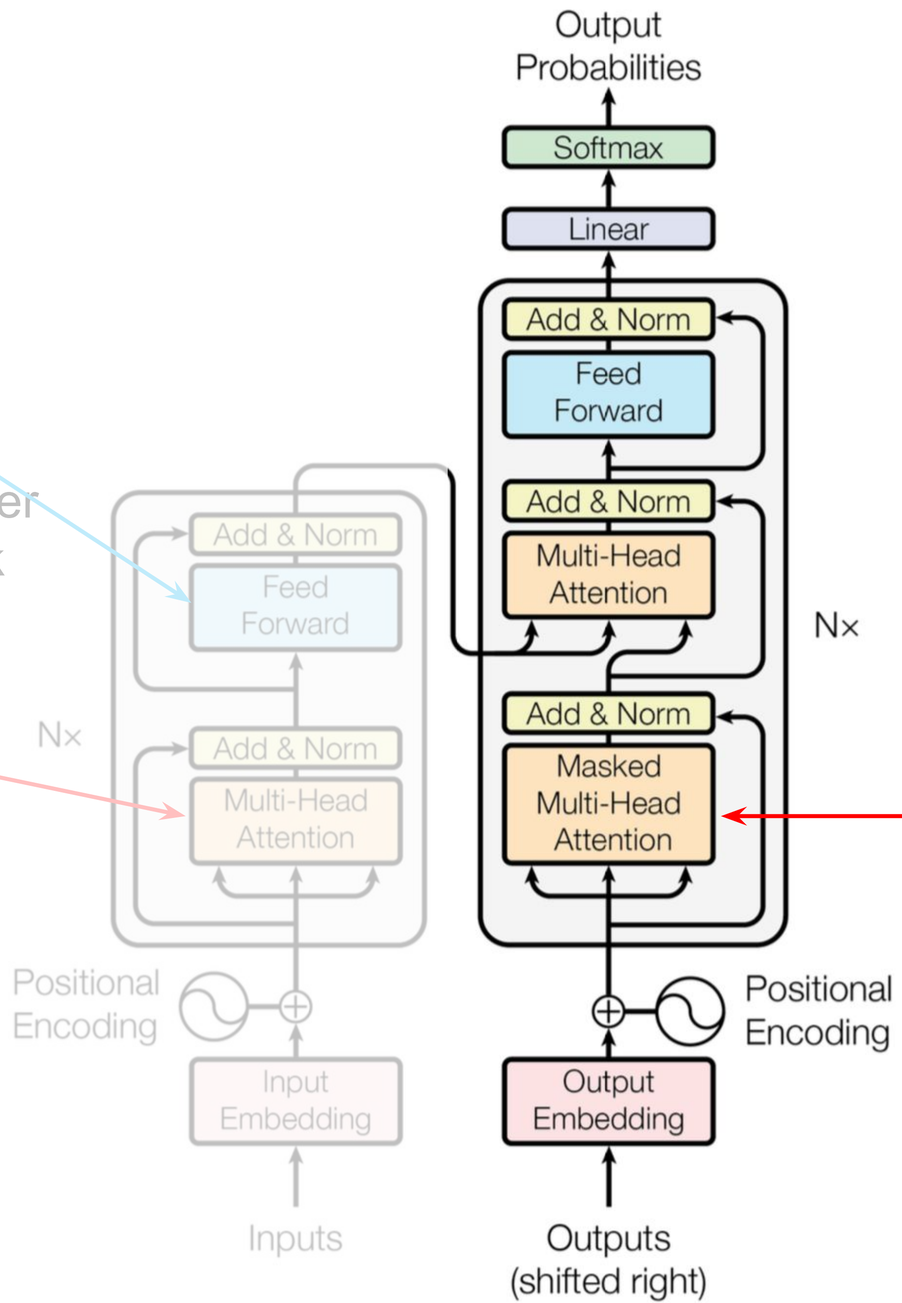


Transformer

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states



Decoder self-attention (masked):
tokens look at the previous tokens

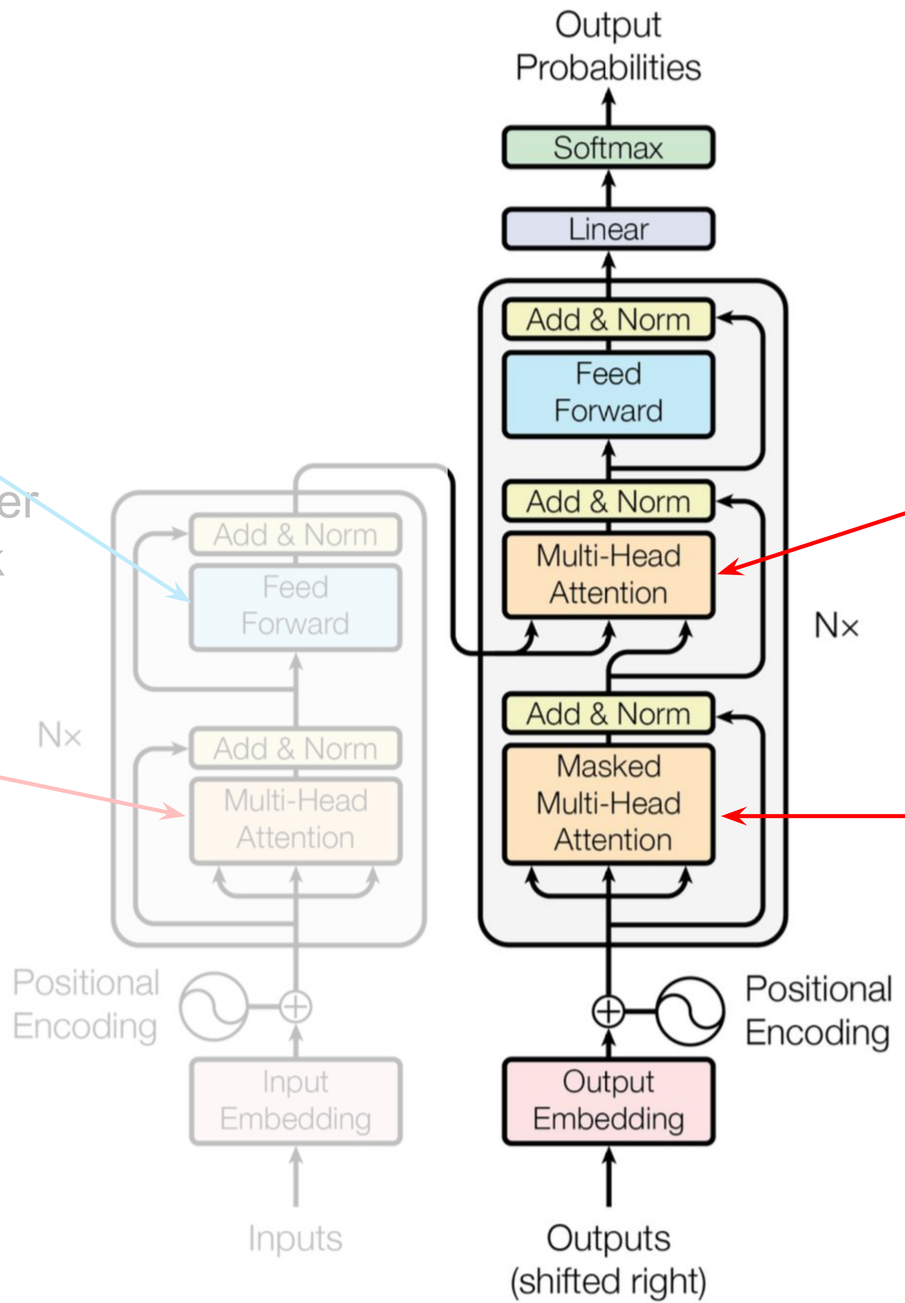
queries, keys, values are computed from decoder states

Transformer

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states



Decoder-encoder attention:
target token looks at the source

queries – from decoder states; keys and values from encoder states

Decoder self-attention (masked):
tokens look at the previous tokens

queries, keys, values are computed from decoder states

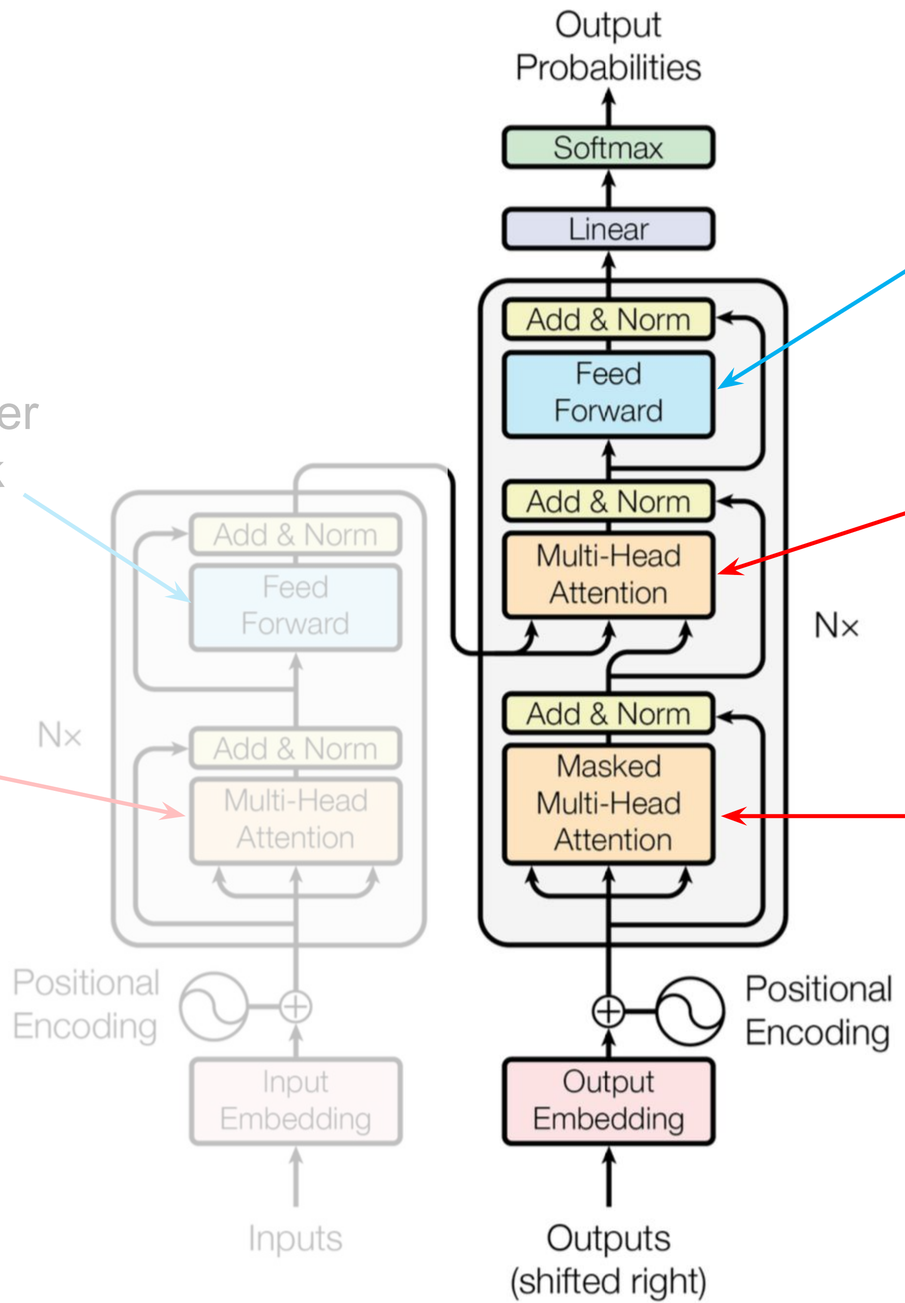
Transformer

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

↑

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states



Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

↑

Decoder-encoder attention:
target token looks at the source

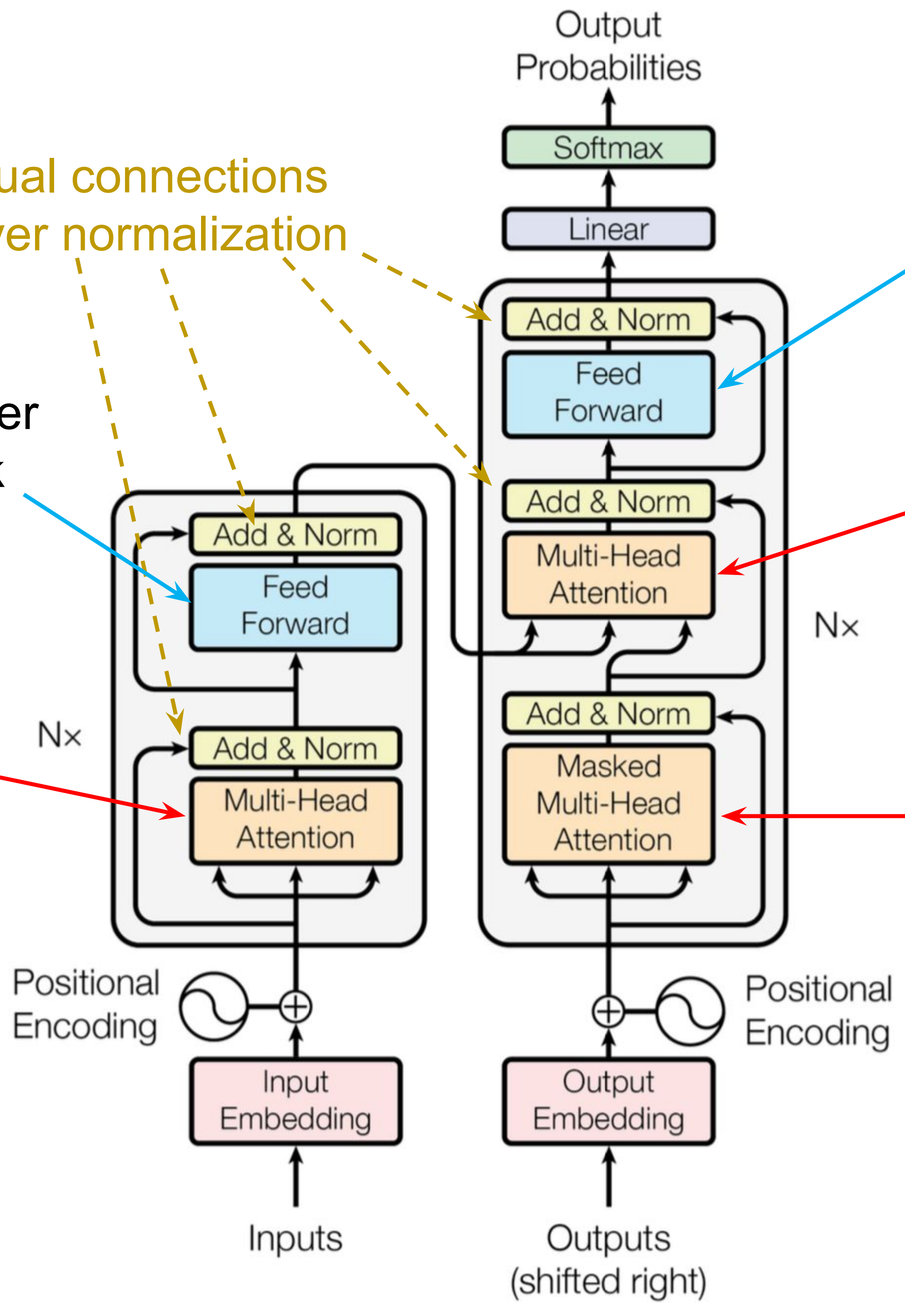
queries – from decoder states; keys and values from encoder states

↑

Decoder self-attention (masked):
tokens look at the previous tokens

queries, keys, values are computed from decoder states

Transformer



Residual connections and layer normalization

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

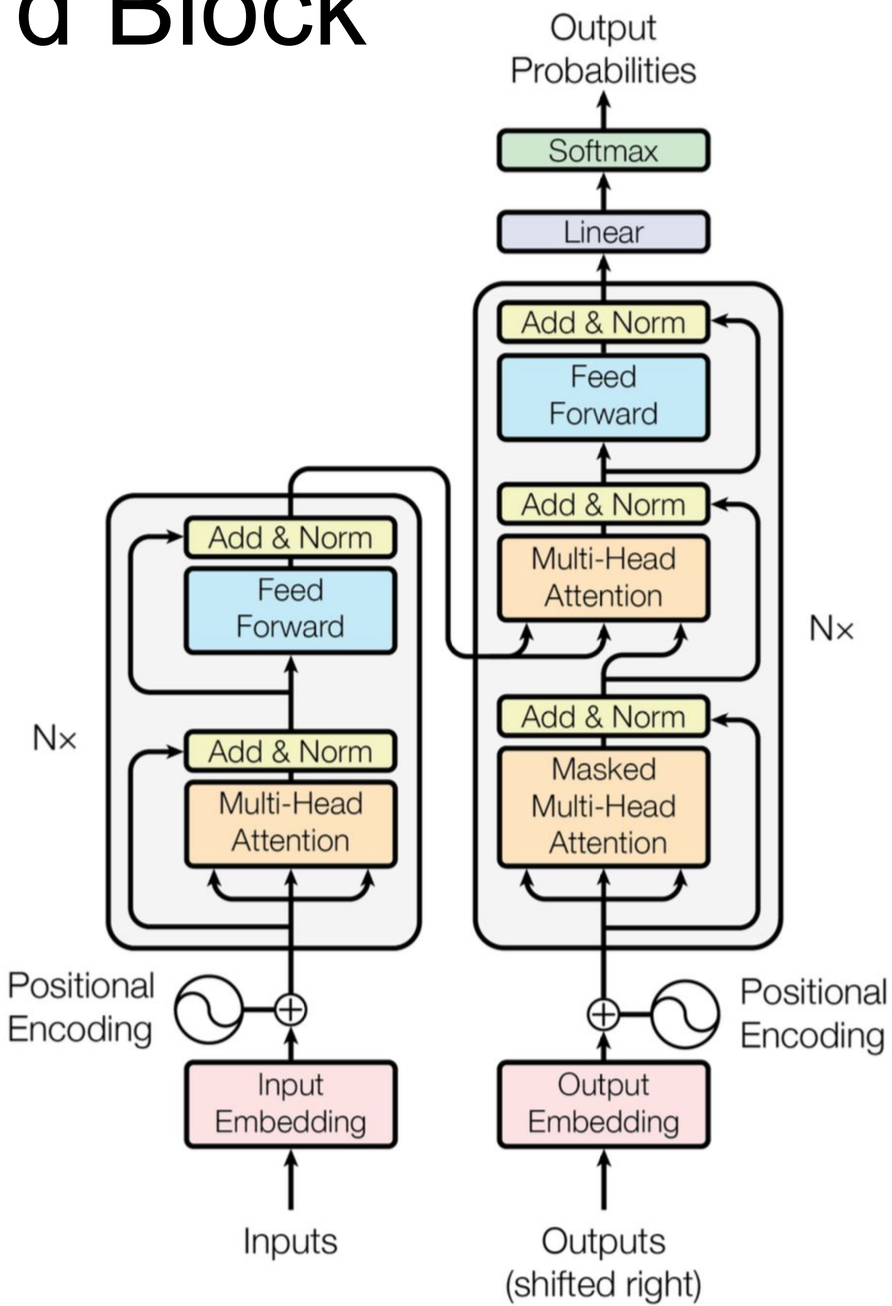
Decoder-encoder attention:
target token looks at the source

queries – from decoder states; keys and values from encoder states

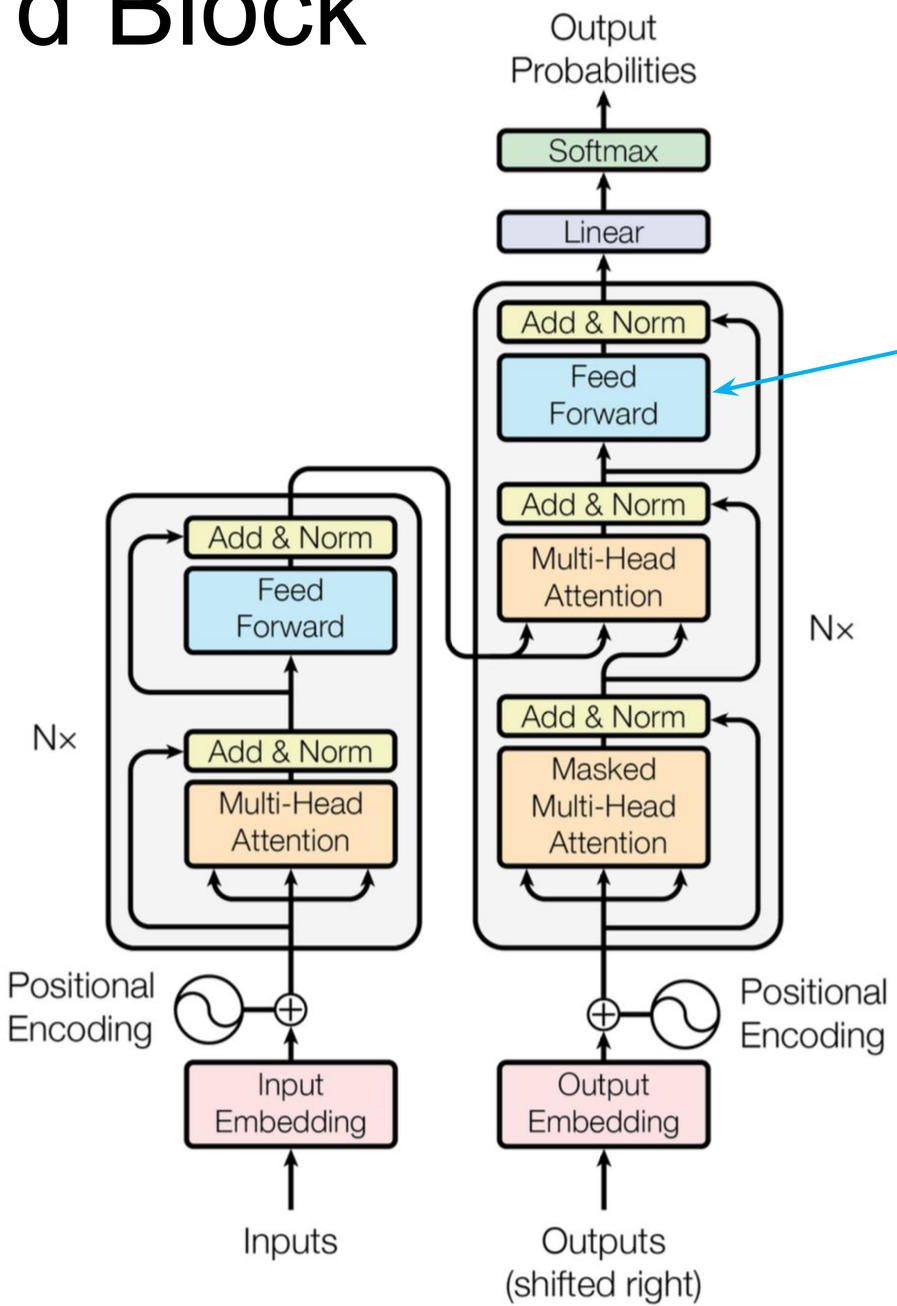
Decoder self-attention (masked):
tokens look at the previous tokens

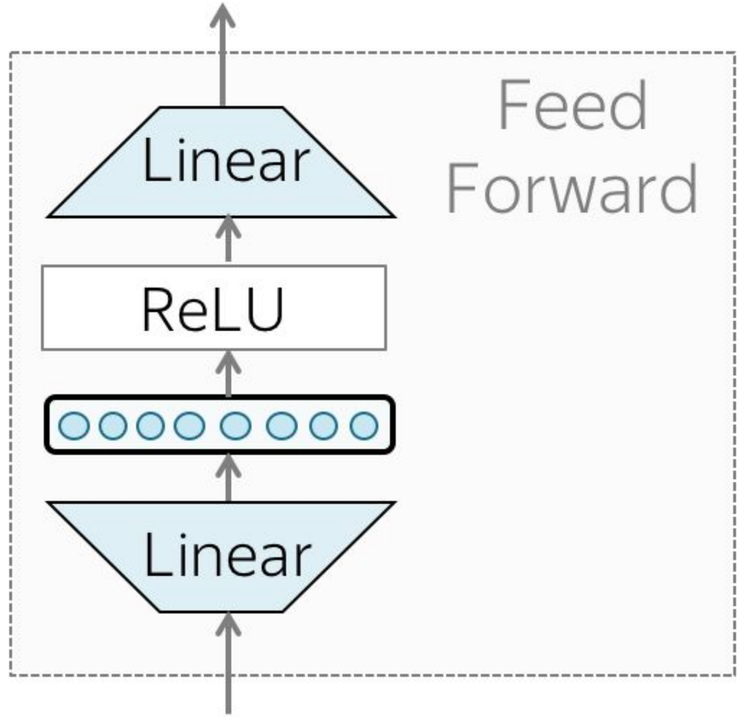
queries, keys, values are computed from decoder states

Feed Forward Block



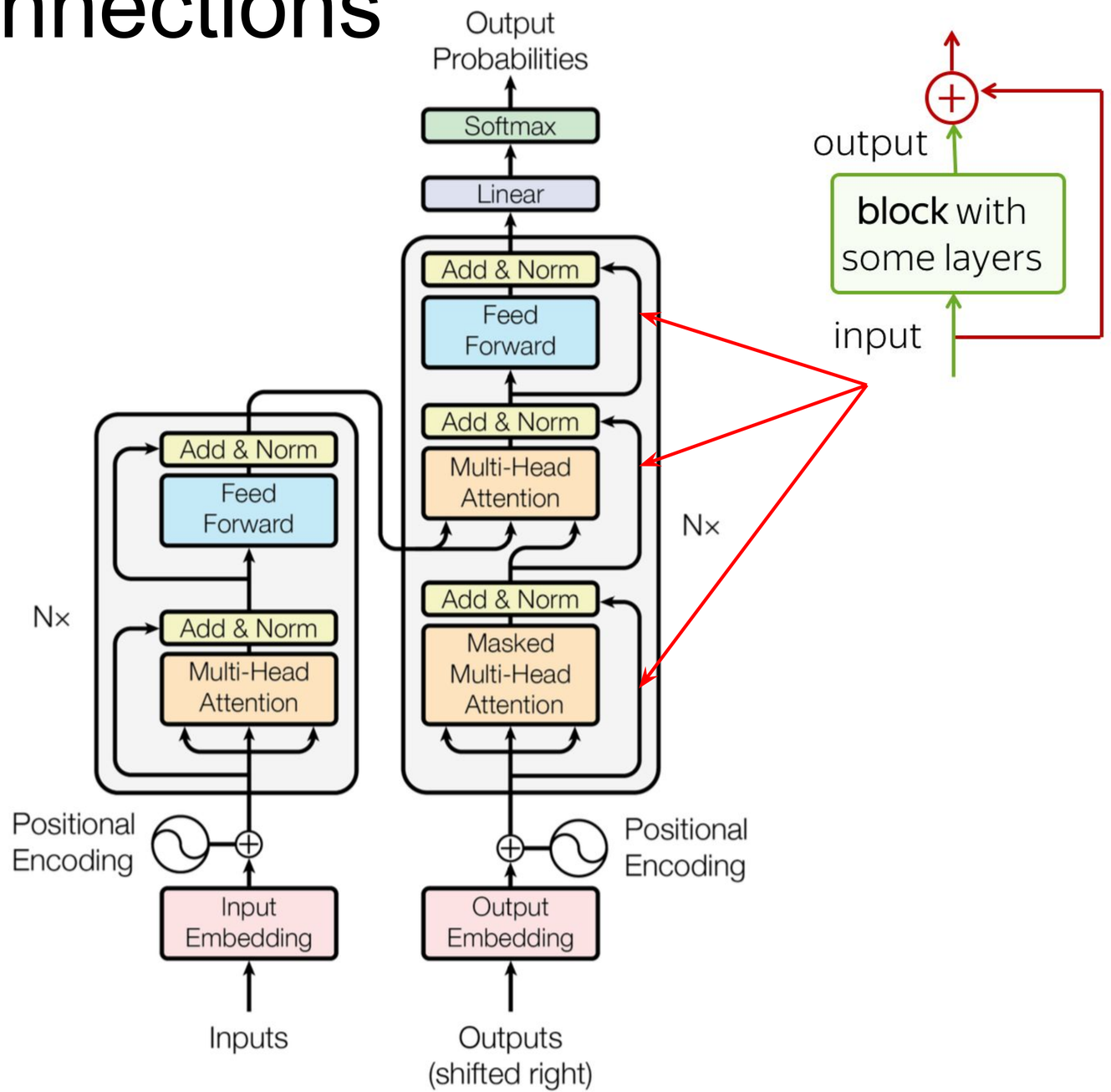
Feed Forward Block



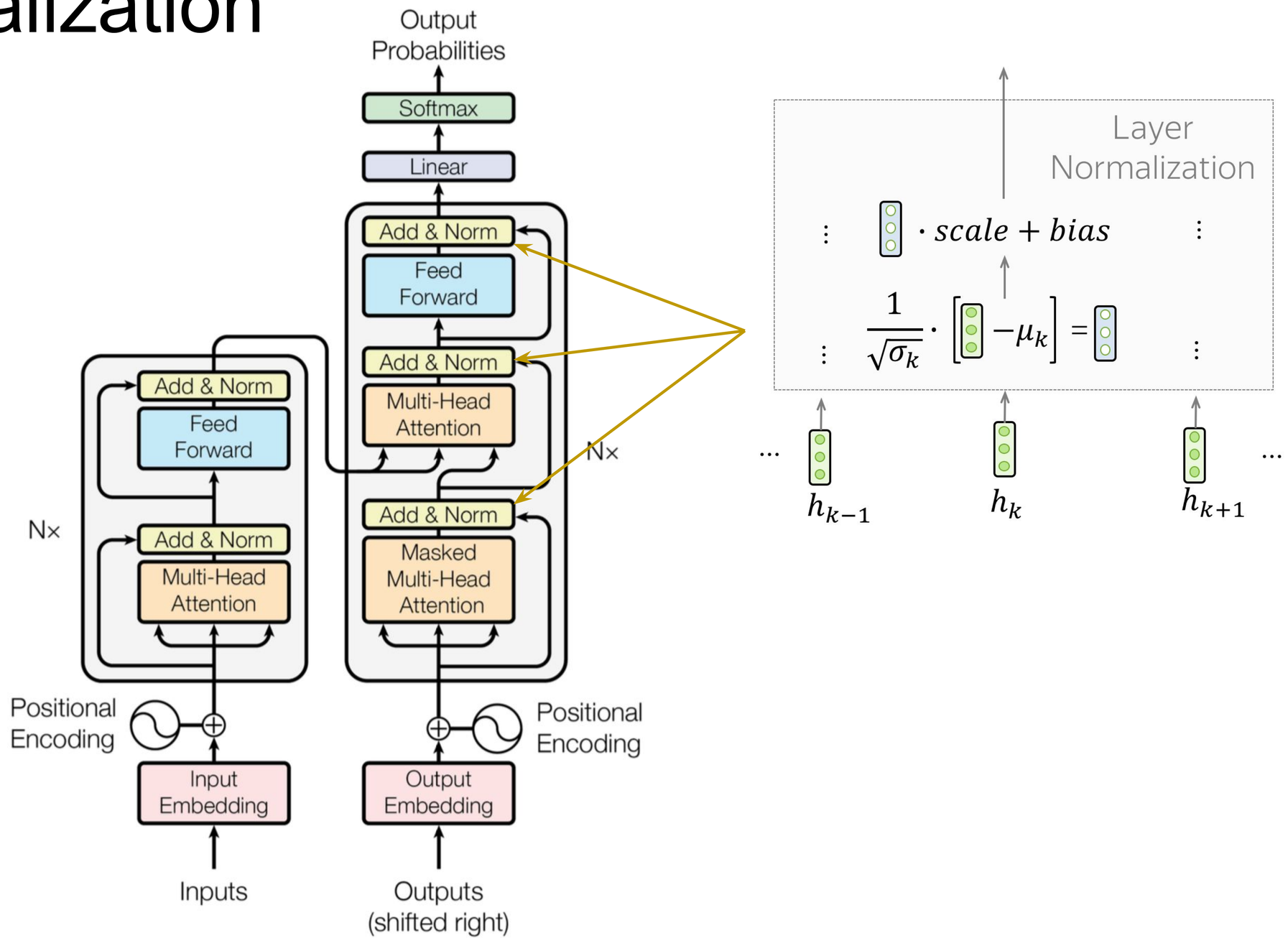


$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

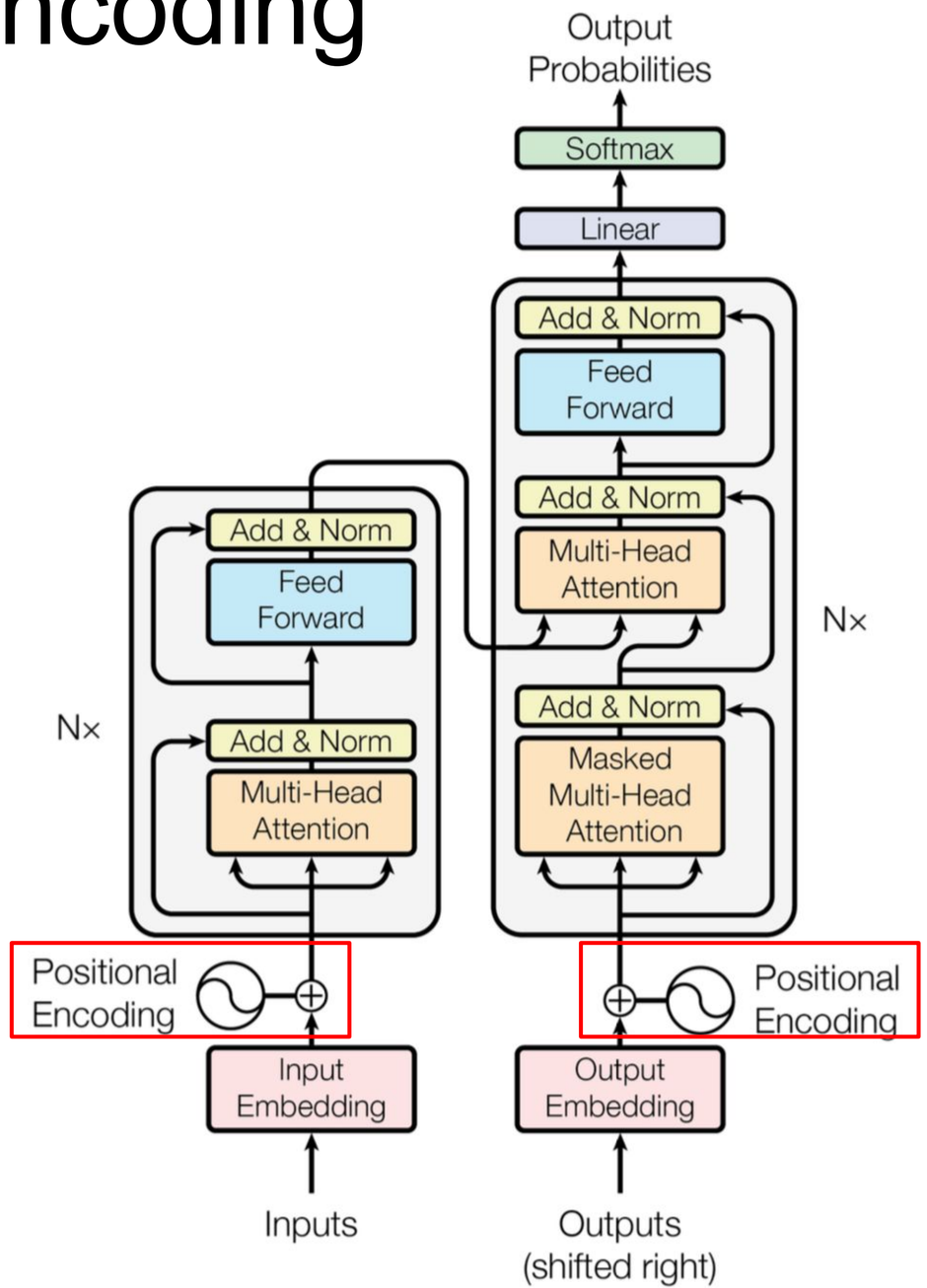
Residual Connections



Layer Normalization



Positional Encoding



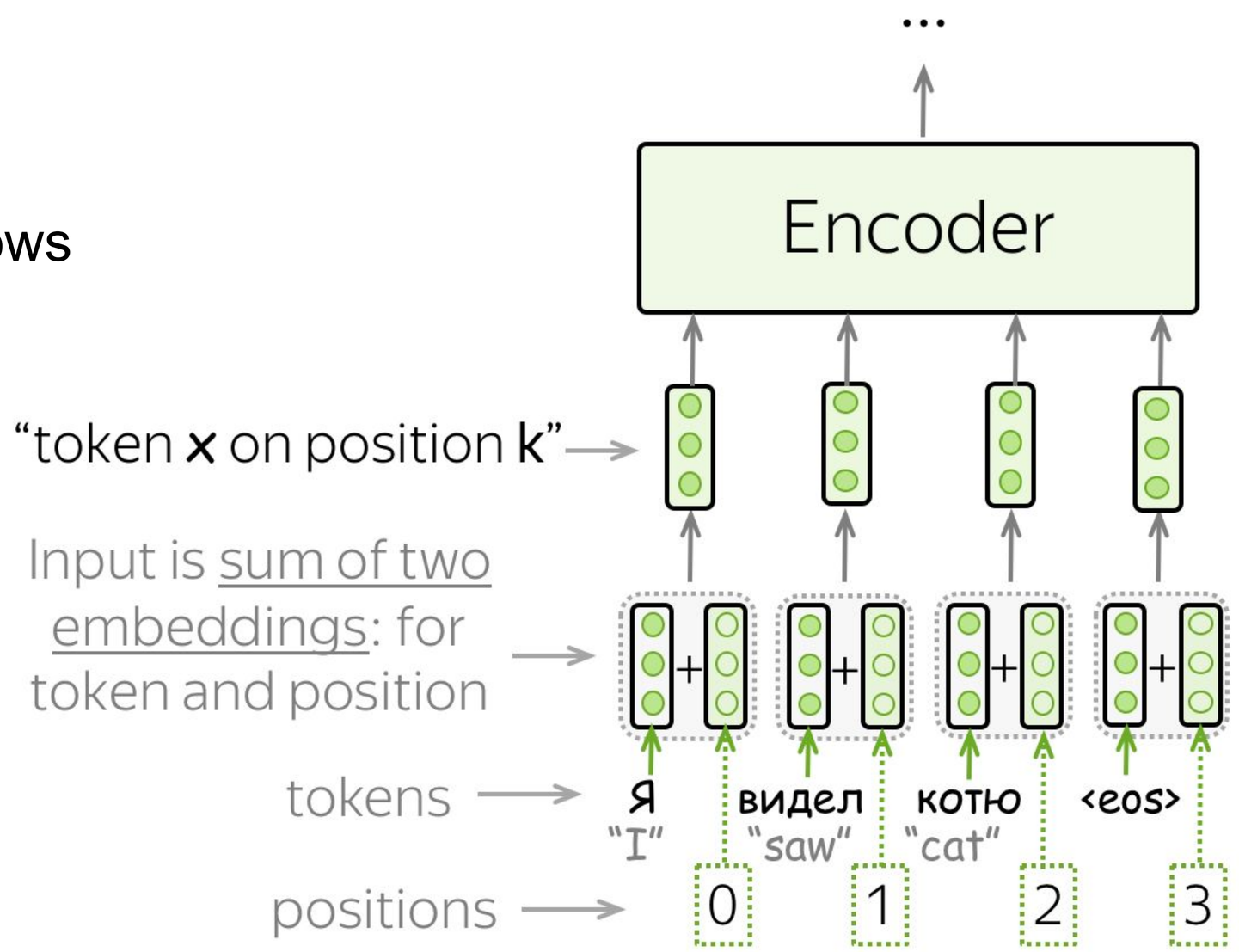
Positional Encoding

Problem:

- without recurrence or convolution, the model knows nothing about position

Solution:

- encode position explicitly and add



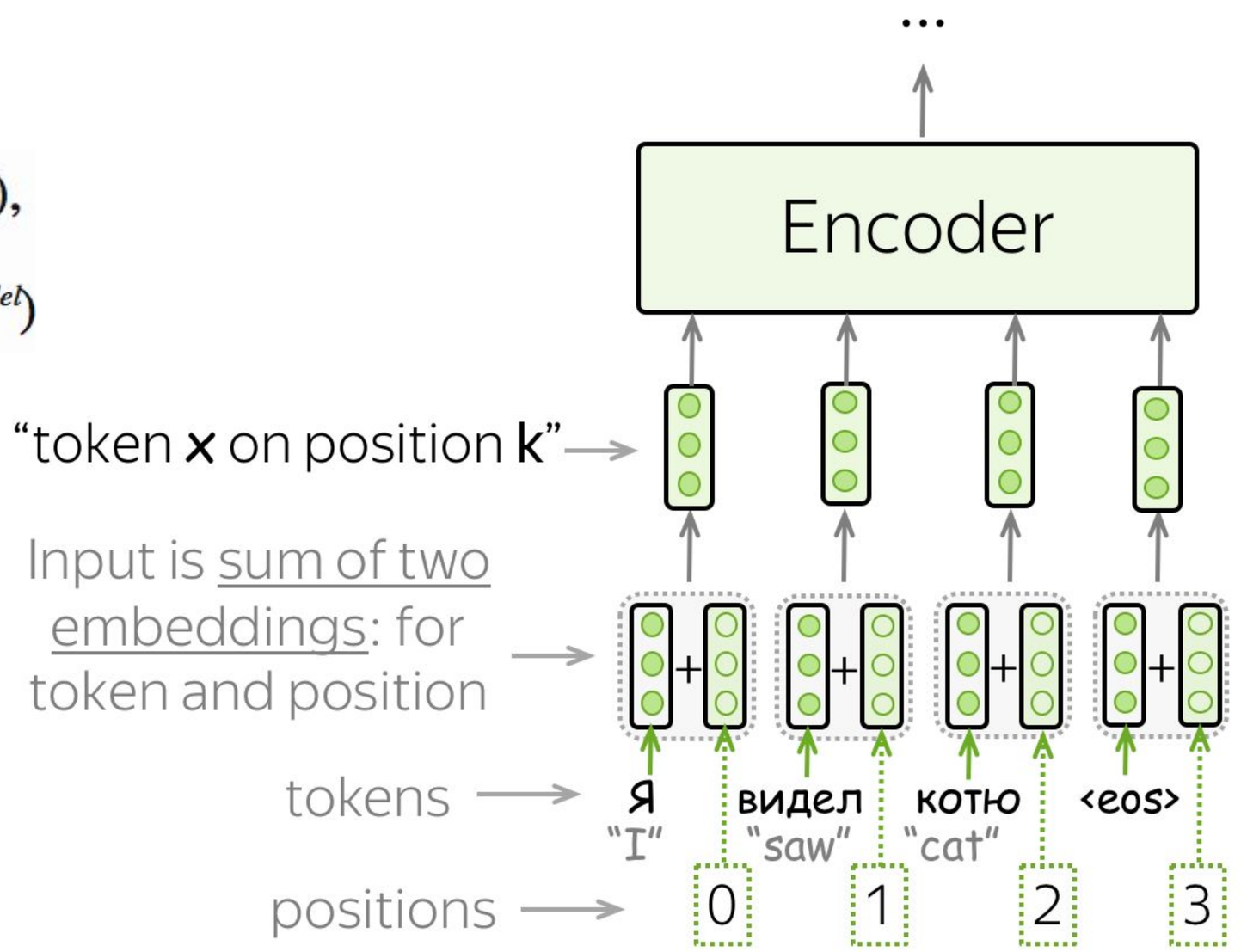
Positional Encoding

Fixed encodings:

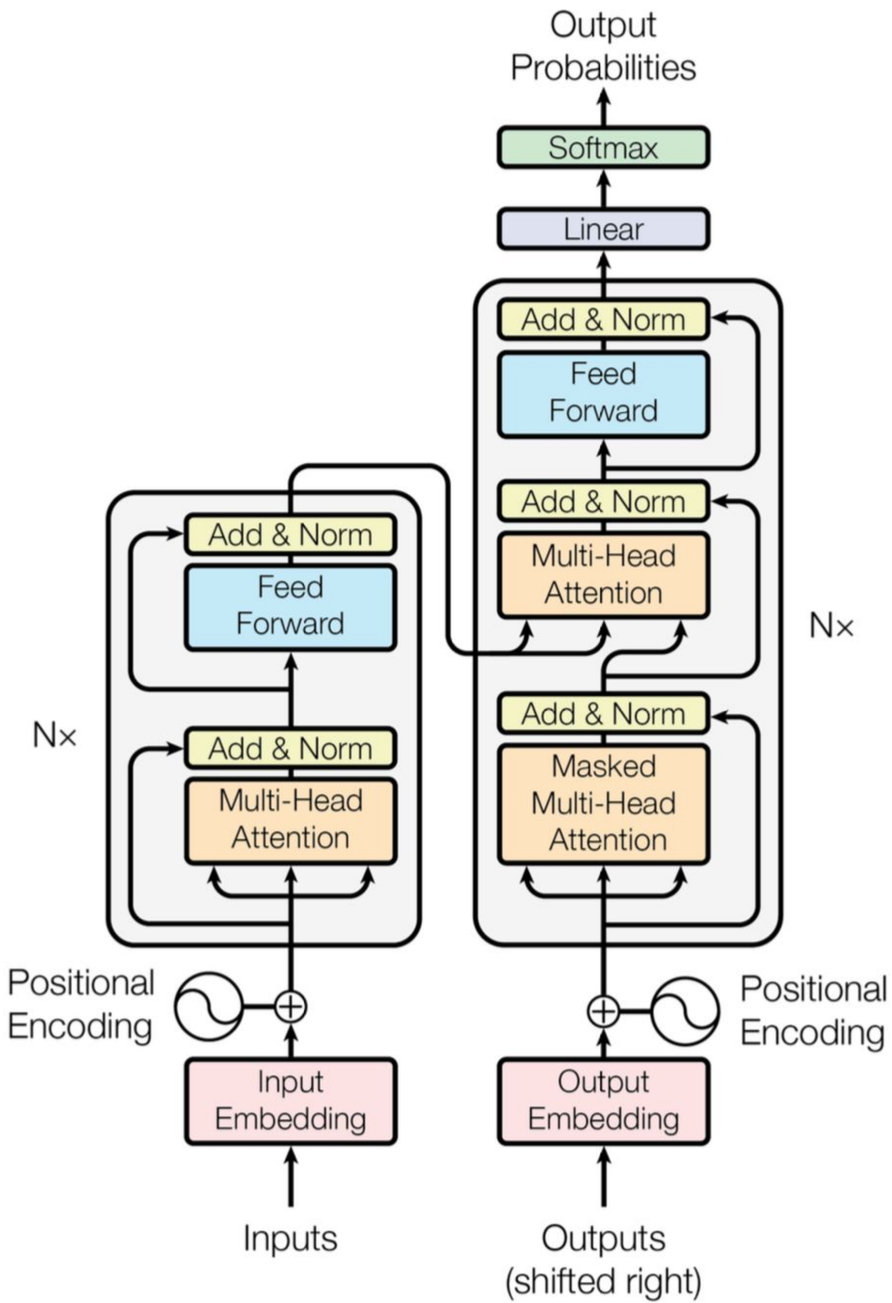
$$\mathrm{PE}_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

$$\mathrm{PE}_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

pos – position, i - dimension



Transformer



Complexity of inference

n - length of input sequence, m - length of output sequence
d - size of embedding

RNN: $O(nd^2 + md^2)$

Transformer without caching: $O(n^2d + m^3d)$

Transformer with caching: $O(n^2d + m^2d)$

What is going to happen:

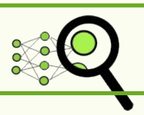
- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE → seminar
-  Analysis and Interpretability

Here could be a joke about attention

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

Analysis Methods

Model-specific:

- Looking at model components
- …

Model-agnostic:

Analysis Methods

Model-specific:

- Looking at model components
- …

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Model-agnostic:

Analysis Methods

Model-specific:

- Looking at model components
- …

Model-agnostic:

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Analysis Methods

Model-specific:

- Looking at model components
- …

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Analysis Methods

Model-specific:

- Looking at model components
- …

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

- Probing: What do representations capture?

Analysis Methods

Model-specific:

- Looking at model components
- …

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

- Probing: What do representations capture?

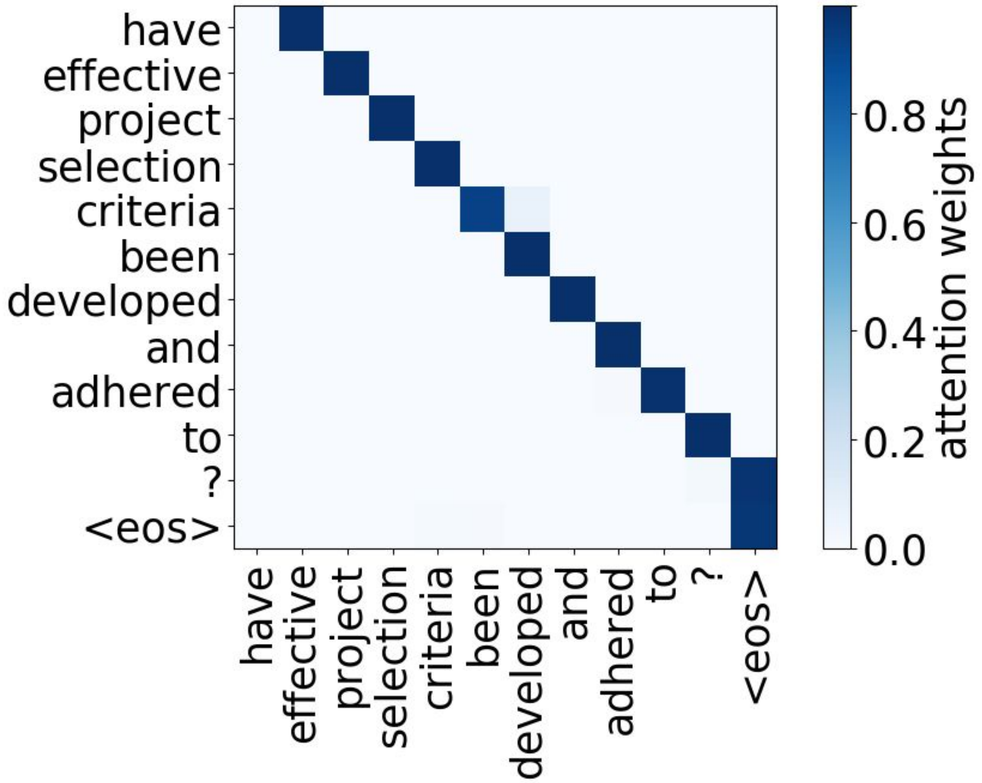
Multi-Head Attention: What are the Heads Doing?

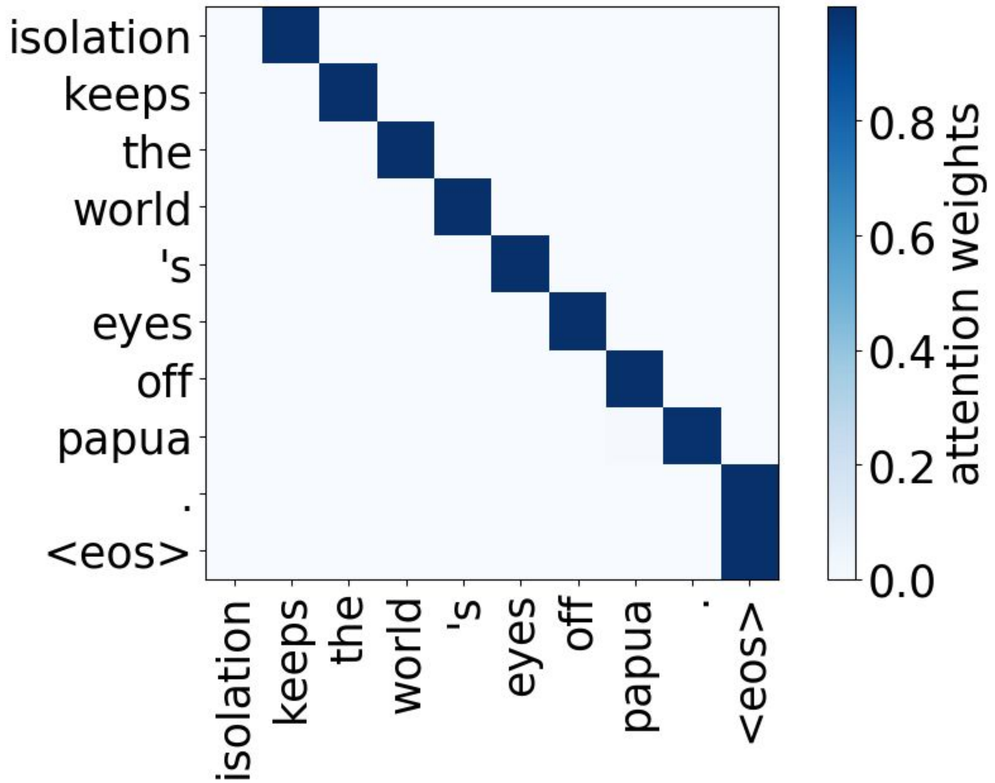
Heads which on average contribute more to generated translations (“important heads”) play interpretable roles:

- Positional
- Syntactic
- Attention to rare tokens

Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

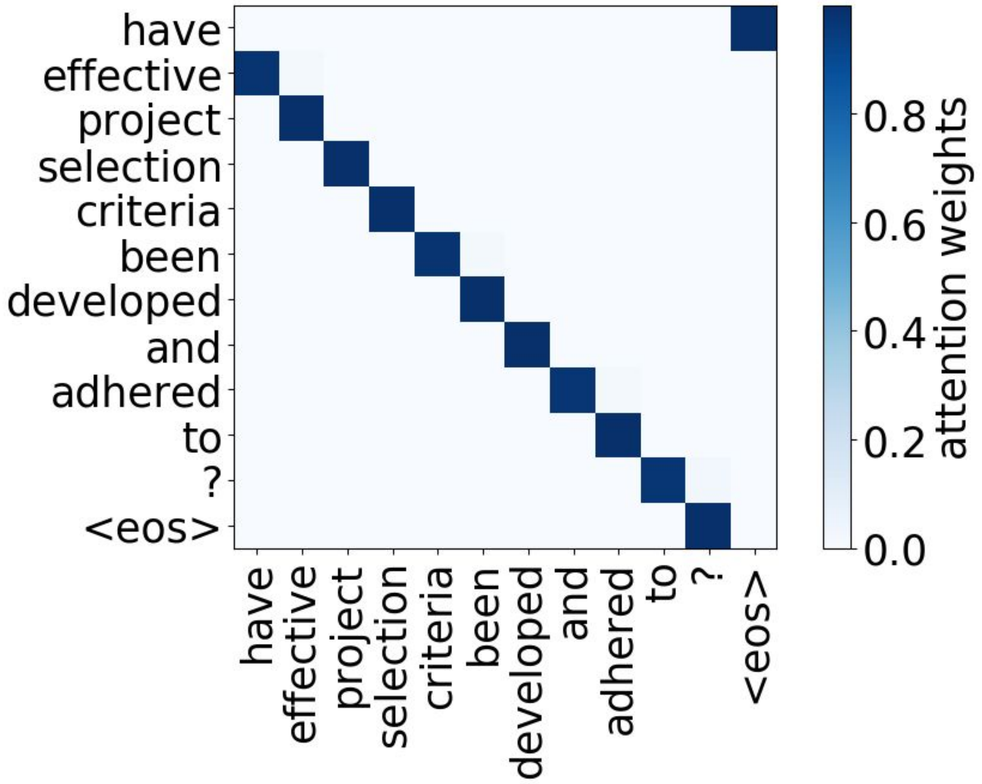
Positional Heads: Attention to Neighbors

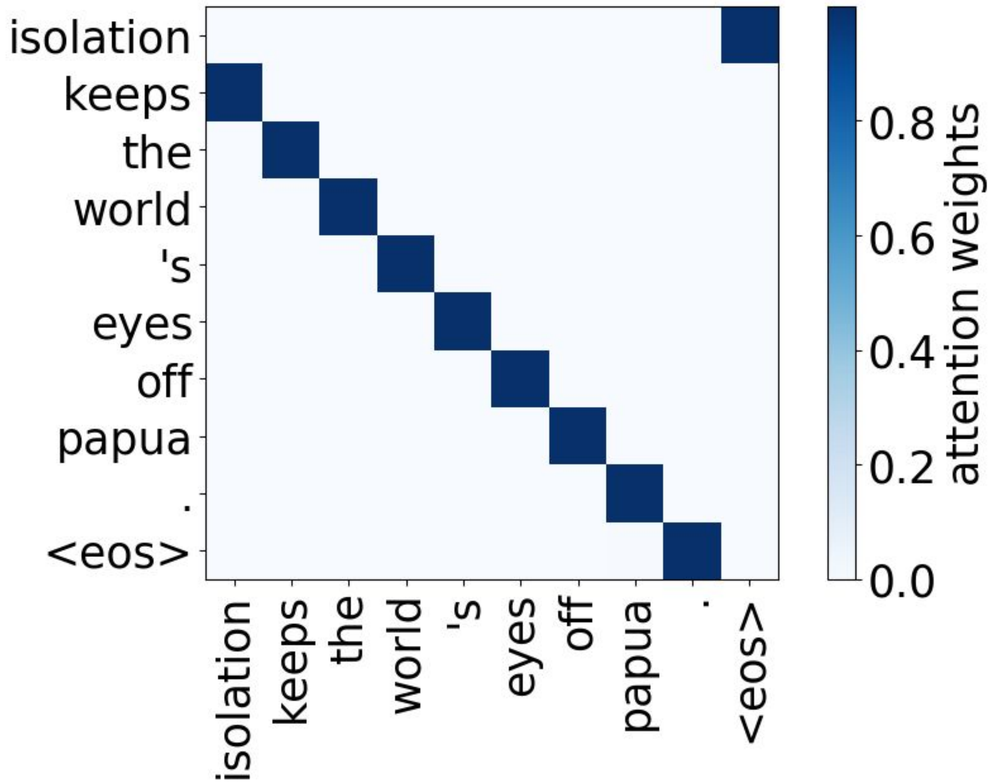




Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Positional Heads: Attention to Neighbors



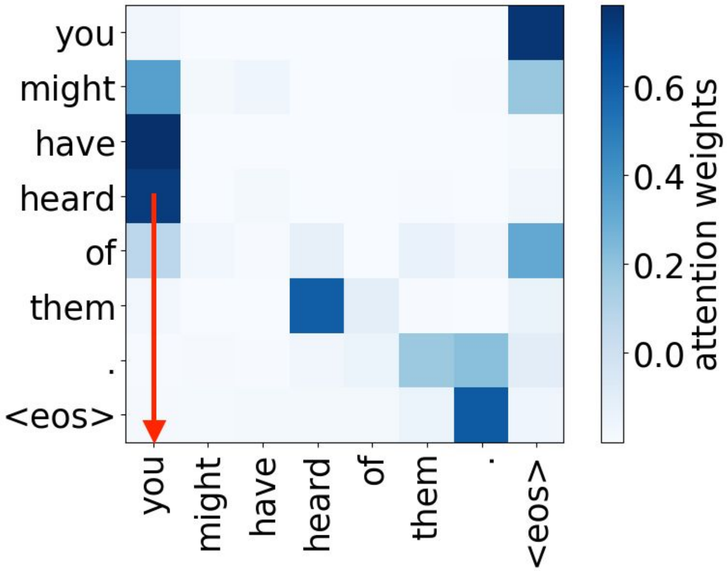


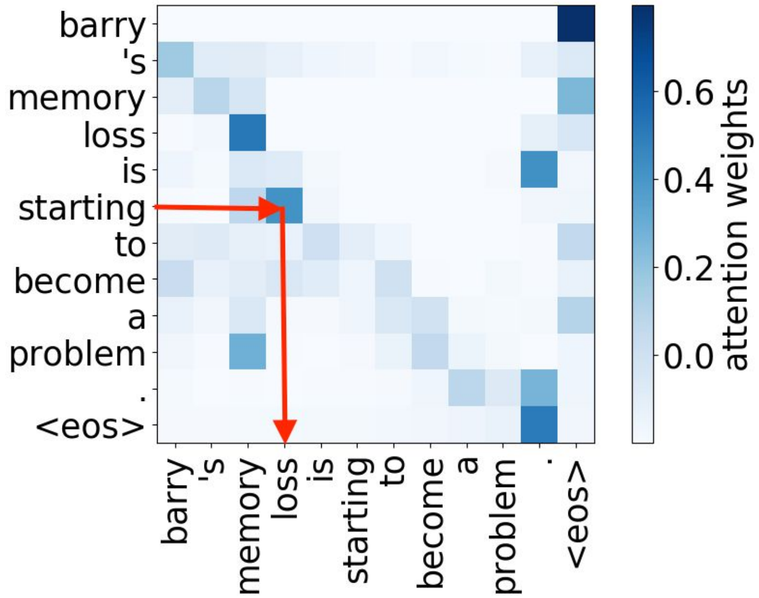
Paper: Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be

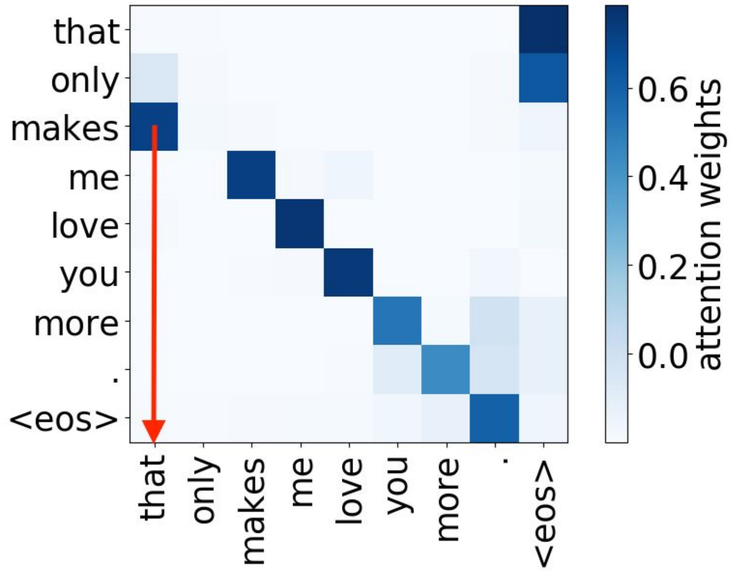
Syntactic Heads: Track Dependencies

- verb->subject





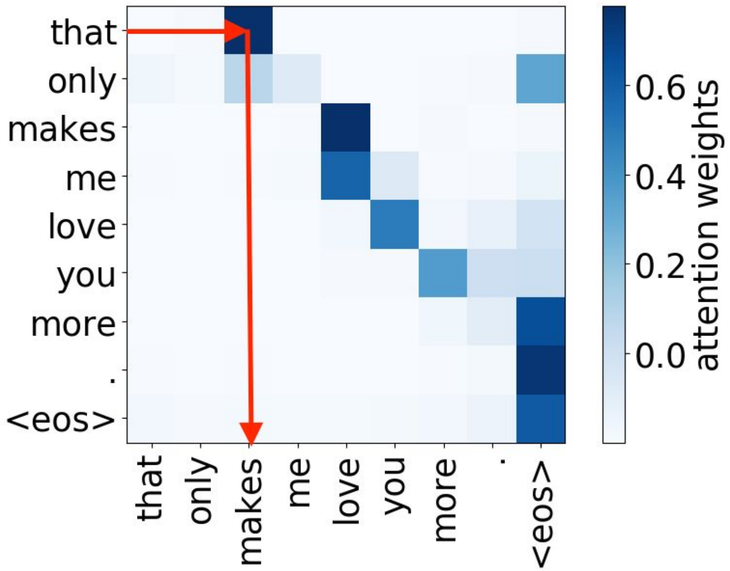


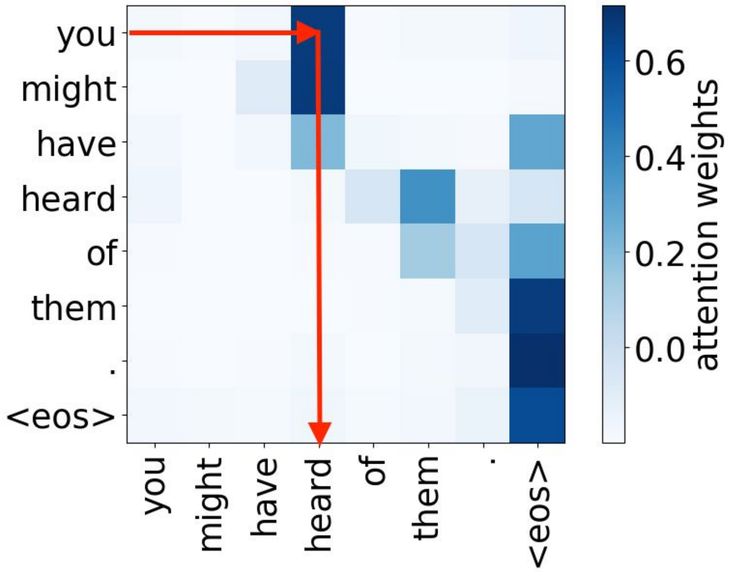


Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Syntactic Heads: Track Dependencies

- subject->verb



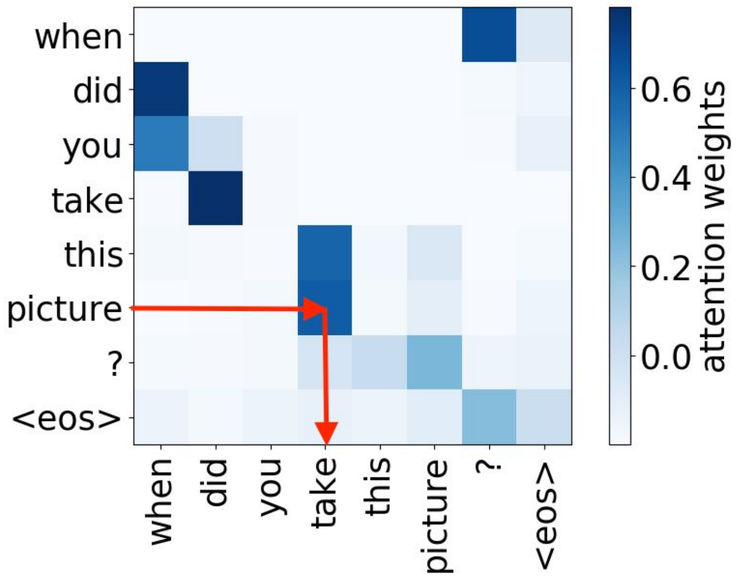


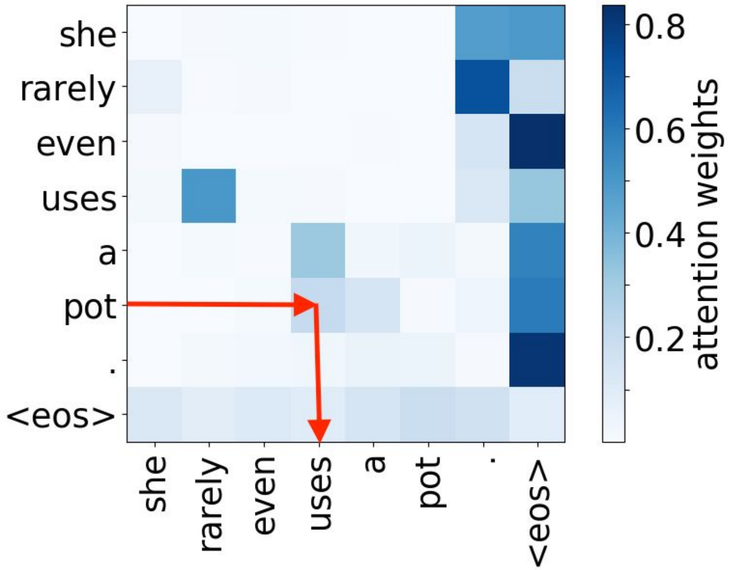


Paper: Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Syntactic Heads: Track Dependencies

- object->verb

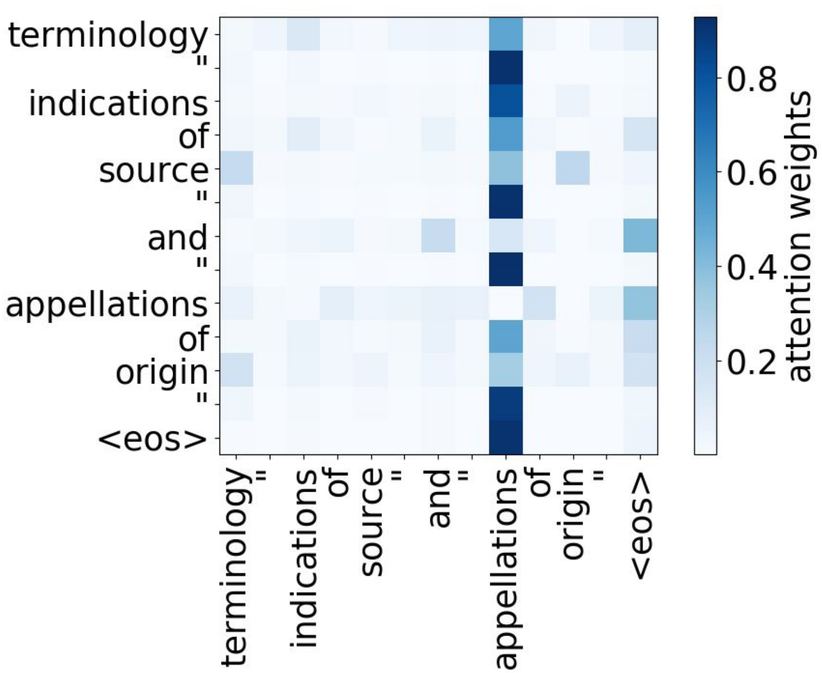


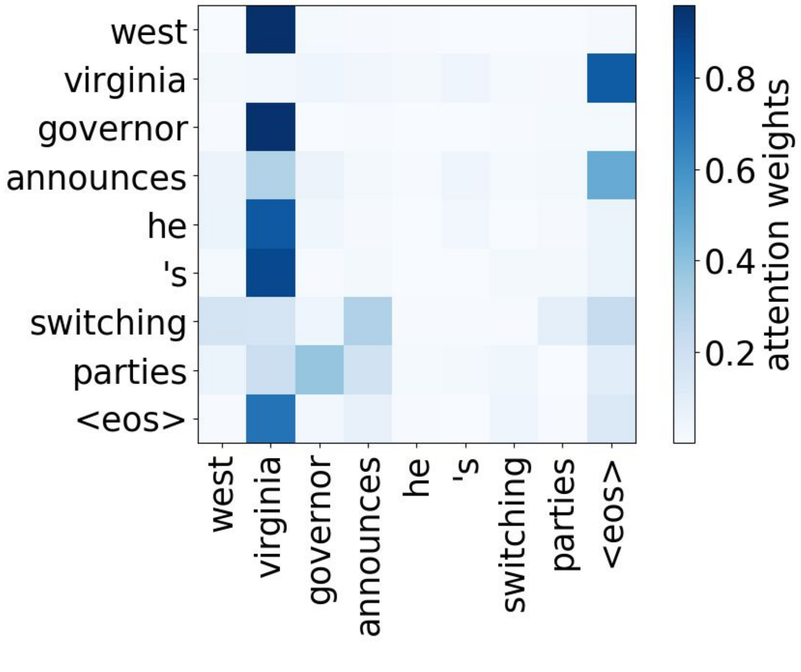


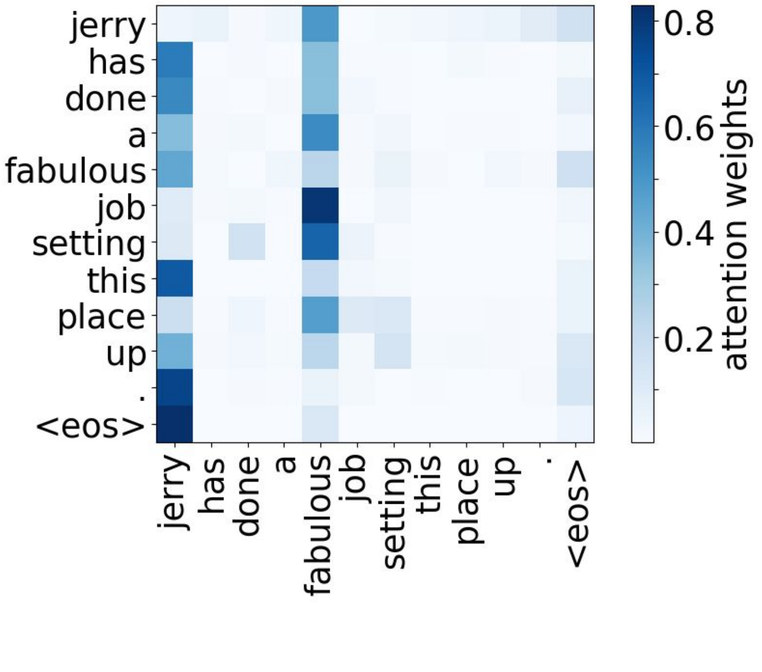


Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Attention to Rare Tokens







Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Analysis Methods

Model-specific:

- Looking at model components
- …

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

- Probing: What do representations capture?

Analysis Methods

Model-specific:

- Looking at model components
- …

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

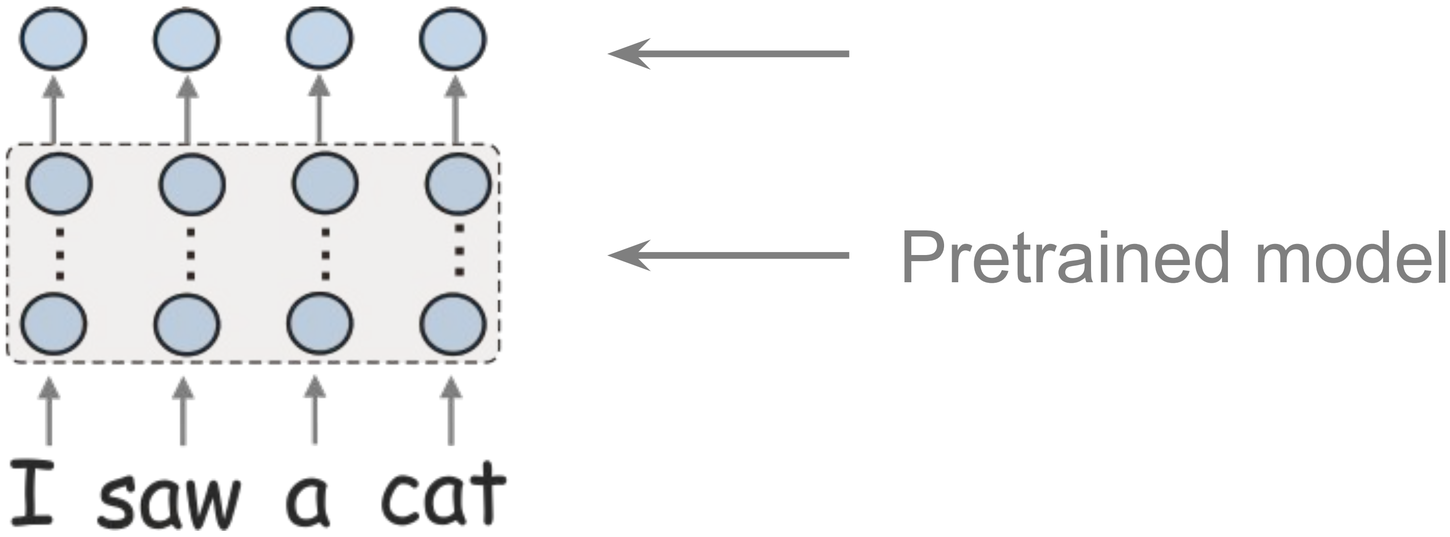
Today:

- Probing: What do representations capture?

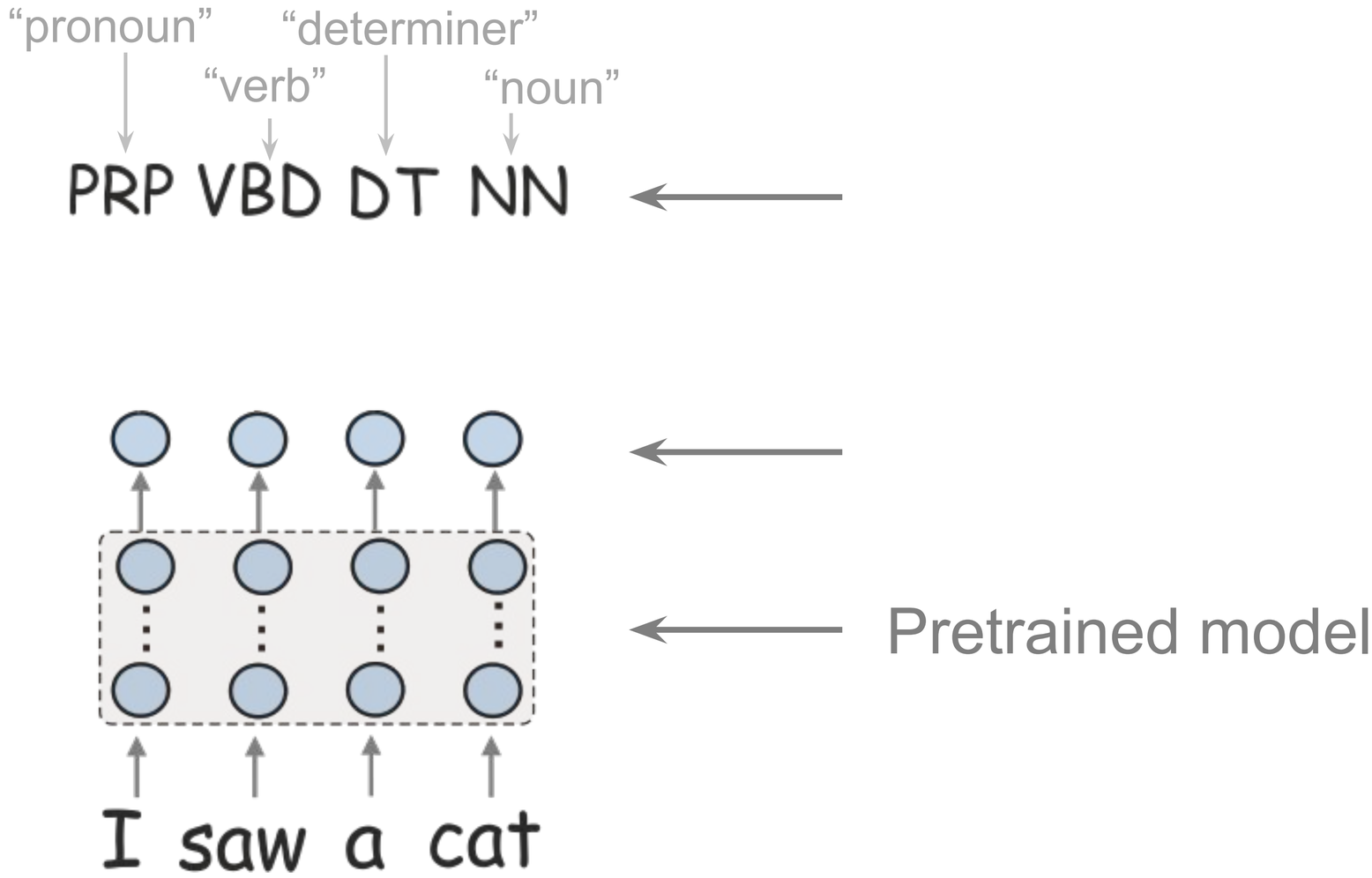
How to understand if a model captures a linguistic property?



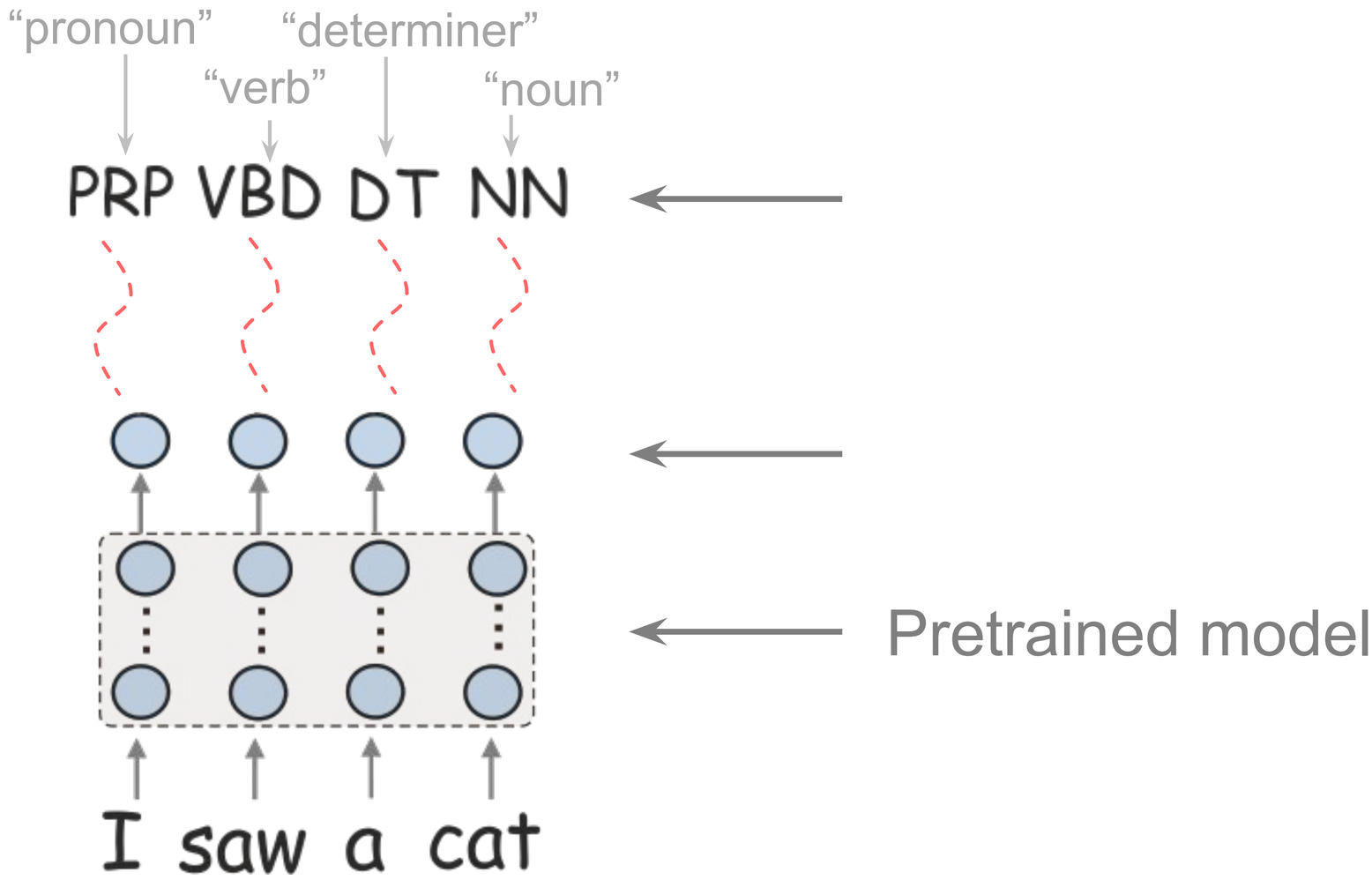
How to understand if a model captures a linguistic property?



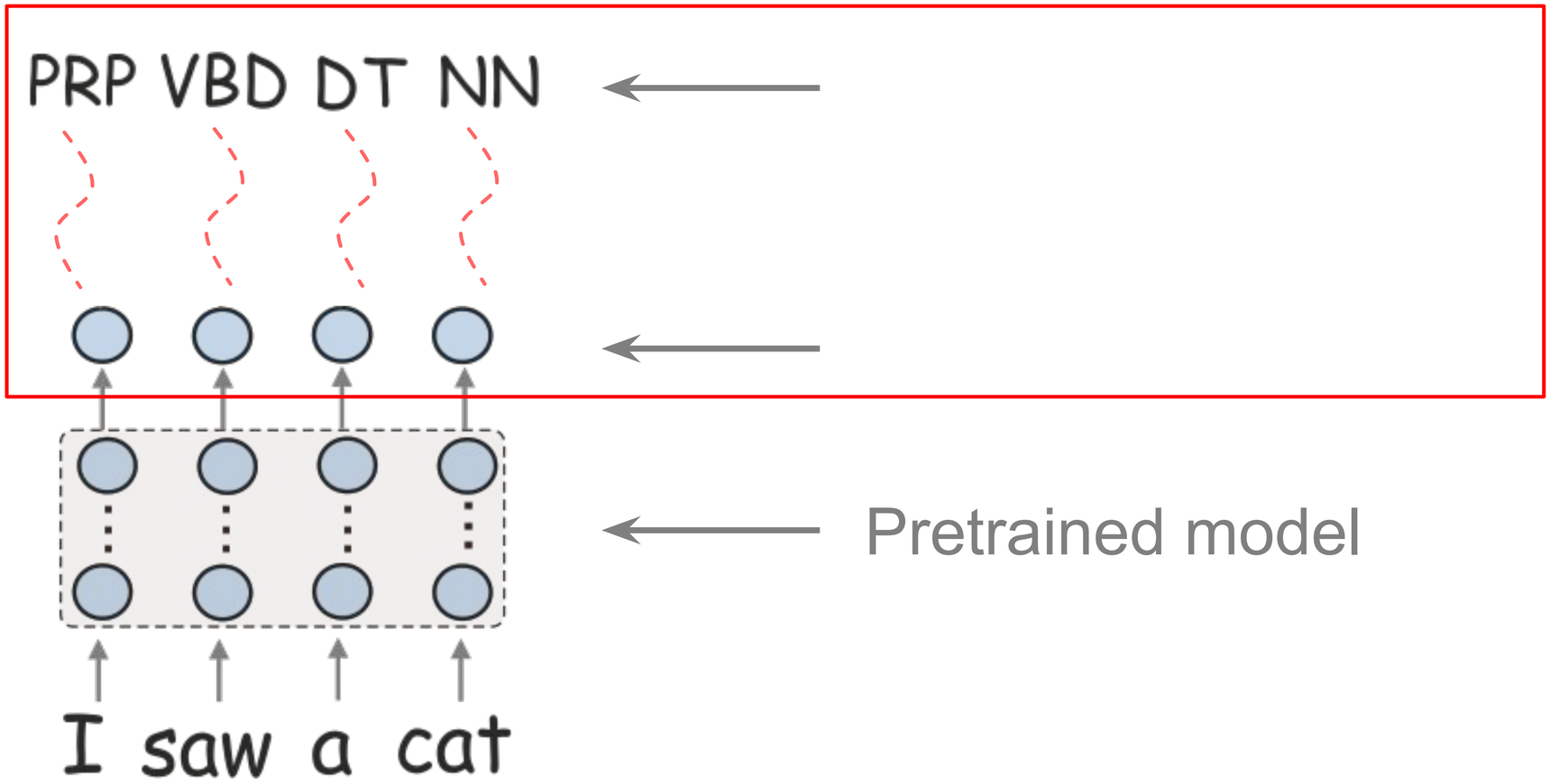
How to understand if a model captures a linguistic property?

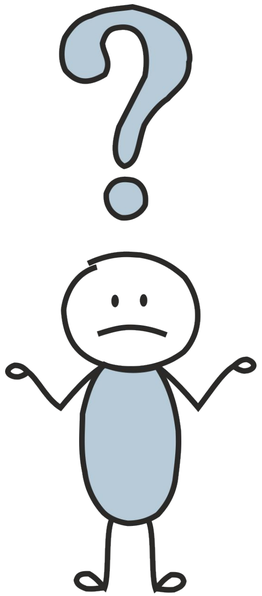


How to understand if a model captures a linguistic property?

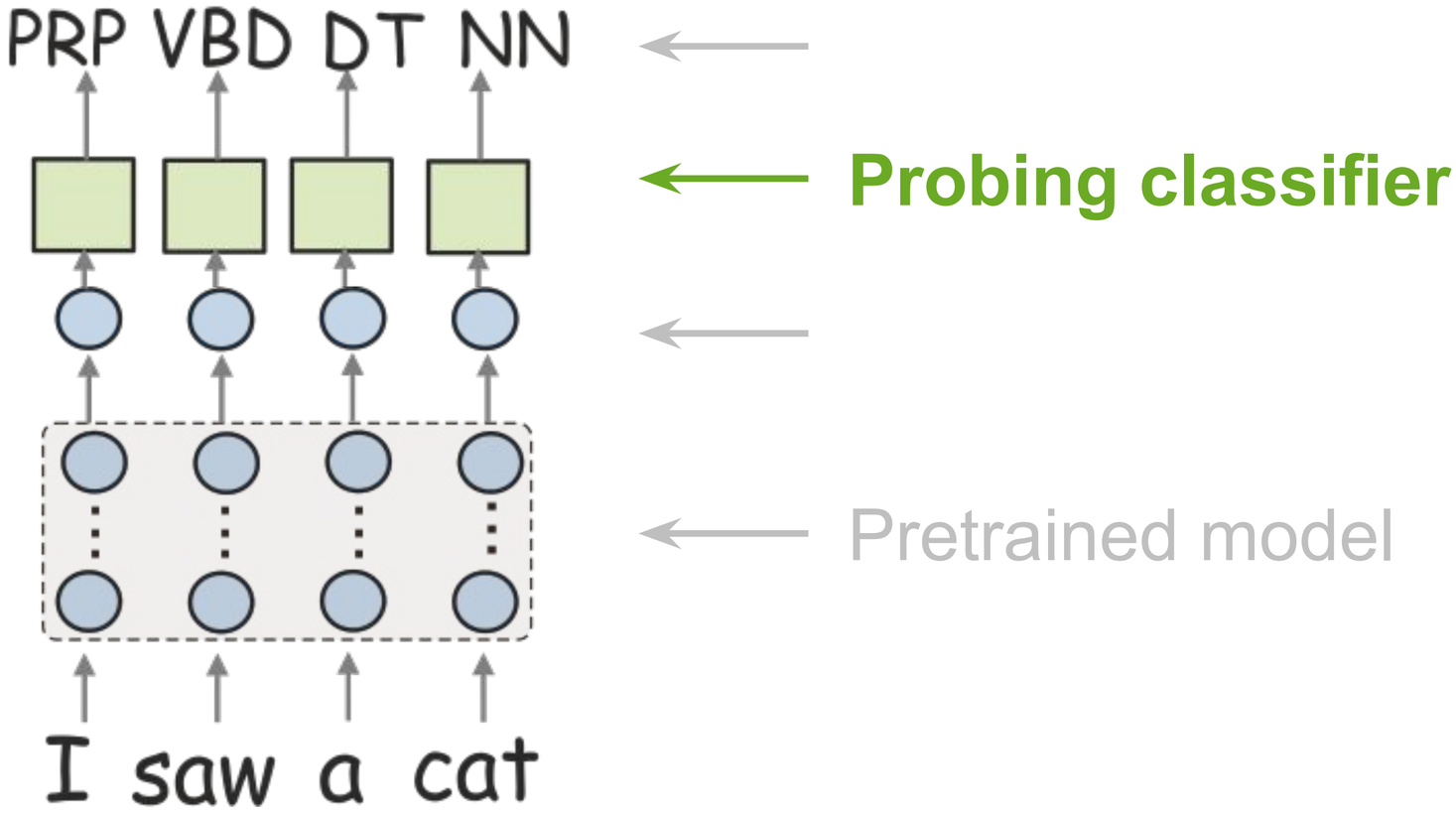


How to understand if a model captures a linguistic property?



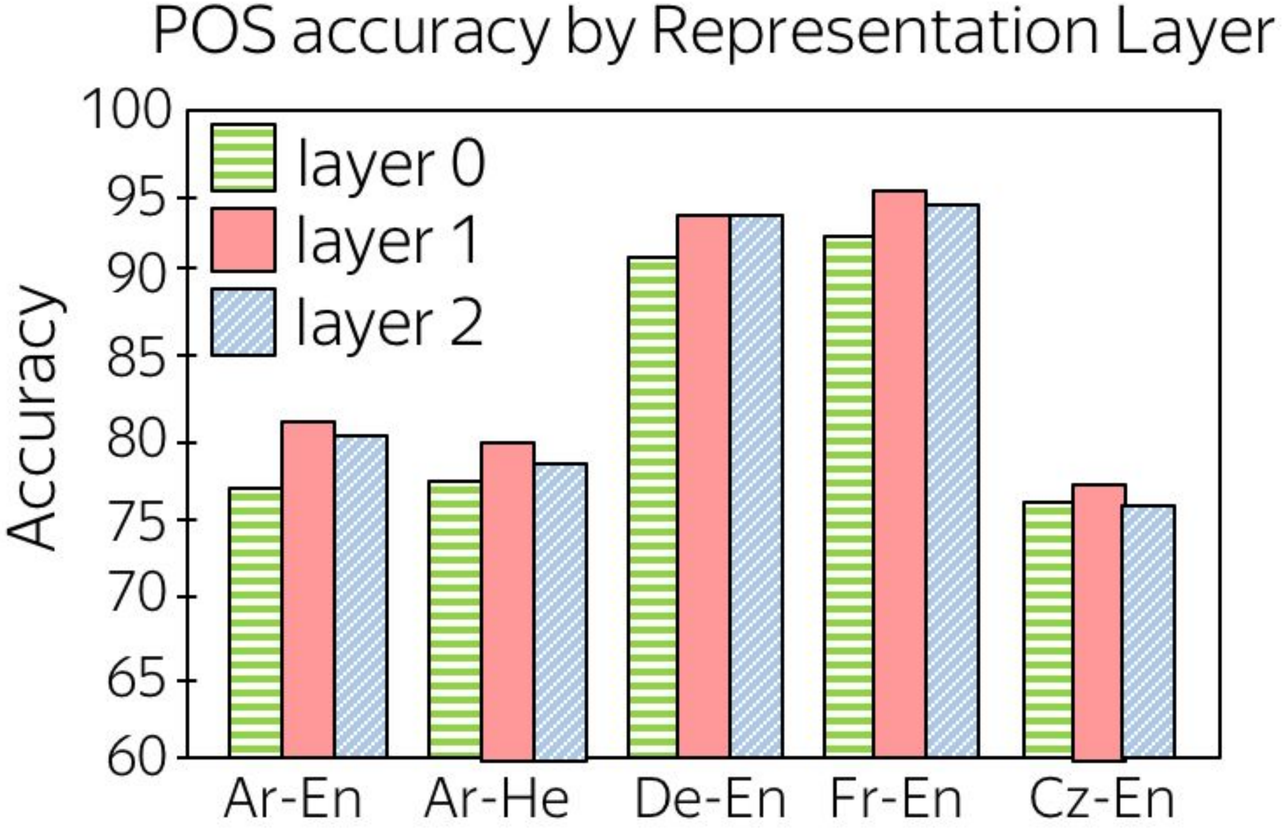


Standard Probing: train a classifier, use its accuracy



What Do NMT Models Learn About Morphology?

- Take NMT models for different language pairs
- Look at the encoder layers



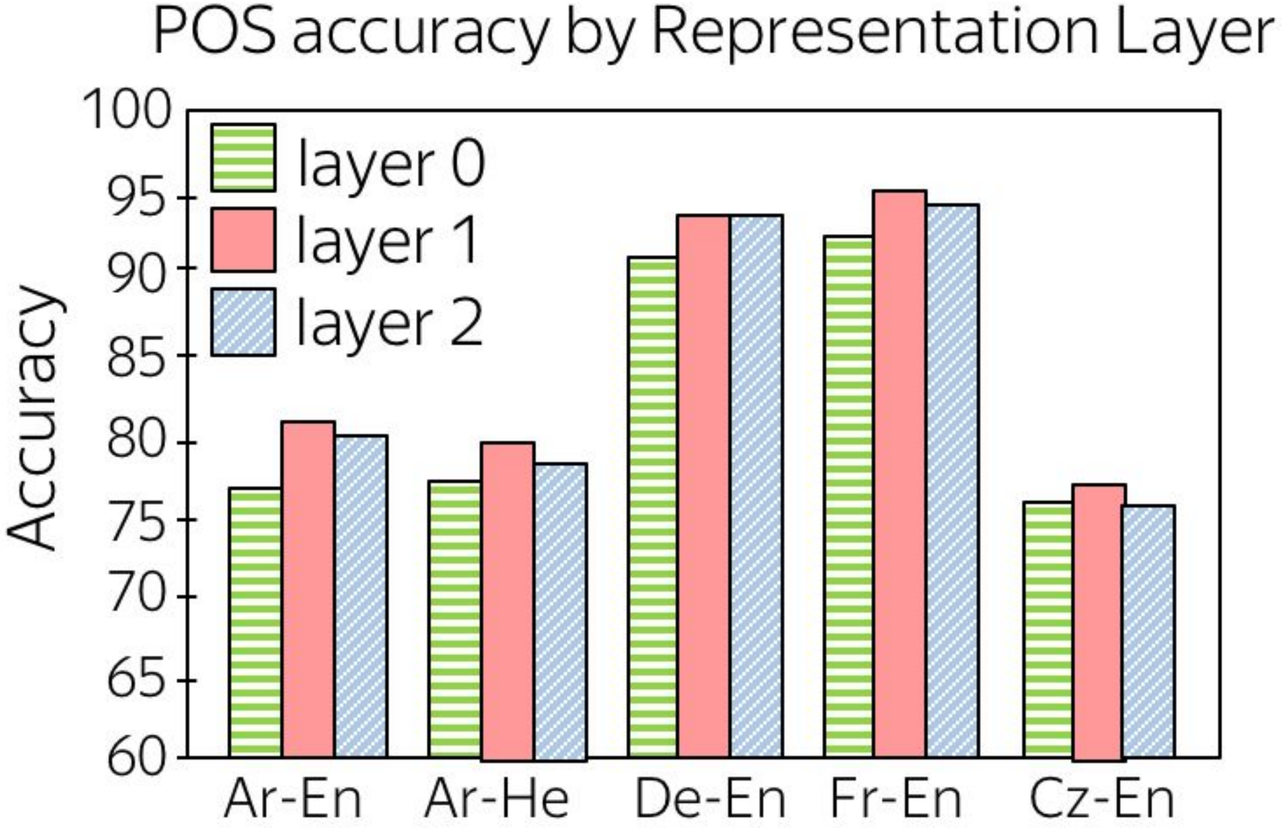
Paper: What do Neural Machine Translation Models Learn about Morphology?

What Do NMT Models Learn About Morphology?

- Take NMT models for different language pairs
- Look at the encoder layers

<u>Results:</u>

- Encoding helps: layer 0 (word embeddings) is the worst



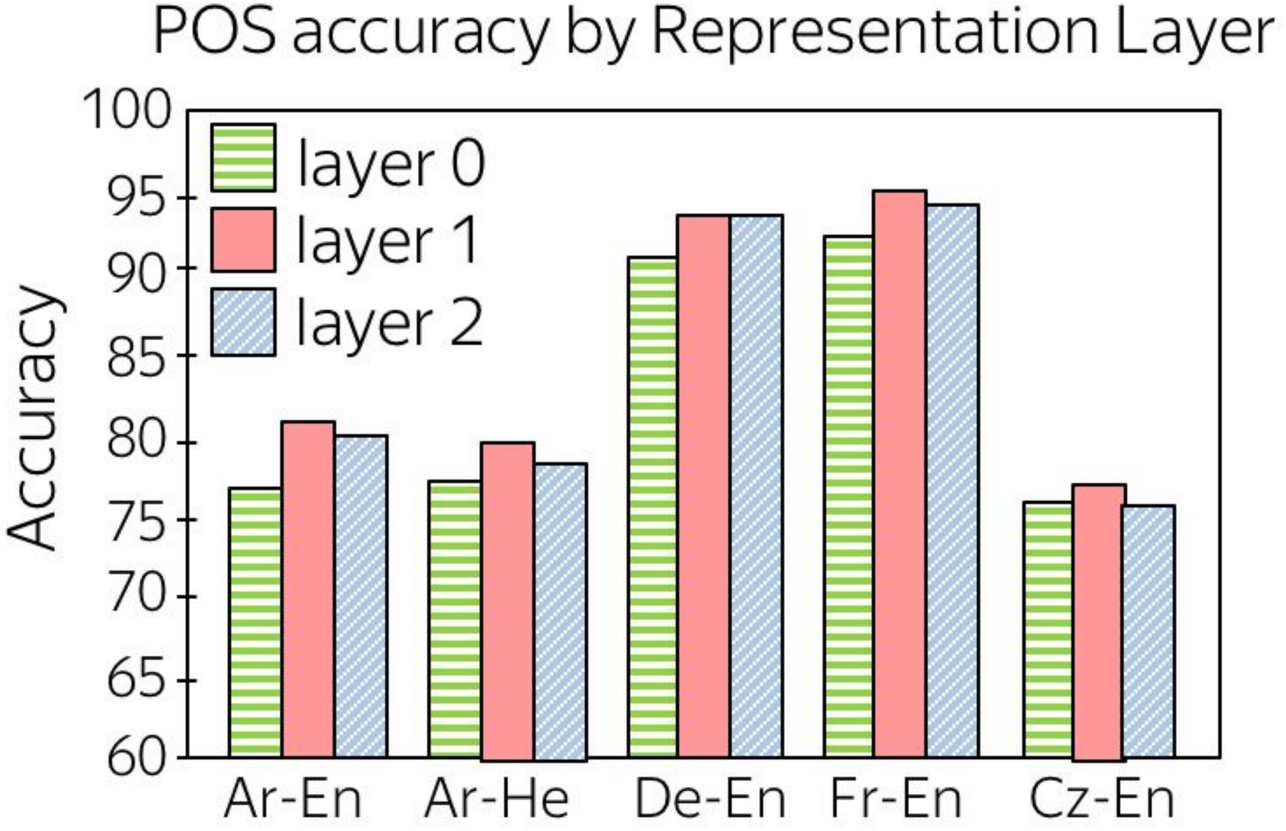
Paper: <u>What do Neural Machine Translation Models Learn about Morphology?</u>

What Do NMT Models Learn About Morphology?

- Take NMT models for different language pairs
- Look at the encoder layers

<u>Results:</u>

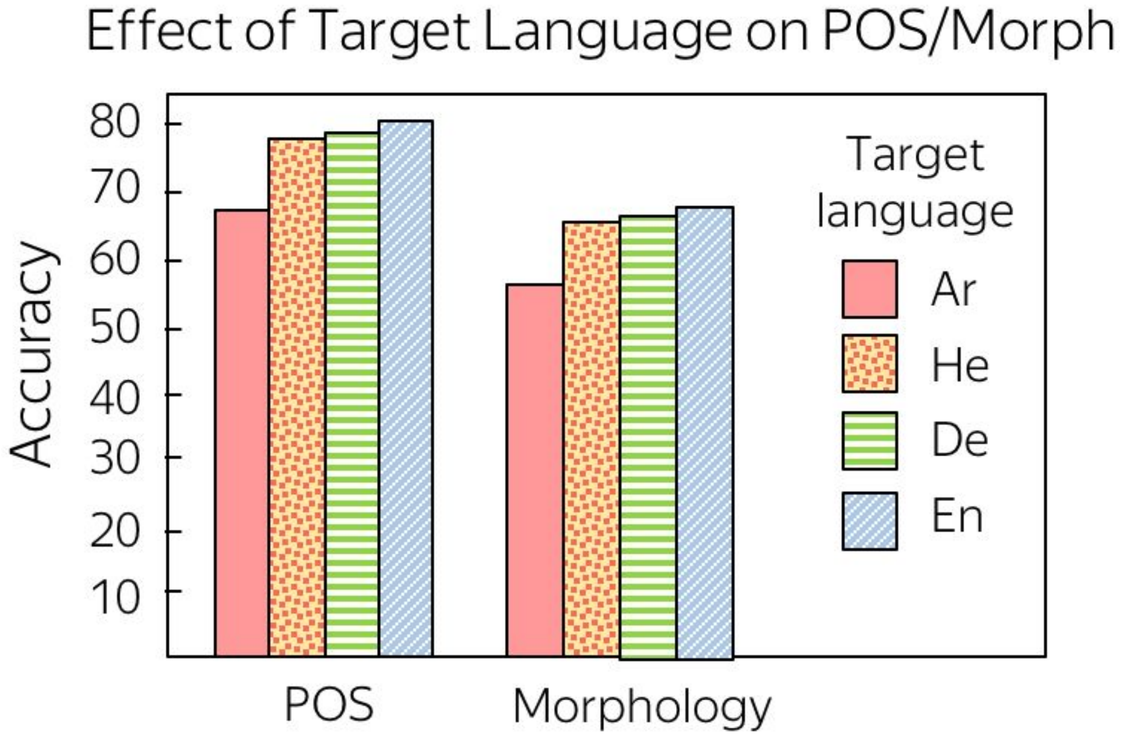
- Encoding helps: layer 0 (word embeddings) is the worst
- Layer 1 is better than layer 2



Paper: <u>What do Neural Machine Translation Models Learn about Morphology?</u>

What Do NMT Models Learn About Morphology?

- Take NMT models with the same source language and different target languages
- Look at the encoder



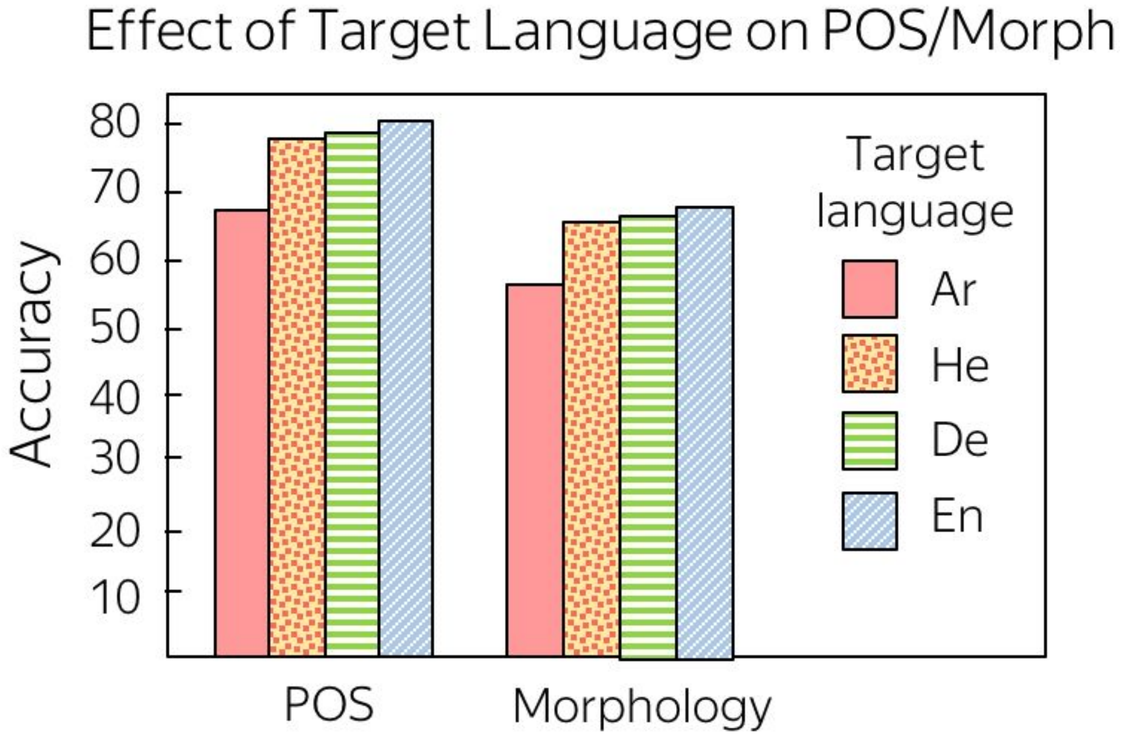
Paper: What do Neural Machine Translation Models Learn about Morphology?

What Do NMT Models Learn About Morphology?

- Take NMT models with the same source language and different target languages
- Look at the encoder

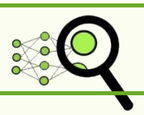
<u>Results:</u>

- Surprising: weaker target morphology leads to stronger encoder



Paper: <u>What do Neural Machine Translation Models Learn about Morphology?</u>

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Subword Segmentation: BPE
-  Analysis and Interpretability

 Learn more in the NLP Course For You → This is up to You!